

# Databases

Generated 9/27/2026, 10:26:34 PM • 61 messages • Source: Shared ChatGPT link  
<https://chatgpt.com/share/6ab94a74-a990-83e8-b92d-26e042b949ec>

USER

Databases

ASSISTANT

Absolutely. We'll now start a **fresh GATE subject: Databases / DBMS**, using the same complete-teacher approach as Computer Networks.

Please paste the **official GATE Database syllabus** here.

Once you send it, I'll convert it into a structured course and teach it **topic by topic**, including:

- Concepts from basics !' GATE depth
- Simple explanations + examples
- Important formulas and rules
- SQL and relational algebra
- Functional dependencies and normalization
- Transactions, concurrency control and recovery
- Indexing and file organization
- GATE traps and commonly confused concepts
- PYQ-oriented problem-solving methods
- Shortcuts/mnemonics
- Notes you can directly write
- Topic-wise practice questions
- Quick revision points

We'll proceed **one manageable topic at a time**. After completing each topic, you can simply type **`NEXT`** to continue.

USER

E R m o d e l: relational model: relational algebra, tuple calculus, SQL. Integrity constraints, normal forms. File organization, indexing (e.g., B and B+ trees). Transactions and concurrency control

This are the complete syllabus copy of gate So i have no flexibility to watch some hours of youtube video i want you to take time and give complete syllabus in detail in which you take your own time and give complete information related to gate and all the stuff give me a perfect prompt ... I keep changing the syllabus i also include different syllabus of gate So that i can make notes and give me some shortcut to remember concepts

ASSISTANT

Yes — since you keep changing the GATE subject, the best thing is **one reusable master prompt** where you only replace the syllabus each time.

Use this prompt exactly:

```
:::writing{variant="document" id="41827"}
```

# GATE SUBJECT MASTER TEACHING PROMPT

---

I am preparing for **GATE** and I do not have enough time to watch many hours of YouTubelectures or search multiple resources.

I will give you the **official GATE syllabus for one subject**. Your job is to act as my **complete GATE teacher + note-making assistant + PYQ-oriented mentor** and teach me the entire syllabus from basics to GATE exam level.

## SUBJECT

---

**Subject Name:** Databases / DBMS

## OFFICIAL GATE SYLLABUS

---

ER-model.

Relational model:

- Relational algebra
- Tuple calculus
- SQL

Integrity constraints.

Normal forms.

File organization.

Indexing:

- B Trees
- B+ Trees
- Other indexing concepts relevant for GATE

Transactions and concurrency control.

---

## IMPORTANT INSTRUCTIONS

---

I have limited preparation time.

Do NOT simply give me a syllabus outline or tell me to watch lectures/books.

You must actually **teach me the complete subject inside this chat**, topic by topic, with enough detail that I can make my own GATE notes from your explanation.

Assume I may be weak in the fundamentals.

Start from the basics, but gradually reach the depth required to solve GATE questions.

Do not unnecessarily go into advanced university-level topics that are outside GATE.

However, if a concept is not explicitly written in the syllabus but is **necessary to understand or solve GATE questions from one of the listed topics**, include it.

---

## FIRST: ANALYSE THE SYLLABUS

---

Before teaching, break the complete syllabus into a logical learning sequence.

---

For every topic, classify it as:

- Foundation
- Important
- Very Important
- High PYQ / High-Scoring

Also identify dependencies.

For example:

Topic A is required before Topic B.

Create a numbered roadmap such as:

1. Database fundamentals
2. ER Model
3. Relational Model
4. Relational Algebra
5. Tuple Relational Calculus
6. SQL
7. Integrity Constraints
8. Functional Dependencies
9. Normalization
10. File Organization
11. Indexing
12. B Tree
13. B+ Tree
14. Transactions
15. Schedules and Serializability
16. Concurrency Control
17. Revision + Mixed Problems

Modify this sequence if a better GATE-oriented order exists.

After showing the roadmap, start **Topic 1 immediately**.

Do NOT dump the entire subject in one huge response.

Teach one meaningful section at a time.

I will type:

**NEXT**

when I want to continue.

---

## FOR EVERY TOPIC, FOLLOW THIS FORMAT

---

### 1. WHY THIS TOPIC EXISTS

---

Before definitions, explain in very simple language:

- What problem does this concept solve?

- Why was it introduced?
- Where does it fit in DBMS?
- Why does GATE ask about it?

Give an intuitive real-life example whenever possible.

---

## 2. COMPLETE CONCEPT EXPLANATION

---

Teach the concept from basics to GATE level.

Explain every important term.

Do not skip intermediate reasoning.

For example, while teaching normalization, don't just define:

1NF 2NF 3NF BCNF

First explain:

- attributes
- tuples
- keys
- super keys
- candidate keys
- prime/non-prime attributes
- functional dependency
- trivial/non-trivial FD
- closure
- minimal/canonical cover
- partial dependency
- transitive dependency

and then build toward normal forms.

Use this same dependency-first approach for every chapter.

---

## 3. NOTE-MAKING VERSION

---

After teaching the concept, give a section called:

### NOTEBOOK NOTES

---

These must be concise, structured notes that I can directly write into my GATE notebook.

Use:

- definitions
  - formulas
  - small diagrams
  - tables
  - important properties
-

- conditions
- differences
- exceptions
- shortcuts

Do not make notes unnecessarily wordy.

---

## 4. MEMORY TRICKS / SHORTCUTS

---

For every topic, wherever possible, give:

- mnemonics
- memory tricks
- pattern-recognition tricks
- calculation shortcuts
- elimination techniques for MCQs
- quick methods for NAT questions

Clearly label them:

### MEMORY TRICK

or

### GATE SHORTCUT

Example:

For ACID:

A !' Atomicity C !' Consistency I !' Isolation D !' Durability

But provide smarter shortcuts wherever possible rather than obvious acronyms only.

---

## 5. GATE TRAPS

---

Give a section:

### & pGATE TRAPS

---

Mention common mistakes such as:

- confusing super key with candidate key
- confusing 3NF and BCNF
- incorrect attribute closure
- confusing lossless join with dependency preservation
- assuming SQL duplicates are removed automatically
- confusing WHERE and HAVING
- confusing clustered and primary index
- B-tree vs B+ tree differences
- conflict serializability vs view serializability
- recoverable vs cascadeless vs strict schedules

- deadlock vs starvation

Whenever GATE uses wording designed to trick students, point it out.

---

## 6. PYQ-ORIENTED UNDERSTANDING

---

Explain:

- What kinds of questions GATE asks from this topic
- Which information is usually given
- What we are expected to find
- Standard solving procedure
- Typical traps
- Fastest reliable method

Do not merely say “practice PYQs.”

Teach me **how to recognize the question type**.

---

## 7. SOLVED GATE-STYLE EXAMPLES

---

Give several examples from easy !' moderate !' difficult.

For numerical or logical questions:

Show:

Given !' Required !' Method !' Calculation / reasoning !' Answer

But also teach me the **fast exam method** after the detailed solution.

Where useful, include questions similar in pattern to actual GATE PYQs without falsely claiming they are exact previous-year questions.

---

## 8. FORMULAS / RULES

---

Whenever formulas or fixed rules exist, create a separate section.

Examples may include:

- attribute closure
- candidate key determination
- normalization rules
- lossless decomposition conditions
- number of possible superkeys
- B/B+ tree height/order calculations
- blocking factor
- number of blocks
- indexing calculations
- transaction schedule conditions
- serializability rules

For every formula explain:

---

1. Meaning
  2. Variables
  3. When to use
  4. Common mistake
  5. One numerical example
- 

## 9. COMPARISON TABLES

---

Whenever two concepts are easily confused, give tables.

Examples:

Primary Key vs Candidate Key vs Super Key

ER Model vs Relational Model

Relational Algebra vs Tuple Calculus

Selection vs Projection

DELETE vs DROP vs TRUNCATE

WHERE vs HAVING

Nested Query vs Correlated Query

3NF vs BCNF

Lossless vs Dependency Preserving

Dense vs Sparse Index

Primary vs Clustering vs Secondary Index

B Tree vs B+ Tree

Serial vs Serializable Schedule

Conflict vs View Serializability

Recoverable vs Cascadeless vs Strict

Lock-Based vs Timestamp-Based Protocols

---

## 10. DIAGRAMS

---

Use simple text/ASCII diagrams wherever diagrams improve understanding.

Example:

```
Student
|
| Enrolls
|
Course
```

For B/B+ Trees, transactions, ER diagrams, database architecture, etc., show simple understandable diagrams.

---

## SPECIAL INSTRUCTIONS FOR EACH MAJOR DBMS AREA

---

### ER MODEL

---

Cover at least:

- entity
  - entity set
  - attributes
  - simple/composite
  - single-valued/multi-valued
  - derived attributes
  - keys
  - relationships
  - degree of relationship
  - cardinality
  - participation
  - weak entities
  - identifying relationship
  - ER notation
  - ER-to-relational mapping
  - common GATE problems
- 

### RELATIONAL MODEL

---

Cover:

- relation
  - tuple
  - attribute
  - domain
  - degree
  - cardinality
  - schema vs instance
  - keys
  - NULL
  - relational properties
- 

### RELATIONAL ALGEBRA

---

Teach thoroughly:

- selection
  - projection
  - union
  - intersection
-



- difference
- Cartesian product
- rename
- joins
- theta join
- equi join
- natural join
- outer joins if GATE-relevant
- division operator

Explain query evaluation step by step.

Give many relational algebra expression problems.

---

## TUPLE RELATIONAL CALCULUS

---

Cover:

- declarative nature
  - tuple variables
  - predicates
  - existential quantifier
  - universal quantifier
  - free/bound variables
  - safe expressions
  - conversion-style GATE questions
  - RA vs TRC
- 

## SQL

---

Teach GATE-relevant SQL thoroughly:

- CREATE
  - INSERT
  - UPDATE
  - DELETE
  - SELECT
  - WHERE
  - DISTINCT
  - ORDER BY
  - GROUP BY
  - HAVING
  - aggregate functions
  - joins
  - nested queries
  - correlated subqueries
-

- EXISTS
- NOT EXISTS
- IN
- ANY
- ALL
- NULL behavior
- views
- constraints
- set operations

Pay special attention to SQL output-prediction questions.

Always show the intermediate table/result when useful.

---

## INTEGRITY CONSTRAINTS

---

Cover:

- domain constraint
  - key constraint
  - entity integrity
  - referential integrity
  - foreign key
  - assertions/check constraints if relevant
  - effects of INSERT/DELETE/UPDATE
- 

## FUNCTIONAL DEPENDENCIES + NORMALIZATION

---

This section must be extremely strong because it is important for GATE.

Cover:

- functional dependency
  - trivial FD
  - non-trivial FD
  - completely non-trivial FD
  - Armstrong axioms
  - attribute closure
  - FD closure
  - superkey
  - candidate keys
  - prime/non-prime attributes
  - minimal/canonical cover
  - decomposition
  - lossless decomposition
  - dependency preservation
  - 1NF
-

- 2NF
- 3NF
- BCNF
- multivalued dependencies / 4NF only if GATE syllabus or PYQs require it

Give plenty of candidate-key and normalization problems.

---

## FILE ORGANIZATION

---

Cover:

- records
  - blocks/pages
  - fixed/variable length records
  - blocking factor
  - spanned/unspanned organization
  - heap files
  - sequential files
  - hashing if GATE-relevant
  - file access cost basics
- 

## INDEXING

---

Cover:

- index basics
- search key
- primary index
- clustering index
- secondary index
- dense index
- sparse index
- multilevel index
- ordered indexing

Explain disk-access calculations carefully.

---

## B TREE AND B+ TREE

---

Teach from fundamentals.

Cover:

- order
  - keys
  - children
  - minimum/maximum occupancy
  - leaf/internal nodes
-

- insertion intuition
- search
- height
- number of records/keys
- B-tree vs B+ tree
- GATE numerical problems

When different textbooks use different definitions of “order,” explicitly warn me and show how to interpret the question.

---

## TRANSACTIONS

---

Cover:

- transaction concept
- read/write operations
- transaction states
- ACID properties
- schedules
- serial schedules
- concurrent schedules
- anomalies

Explain:

- lost update
  - dirty read
  - unrepeatable read
  - incorrect summary
- 

## SERIALIZABILITY

---

Cover in depth:

- conflict operations
- conflict equivalence
- conflict serializability
- precedence/serialization graph
- cycle detection
- equivalent serial order
- view serializability
- blind writes

Give several graph-based problems.

---

## RECOVERABILITY

---

Cover:

- recoverable schedule
-

- unrecoverable schedule
- cascading rollback
- cascadeless schedule
- strict schedule

Give an easy hierarchy/memory method.

---

## CONCURRENCY CONTROL

---

Cover:

- locks
- shared lock
- exclusive lock
- lock compatibility
- two-phase locking
- basic 2PL
- strict 2PL
- rigorous 2PL if GATE-relevant
- lock conversion
- deadlock
- wait-for graph
- timestamp ordering where required

Clearly distinguish:

Serializability guarantee vs Recoverability guarantee.

---

## AFTER EACH TOPIC

---

End with:

## 30-SECOND REVISION

---

Give only the absolute essential facts.

Then:

## CHECK YOUR UNDERSTANDING

---

Ask me approximately **3–5 GATE-style questions**.

Do NOT immediately reveal the answers unless I ask or attempt them.

Finally write:

**Type `NEXT` to continue to the next topic.**

---

## AFTER EVERY MAJOR CHAPTER

---

Give:

---

## CHAPTER REVISION SHEET

containing:

- Must-remember definitions
  - Must-remember formulas
  - Most common GATE traps
  - Most important question patterns
  - 5-minute revision notes
- 

## AFTER COMPLETING THE ENTIRE SYLLABUS

Create a final:

## DBMS GATE MASTER REVISION

Include:

1. Complete formula sheet
2. Complete shortcut sheet
3. Important definitions
4. Most confusing concept comparisons
5. High-frequency GATE traps
6. SQL quick-reference sheet
7. Normalization decision method
8. B/B+ Tree calculation sheet
9. Transaction/serializability decision sheet
10. Last-day revision notes
11. 30 most important conceptual questions
12. 20 mixed GATE-style problems
13. Weak-area checklist

Also show me how all topics are interconnected.

---

## TEACHING STYLE

Use simple English.

Assume that my aim is not merely to memorize definitions.

My goal is:

**UNDERSTAND ! ' MAKE NOTES ! ' REMEMBER ! ' RECOGNIZE QUESTION TYPE ! ' SOLVE GATE QUESTIONS FAST**

Whenever I appear confused, explain using an easier example instead of repeating the same definition.

Avoid unnecessarily long historical explanations.

Prioritize:

**Concept clarity + GATE relevance + problem solving + revision efficiency.**

---

Start now by:

1. Analysing the complete syllabus
2. Creating the GATE-oriented learning roadmap
3. Identifying the highest-scoring/highest-priority areas
4. Explaining how the topics connect
5. Starting Topic 1 immediately

...

## How to reuse this for any GATE subject

You **do not need a new prompt every time**. Just change these two parts:

Subject Name: Databases / DBMS

and

OFFICIAL GATE SYLLABUS: ...

For example, later you can replace it with **Operating Systems, Computer Networks, Algorithms, COA, Digital Logic, Engineering Mathematics**, etc. The rest of the prompt can remain exactly the same.

For **DBMS specifically**, I would make **Normalization/FDs, SQL + Relational Algebra, Transactions/Serializability, and Indexing/B+ Trees** especially strong because those areas contain many reasoning/numerical-style questions where shortcuts and question-pattern recognition matter a lot.

If you paste the prompt into this **Databases conversation itself**, I can start teaching from **Topic 1 and build your notes here continuously**.

### ASSISTANT

Your DBMS master prompt is clear, and I'll follow it as the standing teaching method for this subject. The official syllabus you supplied covers **ER model; relational model including relational algebra, tuple calculus and SQL; integrity constraints; normal forms; file organization; indexing including B/B+ trees; transactions; and concurrency control**.

## GATE DATABASES/ DBMS — COMPLETE COURSE

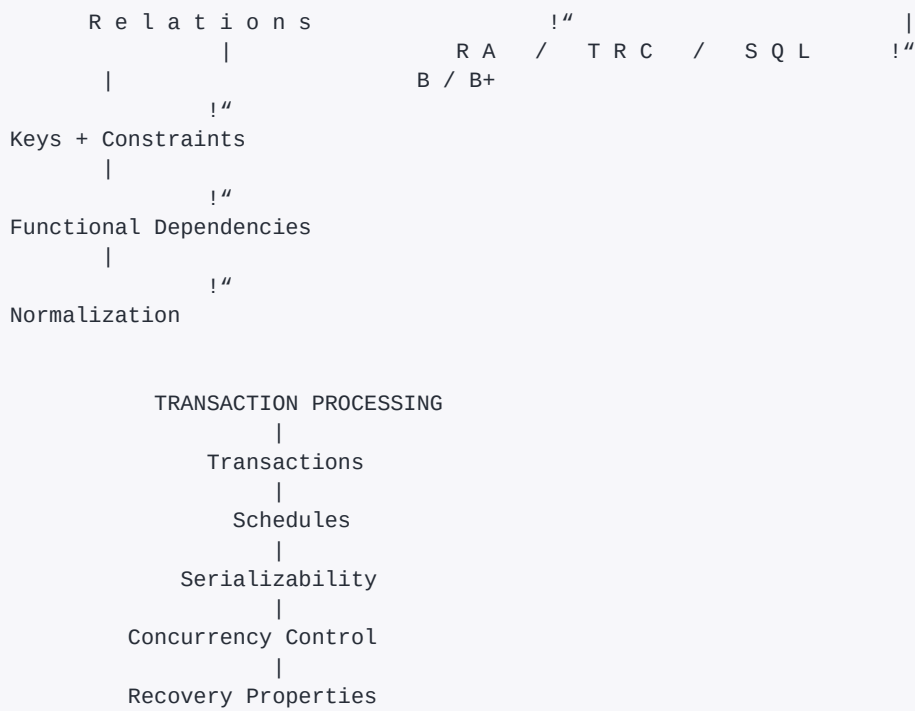
### Part 1 — GATE-oriented roadmap

I'm going to arrange the syllabus slightly differently from the order in which it is written, because some concepts must be understood before others.

| # | Topic                        | Priority           | Why                                      |
|---|------------------------------|--------------------|------------------------------------------|
| 1 | Database Fundamentals        | Foundation         | Vocabulary needed everywhere             |
| 2 | ER Model                     | Important          | Conceptual modelling + mapping questions |
| 3 | Relational Model             | Very Important     | Base for RA, SQL, keys, normalization    |
| 4 | Keys & Integrity Constraints | Very Important     | Required for normalization + SQL         |
| 5 | Relational Algebra           | Ø=Ý% H i g h P Y Q | Logic-heavy and frequently tested        |
| 6 | Tuple Relational Calculus    | Important          | Declarative query reasoning              |



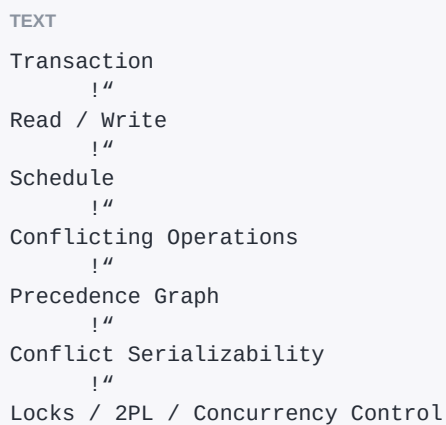




The most important dependency chain is:



And for transactions:



## Where you should give maximum attention

---

If preparation time becomes limited, never treat every chapter equally.

### Tier 1 — Master these

**Normalization + Functional Dependencies**

**SQL**

**Relational Algebra**

**Transactions + Serializability**

**B/B+ Trees**

These contain many questions where knowing the solving technique gives you a major advantage.

### Tier 2 — Know very well

- Keys
- Integrity constraints
- Indexing
- Recoverability
- 2PL
- ER Model

### Tier 3 — Don't ignore, but usually smaller

- Basic database terminology
- Tuple calculus
- Basic file organization

Now we begin properly.

---

## TOPIC 1 — DATABASE FUNDAMENTALS

---

### 1. WHY DOES A DATABASE EXIST?

---

Suppose a college stores student information like this:

TEXT

```
Student1.txt
Student2.txt
Student3.txt
Marks.xlsx
Attendance.xlsx
Fees.xlsx
Placement.xlsx
```

This appears manageable initially.

But soon problems occur.

Suppose student Ravi changes his phone number.

His phone number may occur in:

- Student file

- Hostel file
- Placement file
- Library file

You must change it everywhere.

If one file is forgotten:

TEXT

```

Student file    !'  9 8 7 6 5 4 3 2 1 0
Placement file !'  9 1 2 3 4 5 6 7 8 0

```

Now two different phone numbers exist for the same student.

This is called **data inconsistency**.

A database system exists mainly to manage large amounts of related data **systematically, consistently and efficiently**.

## 2. DATA

**Data = raw facts or values.**

Examples:

TEXT

```

101
Ravi
CSE
85
Hyderabad

```

Individually, these are simply values.

When given context:

TEXT

```

Roll No = 101
Name = Ravi
Department = CSE
Marks = 85
City = Hyderabad

```

they become meaningful information.

## 3. DATABASE

### Definition

A **database** is an organized collection of related data.

Example:

TEXT

STUDENT

| RollNo | Name  | Dept | Marks |
|--------|-------|------|-------|
| 101    | Ravi  | CSE  | 85    |
| 102    | Anu   | AIML | 92    |
| 103    | Kiran | CSE  | 78    |

This table may be part of a college database.

The complete database might contain:

TEXT

STUDENT

COURSE

FACULTY

ENROLLMENT

MARKS

FEES

All are related.

## 4. DBMS

**DBMS = Database Management System**

It is software that allows users/applications to:

TEXT

Create

Store

Retrieve

Modify

Delete

Manage

Protect

data.

Examples of DBMS products include:

- PostgreSQL
- MySQL
- Oracle Database
- SQL Server

For GATE, product-specific knowledge is usually much less important than the **underlying DBMS concepts**.

## 5. DATABASE vs DBMS

Very important distinction.

**Database**

The **actual data**.

## DBMS

The **software used to manage that data.**

Memory analogy:

TEXT

Database = Books  
DBMS = Librarian/System managing books

## 6. WHY NOT JUST USE FILES?

Before DBMSs, applications often relied heavily on traditional file-processing systems.

Example:

TEXT

Students.txt  
Marks.txt  
Courses.txt  
Faculty.txt

Several major problems arise.

### Problem 1 — Data redundancy

**Redundancy = same data stored repeatedly.**

Example:

TEXT

| Roll | Name | Dept |
|------|------|------|
| 1    | Ravi | CSE  |

Ravi's department might appear in:

TEXT

Student file  
Fee file  
Placement file  
Hostel file

The same value gets repeated.

### Problem

Extra storage + harder maintenance.

## 7. DATA INCONSISTENCY

Suppose:

TEXT

Student file:  
R a v i ! ' C S E

Placement file:  
R a v i ! ' A I M L

Which is correct?

The same fact has different values.

That is **data inconsistency**.

## Relationship

TEXT

Redundancy  
! "  
Multiple copies  
! "  
Different updates  
! "  
Inconsistency

## Memory trick

**Redundancy creates copies; inconsistency creates contradictions.**

Do not confuse them.

## 8. DIFFICULT DATA ACCESS

Suppose the principal asks:

**Find all CSE students with marks above 80 who live in Hyderabad.**

With independent files, programmers may need to write a special program to search and combine information.

DBMS query languages make such retrieval easier.

Example conceptually:

SQL

```
SELECT *  
FROM Student  
WHERE Dept = 'CSE'  
AND Marks > 80  
AND City = 'Hyderabad';
```

We will learn SQL later in depth.

## 9. DATA ISOLATION

Suppose:

TEXT

```
M a r k s           !'   E x c e l
S t u d e n t s     !'   T e x t   f i l e
F e e s             !'   a n o t h e r   f o r m a t
P l a c e m e n t   !'   s e p a r a t e   a p p l i c a t i o n
```

Data exists at different locations and perhaps in different formats.

Combining them becomes difficult.

This is called **data isolation** in the file-system problem context.

## 10. INTEGRITY PROBLEMS

Suppose the university rule is:

TEXT

Marks must be between 0 and 100.

If somebody enters:

TEXT

Marks = 135

the database has invalid data.

Such correctness rules are called **integrity constraints**.

Another example:

TEXT

Student Roll Number must be unique.

Later we will study:

- Domain constraints
- Key constraints
- Entity integrity
- Referential integrity

## 11. ATOMICITY PROBLEM

This becomes extremely important when we study transactions.

Imagine transferring  $11000$ :

TEXT

```
A c c o u n t   A   !'   s u b t r a c t   1 1 0 0 0
A c c o u n t   B   !'   a d d       1 1 0 0 0
```

Suppose:

TEXT

A = 1 5 0 0 0  
B = 1 2 0 0 0

Step 1:

TEXT

A = 1 4 0 0 0

Then the system crashes before Step 2.

B remains:

TEXT

1 2 0 0 0

1 1 0 0 0 effectively vanished from the logical transaction.

Correct behaviour should be:

TEXT

Either BOTH operations happen

OR

NEITHER happens.

That principle is called **atomicity**.

We'll study this under ACID properties.

---

## 12. CONCURRENT ACCESS PROBLEMS

---

Imagine:

TEXT

B a n k   b a l a n c e   =   1 1 0 , 0 0 0

Two transactions execute simultaneously.

TEXT

T 1   w i t h d r a w s   1 2 , 0 0 0  
T 2   w i t h d r a w s   1 3 , 0 0 0

Poor coordination could produce an incorrect final balance.

Correct:

TEXT

10,000 - 2,000 - 3,000 = 5,000

Concurrent access is useful for speed, but needs **concurrency control**.

This directly connects to the last part of your GATE syllabus.



---

## 13. SECURITY PROBLEMS

Not every user should see everything.

Example:

Student:

TEXT

Can view own marks

Teacher:

TEXT

Can update marks

Accounts office:

TEXT

Can view fees

Database administrator:

TEXT

Broader control

A DBMS supports controlled access.

---

## File System vs DBMS

| Feature       | File System                | DBMS                         |
|---------------|----------------------------|------------------------------|
| Redundancy    | Usually higher             | Can be controlled            |
| Consistency   | Harder                     | Better controlled            |
| Querying      | Application-dependent      | Query languages              |
| Constraints   | Harder to enforce globally | Built-in mechanisms          |
| Concurrency   | Difficult                  | Supported                    |
| Security      | Basic/application-specific | Stronger control possible    |
| Recovery      | Harder                     | Transaction/recovery support |
| Relationships | Manual                     | Systematically represented   |

& Do **not** memorize this as “DBMS has zero redundancy.”

Some redundancy may exist.

The better statement is:

DBMS helps control/reduce unnecessary redundancy.

## 14. DATABASE SYSTEM

There is another distinction worth remembering.

```
TEXT
Database
+
DBMS
+
Applications
+
Users
```

together form a broader **database system**.

## 15. SCHEMA

This is extremely important.

A **schema** describes the structure/design of the database.

Example:

```
TEXT
STUDENT(
    RollNo,
    Name,
    Department,
    Marks
)
```

This describes **what attributes exist**.

Think:

**Schema = blueprint.**

## 16. INSTANCE

An **instance** is the actual data stored at a particular point in time.

Example:

```
TEXT
STUDENT

RollNo  Name  Dept
-----
101     Ravi  CSE
102     Anu   AIML
```

Tomorrow:

TEXT

103      Kiran      ECE

may be added.

Now the instance has changed.

But the schema can remain:

TEXT

STUDENT(RollNo, Name, Dept)

## Schema vs Instance

| Schema               | Instance                                |
|----------------------|-----------------------------------------|
| Structure            | Actual content                          |
| Database design      | Current database state                  |
| Changes rarely       | Changes frequently                      |
| Similar to blueprint | Similar to constructed/current contents |

### MEMORY TRICK

**Schema = Skeleton    Instance = Information currently inside it**

Both start conveniently with:

TEXT

S !' S t r u c t u r e  
I !' I n f o r m a t i o n

## 17. DATAMODEL

A **data model** provides concepts used to describe:

- data
- relationships
- constraints

Your syllabus contains two important modelling ideas:

TEXT

ER Model  
Relational Model

## ER Model

Used mainly for conceptual database design.

Example:

```
TEXT
S T U D E N T   % %       E n r o l l s   % %       C O U R S E
```

## Relational Model

Represents data using **relations/tables**.

```
TEXT
STUDENT

RollNo | Name | Dept
```

We will study both separately.

---

## 18. ABSTRACTION — WHY USERS DON'T SEE EVERYTHING

A database internally contains complex details:

- files
- records
- blocks
- indexes
- pages
- storage structures

But ordinary users don't need all of this.

DBMS therefore provides levels of abstraction.

Common conceptual representation:

```
TEXT

View Level
      !"
Logical Level
      !"
Physical Level
```

---

## 19. PHYSICAL LEVEL

Lowest abstraction level.

Describes **how data is physically stored**.

Examples:

- files
- blocks
- indexes

- record placement

Later:

TEXT

File Organization  
B Trees  
B+ Trees

belong largely to this storage-oriented side.

---

## 20. LOGICAL LEVEL

---

Describes:

- what data is stored
- relationships between data

Example:

TEXT

```
STUDENT(  
  RollNo,  
  Name,  
  Department  
)
```

Most relational-design concepts operate around this level.

---

## 21. VIEW LEVEL

---

Highest abstraction.

Different users can see different portions of the database.

Example:

Student sees:

TEXT

Name  
RollNo  
Marks

Accounts staff sees:

TEXT

Name  
RollNo  
Fees

Principal may see broader information.

---

## MEMORY TRICK — 3 LEVELS

Think:

TEXT

V I E W            ! '   W h a t   u s e r   s e e s  
L O G I C A L   ! '   W h a t   d a t a b a s e   c o n t a i n s   s t r u c t u r a l l y  
P H Y S I C A L   ! '   H o w   i t   i s   s t o r e d

Or:

**VLP = View !' Logical !' Physical**

Top to bottom.

## 22. DATA INDEPENDENCE

Very useful GATE concept.

Data independence means changes at one database abstraction level should require minimal/no changes at higher levels.

There are two major forms:

TEXT

Physical Data Independence  
Logical Data Independence

## 23. PHYSICAL DATA INDEPENDENCE

Change physical storage **without changing the logical schema**.

Example:

Initially:

TEXT

Sequential file

Later:

TEXT

B+ Tree index

But logically:

TEXT

STUDENT(RollNo, Name, Dept)

remains unchanged.

That is physical data independence.

---

## 24. LOGICAL DATA INDEPENDENCE

---

Change logical structure **without forcing changes in external views/applications**, where supported.

Example:

Original:

```
TEXT
STUDENT(RollNo, Name)
```

Later add:

```
TEXT
STUDENT(RollNo, Name, Email)
```

Existing user views may continue operating without modification.

---

### Which one is easier to achieve?

---

Usually:

**Physical data independence is easier to achieve than logical data independence.**

This is an important conceptual fact.

#### Why?

Changing internal storage usually doesn't alter the conceptual meaning of the data.

But changing the logical schema may directly affect applications and views.

---

## MEMORY TRICK

---

### Physical independence

**Change HOW.**

### Logical independence

**Change WHAT.**

```
TEXT
P h y s i c a l   ! '   H O W   s t o r e d
L o g i c a l     ! '   W H A T   s t r u c t u r e
```

---

## 25. DATABASEUSERS

---

You should recognize the terminology, though GATE generally emphasizes concepts more heavily.

### DBA

#### Database Administrator

Responsibilities can include:

- schema management
- authorization
- backup/recovery
- performance/storage management

#### Application programmer

Writes applications accessing the database.

#### End user

Uses applications interacting with database data.

---

## 26. DBA vs DBMS

---

Don't confuse these.

TEXT

DBMS = Software

DBA = Person / administrative role

---

## NOTEBOOK NOTES

---

Write this compact version in your notes.

---

## DATABASEFUNDAMENTALS

---

### Data

Raw facts/values.

### Database

Organized collection of related data.

### DBMS

Software used to create, store, retrieve, update and manage databases.

---

### Problems with traditional file processing

1. Data redundancy
  2. Data inconsistency
-



3. Difficult data access
  4. Data isolation
  5. Integrity problems
  6. Atomicity problems
  7. Concurrent access problems
  8. Security problems
- 

## Redundancy vs Inconsistency

TEXT

Redundancy = duplicate copies

Inconsistency = conflicting values

---

## Schema

Database structure/design.

TEXT

STUDENT(RollNo, Name, Dept)

---

## Instance

Actual database contents at a particular time.

TEXT

101 Ravi CSE  
102 Anu AIML

---

## Abstraction

TEXT

View  
!"  
Logical  
!"  
Physical

View !" user-specific representation Logical !" database structure Physical !" storage implementation

---

## Data Independence

TEXT

Physical DI:  
Physical changes  
!"  
Logical schema unchanged  
  
Logical DI:  
Logical changes  
!"

---

External views/programs unaffected

Physical DI generally easier than Logical DI.

## MEMORY SHEET

Remember just these phrases:

### TEXT

DATABASE = Data

DBMS = Manager

Schema = Structure

Instance = Current Information

Physical = HOW

Logical = WHAT

View = WHAT USER SEES

## & pGATE TRAPS

### Trap 1

Database "DBMS

Database contains the data.

DBMS manages it.

### Trap 2

Schema "Instance.

If rows change:

### TEXT

Instance changes.

Not necessarily schema.

If attributes/table structure changes:

### TEXT

Schema changes.

### Trap 3

Redundancy "Inconsistency.

TEXT

Repeated same information  
!"  
Redundancy

TEXT

Different values for same fact  
!"  
Inconsistency

#### Trap 4

Physical data independence does **not** mean physical storage never changes.

It means:

**Physical storage can change without requiring change to the logical level.**

#### Trap 5

Logical data independence is generally **harder** than physical data independence.

#### Trap 6

Concurrency itself is not bad.

Concurrency:

TEXT

!' throughput  
!' resource utilization

But uncontrolled concurrency can produce incorrect results.

## PYQ-ORIENTED QUESTION RECOGNITION

### Pattern 1 — Schema vs Instance

Question says:

**Student records were inserted.**

Think:

TEXT

Instance changed.

Question says:

**A new attribute was added.**

Think:

TEXT

Schema changed.

## Pattern 2 — Data independence

Question says:

**Index changed from one structure to another without changing tables.**

Answer:

TEXT

Physical data independence

Question says:

**Logical schema changed while external user view remains unaffected.**

Answer:

TEXT

Logical data independence

## Pattern 3 — File-system disadvantage

Look for:

TEXT

same value repeated

!' r e d u n d a n c y

different copies have different values

!' i n c o n s i s t e n c y

rule cannot be maintained

!' i n t e g r i t y

simultaneous users produce wrong result

!' c o n c u r r e n c y   p r o b l e m

multi-step operation partially executes

!' a t o m i c i t y   p r o b l e m

This recognition saves exam time.

## SOLVED GATE-STYLE EXAMPLE 1

---

Suppose:

TEXT

```
EMPLOYEE(Eid, Name, Salary)
```

Initially:

TEXT

```
1 Ravi 50000
2 Anu 60000
```

Later:

TEXT

```
3 Raj 55000
```

is inserted.

What changed?

### Reasoning

The structure remains:

TEXT

```
EMPLOYEE(Eid, Name, Salary)
```

Only the contents changed.

### Answer

**Database instance changed.**

---

## Example 2

---

A database initially stores records sequentially.

Later, a B+ tree index is created, while all applications continue using the same logical tables.

What property is demonstrated?

### Reasoning

Only storage/access organization changed.

Logical structure remains unchanged.

### Answer

**Physical data independence.**

### GATE shortcut

If you see:

TEXT

index  
disk  
file  
storage  
block

and applications/schema remain unaffected !'

Think **PHYSICAL**.

---

## Example 3

The same student's address is stored in three files.

Two files show:

TEXT

Hyderabad

while another shows:

TEXT

Warangal

What problems exist?

Repeated storage:

**Redundancy**

Different values:

**Inconsistency**

So both exist.

---

## 30-SECOND REVISION

If you remember only this, remember:

TEXT

Database = organized related data

DBMS = software managing database

Redundancy = repeated data

Inconsistency = conflicting copies

Schema = structure

Instance = actual current data

View !' user sees  
Logical !' database structure  
Physical !' storage

Physical Independence = change HOW  
Logical Independence = change WHAT

Physical DI generally easier than Logical DI

---

## CHECK YOUR UNDERSTANDING

---

Don't look for answers yet. Try these yourself.

### Q1

A table contains 10,000 records. Another 100 records are inserted without changing its attributes.

What changes?

A. Schema only B. Instance only C. Both schema and instance D. Neither

---

### Q2

An organization changes its database storage from sequential files to indexed files without modifying application programs.

Which property is illustrated?

A. Logical data independence B. Physical data independence C. Data redundancy D. Referential integrity

---

### Q3

Customer address is stored independently in three files. One file contains an old address while the other two contain the new address.

Identify:

1. What causes unnecessary duplication?
  2. What describes the conflicting values?
- 

### Q4

Arrange from highest abstraction to lowest:

TEXT

Physical  
View  
Logical

---

### Q5

Which is generally harder to achieve?

A. Physical data independence B. Logical data independence

You can answer these five, or type ``NEXT`` and we will move to **Topic 2 — ER Model**, starting from **entity !' attributes !' relationships !' cardinality !' participation !' weak entities !' ER-to-relational mapping !' GATE problems**.

USER  
NEXT

ASSISTANT

## TOPIC 2 — ER MODEL

The **ER Model** is explicitly part of your GATE DBMS syllabus.

This topic is important because it teaches you how to convert a real-world situation such as a **college, hospital, bank, company, railway system, or e-commerce system** into a database design.

For GATE, focus especially on:

**Entity + Attributes + Keys + Relationships + Cardinality + Participation + Weak Entity + ER-to-Relational Mapping**

### 1. WHY DOES THE ER MODEL EXIST?

Imagine we want to build a college database.

We need to store:

- Students
- Courses
- Teachers
- Departments
- Enrollments

Before directly creating tables, we first need to answer questions such as:

**What objects exist?**

**What information should we store about them?**

**How are these objects connected?**

For example:

```
TEXT
Student
  |
  | enrolls in
  |
Course
```

This conceptual representation is the basic idea of an **ER model**.



## 2. ER MODEL

### ER = Entity-Relationship

An ER model represents a real-world system using:

1. Entities
2. Attributes
3. Relationships

The broad structure is:

```
TEXT
ENTITY
    ! "
has
    ! "
ATTRIBUTES

ENTITY
    ! "
participates in
    ! "
RELATIONSHIP
```

Example:

```
TEXT
S t u d e n t   % % %   E n r o l l s   % % %   C o u r s e
```

Here:

- Student = Entity
- Course = Entity
- Enrolls = Relationship

## 3. ENTITY

An **entity** is a distinguishable real-world object.

Examples:

```
TEXT
Student Ravi
Course DBMS
Employee 101
Department CSE
Car TS09AB1234
```

Each individual object is an entity.

## 4. ENTITY TYPE / ENTITY SET

Suppose:

TEXT

Ravi  
Anu  
Kiran  
Rahul

are students.

Each person is an **entity**.

The collection of all students is an **entity set**.

TEXT

STUDENT

|     |           |
|-----|-----------|
| %%% | R a v i   |
| %%% | A n u     |
| %%% | K i r a n |
| %%% | R a h u l |

So:

**Entity = one object**

**Entity set = collection of similar entities**

## MEMORY TRICK

Think:

TEXT

Entity = ONE  
Entity Set = MANY similar entities

Example:

TEXT

|               |   |                   |
|---------------|---|-------------------|
| R a v i       | ! | E n t i t y       |
| S T U D E N T | ! | E n t i t y S e t |

## 5. ER DIAGRAM SYMBOLS

Classical ER notation uses some important symbols.

TEXT

|                   |   |                         |
|-------------------|---|-------------------------|
| R e c t a n g l e | ! | E n t i t y S e t       |
| E l l i p s e     | ! | A t t r i b u t e       |
| D i a m o n d     | ! | R e l a t i o n s h i p |
| U n d e r l i n e | ! | K e y A t t r i b u t e |

Double Ellipse !' Multivalued Attribute  
Dashed Ellipse !' Derived Attribute  
Double Rectangle !' Weak Entity  
Double Diamond !' Identifying Relationship

We'll understand each one.

## 6. ATTRIBUTE

An **attribute** describes a property of an entity.

Consider:

TEXT  
STUDENT

Possible attributes:

TEXT  
RollNo  
Name  
Age  
Phone  
Address  
DateOfBirth

ER-style representation:

TEXT

```
      Name
      |
RollNo ---- [STUDENT] ---- Age
      |
      Phone
```

So:

**Entity = object**

**Attribute = property describing that object**

## 7. TYPES OF ATTRIBUTES

This is important for GATE.

Major types:

1. Simple

2. Composite
3. Single-valued
4. Multi-valued
5. Derived
6. Stored
7. Key attribute

Let's understand each.

---

## 8. SIMPLE ATTRIBUTE

An attribute that **cannot meaningfully be divided further**.

Example:

```
TEXT
Age
Salary
Gender
RollNo
```

Usually treated as atomic values in the given model.

---

## 9. COMPOSITE ATTRIBUTE

An attribute that can be divided into smaller components.

Example:

```
TEXT
Name
```

can be:

```
TEXT
Name
% % %   F i r s t N a m e
% % %   M i d d l e N a m e
% % %   L a s t N a m e
```

Another common example:

```
TEXT
Address
% % %   H o u s e N o
% % %   S t r e e t
% % %   C i t y
% % %   S t a t e
% % %   P I N
```

## Simple vs Composite

| Simple                         | Composite      |
|--------------------------------|----------------|
| Cannot be meaningfully divided | Can be divided |
| Age                            | Name           |
| Salary                         | Address        |
| RollNo                         | FullName       |

## MEMORY TRICK

**Composite = Components**

If an attribute contains smaller meaningful parts, think **composite**.

## 10. SINGLE-VALUED ATTRIBUTE

An attribute that normally has **one value for each entity**.

Example:

TEXT

Student RollNo = 101  
DateOfBirth = 10-05-2004

One student has one RollNo.

## 11. MULTI-VALUED ATTRIBUTE

An attribute that can have **multiple values for the same entity**.

Example:

A student may have:

TEXT

PhoneNumbers :  
9876543210  
9123456780

So PhoneNumber can be multivalued.

ER notation:

TEXT

((PhoneNumber))

Conceptually, it is drawn using a **double ellipse**.

---

## GATE TRAP

Do not confuse:

### Composite attribute

One value containing components.

TEXT

A d d r e s s ! ' C i t y + S t a t e + P I N

### Multivalued attribute

Multiple separate values.

TEXT

P h o n e ! ' P 1 , P 2 , P 3

---

## 12. STORED ATTRIBUTE

An attribute physically stored in the database.

Example:

TEXT

DateOfBirth

---

## 13. DERIVED ATTRIBUTE

An attribute calculated from another attribute.

Example:

TEXT

DateOfBirth = 10-05-2004

Age = CurrentDate - DateOfBirth

Age can be derived from DateOfBirth.

So:

TEXT

S t o r e d   A t t r i b u t e   ! '   D a t e O f B i r t h  
D e r i v e d   A t t r i b u t e   ! '   A g e

Classical ER notation uses a **dashed ellipse** for a derived attribute.

---

## MEMORY TRICK

Stored gives source; Derived does calculation.

Example:

TEXT

```
D O B !'   s t o r e d
A g e !'   d e r i v e d
```

## 14. KEY ATTRIBUTE

An attribute whose value uniquely identifies an entity.

Example:

TEXT

STUDENT

RollNo = 101

If RollNo uniquely identifies each student:

TEXT

```
R o l l N o !'   K e y   A t t r i b u t e
```

It is conventionally **underlined** in an ER diagram.

TEXT

STUDENT

-----

RollNo

Name

Age

Later, while studying relational models, we will go much deeper into:

- Super key
- Candidate key
- Primary key

For ER modelling, understand that a **key attribute distinguishes entities**.

## 15. NULL ATTRIBUTE VALUE

Sometimes an attribute may not currently have a value.

Example:

TEXT

MiddleName = NULL

NULL can conceptually mean things such as:

- Unknown
- Not available
- Not applicable

But SQL NULL has special semantics, which we'll study later.

## 16. RELATIONSHIP

A **relationship** represents an association among entities.

Example:

TEXT

S T U D E N T   % % %   E N R O L L S   % % %   C O U R S E

Meaning:

**A student enrolls in a course.**

Here:

TEXT

STUDENT = Entity set  
COURSE = Entity set  
ENROLLS = Relationship

## 17. RELATIONSHIP SET

Just as an entity set contains many entities, a relationship set contains many relationship instances.

Example:

TEXT

R a v i   ! '   D B M S  
R a v i   ! '   O S  
A n u   ! '   D B M S  
K i r a n   ! '   C N

All these together belong to the relationship set:

TEXT

ENROLLS

## 18. DEGREE OF A RELATIONSHIP

Degree = number of entity sets participating in the relationship.

There are three common types:



TEXT

Unary  
Binary  
Ternary

## 19. UNARY RELATIONSHIP

Only **one entity set** participates.

Example:

TEXT

```
EMPLOYEE
  |
supervises
  |
EMPLOYEE
```

One employee supervises another employee.

This is also called a **recursive relationship**.

## 20. BINARY RELATIONSHIP

Two entity sets participate.

Example:

TEXT

S T U D E N T   %   %   E N R O L L S   %   %   C O U R S E

This is the most common type.

## 21. TERNARY RELATIONSHIP

Three entity sets participate.

Example:

TEXT

```
SUPPLIER
  \
   \
    SUPPLIES
   /  \
PRODUCT PROJECT
```

This relationship simultaneously involves:

TEXT

Supplier  
Product

Project

Therefore degree = 3.

## MEMORY TRICK

TEXT

```
U n a r y      !'   1   e n t i t y   s e t
B i n a r y    !'   2
T e r n a r y  !'   3
```

Simple but important.

## & pGATE TRAP — DEGREE

Do not confuse:

**Degree of relationship**

with

**Degree of relation/table**

For a relationship:

TEXT

Degree = number of participating entity sets.

For a relational table later:

TEXT

Degree = number of attributes/columns.

Same word, different context.

## 22. CARDINALITY / MAPPING CONSTRAINT

This is one of the most important ER topics.

Cardinality tells us **how many entities can be associated with entities on the other side of a relationship.**

Four common types:

TEXT

```
1 : 1
1 : N
N : 1
M : N
```

## 23. ONE-TO-ONE — 1:1

Example:

```
TEXT
P E R S O N   % % % % %   P A S S P O R T
1               1
```

Suppose:

- One person has at most one passport.
- One passport belongs to one person.

Then:

```
TEXT
PERSON : PASSPORT = 1 : 1
```

## 24. ONE-TO-MANY — 1:N

Example:

```
TEXT
D E P A R T M E N T   % % % % %   E M P L O Y E E
1               N
```

One department can have many employees.

But each employee belongs to one department.

Therefore:

```
TEXT
Department : Employee = 1 : N
```

## 25. MANY-TO-ONE — N:1

Same relationship viewed from opposite direction:

```
TEXT
E M P L O Y E E   % % % % %   D E P A R T M E N T
N               1
```

Many employees belong to one department.

## 26. MANY-TO-MANY — M:N

Example:

TEXT

S T U D E N T   % % % %   C O U R S E  
M                      N

A student can enroll in many courses.

A course can contain many students.

So:

TEXT

STUDENT : COURSE = M : N

## CARDINALITY MEMORY TRICK

Ask two questions.

For:

TEXT

A   %   R e l a t i o n s h i p   %   B

Ask:

### Question 1

For **one A**, how many B can exist?

### Question 2

For **one B**, how many A can exist?

Example:

TEXT

S t u d e n t   %   E n r o l l s   %   C o u r s e

One student !' many courses.

One course !' many students.

Therefore:

TEXT

M : N

This method is much safer than memorizing diagrams.

## 27. PARTICIPATIONCONSTRAINT

Participation answers a different question:

## Is participation in a relationship mandatory or optional?

Two types:

TEXT

Total Participation  
Partial Participation

## 28. TOTAL PARTICIPATION

Every entity in the entity set **must participate** in the relationship.

Example:

Suppose every employee must belong to some department.

TEXT

E M P L O Y E E   % P % P   W O R K S \_ F O R   % %   D E P A R T M E N T

Employee has **total participation** in WORKS\_FOR.

Classically represented by a **double line**.

## 29. PARTIAL PARTICIPATION

Some entities may not participate.

Suppose not every employee manages a department.

TEXT

E M P L O Y E E   % %   M A N A G E S   % %   D E P A R T M E N T

Only some employees are managers.

Therefore EMPLOYEE may have **partial participation** in MANAGES.

Classically represented with a **single line**.

## Cardinality vs Participation

Very important.

| Cardinality             | Participation              |
|-------------------------|----------------------------|
| How many?               | Mandatory or optional?     |
| 1:1, 1:N, M:N           | Total / Partial            |
| Maximum-type constraint | Minimum participation idea |

## MEMORY TRICK

**Cardinality = COUNT**

**Participation = COMPULSORY?**

## 30. MIN-MAX NOTATION

Sometimes ER constraints are written using:

TEXT  
(min, max)

Example:

TEXT  
EMPLOYEE (1,1)

Means employee participates:

- minimum 1 time
- maximum 1 time

Example:

TEXT  
STUDENT (0,N)

Means:

- student may participate zero times
- or many times

Important interpretation:

TEXT  
Minimum = 0 !' Partial participation  
Minimum "e 1 !' Total participation

This is a very useful shortcut.

## GATE SHORTCUT

When given:

TEXT  
(0, N)

think:

**Optional participation.**

When given:

TEXT  
(1, N)

think:

**Mandatory participation.**

Because the **minimum** determines participation.

### 31. ROLE NAMES

Consider:

TEXT

|          |    |             |    |          |
|----------|----|-------------|----|----------|
| EMPLOYEE | %% | SUPERVISERS | %% | EMPLOYEE |
|----------|----|-------------|----|----------|

Both sides are the same entity set.

We need to distinguish roles:

TEXT

Supervisor  
Subordinate

These are called **role names**.

## 32. RELATIONSHIP ATTRIBUTES

A relationship itself can have attributes.

Example:

TEXT

S T U D E N T   %   %   %   E N R O L L S   %   %   %   C O U R S E

|

Grade

Grade depends on the combination:

TEXT

Student + Course

A student's grade is not simply a property of the student.

And it is not simply a property of the course.

It belongs to their **enrollment relationship**.

Other examples:

| Employee % | Works_On % | Project |
|------------|------------|---------|
| Hours      |            |         |

## & pGATE TRAP

**Does this information describe one entity independently?**

**Does it describe the association between entities?**

```
Student Name !' Student attribute
Course Name !' Course attribute
Enrollment Date !' Enrollment relationship attribute
```

### 33. STRONG ENTITY

| EmpID | Name | Salary |
|-------|------|--------|
|-------|------|--------|

### 34. WEAK ENTITY

Example:



Suppose:

TEXT  
EMPLOYEE  
EmpID  
Name

Each employee may have dependents:

TEXT  
DEPENDENT  
Name  
Age  
Relationship

Problem:

Dependent Name alone may not be globally unique.

Two employees may both have a dependent named:

TEXT  
Anu

So:

TEXT  
DependentName

alone cannot identify DEPENDENT.

But:

TEXT  
(EmpID, DependentName)

can.

Therefore DEPENDENT can be modelled as a weak entity.

---

## 35. PARTIAL KEY

A weak entity often has a **partial key**.

Example:

TEXT  
DependentName

may distinguish dependents **within one employee**, but not across all employees.

So:

TEXT

Owner key + Partial key

uniquely identifies a weak entity.

Example:

TEXT

EmpID + DependentName

## 36. IDENTIFYING RELATIONSHIP

The relationship connecting a weak entity to its owner is called an **identifying relationship**.

Example:

TEXT

E M P L O Y E E   %P%P%P   H A S   %P%P%P   D E P E N D E N T

Classical notation:

TEXT

W e a k   E n t i t y   ! '   D o u b l e   R e c t a n g l e  
I d e n t i f y i n g   R e l a t i o n s h i p   ! '   D o u b l e   D i a m o n d

## Important properties of weak entities

Usually:

1. Weak entity has no complete key of its own.
2. It depends on an owner entity.
3. It has a partial key/discriminator.
4. It has **total participation** in the identifying relationship.
5. Owner key becomes part of weak entity's identification.

## MEMORY TRICK — WEAK ENTITY

Remember:

**Weak needs Owner + Partial Key**

TEXT

Weak Entity ID  
=  
Owner's Key  
+  
Partial Key

Example:

TEXT

Dependent identity  
=  
EmpID + DependentName

## Strong Entity vs Weak Entity

| Strong Entity                | Weak Entity                        |
|------------------------------|------------------------------------|
| Has own complete key         | No complete independent key        |
| Exists independently         | Depends on owner                   |
| Single rectangle             | Double rectangle                   |
| Normal relationship possible | Identifying relationship           |
| Own key identifies entity    | Owner key + partial key identifies |

## 37. EXISTENCE DEPENDENCY

A weak entity is generally **existence-dependent** on its owner.

For example:

If employee 101 does not exist, an associated dependent record identified through employee 101 has no independent identity in that weak-entity design.

This is why weak entities have total participation in their identifying relationship.

## 38. SPECIALIZATION AND GENERALIZATION

These concepts can appear in ER modelling, though your direct GATE syllabus only says ER-model rather than listing every enhanced-ER construct. We will understand them only to the useful level.

Suppose:

TEXT

EMPLOYEE

|

% % %    M A N A G E R  
% % %    E N G I N E E R

EMPLOYEE is a general entity.

MANAGER and ENGINEER are specialized entities.

## Specialization

Top-down.

Start with a general entity and create subtypes.

```
TEXT
EMPLOYEE
! "
Manager
Engineer
```

## Generalization

Bottom-up.

Combine similar entity types into a common supertype.

```
TEXT
CAR
TRUCK
! "
VEHICLE
```

## MEMORY TRICK

**Specialization = General !' Specific**

**Generalization = Specific !' General**

## 39. ISA RELATIONSHIP

Specialization/generalization is often represented using **ISA**.

Example:

```
TEXT
MANAGER ISA EMPLOYEE
```

Meaning:

Every manager is an employee.

But every employee is not necessarily a manager.

## 40. INHERITANCE

A subtype inherits attributes of its supertype.

Example:

TEXT

EMPLOYEE

EmpID

Name

Salary

TEXT

MANAGER

Bonus

Manager conceptually has:

TEXT

EmpID

Name

Salary

Bonus

because it inherits Employee properties.

## 41. DISJOINT vs OVERLAPPING

Suppose:

TEXT

PERSON

% % %

S T U D E N T

% % %

E M P L O Y E E

Can one person be both?

If no:

TEXT

Disjoint

If yes:

TEXT

Overlapping

## 42. TOTAL vs PARTIAL SPECIALIZATION

Suppose:

TEXT

EMPLOYEE

% % %

M A N A G E R

% % %

E N G I N E E R

## Total

Every employee must belong to at least one subtype.

## Partial

Some employees may belong to none of the shown subtypes.

Again, don't confuse this with relationship participation, though the "total/partial" terminology expresses a similar mandatory-coverage idea.

---

## 43. ER TO RELATIONAL MODEL MAPPING

This is where ER becomes database tables.

Very important.

---

### RULE 1 — STRONG ENTITY !' TABLE

ER entity:

```
TEXT
STUDENT

RollNo
Name
Age
```

becomes:

```
TEXT
STUDENT(
  RollNo,
  Name,
  Age
)
```

Primary key:

```
TEXT
RollNo
```

---

### RULE 2 — COMPOSITE ATTRIBUTE

Suppose:

```
TEXT
Name
  % % %   F i r s t N a m e
  % % %   L a s t N a m e
```

Usually store its components:

TEXT

```
STUDENT(  
  RollNo,  
  FirstName,  
  LastName  
)
```

rather than storing the conceptual composite attribute as an additional redundant field.

---

## RULE 3 — MULTIVALUED ATTRIBUTE !' SEPARATE TABLE

---

Suppose:

TEXT

```
STUDENT  
RollNo  
Name  
Phone {multiple}
```

Create:

TEXT

```
STUDENT(  
  RollNo,  
  Name  
)
```

and:

TEXT

```
STUDENT_PHONE(  
  RollNo,  
  Phone  
)
```

Primary key can often be:

TEXT

```
(RollNo, Phone)
```

---

## GATE SHORTCUT

---

Whenever you see a true multivalued attribute:

**Think new relation/table.**

---

## RULE 4 — DERIVED ATTRIBUTE

---

Usually a derived value need not be physically stored if it can be calculated.

Example:

```
TEXT
D O B ! '   s t o r e d
A g e ! '   d e r i v e   f r o m   D O B
```

## RULE 5 — 1:1 RELATIONSHIP

Example:

```
TEXT
P E R S O N   % % %   H A S   % % %   P A S S P O R T
1               1
```

A foreign key can be placed in one of the relations.

A good placement is often guided by participation constraints to avoid unnecessary NULLs.

For example, if every passport must belong to a person:

```
TEXT
PASSPORT(
  PassportNo,
  IssueDate,
  PersonID
)
```

where PersonID references PERSON.

For GATE, participation constraints may determine which side is preferable.

## RULE 6 — 1:N RELATIONSHIP

This rule is extremely important.

```
TEXT
D E P A R T M E N T   % % %   E M P L O Y E E
1                     N
```

Place the primary key of the **1-side** into the **N-side** as a foreign key.

Example:

```
TEXT
DEPARTMENT(
  DeptID,
  DeptName
)
```

```
TEXT
EMPLOYEE(
  EmpID,
```



`DeptID` becomes a foreign key in `EMPLOYEE`.

1 gives key to N

$$1 \quad !' \quad N$$

Primary key travels:

TEXT

ONE side !' MANY side

ONE side !' MANY side

This is one of the most useful ER-mapping shortcuts.

Example:

**TEXT**

S T U D E N T   % % %   E N R O L L S   % % %   C O U R S E

M                          N

| S T U D E N T | % % % | E N R O L L S | % % % | C O U R S E |
|---------------|-------|---------------|-------|-------------|
|---------------|-------|---------------|-------|-------------|

N

We cannot simply put one foreign key on one side.

Create a **new relation**:

```
TEXT
ENROLLMENT(
  StudentID,
  CourseID
)
```

```
ENROLLMENT(  
  StudentID,  
  CourseID  
)
```

Usually:

TEXT

(StudentID, CourseID)

(StudentID, CourseID)

is the composite primary key.

If the relationship has an attribute:

TEXT

Grade

then:

TEXT

```
ENROLLMENT(  
  StudentID,  
  CourseID,  
  Grade  
)
```

## MEMORY TRICK — M:N

**Many-Many makes a Middle table**

All three begin conceptually with M:

TEXT

M a n y - M a n y ! ' M i d d l e r e l a t i o n

## RULE 8 — WEAK ENTITY

Suppose:

TEXT

EMPLOYEE  
EmpID

OWNS:

TEXT

DEPENDENT  
DependentName  
Age

Create:

TEXT

```
EMPLOYEE(  
  EmpID,  
  ...  
)
```

and:

TEXT

```
DEPENDENT(  
  EmpID,
```

```
DependentName,  
Age  
)
```

Primary key:

```
TEXT  
(EmpID, DependentName)
```

Because:

```
TEXT  
Owner key + Partial key
```

identifies the weak entity.

## ER MAPPING MASTER TABLE

| ER Construct          | Relational Mapping        |
|-----------------------|---------------------------|
| Strong entity         | Create relation           |
| Simple attribute      | Column                    |
| Composite attribute   | Store components          |
| Multivalued attribute | Separate relation         |
| Derived attribute     | Usually calculate         |
| 1:1 relationship      | FK on suitable side       |
| 1:N relationship      | 1-side key goes to N-side |
| M:N relationship      | Create separate relation  |
| Weak entity           | Owner key + partial key   |

This table is worth writing into your notebook.

## 44. GATE-STYLE CARDINALITY REASONING

Suppose:

```
TEXT  
E M P L O Y E E % % %   W O R K S _ F O R % % %   D E P A R T M E N T
```

Conditions:

- Every employee works for exactly one department.
- Each department may have many employees.

Then:

TEXT

E m p l o y e e ! ' D e p a r t m e n t = N : 1

Employee participation:

TEXT

Total

because every employee must work for a department.

Department participation could be:

TEXT

Partial

if a department can exist without employees.

## 45. SOLVED GATE-STYLE EXAMPLE

Consider:

TEXT

S T U D E N T % E N R O L L S % C O U R S E

Each student can enroll in many courses.

Each course can contain many students.

Each enrollment stores a grade.

### Required

Convert into relations.

### Step 1 — Entities

TEXT

```
STUDENT(  
  StudentID,  
  Name  
)
```

TEXT

```
COURSE(  
  CourseID,  
  Title  
)
```

### Step 2 — Cardinality

M:N.

### Step 3 — M:N rule

Create a new relation.

TEXT

```
ENROLLMENT(  
  StudentID,  
  CourseID,  
  Grade  
)
```

Primary key:

TEXT

```
(StudentID, CourseID)
```

Foreign keys:

TEXT

```
S t u d e n t I D ! ' S T U D E N T  
C o u r s e I D ! ' C O U R S E
```

## Answer

Three relations are needed.

## GATE FAST METHOD

When converting ER to tables:

TEXT

```
S t r o n g   E n t i t y ! '   T a b l e  
  
1 : N ! '   F K   t o   N  
  
M : N ! '   N e w   t a b l e  
  
M u l t i v a l u e d   a t t r i b u t e ! '   N e w   t a b l e  
  
W e a k   e n t i t y ! '   O w n e r   k e y   +   P a r t i a l   k e y
```

Memorize these five first.

## 46. SOLVED EXAMPLE — WEAK ENTITY

Suppose:

TEXT

```
EMPLOYEE(EmpID, Name)
```

Each employee can have multiple dependents.

A dependent has:

TEXT

```
DependentName  
Age
```

DependentName is unique only within one employee.

## Analysis

DEPENDENT cannot be uniquely identified by DependentName globally.

Therefore:

TEXT

DEPENDENT = Weak Entity

Identification:

TEXT

EmpID + DependentName

Relational schema:

TEXT

```
DEPENDENT(  
  EmpID,  
  DependentName,  
  Age  
)
```

Primary key:

TEXT

(EmpID, DependentName)

---

## 47. SOLVED EXAMPLE — COMPOSITE vs MULTIVALUED

Consider:

TEXT

Address  
PhoneNumber

A student has:

TEXT

Address:  
HouseNo  
Street  
City  
PIN

but may have three phone numbers.

Then:

TEXT

Address = Composite Attribute  
Phone = Multivalued Attribute

Why?

Address is one conceptual value with components.

Phone contains multiple independent values.

## 48. POSSIBLE GATE QUESTION — NUMBER OF RELATIONS

A common question style gives an ER diagram and asks:

**How many relations will be generated?**

Useful heuristic:

Start with:

TEXT

O n e   s t r o n g   e n t i t y ! '   o n e   r e l a t i o n

Then add relations when necessary for:

TEXT

M:N relationships  
Multivalued attributes  
Weak entities  
Certain higher-degree relationships

Do **not** automatically create a separate table for every 1:N relationship.

A 1:N relationship can usually be represented using a foreign key on the N-side.

## & pGATE TRAPS

### Trap 1 — Entity vs Entity Set

TEXT

Ravi = Entity  
STUDENT = Entity Set

### Trap 2 — Composite vs Multivalued

TEXT

N a m e ! '   c o m p o n e n t s  
P h o n e ! '   m u l t i p l e   v a l u e s

### Trap 3 — Cardinality vs Participation

TEXT

C a r d i n a l i t y ! ' H O W M A N Y

P a r t i c i p a t i o n ! ' M U S T P A R T I C I P A T E ?

## Trap 4 — Weak entity

Weak entity does **not** mean:

**entity having no attributes.**

It means it **cannot be uniquely identified independently by a full key of its own.**

## Trap 5 — Weak entity participation

A weak entity has **total participation** in its identifying relationship.

## Trap 6 — 1:N mapping

Do not put N-side primary key into the 1-side as a repeating field.

Correct shortcut:

TEXT

1 - s i d e k e y ! ' N - s i d e f o r e i g n k e y

## Trap 7 — M:N mapping

An M:N relationship usually requires a **separate relation**.

## Trap 8 — Relationship attribute

For:

TEXT

S T U D E N T % E N R O L L S % C O U R S E

**Grade** belongs to ENROLLS, not STUDENT or COURSE alone.

## 49. PYQ-ORIENTED RECOGNITION

When you see these words, think immediately:



#### TEXT

"uniquely identify"  
!' key

"many values"  
!' multivalued attribute

"calculated from"  
!' derived attribute

"cannot exist independently"  
!' weak entity

"one employee, many projects"  
!' cardinality question

"every employee must..."  
!' total participation

"may..."  
!' likely partial participation

"M:N"  
!' new relation

"1:N"  
!' foreign key on N-side

## 50. NOTEBOOK NOTES

Write this compactly.

### ER MODEL

**ER = Entity Relationship Model**

Used for conceptual database design.

Main components:

#### TEXT

Entity  
Attribute  
Relationship

### Symbols

#### TEXT

|                  |                             |
|------------------|-----------------------------|
| Rectangle        | !' Entity                   |
| Ellipse          | !' Attribute                |
| Diamond          | !' Relationship             |
| Underline        | !' Key                      |
| Double ellipse   | !' Multivalued              |
| Dashed ellipse   | !' Derived                  |
| Double rectangle | !' Weak entity              |
| Double diamond   | !' Identifying relationship |

## Attributes

### TEXT

```
Simple      !' indivisible
Composite   !' components
Single-valued !' one value
Multivalued !' many values
Stored      !' physically stored
Derived     !' computed
Key         !' uniquely identifies
```

## Relationship Degree

### TEXT

```
Unary   = 1 entity set
Binary  = 2
Ternary = 3
```

## Cardinality

### TEXT

```
1:1
1:N
N:1
M:N
```

## Participation

### TEXT

```
Total      !' every entity participates
Partial    !' some may not participate
```

Min-max shortcut:

### TEXT

```
min = 0 !' Partial
min "e 1 !' Total
```

## Weak Entity

### TEXT

```
No complete independent key
Depends on owner
```

```
Identification =
Owner Key + Partial Key
```

Weak entity has total participation in identifying relationship.

## ER !' Relational Mapping

TEXT

Strong entity !' Relation  
Composite !' components  
Multivalued !' separate relation  
1 : N !' key of 1 goes to N as FK  
M : N !' separate relation  
Weak entity !' owner key + partial key

## MEMORY PAGE

Remember:

TEXT

E-A-R

Entity  
Attribute  
Relationship

For attributes:

TEXT

S C S M D K

Simple  
Composite  
Single  
Multi  
Derived  
Key

Don't force the acronym if it becomes confusing; conceptual recognition matters more.

For mapping:

**1 gives to N.**

**M:N makes a middle table.**

**Multi-value makes a new table.**

**Weak = Owner + Partial.**

These four shortcuts are highly useful.

## 30-SECOND REVISION

### TEXT

Entity = real-world object

Attribute = property

Relationship = association

Cardinality = how many

Participation = mandatory or optional

T o t a l ! ' m u s t p a r t i c i p a t e  
P a r t i a l ! ' m a y n o t

Weak entity:  
cannot be identified independently

Weak ID =  
Owner key + Partial key

1:N mapping:  
1 - s i d e P K ! ' N - s i d e F K

M:N mapping:  
create new relation

Multivalued attribute:  
create new relation

## CHECK YOUR UNDERSTANDING

Try these before checking explanations.

### Q1

A student may have multiple phone numbers.

PhoneNumber is:

A. Simple attribute B. Composite attribute C. Multivalued attribute D. Derived attribute

### Q2

Consider:

### TEXT

D E P A R T M E N T % % % E M P L O Y E E

One department may contain many employees, while each employee belongs to exactly one department.

What is the cardinality from Department to Employee?

A. 1:1 B. 1:N C. N:1 D. M:N

### Q3

Every employee must belong to at least one department.

Employee's participation in `WORKS_FOR` is:

A. Partial B. Total C. Weak D. Multivalued

### Q4

A weak entity `DEPENDENT` has partial key `DependentName` and is owned by `EMPLOYEE` whose primary key is `EmpID`.

What will normally identify `DEPENDENT`?

A. `DependentName` only B. `EmpID` only C. `(EmpID, DependentName)` D. Name of employee

### Q5

Consider:

TEXT

S T U D E N T   % % %   E N R O L L S   % % %   C O U R S E

where the relationship is M:N.

When converted to the relational model, what should normally be done?

A. Store all CourseIDs in one STUDENT field B. Put StudentID only in COURSE C. Create an ENROLLMENT relation D. Remove the relationship

Next is **Topic 3 — Relational Model**, where we'll build the vocabulary used throughout the rest of DBMS:

**relation !' tuple !' attribute !' domain !' degree !' cardinality !' schema vs instance !' super key !' candidate key !' primary key !' foreign key !' prime/non-prime attributes !' NULL and relational properties.**

Type ``NEXT``.

USER

NEXT

USER

NEXT

ASSISTANT

## TOPIC 3 — RELATIONAL MODEL

The relational model is one of the core areas in your GATE DBMS syllabus, and it is the foundation for **relational algebra, SQL, keys, integrity constraints, functional dependencies, and normalization**.

This topic is extremely important because if these terms are weak, later topics like **candidate keys, normalization, SQL, and relational algebra** become confusing.

## 1. WHY DOES THE RELATIONAL MODEL EXIST?

In the ER model, we represented data conceptually.

Example:

TEXT

S T U D E N T   % % %   E N R O L L S   % % %   C O U R S E

But ultimately, the database has to store data in a practical structure.

The relational model represents data using:

**Relations, which we commonly visualize as tables.**

Example:

TEXT

STUDENT

| RollNo | Name  | Dept |
|--------|-------|------|
| 101    | Ravi  | CSE  |
| 102    | Anu   | AIML |
| 103    | Kiran | CSE  |

This table-like representation is the basis of the relational model.

## 2. RELATIONAL MODEL

The relational model organizes data as a collection of **relations**.

Important vocabulary:

TEXT

Relation  
Tuple  
Attribute  
Domain  
Degree  
Cardinality  
Schema  
Instance  
Keys

These words are extremely important for GATE.

## 3. RELATION

A **relation** is a set of tuples defined over a set of attributes.

In simple terms:

**A relation is similar to a table.**

Example:

TEXT

STUDENT

| RollNo | Name  | Dept |
|--------|-------|------|
| 101    | Ravi  | CSE  |
| 102    | Anu   | AIML |
| 103    | Kiran | CSE  |

Here:

TEXT

Relation name = STUDENT

## 4. ATTRIBUTE

An **attribute** represents a property or column of a relation.

Example:

TEXT

STUDENT(RollNo, Name, Dept)

Attributes are:

TEXT

RollNo  
Name  
Dept

So:

**Attribute = column.**

## 5. TUPLE

A **tuple** is one row of a relation.

Example:

TEXT

101    Ravi    CSE

is one tuple.

Similarly:

```
TEXT
102   Anu   AIMA
```

is another tuple.

## MEMORY TRICK

```
TEXT
Tuple = Row
Attribute = Column
```

Think:

**T !' Tuple travels horizontally**

**A !' Attribute arranged vertically**

## 6. DOMAIN

A **domain** is the set of allowed values for an attribute.

Example:

For:

```
TEXT
Age
```

domain may be:

```
TEXT
{1, 2, 3, ..., 120}
```

For:

```
TEXT
Gender
```

the defined domain might be some permitted set of codes.

For:

```
TEXT
Marks
```

domain may be:



TEXT

0 to 100

Therefore:

**Domain specifies what values an attribute is allowed to contain.**

## EXAMPLE

Consider:

TEXT

```
STUDENT(  
  RollNo,  
  Name,  
  Age  
)
```

Possible domains:

TEXT

```
R o l l N o   ! '   p o s i t i v e   i n t e g e r s  
N a m e       ! '   c h a r a c t e r   s t r i n g s  
A g e         ! '   i n t e g e r s   f r o m   0   t o   1 2 0
```

If:

TEXT

Age = -50

it violates the intended domain.

## 7. RELATION SCHEMA

A **relation schema** defines the structure of a relation.

Example:

TEXT

```
STUDENT(RollNo, Name, Dept, Age)
```

This tells us:

- relation name
- attributes

but not the current rows.

Notation:

TEXT

$R(A_1, A_2, A_3, \dots, A_n)$

where:

TEXT

R = relation name  
A1...An = attributes

## 8. RELATION INSTANCE

The actual set of tuples present at a particular moment is the **relation instance** or state.

Schema:

TEXT

STUDENT(RollNo, Name, Dept)

Instance:

TEXT

|     |       |      |
|-----|-------|------|
| 101 | Ravi  | CSE  |
| 102 | Anu   | AIML |
| 103 | Kiran | ECE  |

Tomorrow, another row might be inserted.

Then:

TEXT

Instance changes

but the schema may remain unchanged.

We saw the same idea in database fundamentals.

## 9. DEGREE OF A RELATION

Very important GATE term.

**Degree = number of attributes in a relation.**

Example:

TEXT

STUDENT(  
RollNo,  
Name,  
Dept,  
Age  
)

Number of attributes = 4.

Therefore:

TEXT

Degree = 4

## 10. CARDINALITY OF A RELATION

**Cardinality = number of tuples in a relation.**

Example:

TEXT

| RollNo | Name  |
|--------|-------|
| 101    | Ravi  |
| 102    | Anu   |
| 103    | Raj   |
| 104    | Kiran |

Number of rows = 4.

Therefore:

TEXT

Cardinality = 4

## & VERY IMPORTANT GATE TRAP

Do not confuse:

TEXT

Degree = Columns  
Cardinality = Rows

## MEMORY TRICK

**Degree != Dimensions/columns**

**Cardinality != Count of rows**

A simpler one:

TEXT

D != number of attributes  
C != count of tuples

## 11. RELATION vs TABLE

In theory:

TEXT

```
R e l a t i o n "H   T a b l e
T u p l e      "H   R o w
A t t r i b u t e "H   C o l u m n
```

But strictly, a mathematical relation has stronger properties than an arbitrary spreadsheet/table.

For GATE, understand the theoretical relational properties too.

## 12. IMPORTANT PROPERTIES OF A RELATION

A theoretical relation generally follows these properties:

### 1. Tuple order is irrelevant.

Example:

TEXT

```
101 Ravi
102 Anu
```

and:

TEXT

```
102 Anu
101 Ravi
```

represent the same relation if all tuples are the same.

### 2. Attribute order is logically irrelevant.

The relation:

TEXT

```
STUDENT(RollNo, Name)
```

contains the same mapping of attributes even if a presentation reorders columns, provided attribute names are correctly associated.

### 3. No duplicate tuples

A relation is mathematically a **set** of tuples.

Sets do not contain duplicates.

Therefore pure relational theory does not allow duplicate tuples.

### 4. Each cell contains an atomic value under the classical 1NF view

Example:

Instead of:

TEXT

Phone = 9876, 9123

in one field, values should be atomic for 1NF.

We will study this deeply in normalization.

---

## & IMPORTANT SQL TRAP

Pure relational model:

TEXT

Duplicates are not allowed.

But SQL tables/results can behave like multisets/bags unless `DISTINCT` or constraints remove duplicates.

We will revisit this in SQL.

---

## 13. NULL VALUES

NULL is very important.

NULL does **not** simply mean:

TEXT

0

It also does not mean:

TEXT

empty string

NULL represents absence of a known/applicable value.

Possible interpretations include:

TEXT

Unknown  
Not available  
Not applicable  
Missing

Example:

TEXT

Student:

RollNo = 101  
Name = Ravi  
Phone = NULL

This may mean the phone number is not known or not provided.

---

## & pGATE TRAP

---

Do not assume:

```
TEXT
NULL = 0
```

or:

```
TEXT
NULL = ''
```

They are different.

SQL uses special three-valued logic with NULL, which we will study later.

---

## 14. KEY

---

A **key** is one or more attributes used to identify tuples.

Keys are one of the most important foundations for GATE.

Types we must understand:

```
TEXT
Super Key
Candidate Key
Primary Key
Alternate Key
Composite Key
Foreign Key
```

Let's build these carefully.

---

## 15. SUPER KEY

---

A **super key** is any set of attributes that uniquely identifies each tuple in a relation.

Suppose:

```
TEXT
STUDENT(
  RollNo,
  Email,
  Name,
  Dept
)
```

Assume both:

TEXT

RollNo  
Email

are individually unique.

Then examples of super keys include:

TEXT

{RollNo}  
  
{Email}  
  
{RollNo, Name}  
  
{RollNo, Dept}  
  
{Email, Name}  
  
{RollNo, Email}  
  
{RollNo, Email, Name}

Why?

Because if RollNo already uniquely identifies a tuple, adding extra attributes still preserves uniqueness.

## GATE KEY IDEA

A super key may contain **unnecessary attributes**.

Example:

TEXT

{RollNo, Name}

If RollNo alone is enough, then Name is unnecessary.

Still, the combination is a super key.

## 16. CANDIDATE KEY

A **candidate key** is a **minimal super key**.

Minimal means:

**No proper subset of it should also be a super key.**

Example:

TEXT

{RollNo}

is a candidate key.

TEXT

{RollNo, Name}

is not a candidate key if RollNo alone is sufficient.

## SUPER KEY vs CANDIDATE KEY

This is one of the biggest GATE traps.

TEXT

Super Key

!"

uniquely identifies tuples

Candidate Key

!"

uniquely identifies tuples

+

minimal

## MEMORY TRICK

**Candidate = Clean super key**

No extra attributes.

Or:

**Every candidate key is a super key, but every super key is not a candidate key.**

Very important.

## 17. PRIMARY KEY

A relation can have multiple candidate keys.

One of them is selected as the **primary key**.

Example:

Candidate keys:

TEXT

RollNo

Email

Suppose we choose:

TEXT

RollNo

Then:



TEXT

Primary Key = RollNo

## 18. ALTERNATE KEY

Candidate keys not selected as the primary key are called **alternate keys**.

Example:

Candidate keys:

TEXT

RollNo

Email

Primary key:

TEXT

RollNo

Then:

TEXT

Email = Alternate Key

## 19. COMPOSITE KEY

A key consisting of more than one attribute is a **composite key**.

Example:

TEXT

```
ENROLLMENT(  
  StudentID,  
  CourseID,  
  Grade  
)
```

Suppose a student can enroll in each course only once.

Then:

TEXT

(StudentID, CourseID)

may be a composite key.

Neither attribute alone uniquely identifies a row.

Together they do.

## 20. FOREIGN KEY

A **foreign key** is an attribute/set of attributes in one relation that references a key—typically a primary or candidate key—of another relation.

Example:

```
TEXT
DEPARTMENT(
  DeptID,
  DeptName
)
```

```
TEXT
STUDENT(
  RollNo,
  Name,
  DeptID
)
```

Here:

```
TEXT
STUDENT.DeptID
```

references:

```
TEXT
DEPARTMENT.DeptID
```

Therefore:

```
TEXT
STUDENT.DeptID = Foreign Key
```

## MEMORY TRICK

Foreign key points outside its own relation to another relation's key.

## 21. FOREIGN KEY CAN HAVE DUPLICATES

This is important.

Suppose:

```
TEXT
DEPARTMENT

DeptID
-----
```

CSE  
ECE

Student table:

```
TEXT
RollNo  DeptID
-----
101     CSE
102     CSE
103     CSE
104     ECE
```

DeptID repeats.

That is allowed because foreign keys do **not necessarily have to be unique**.

## & PGATE TRAP

Primary key:

```
TEXT
must uniquely identify rows
```

Foreign key:

```
TEXT
can repeat
```

unless additional constraints make it unique.

## 22. CAN A FOREIGN KEY BE NULL?

Potentially yes, depending on the relationship and schema constraints.

Example:

```
TEXT
EMPLOYEE(
  EmpID,
  ManagerID
)
```

Top-level CEO may have:

```
TEXT
ManagerID = NULL
```

But if the relationship is mandatory, NULL may not be allowed.

So:

Foreign key can be NULL unless constrained otherwise.

## 23. PRIME ATTRIBUTE

A **prime attribute** is an attribute that belongs to **at least one candidate key**.

Suppose candidate keys are:

TEXT  
{A, B}  
{C}

Then prime attributes are:

TEXT  
A  
B  
C

because each appears in at least one candidate key.

## 24. NON-PRIME ATTRIBUTE

An attribute that does not belong to any candidate key.

If relation is:

TEXT  
 $R(A, B, C, D, E)$

and candidate keys are:

TEXT  
AB  
C

then:

Prime:

TEXT  
A, B, C

Non-prime:

TEXT  
D, E

This becomes extremely important in:

TEXT

2NF  
3NF  
BCNF

## MEMORY TRICK

**Prime = Part of some candidate key**

Very important wording:

Not merely primary key.

Prime means:

TEXT

member of ANY candidate key

## & ~~p~~HUGE GATE TRAP

Prime attribute does **not** mean:

**attribute belonging to primary key only.**

Wrong.

Correct:

**Attribute belonging to at least one candidate key.**

## 25. KEY COMPARISON TABLE

| Key           | Meaning                            |
|---------------|------------------------------------|
| Super Key     | Uniquely identifies tuple          |
| Candidate Key | Minimal super key                  |
| Primary Key   | Chosen candidate key               |
| Alternate Key | Candidate key not chosen           |
| Composite Key | Key with >1 attribute              |
| Foreign Key   | References key in another relation |

This is a must-write table.

## 26. EXAMPLE — IDENTIFY ALL KEY TYPES

Consider:

```
TEXT
EMPLOYEE(
  EmpID,
  AadhaarNo,
  Email,
  Name,
  Dept
)
```

Assume:

```
TEXT
EmpID unique
AadhaarNo unique
Email unique
```

Then candidate keys:

```
TEXT
{EmpID}
{AadhaarNo}
{Email}
```

Suppose primary key:

```
TEXT
EmpID
```

Then alternate keys:

```
TEXT
AadhaarNo
Email
```

Super keys include:

```
TEXT
{EmpID}
{EmpID, Name}
{Email}
{Email, Dept}
{AadhaarNo, Name, Dept}
...
```

## 27. NUMBER OF SUPER KEYS — IMPORTANT GATE IDEA

Suppose a relation has:

TEXT

n total attributes

and a candidate key contains:

TEXT

k attributes

If there is exactly one candidate key and every super key is formed by including all candidate-key attributes plus any subset of the remaining attributes, then:

TEXT

Number of super keys =  $2^{(n-k)}$

because the remaining:

TEXT

n - k

attributes may be either:

TEXT

included  
or  
not included

---

## EXAMPLE

Relation:

TEXT

R(A, B, C, D, E)

Suppose only candidate key:

TEXT

AB

Here:

TEXT

n = 5  
k = 2

Remaining attributes:

TEXT

C, D, E

Each can be chosen or not chosen.

Therefore:

TEXT

Number of super keys  
 $= 2^{(5-2)}$   
 $= 2^3$   
 $= 8$

## The 8 super keys

TEXT

AB  
ABC  
ABD  
ABE  
ABCD  
ABCE  
ABDE  
ABCDE

## & pGATE TRAP — FORMULA CONDITION

Do **not** blindly use:

TEXT

$2^{(n-k)}$

when there are multiple candidate keys that overlap in complex ways.

Then different families of superkeys can overlap and require counting carefully, often using inclusion-exclusion.

We'll practice this when we study candidate-key problems.

## 28. INTEGRITY CONSTRAINTS — INTRODUCTION

Your syllabus separately includes integrity constraints.

We will teach them in depth soon, but relational model vocabulary requires an early overview.

Four major categories:

TEXT

Domain Constraint  
Key Constraint  
Entity Integrity  
Referential Integrity

## 29. DOMAIN CONSTRAINT

Values must belong to their allowed domain.



Example:

TEXT

M a r k s " [ 0 , 1 0 0 ]

Invalid:

TEXT

M a r k s = 150

## 30. KEY CONSTRAINT

Candidate key values must uniquely identify tuples.

Example:

TEXT

R o l l N o

should not be duplicated if it is a key.

## 31. ENTITY INTEGRITY

A primary key cannot be NULL.

Why?

Because a primary key must identify every tuple.

If primary key is unknown:

TEXT

N U L L

the tuple cannot be properly identified.

Therefore:

**Primary key attributes cannot be NULL.**

## 32. REFERENTIAL INTEGRITY

A foreign-key value must correspond to an existing referenced key value, or be NULL when NULL is permitted.

Example:

DEPARTMENT:

TEXT

D e p t I D

-----

CSE  
ECE

STUDENT:

```
TEXT
RollNo  DeptID
-----
101     CSE
102     ECE
```

Valid.

But:

```
TEXT
103     CIVIL
```

is invalid if **CIVIL** does not exist in DEPARTMENT and there is no matching referenced value.

## MEMORY TRICK

```
TEXT
Entity Integrity
!' Primary key cannot be NULL

Referential Integrity
!' Foreign key must point to existing referenced value
```

## 33. BASE RELATION vs DERIVED RELATION

Useful terminology.

### Base relation

Actually stored.

Example:

```
TEXT
STUDENT
COURSE
```

### Derived relation

Obtained from other relations through operations/querying.

Example:

```
TEXT
Students with marks > 90
```

The result may be produced using relational algebra or SQL.

## 34. RELATIONAL DATABASE

A relational database contains a collection of related relations.

Example:

```
TEXT
STUDENT(
  StudentID,
  Name,
  DeptID
)

DEPARTMENT(
  DeptID,
  DeptName
)

COURSE(
  CourseID,
  Title
)

ENROLLMENT(
  StudentID,
  CourseID
)
```

Relationships are represented primarily through common key values and constraints.

## 35. GATE-STYLE EXAMPLE — DEGREE & CARDINALITY

Given:

```
TEXT
EMPLOYEE

ID   Name   Dept   Salary
-----
1    Ravi    CSE    50000
2    Anu     AIML   60000
3    Raj     ECE    45000
```

How many attributes?

```
TEXT
4
```

Therefore:

```
TEXT
Degree = 4
```

How many tuples?

TEXT

3

Therefore:

TEXT

Cardinality = 3

## Answer

TEXT

Degree = 4

Cardinality = 3

## 36. GATE-STYLE EXAMPLE — CANDIDATE KEY

Relation:

TEXT

R(A, B, C)

Suppose:

TEXT

A uniquely identifies every tuple.

Then:

TEXT

A = Candidate Key

Super keys include:

TEXT

A

AB

AC

ABC

Why isn't AB a candidate key?

Because:

TEXT

A alone

already uniquely identifies tuples.

Therefore B is unnecessary.

## 37. GATE-STYLE EXAMPLE — COMPOSITE KEY

Consider:

TEXT

ENROLLMENT

| StudentID | CourseID | Grade |
|-----------|----------|-------|
| 1         | DBMS     | A     |
| 1         | OS       | B     |
| 2         | DBMS     | A     |

StudentID alone:

TEXT

not unique

CourseID alone:

TEXT

not unique

But:

TEXT

(StudentID, CourseID)

is unique.

Therefore:

TEXT

Composite Candidate Key

## 38. GATE-STYLE EXAMPLE — PRIME ATTRIBUTES

Suppose:

TEXT

R(A, B, C, D, E)

Candidate keys:

TEXT

AB  
CD

Prime attributes:

TEXT

A  
B  
C  
D

Non-prime:

TEXT

E

### GATE shortcut

Take:

union of all candidate-key attributes.

TEXT

A B " \* C D = A B C D

Those are the prime attributes.

## 39. GATE-STYLE EXAMPLE — SUPERKEY COUNT

Relation:

TEXT

R(A, B, C, D, E, F)

Only candidate key:

TEXT

AB

Then:

TEXT

n = 6  
k = 2

Hence:

TEXT

$2^{(6-2)}$   
=  $2^4$   
= 16

superkeys.

## 40. RELATION PROPERTIES — EXAM VIEW

Remember:

TEXT

Relation = Set of tuples

Therefore:

TEXT

Duplicate tuples !' not allowed theoretically  
Tuple order !' irrelevant  
Attribute ordering !' not semantically important

But:

**SQL does not always behave exactly like mathematical relational theory.**

This distinction becomes important later.

## NOTEBOOK NOTES

Write this compact version.

## RELATIONAL MODEL

Data represented as **relations (tables)**.

Terminology:

TEXT

Relation = Table  
Tuple = Row  
Attribute = Column  
Domain = Allowed values

## DEGREE

TEXT

Degree = Number of attributes

## CARDINALITY

TEXT

Cardinality = Number of tuples

Memory:

TEXT

D e g r e e ! ' C o l u m n s  
C a r d i n a l i t y ! ' R o w s

---

## SCHEMA vs INSTANCE

TEXT

Schema = Structure

Instance = Current data

---

## KEYS

### Super Key

Any attribute set uniquely identifying tuple.

### Candidate Key

Minimal super key.

### Primary Key

Selected candidate key.

### Alternate Key

Unselected candidate key.

### Composite Key

Key containing multiple attributes.

### Foreign Key

Attribute(s) referencing another relation's candidate/primary key.

---

## PRIME ATTRIBUTE

Attribute belonging to **at least one candidate key**.

---

## NON-PRIME ATTRIBUTE

Attribute belonging to no candidate key.

---

## SUPERKEY COUNT

If exactly one candidate key has **k** attributes in a relation with **n** attributes:



TEXT

#Superkeys =  $2^{(n-k)}$

---

## RELATIONAL PROPERTIES

TEXT

No duplicate tuples  
Tuple order irrelevant  
Values atomic under 1NF

---

## MEMORY TRICKS

---

### Tuple vs Attribute

TEXT

T u p l e            ! '    R o w  
A t t r i b u t e   ! '    C o l u m n

---

### Degree vs Cardinality

TEXT

Degree = Columns  
Cardinality = Rows

---

### Super Key vs Candidate Key

TEXT

Super Key = Unique

Candidate Key =  
Unique + Minimal

---

### Primary Key

TEXT

Primary = Picked Candidate

---

### Prime Attribute

TEXT

Prime = Part of any Candidate Key

## & pGATE TRAPS

---

### Trap 1

Every candidate key is a super key.

But:

**Every super key is NOT a candidate key.**

### Trap 2

A candidate key may contain multiple attributes.

Example:

TEXT  
AB

### Trap 3

A relation may have multiple candidate keys.

But only one is normally selected as the primary key.

### Trap 4

Foreign keys do not have to be unique.

### Trap 5

Prime attribute means member of **any candidate key**, not only primary key.

### Trap 6

Primary key:

TEXT  
cannot be NULL

Foreign key:

TEXT  
may be NULL

if allowed by schema/relationship constraints.

### Trap 7

Pure relational model does not contain duplicate tuples.

SQL query results, however, may contain duplicates.

## Trap 8

Degree and cardinality depend on different dimensions:

TEXT

Degree = columns

Cardinality = rows

## PYQ-ORIENTED QUESTION RECOGNITION

When GATE gives:

**“minimum subset uniquely identifying tuples”**

think:

TEXT

Candidate Key

When it says:

**“any set uniquely identifying tuples”**

think:

TEXT

Super Key

When it says:

**“attribute occurs in some candidate key”**

think:

TEXT

Prime Attribute

When it says:

**“references another relation”**

think:

TEXT

Foreign Key

When it asks:

**“number of columns”**

think:

TEXT

Degree

When it asks:

**“number of tuples”**

think:

TEXT

Cardinality

## 30-SECOND REVISION

TEXT

Relation = Table  
Tuple = Row  
Attribute = Column  
Domain = Allowed values

Degree = #columns  
Cardinality = #rows

Super Key = Unique  
Candidate Key = Unique + Minimal  
Primary Key = Chosen Candidate Key  
Alternate Key = Remaining Candidate Key  
Composite Key = Multi-attribute key  
Foreign Key = References another relation

Prime attribute = Part of ANY candidate key  
Non-prime = Part of none

Primary key cannot be NULL

If one candidate key of size k  
among n attributes:

Superkeys =  $2^{(n-k)}$

## CHECK YOUR UNDERSTANDING

### Q1

For the relation:

TEXT

R(A, B, C, D)

with 100 tuples, what are:

1. Degree
2. Cardinality

## Q2

Suppose A alone uniquely identifies every tuple.

Which of these are definitely super keys?

A. A B. AB C. AC D. ABCD E. All of the above

## Q3

Candidate keys of:

TEXT

R(A, B, C, D, E)

are:

TEXT

AB  
CD

Identify:

1. Prime attributes
2. Non-prime attributes

## Q4

A relation has 7 attributes and exactly one candidate key consisting of 3 attributes.

How many super keys exist?

## Q5

Which statement is correct?

A. Every super key is a candidate key. B. Every candidate key is a super key. C. Foreign keys must always be unique. D. Primary keys may contain NULL.

Next we move into one of the first **major GATE problem-solving chapters**:

## TOPIC 4 — RELATIONAL ALGEBRA

We will learn:

**Selection !' Projection !' Union !' Intersection !' Difference !' Cartesian Product !' Rename !' Joins !' Natural Join**

!' Theta Join !' Division !' query-expression solving !' cardinality tricks !' GATE traps.

Type `NEXT`.

USER

NEXT

ASSISTANT

## TOPIC 4 — RELATIONAL ALGEBRA

Relational Algebra is explicitly part of your GATE DBMS syllabus.

This is one of the first **real problem-solving chapters** in DBMS. GATE can give you relations and expressions such as:

$\pi_{\text{Name}}(\sigma_{\text{Dept}='CSE'}(\text{STUDENT}))$

and ask:

- What does the expression return?
- How many tuples are produced?
- Which relational algebra expression represents a query?
- What is the difference between join, product, projection, selection, division, etc.?

So don't memorize symbols blindly. We will learn to **read an expression like a sentence**.

## 1. WHY RELATIONAL ALGEBRA EXISTS

Suppose we have:

### STUDENT

| Sid | Name  | Dept | Age |
|-----|-------|------|-----|
| 1   | Ravi  | CSE  | 20  |
| 2   | Anu   | AIML | 21  |
| 3   | Kiran | CSE  | 19  |
| 4   | Riya  | ECE  | 20  |

Now someone asks:

**Give me the names of all CSE students.**

We need two operations:

1. Keep only students whose Dept = CSE.
2. Keep only their Name column.

Conceptually:

TEXT

STUDENT

!"

Choose rows where Dept = CSE

!"

Choose Name column

Relational algebra writes this mathematically:

$\pi_{\text{Name}}(\sigma_{\text{Dept}='CSE'}(\text{STUDENT}))$

This is relational algebra.

## 2. WHAT IS RELATIONAL ALGEBRA?

**Relational Algebra (RA)** is a **procedural query language** for the relational model.

Procedural means:

**We specify what operations to perform and in what sequence to obtain the result.**

That is important because later:

**Tuple Relational Calculus = declarative/non-procedural**

## MEMORY TRICK

### Relational Algebra

**HOW to get the answer**

### Relational Calculus

**WHAT answer I want**

We'll compare them later.

## 3. CLOSURE PROPERTY

One beautiful property of relational algebra:

**Every relational algebra operation takes relation(s) as input and produces another relation as output.**

Example:

TEXT

Relation

!"

Selection

!"

```
Relation
  ! "
Projection
  ! "
Relation
```

Therefore operations can be combined/nested.

This is called the **closure property**.

## 4. BASIC RELATIONAL ALGEBRA OPERATORS

Core operators include:

```
TEXT
Ã      S e l e c t i o n
À      P r o j e c t i o n
"*     U n i o n
"      S e t   D i f f e r e n c e
×      C a r t e s i a n   P r o d u c t
Á      R e n a m e
```

Other useful/derived operators:

```
TEXT
")     I n t e r s e c t i o n
"È     J o i n
÷      D i v i s i o n
```

We will learn each carefully.

## PART A — SELECTION

### 5. SELECTION — Ã

Selection chooses **rows/tuples** satisfying a condition.

Symbol:

$\sigma$

Syntax:

$\sigma_{\text{condition}}(R)$

### Example

STUDENT:

| Sid | Name | Dept | Age |
|-----|------|------|-----|
| 1   | Ravi | CSE  | 20  |



| Sid | Name  | Dept | Age |
|-----|-------|------|-----|
| 2   | Anu   | AIML | 21  |
| 3   | Kiran | CSE  | 19  |
| 4   | Riya  | ECE  | 20  |

Query:

**Find CSE students.**

$\sigma_{\text{Dept}='CSE'}(\text{STUDENT})$

Result:

| Sid | Name  | Dept | Age |
|-----|-------|------|-----|
| 1   | Ravi  | CSE  | 20  |
| 3   | Kiran | CSE  | 19  |

## 6. WHAT DOES SELECTION CHANGE?

Selection removes rows.

It does **not** remove columns.

If original relation has 4 attributes:

TEXT

Sid, Name, Dept, Age

selection result still has 4 attributes.

Therefore:

$\text{Degree}(\sigma_C(R)) = \text{Degree}(R)$

## CARDINALITY

Selection may reduce the number of tuples.

Therefore:

$\sigma_C(R)$

|               |        |   |
|---------------|--------|---|
| $\sigma_C(R)$ | $\leq$ | R |
|---------------|--------|---|

$\sigma_C(R)$

It can return:

TEXT

0 tuples  
some tuples  
all tuples

## MEMORY TRICK

**Selection selects rows.**

Think:

TEXT

Ä ! ' h o r i z o n t a l f i l t e r i n g

## 7. SELECTION CONDITIONS

We can use:

TEXT

=  
" "  
<  
>  
"d  
"e  
AND  
OR  
NOT

Example:

**CSE students older than 19.**

$\sigma_{\text{Dept}='CSE' \wedge \text{Age} > 19}(\text{STUDENT})$

Result:

TEXT

Ravi

## 8. CASCADING SELECTION

Suppose:

$\sigma_{\text{Age} > 19}(\sigma_{\text{Dept}='CSE'}(\text{STUDENT}))$

This means:

First:

TEXT

Dept = CSE

then:

TEXT

Age > 19

Equivalent to:

$\sigma_{\text{Dept}='CSE' \wedge \text{Age}>19}(\text{STUDENT})$

## IMPORTANT PROPERTY

$\sigma_{C_1}(\sigma_{C_2}(R)) = \sigma_{C_1 \wedge C_2}(R)$

Also selections commute:

$\sigma_{C_1}(\sigma_{C_2}(R)) = \sigma_{C_2}(\sigma_{C_1}(R))$

This becomes useful in query optimization.

## PART B — PROJECTION

### 9. PROJECTION — $\pi$

Projection chooses **columns/attributes**.

Symbol:

$\pi$

Syntax:

$\pi_{\text{attributes}}(R)$

Example:

**Display only Name and Dept.**

$\pi_{\text{Name,Dept}}(\text{STUDENT})$

Result:

| Name  | Dept |
|-------|------|
| Ravi  | CSE  |
| Anu   | AIML |
| Kiran | CSE  |
| Riya  | ECE  |

## MEMORY TRICK

Projection projects columns.

TEXT

À !' v e r t i c a l f i l t e r i n g

So:

TEXT

S e l e c t i o n !' R o w s  
P r o j e c t i o n !' C o l u m n s

This distinction is absolutely essential.

## 10. WHAT DOES PROJECTION CHANGE?

Projection usually reduces **degree**.

If:

TEXT

STUDENT(Sid, Name, Dept, Age)

has degree 4,

then:

$\pi_{\{Name, Dept\}}(STUDENT)$

has degree:

TEXT

2

## 11. PROJECTION REMOVES DUPLICATES

This is extremely important for GATE.

Suppose:

| Sid | Dept |
|-----|------|
| 1   | CSE  |
| 2   | AIML |
| 3   | CSE  |
| 4   | ECE  |

Now:

$\pi_{\text{Dept}}(\text{STUDENT})$

The raw values are:

```
TEXT
CSE
AIML
CSE
ECE
```

But relations are sets.

Therefore duplicate CSE is removed.

Result:

| Dept |
|------|
| CSE  |
| AIML |
| ECE  |

So cardinality becomes 3.

## & pMAJOR GATE TRAP

### Relational Algebra projection

removes duplicate tuples automatically.

### SQL SELECT

Does not necessarily remove duplicates unless you use:

```
SQL
DISTINCT
```

Remember this difference.

## 12. PROJECTION CARDINALITY

For:

$\pi_X(R)$

we know:

$|\pi_X(R)| \leq |R|$

|            |        |     |
|------------|--------|-----|
| $\pi_X(R)$ | $\leq$ | $R$ |
|------------|--------|-----|

$|\pi_X(R)| \leq |R|$

because duplicates may disappear.

## 13. CASCADING PROJECTION

Suppose:

$\pi_A(\pi_{\{A,B\}}(R))$

Equivalent to:

$\pi_A(R)$

because after the inner projection we still retain A.

But this is invalid:

$\pi_A(\pi_B(R))$

if A is not available after the first projection.

## SELECTION vs PROJECTION

| Selection              | Projection                                          |
|------------------------|-----------------------------------------------------|
| Symbol $\sigma$        | Symbol $\pi$                                        |
| Selects rows           | Selects columns                                     |
| Degree unchanged       | Degree may reduce                                   |
| Cardinality may reduce | Cardinality may reduce because duplicates disappear |
| Uses condition         | Uses attribute list                                 |

### Memory

TEXT

$\sigma$  !' Rows  
 $\pi$  !' Columns

## 14. COMBINING SELECTION + PROJECTION

Question:

**Find names of CSE students.**

Step 1:

$\sigma_{\text{Dept}='CSE'}(\text{STUDENT})$

Step 2:

$\pi_{\text{Name}}(\dots)$

Final:

$\pi_{\text{Name}}(\sigma_{\text{Dept}='CSE'}(\text{STUDENT}))$

## HOW TO READ NESTED EXPRESSIONS

Always evaluate **inside !' outside**.

For:

$\pi_{\text{Name}}(\sigma_{\text{Dept}='CSE'}(\text{STUDENT}))$

read:

TEXT

Start with STUDENT

!"

Select Dept = CSE

!"

Project Name

## GATE SHORTCUT

RA expressions are read from the innermost brackets outward.

## PART C — UNION

### 15. UNION — " $\cup$ "

Union combines tuples from two relations.

$R \cup S$

Result contains tuples that are:

TEXT

in R

OR

in S

OR

in both

Duplicates are removed because relations are sets.

## Example

R:

| Sid |
|-----|
| 1   |
| 2   |
| 3   |

S:

| Sid |
|-----|
| 3   |
| 4   |
| 5   |

Then:

$\mathcal{R} \cup \mathcal{S}$

Result:

TEXT

1  
2  
3  
4  
5

Not:

TEXT

1  
2  
3  
3  
4  
5

## 16. UNION COMPATIBILITY

This is critical.

For:

$\mathcal{R} \cup \mathcal{S}$

the relations must be **union compatible**.

That means:

1. Same number of attributes.
2. Corresponding attributes must have compatible domains.

Example:

TEXT

$R(A, B)$   
 $S(C, D)$

can potentially be union-compatible if:



TEXT

domain(A) compatible with domain(C)  
domain(B) compatible with domain(D)

Attribute names themselves do not necessarily need to be identical in the mathematical idea, but corresponding domains/structure must be compatible.

## & pGATE TRAP

You cannot generally union:

TEXT

STUDENT(Sid, Name, Dept)

with:

TEXT

COURSE(Cid, Title)

because degrees differ:

TEXT

3 vs 2

## 17. UNION CARDINALITY FORMULA

For sets:

$\setminus$

$R \setminus \cup S$

=

|   |   |   |   |                      |
|---|---|---|---|----------------------|
| R | + | S | - | $R \setminus \cap S$ |
|---|---|---|---|----------------------|

$\setminus$

Example:

TEXT

$|R| = 10$   
 $|S| = 8$   
 $|R \setminus S| = 3$

Then:

$|R \setminus \cup S| = 10 + 8 - 3 = 15$

## Bounds

Minimum possible:

$$\lceil \max(|R|, |S|) \rceil$$

if the smaller relation is contained in larger.

Maximum:

$$\lceil$$

|   |   |   |
|---|---|---|
| R | + | S |
|---|---|---|

$$\rceil$$

if they are disjoint.

## PART D — INTERSECTION

### 18. INTERSECTION — "

$$\lceil R \cap S \rceil$$

returns tuples appearing in **both** relations.

Example:

TEXT

$$R = \{1, 2, 3\}$$

$$S = \{3, 4, 5\}$$

Then:

$$\lceil R \cap S = \{3\} \rceil$$

Relations must be union compatible.

### 19. INTERSECTION CAN BE DERIVED

Intersection is not required as a primitive operator because:

$$\lceil R \cap S = R - (R - S) \rceil$$

or equivalently:

$$\lceil R \cap S = S - (S - R) \rceil$$

This is useful theoretically.

## PART E — SET DIFFERENCE

### 20. SET DIFFERENCE — "

$$\lceil R - S \rceil$$

returns tuples present in R but not in S.

Example:

TEXT

R = {1, 2, 3}  
S = {3, 4}

Then:

$R - S = \{1, 2\}$

## IMPORTANT

Set difference is **not commutative**.

Generally:

$R - S \neq S - R$

Example:

TEXT

R - S = {1, 2}  
S - R = {4}

## MEMORY TRICK

In " $R - S$ ", start from R, then remove anything also found in S.

## Union / Intersection / Difference

| Operation  | Meaning     |
|------------|-------------|
| $R \cup S$ | R OR S      |
| $R \cap S$ | R AND S     |
| $R - S$    | R but NOT S |

## PART F — CARTESIAN PRODUCT

### 21. CARTESIAN PRODUCT — $\times$

Cartesian product combines **every tuple of R with every tuple of S**.

$R \times S$

Suppose:

R:

| A |
|---|
| 1 |
| 2 |

S:

| B |
|---|
| X |
| Y |
| Z |

Then:

$\forall [R \times S]$

gives:

| A | B |
|---|---|
| 1 | X |
| 1 | Y |
| 1 | Z |
| 2 | X |
| 2 | Y |
| 2 | Z |

There are:

TEXT

$2 \times 3 = 6$  tuples

## 22. CARTESIAN PRODUCT CARDINALITY

If:

$\forall [R=m]$

and:

$\forall [S=n]$

then:

$\forall [R \times S = mn]$

---

## 23. DEGREE OF CARTESIAN PRODUCT

---

If:

TEXT

degree(R) = p  
degree(S) = q

then:

$\forall \text{ degree}(R \times S) = p + q$

Example:

TEXT

R has 3 attributes  
S has 4 attributes

Then:

TEXT

$R \times S$  has 7 attributes

---

## VERY IMPORTANT FORMULA

---

For Cartesian product:

$\forall \boxed{|R \times S| = |R| \times |S|}$

and:

$\forall \boxed{\text{degree}(R \times S) = \text{degree}(R) + \text{degree}(S)}$

Worth memorizing.

---

## 24. ATTRIBUTE NAME CONFLICT

---

Suppose:

TEXT

R(A, B)  
S(B, C)

Then **B** exists in both.

In Cartesian product we may distinguish using qualified names:

TEXT

R.B  
S.B

or rename attributes.

This leads to the rename operator.

---

## PART G — RENAME

---

### 25. RENAME — $\rho$

---

Symbol:

$\rho$

Used to rename:

- Relation
- Attributes
- Both

Example:

$\rho_S(STUDENT)$

renames STUDENT as S.

---

### Why do we need rename?

---

Especially useful when:

1. The same relation participates more than once.
2. Attribute names conflict.
3. We want temporary aliases.

Example:

Employee:

TEXT

EMPLOYEE(Eid, Name, ManagerID)

To compare employees with other employees, we might use:

TEXT

E1 = EMPLOYEE

E2 = EMPLOYEE

conceptually using rename.

This is useful for **self joins**.

---

## PART H — JOIN

---

### 26. WHY JOIN EXISTS

---

Suppose:

## STUDENT

| Sid | Name | DeptID |
|-----|------|--------|
| 1   | Ravi | 10     |
| 2   | Anu  | 20     |

## DEPARTMENT

| DeptID | DeptName |
|--------|----------|
| 10     | CSE      |
| 20     | AIML     |

We want:

TEXT

```
R a v i   ! '   C S E
A n u     ! '   A I M L
```

Information is split across two relations.

A **join** combines related tuples.

## 27. JOIN IS PRODUCT + SELECTION

Conceptually:

$\bowtie_{\text{condition}} R, S$

can be thought of as:

$\sigma_{\text{condition}}(R \times S)$

Very important.

### MEMORY TRICK

**Join = Cartesian Product + Match Condition**

## 28. THETA JOIN

A **theta join** uses a comparison condition.

$\bowtie_{R.A \theta S.B} R, S$

where theta can be:

TEXT

```
=
<
>
```

```
"d
"e
" `
```

Example:

```
\[ STUDENT \bowtie_{STUDENT.DeptID=DEPARTMENT.DeptID} DEPARTMENT \]
```

## 29. EQUI JOIN

An **equi join** is a theta join where the comparison operator is equality.

So:

```
\[ R \bowtie_{R.A=S.B} S \]
```

is an equi join.

## Key relationship

TEXT

Theta Join

|

% % %

|

E q u i J o i n

% % %

N a t u r a l J o i n ( s p e c i a l c a s e )

Natural join has additional duplicate-column handling.

## 30. EQUI JOIN EXAMPLE

STUDENT:

| Sid | Name | DeptID |
|-----|------|--------|
| 1   | Ravi | 10     |
| 2   | Anu  | 20     |

DEPT:

| DeptID | DeptName |
|--------|----------|
| 10     | CSE      |
| 20     | AIML     |

Equi join on DeptID:

| Sid | Name | STUDENT.DeptID | DEPT.DeptID | DeptName |
|-----|------|----------------|-------------|----------|
| 1   | Ravi | 10             | 10          | CSE      |



| Sid | Name | STUDENT.DeptID | DEPT.DeptID | DeptName |
|-----|------|----------------|-------------|----------|
| 2   | Anu  | 20             | 20          | AIML     |

Notice:

Both copies of the joining attribute can remain in an equi join.

### 31. NATURAL JOIN — "È

Natural join:

- 1. Automatically identifies attributes with the same name.
- 2. Performs equality matching on those common attributes.
- 3. Keeps only one copy of each common attribute.

Example:

```
TEXT
STUDENT(Sid, Name, DeptID)
DEPT(DeptID, DeptName)
```

Then:

\[ STUDENT \bowtie DEPT \]

result:

| Sid | Name | DeptID | DeptName |
|-----|------|--------|----------|
| 1   | Ravi | 10     | CSE      |
| 2   | Anu  | 20     | AIML     |

Only one DeptID .

### Equi Join vs Natural Join

| Equi Join                             | Natural Join                                |
|---------------------------------------|---------------------------------------------|
| Equality condition explicitly given   | Matches same-named attributes automatically |
| Both join columns may remain          | Duplicate common column removed             |
| Can join differently named attributes | Based on common attribute names             |

### & p<HUGE GATE TRAP — NATURAL JOIN

Natural join uses **all common attribute names**.

Suppose:

TEXT

$R(A, B, C)$   
 $S(B, C, D)$

Natural join requires matching on:

TEXT

$B \text{ AND } C$

Not merely B.

So:

$\forall [R \bowtie S]$

is essentially based on:

$\forall [R.B=S.B \wedge R.C=S.C]$

Then duplicate B and C columns are retained only once.

---

## 32. DEGREE OF NATURAL JOIN

Suppose:

TEXT

$\text{degree}(R) = p$   
 $\text{degree}(S) = q$

and they share:

TEXT

$k$  common attributes

Then:

$\forall [\text{degree}(R \bowtie S) = p + q - k]$

provided those  $k$  common attributes are the natural-join columns.

---

## Example

TEXT

$R(A, B, C)$   
 $S(B, C, D, E)$

Then:

TEXT

$p=3$   
 $q=4$   
 $k=2$

Therefore natural join degree:

$\lfloor 3+4-2=5 \rfloor$

Attributes:

TEXT

A, B, C, D, E

### 33. NATURAL JOIN CARDINALITY

This is trickier.

There is no simple universal exact formula because result size depends on matching values.

But bounds are often useful.

If joining on common attribute(s):

TEXT

minimum can be 0

because no values may match.

Maximum depends on duplicates/value distributions.

In many general cases maximum can approach:

$\lfloor$

|   |          |   |
|---|----------|---|
| R | $\times$ | S |
|---|----------|---|

$\rfloor$

if all tuples match on common join attributes.

However, if a join attribute is a key in one relation, stronger bounds exist.

### 34. KEY-FOREIGN KEY JOIN

Very important GATE scenario.

Suppose:

TEXT

DEPARTMENT(DepartmentID, DepartmentName)

DepartmentID = primary key.

STUDENT:

TEXT

STUDENT(StudentID, Name, DepartmentID)

Every student has a valid non-NULL DepartmentID foreign key.

Then:

$\{ \text{STUDENT} \bowtie \text{DEPARTMENT} \}$

has exactly:

$\{$

STUDENT

$\}$

tuples.

Why?

Each student matches exactly one department.

## GATE SHORTCUT

For a proper FK !' referenced key join:

**Each referencing tuple normally matches at most one referenced tuple.**

If every FK is valid and non-NULL:

**each referencing tuple matches exactly one.**

Therefore result cardinality = number of referencing tuples.

## 35. SELF JOIN

A relation joined with itself.

Example:

TEXT

EMPLOYEE(Eid, Name, ManagerID)

To find:

TEXT

Employee Name + Manager Name

we use two renamed versions:

TEXT

E1

E2

and join:

TEXT

```
E1.ManagerID = E2.Eid
```

Rename is essential to distinguish the two roles.

## PART I — OUTER JOINS

Outer joins are often more SQL-oriented, but knowing the idea helps.

Normal joins can discard nonmatching tuples.

Outer joins preserve them.

Types:

TEXT

```
Left Outer Join
Right Outer Join
Full Outer Join
```

### 36. LEFT OUTER JOIN

Keeps:

- all matching tuples
- all tuples from left relation even if no match

Missing right-side attributes become NULL.

Symbol sometimes:

$\bowtie_{\text{leftouterjoin}}$  R S

Notation varies by text.

### 37. RIGHT OUTER JOIN

Keeps all tuples from right relation.

### 38. FULL OUTER JOIN

Keeps unmatched tuples from both sides.

## INNER vs OUTER JOIN

TEXT

```
Inner Join
! ' only matching tuples

Outer Join
```

!' also retains certain unmatched tuples

SQL will make this much clearer later.

## PART J — DIVISION

### 39. DIVISION — $\div$

Division is one of the most confusing relational algebra operators, but there is a simple pattern.

Whenever the English query contains:

**“for all”**

start thinking about **division**.

### Example

Suppose:

ENROLLMENT:

| Student | Course |
|---------|--------|
| Ravi    | DBMS   |
| Ravi    | OS     |
| Ravi    | CN     |
| Anu     | DBMS   |
| Anu     | OS     |
| Raj     | DBMS   |

REQUIRED:

| Course |
|--------|
| DBMS   |
| OS     |

Question:

**Find students who have taken all courses in REQUIRED.**

Answer:

$\pi [ \text{ENROLLMENT} \div \text{REQUIRED} ]$

Result:

TEXT

Ravi  
Anu

Why?

Ravi has:

TEXT

D B M S ' '  
O S ' '

Anu has:

TEXT

D B M S ' '  
O S ' '

Raj has:

TEXT

D B M S ' '  
O S ' '

Therefore Raj is excluded.

## MEMORY TRICK

**Division = ALL**

Whenever you see:

TEXT

all courses  
every project  
all subjects  
every required skill

division is a strong candidate.

## 40. GENERAL DIVISION STRUCTURE

Suppose:

TEXT

$R(X, Y)$   
 $S(Y)$

Then:

`[ R \div S ]`

returns values of `X` that are paired in `R` with **every value of `Y` appearing in `S`.**

So:

TEXT

```
R ( X , Y )  ÷  S ( Y )  !'  R e s u l t ( X )
```

## 41. DIVISION EXAMPLE

`R`:

| A | B |
|---|---|
| 1 | x |
| 1 | y |
| 1 | z |
| 2 | x |
| 2 | y |
| 3 | x |

`S`:

| B |
|---|
| x |
| y |

Then:

`[ R \div S ]`

returns:

TEXT

```
1
2
```

because:

TEXT

```
1  !'  x   a n d   y
2  !'  x   a n d   y
3  !'  o n l y   x
```

## 42. DIVISION USING BASIC OPERATORS



Division is a derived operator.

For:

```
TEXT
R(X, Y)
S(Y)
```

one common relational-algebra construction is:

$$\pi_X(R) - \pi_X(\pi_X(R) \times S) \div R$$

This looks scary, but conceptually:

1. Find all candidate X values.
2. Generate all X–Y combinations they should have.
3. Find combinations that are missing from R.
4. Remove X values having any missing required combination.

You do not need to memorize this immediately unless a question asks expression equivalence.

## 43. DIVISION MEMORY STORY

Imagine a placement eligibility table.

```
TEXT
STUDENT_SKILL(Student, Skill)
```

Required skills:

```
TEXT
REQUIRED(Skill)
```

Question:

**Which student has every required skill?**

Use:

$$STUDENT\_SKILL \div REQUIRED$$

The word **every/all** is the clue.

## PART K — DERIVING OPERATORS

### 44. INTERSECTION USING DIFFERENCE

$$R \cap S = R - (R - S)$$

### 45. JOIN USING PRODUCT + SELECTION

$\sigma_C(R \bowtie S) = \sigma_C(R \times S)$

This is one of the most important identities.

## 46. NATURAL JOIN CONCEPTUALLY

Natural join can be viewed as:

TEXT

Cartesian Product  
!"  
Select equality on all same-name attributes  
!"  
Remove duplicate copies of join columns

So:

**Natural Join = Product + Selection + Projection**

Conceptually.

## PART L — OPERATOR CLASSIFICATION

Relational algebra operations can also be classified by number of input relations.

### Unary operators

Take one relation.

TEXT

Selection  
Projection  
Rename

### Binary operators

Take two relations.

TEXT

Union  
Difference  
Intersection  
Cartesian Product  
Join  
Division

## MEMORY TRICK

Unary = **one** input

TEXT

Ã  
À  
Á

Everything combining relations is generally binary.

## 47. PROPERTIES YOU SHOULD KNOW

### Selection is commutative

$$\sigma_{C_1}(\sigma_{C_2}(R)) = \sigma_{C_2}(\sigma_{C_1}(R))$$

### Selection cascade

$$\sigma_{C_1 \wedge C_2}(R) = \sigma_{C_1}(\sigma_{C_2}(R))$$

### Union is commutative

$$R \cup S = S \cup R$$

### Intersection is commutative

$$R \cap S = S \cap R$$

### Difference is NOT commutative

$$R - S \neq S - R$$

### Cartesian product is not literally identical under fixed attribute ordering/naming

But after appropriate reordering/renaming, it has a symmetric mathematical relationship.

For GATE, don't casually treat:

$$R \times S$$

and:

$$S \times R$$

as syntactically identical relations without considering schema/attribute ordering and names.

## 48. SELECTION DISTRIBUTES OVER UNION

$$\sigma_C(R \cup S) = \sigma_C(R) \cup \sigma_C(S)$$

provided schemas are compatible.

---

## 49. SELECTION DISTRIBUTES OVER DIFFERENCE

---

$$\sigma_C(R-S) = \sigma_C(R) - \sigma_C(S)$$

when the condition applies appropriately.

---

## 50. PROJECTION DOES NOT ALWAYS COMMUTE WITH EVERYTHING

---

Be careful.

Once projection removes an attribute, later operations needing that attribute may become impossible.

Example:

TEXT

$R(A, B, C)$

If you first perform:

$$\pi_A(R)$$

then attribute B is gone.

So you cannot later evaluate:

$$\sigma_{B=10}$$

on that result.

---

## GATE QUERY OPTIMIZATION IDEA

---

Selection should often be pushed down early because it reduces tuples before expensive operations.

Example:

Instead of:

$$\sigma_{R.A=5}(R \times S)$$

we can often perform:

$$\sigma_{A=5}(R) \times S$$

if the condition only involves R.

Why is it better?

If:

TEXT

$R = 1,000,000$  rows

and selection leaves:

---

TEXT

100 rows

then product becomes dramatically smaller.

This is the intuition behind **selection pushdown**.

## 51. GATE-STYLE SOLVED EXAMPLE 1

Given:

### STUDENT

| Sid | Name | Dept |
|-----|------|------|
| 1   | Ravi | CSE  |
| 2   | Anu  | ECE  |
| 3   | Riya | CSE  |
| 4   | Raj  | AIML |

Evaluate:

$\pi_{\text{Name}}(\sigma_{\text{Dept}='CSE'}(\text{STUDENT}))$

### Step 1 — Selection

TEXT

Ravi CSE  
Riya CSE

### Step 2 — Projection

TEXT

Ravi  
Riya

### Answer

| Name |
|------|
| Ravi |
| Riya |

## 52. GATE-STYLE EXAMPLE 2 — DUPLICATES

Relation:

| A | B |
|---|---|
| 1 | X |
| 2 | X |
| 3 | Y |
| 4 | Y |
| 5 | Z |

Find cardinality of:

$\pi_B(R)$

Values:

TEXT

X  
X  
Y  
Y  
Z

Projection removes duplicates.

Result:

TEXT

X  
Y  
Z

Therefore:

$|\pi_B(R)|=3$

## 53. GATE-STYLE EXAMPLE 3 — PRODUCT

Suppose:

TEXT

$|R| = 10$   
 $\text{degree}(R) = 3$

$|S| = 20$   
 $\text{degree}(S) = 4$

For:

$R \times S$

Cardinality:

$10 \times 20 = 200$

Degree:

$\backslash$  3+4=7  $\backslash$

### Answer

TEXT

Cardinality = 200  
Degree = 7

## 54. GATE-STYLE EXAMPLE 4 — UNION

Suppose:

TEXT

$|R| = 30$   
 $|S| = 20$   
 $|R \cap S| = 5$

Find:

$\backslash$

$R \cup S$

$\backslash$

Formula:

$\backslash$  30+20-5=45  $\backslash$

### Answer

$\backslash$  45  $\backslash$

## 55. GATE-STYLE EXAMPLE 5 — NATURAL JOIN DEGREE

Relations:

TEXT

$R(A, B, C, D)$   
 $S(C, D, E, F, G)$

Common attributes:

TEXT

C, D

Therefore:

TEXT

degree(R)=4  
degree(S)=5  
common=2

Natural join degree:

$\lfloor 4+5-2=7 \rfloor$

Result schema conceptually:

TEXT

A, B, C, D, E, F, G

---

## 56. GATE-STYLE EXAMPLE 6 — KEY-FK JOIN

---

Relation:

TEXT

EMPLOYEE(Eid, Name, DeptID)

contains 500 employees.

DEPARTMENT:

TEXT

DEPARTMENT(DeptID, DeptName)

contains 10 departments.

Assume:

- DeptID is primary key of DEPARTMENT.
- Every EMPLOYEE contains a valid non-NULL DeptID.

What is cardinality of:

$\lfloor \text{EMPLOYEE} \bowtie \text{DEPARTMENT} \rfloor$

Every employee maps to exactly one department.

Therefore:

$\lfloor 500 \rfloor$

Not:

TEXT

$500 \times 10$

That would be Cartesian product.

---

## & ⚠ MAJOR GATE TRAP

---

Don't confuse:

$\lfloor R \times S \rfloor$

with:

$\lfloor R \bowtie S \rfloor$



Product creates **all combinations**.

Join retains **matching combinations**.

---

## 57. GATE-STYLE EXAMPLE 7 — DIVISION

Relation:

TEXT  
WORKS(Employee, Project)

Required:

TEXT  
REQUIRED(Project)

Query:

**Find employees working on every required project.**

Immediately recognize:

TEXT  
EVERY

Therefore:

$\forall$  [ WORKS  $\div$  REQUIRED ]

---

## 58. HOW TO TRANSLATE ENGLISH TO RA

This is very useful.

**“Find all CSE students”**

TEXT  
condition

!' Selection

$\forall$  [  $\sigma_{\text{Dept}='CSE'}$  (STUDENT) ]

**“Find names of students”**

TEXT  
specific columns

!' Projection

$\forall$  [  $\pi_{\text{Name}}$  (STUDENT) ]

---

**“Find students who are either in club A or club B”**

!' Union

$\backslash [ A \cup B \backslash ]$

---

**“Find students belonging to both clubs”**

!' Intersection

$\backslash [ A \cap B \backslash ]$

---

**“Find students in club A but not B”**

!' Difference

$\backslash [ A - B \backslash ]$

---

**“Combine every student with every course”**

!' Cartesian Product

$\backslash [ STUDENT \times COURSE \backslash ]$

---

**“Combine student with matching department”**

!' Join

---

**“Find students taking all compulsory courses”**

!' Division

---

## MASTER ENGLISH-TO-OPERATOR TABLE

---

| English clue             | Operator                   |
|--------------------------|----------------------------|
| where / satisfying       | Selection $\tilde{A}$      |
| show only columns        | Projection $\tilde{A}$     |
| either / or              | Union $\cup$               |
| both                     | Intersection $\cap$        |
| but not                  | Difference $-$             |
| every combination        | Cartesian Product $\times$ |
| matching related records | Join $\Join$               |
| all/every required       | Division $\div$            |
| alias/rename             | Rename $\rho$              |

This table is extremely useful for revision.

---

## 59. RELATIONAL ALGEBRA vs SQL PREVIEW

---

RA:

$\pi_{\text{Name}}(\sigma_{\text{Dept}='CSE'}(\text{STUDENT}))$

SQL equivalent:

SQL

```
SELECT Name
FROM STUDENT
WHERE Dept = 'CSE';
```

Conceptually:

TEXT

```
Ã  c o n d i t i o n !"  W H E R E

À  a t t r i b u t e s !"  S E L E C T  c o l u m n  l i s t
```

This connection will make SQL much easier later.

---

## 60. RELATIONAL ALGEBRA vs RELATIONAL CALCULUS PREVIEW

---

### Relational Algebra

Procedural.

TEXT

```
Select rows
then project columns
then join...
```

### Tuple Relational Calculus

Declarative.

TEXT

```
Give tuples that satisfy condition X.
```

Memory:

**Algebra = Operations**

**Calculus = Conditions**

---

## FORMULA / RULE SHEET

---

## Selection

$\sigma_{\text{degree}(\sigma_C(R))=\text{degree}(R)}$

[

|               |        |   |
|---------------|--------|---|
| $\sigma_C(R)$ | $\leq$ | R |
|---------------|--------|---|

]

## Projection

$\pi_{\text{degree}(\pi_X(R))=|X|}$

[

|            |        |   |
|------------|--------|---|
| $\pi_X(R)$ | $\leq$ | R |
|------------|--------|---|

]

Duplicates are removed.

## Cartesian Product

[

|              |   |   |  |   |
|--------------|---|---|--|---|
| $R \times S$ | = | R |  | S |
|--------------|---|---|--|---|

]

$\text{degree}(R \times S) = \text{degree}(R) + \text{degree}(S)$

## Union

[

|            |
|------------|
| $R \cup S$ |
|------------|

=

|   |   |   |   |            |
|---|---|---|---|------------|
| R | + | S | - | $R \cap S$ |
|---|---|---|---|------------|

]

## Natural Join Degree

If  $k$  common join attributes:

$\text{degree}(R \bowtie S) = \text{degree}(R) + \text{degree}(S) - k$

---

## Join

---

$\sigma_{R \bowtie_C S} = \sigma_C(R \times S)$

---

## 61. GATE TRAPS — MUST REMEMBER

---

### & pTrap 1 — Selection vs Projection

---

TEXT

Selection = rows  
Projection = columns

---

### & pTrap 2 — Projection duplicates

---

Projection removes duplicates in relational algebra.

---

### & pTrap 3 — SQL difference

---

SQL:

SQL

SELECT Dept

---

may contain duplicates.

RA:

$\pi_{\{Dept\}}$

does not.

---

### & pTrap 4 — Set-operation compatibility

---

Union, intersection and set difference require union-compatible relations.

---

### & pTrap 5 — Difference

---

$R - S \neq S - R$

---

### & pTrap 6 — Product cardinality

---

Always:

$\bowtie$

---

|           |   |   |  |   |
|-----------|---|---|--|---|
| R times S | = | R |  | S |
|-----------|---|---|--|---|

\]

### & pTrap 7 — Natural join

Matches on **all same-named attributes**, not arbitrarily one.

### & pTrap 8 — Natural join removes duplicate join columns

Equi join may retain both.

### & pTrap 9 — Join " Cartesian product

Join contains only tuples satisfying matching condition.

### & pTrap 10 — Division

Look for:

ALL / EVERY

## 62. PYQ SOLVING PROCEDURE

When GATE gives a complicated RA expression:

### STEP 1

Start from innermost relation/operator.

### STEP 2

For every operation, write what changes:

TEXT

```

~ !' r o w s
^ !' c o l u m n s
x !' r o w s m u l t i p l y , c o l u m n s a d d
"È !' m a t c h i n g r o w s
"* !' c o m b i n e
" !' r e m o v e
÷ !' s a t i s f y A L L

```

### STEP 3

Track the relation schema after every projection/join.

This prevents attribute-reference errors.

### STEP 4

For cardinality questions, don't immediately enumerate rows.

First check:

TEXT

```
selection?
projection?
product?
key-FK join?
set operation?
```

There may be a shortcut formula.

## NOTEBOOK NOTES

### RELATIONAL ALGEBRA

Procedural relational query language.

Input:

TEXT

```
Relation(s)
```

Output:

TEXT

```
Relation
```

Closure property.

### Basic operators

TEXT

```
σ Selection
π Projection
⋈ Union
⋈ Difference
⋈ Cartesian Product
ρ Rename
```

Derived:

TEXT

```
) Intersection
⋈ Join
÷ Division
```

---

## SELECTION

---

$\sigma_C(R)$

Select rows satisfying C.

TEXT

Degree unchanged  
Cardinality "d o r i g i n a l"

---

## PROJECTION

---

$\pi_A(R)$

Select columns.

TEXT

Degree reduces  
Duplicates removed

---

## SET OPERATIONS

---

Require union compatibility.

TEXT

R  $\cap$  S = e i t h e r  
R  $\cup$  S = b o t h  
R  $\setminus$  S = R b u t n o t S

---

## CARTESIAN PRODUCT

---

$\times$

|              |   |   |  |   |
|--------------|---|---|--|---|
| $R \times S$ | = | R |  | S |
|--------------|---|---|--|---|

$\times$

$\deg(R \times S) = \deg(R) + \deg(S)$

---

## JOIN

---

TEXT

Join = Product + Selection

Theta join:



TEXT

any comparison

Equi join:

TEXT

equality comparison

Natural join:

TEXT

same-name attributes matched  
duplicate join columns removed

## DIVISION

TEXT

Division = ALL / EVERY

For:

TEXT

$R(X,Y) \div S(Y)$

result:

TEXT

X values paired with every Y in S

## MEMORY PAGE

Remember this one line:

TEXT

Ã R o w s  
À C o l u m n s  
"\* E i t h e r  
") B o t h  
" B u t n o t  
× A l l c o m b i n a t i o n s  
"È M a t c h i n g c o m b i n a t i o n s  
÷ A L L r e q u i r e d  
Á R e n a m e

If you remember this, most basic RA expressions become readable.

## 30-SECOND REVISION

#### TEXT

RA = Procedural query language

Ã = r o w s  
Ã = c o l u m n s

Projection removes duplicates

Union/intersection/difference  
need union compatibility

$|R \times S| = |R| |S|$

degree( $R \times S$ )  
= degree(R)+degree(S)

Join = product + selection

Natural join:  
matches all common attributes  
and removes duplicate common columns

Division = ALL / EVERY

Evaluate nested expressions  
i n s i d e ! ' o u t s i d e

## CHECK YOUR UNDERSTANDING

### Q1

Given:

#### TEXT

R(A, B, C)

with 50 tuples.

What is the maximum possible cardinality of:

$\pi_A(R)$

A. 1 B. 3 C. 50 D. 150

### Q2

Suppose:

#### TEXT

$|R| = 20$   
degree(R) = 4

$|S| = 30$   
degree(S) = 3

Find for:

$R \times S$

1. Cardinality
2. Degree

### Q3

Relations:

TEXT

$R(A, B, C, D)$   
 $S(B, C, E)$

What is the degree of:

$\bowtie R \bowtie S$

assuming natural join?

### Q4

Which operation immediately comes to mind for:

**Find students who completed all compulsory courses.**

A. Selection B. Union C. Division D. Cartesian product

### Q5

Consider:

$\pi_{\text{Name}}(\sigma_{\text{Age} > 20}(\text{STUDENT}))$

Explain the query in one English sentence.

### Q6 — Important

Which statement is true?

A. Projection retains duplicate tuples. B. Selection changes relation degree. C. Cartesian product multiplies cardinalities. D. Set difference is commutative.

Next:

## TOPIC 5 — TUPLE RELATIONAL CALCULUS

We will learn it **without unnecessary mathematical complication**:

**procedural vs declarative ! ' tuple variables ! ' predicates ! ' " ! ' " ! ' free/bound variables ! ' safe expressions ! ' RA vs TRC ! ' "for all" questions ! ' GATE traps.**

Type ``NEXT``.

USER

NEXT

## TOPIC 5 — TUPLE RELATIONAL CALCULUS (TRC)

Tuple Relational Calculus is explicitly part of your GATE DBMS syllabus.

This topic is smaller than SQL or normalization, but GATE can test whether you understand:

- procedural vs non-procedural query languages
- tuple variables
- predicates
- existential quantifier " $\exists$ "
- universal quantifier " $\forall$ "
- free vs bound variables
- safe vs unsafe expressions
- equivalence between Relational Algebra and Tuple Relational Calculus

The good news: once you understand the logic, TRC is much easier than it initially looks.

### 1. WHY DOES TUPLE RELATIONAL CALCULUS EXIST?

In Relational Algebra, we explicitly tell the system **how to obtain the answer**.

Example:

**Find names of CSE students.**

Relational Algebra:

$\pi_{\text{Name}}(\sigma_{\text{Dept}='CSE'}(\text{STUDENT}))$

We explicitly say:

1. Select CSE rows.
2. Project Name.

TRC takes a different approach.

We say:

**Give me all tuples whose department is CSE.**

We describe the **condition the desired result must satisfy**, not the step-by-step procedure.

That is the central idea.

### 2. RELATIONAL ALGEBRA vs RELATIONAL CALCULUS

#### Relational Algebra

##### Procedural

We specify:

**HOW to obtain the result.**

Example:

TEXT

Select CSE rows  
! "  
Project Name

## Relational Calculus

**Non-procedural / Declarative**

We specify:

**WHAT properties the result should satisfy.**

## MEMORY TRICK

TEXT

A L G E B R A   ! '   A c t i o n s   /   o p e r a t i o n s  
C A L C U L U S   ! '   C o n d i t i o n s

Or:

**RA = HOW**

**TRC = WHAT**

This distinction is extremely important.

## 3. TYPES OF RELATIONAL CALCULUS

Two major forms exist:

TEXT

T u p l e   R e l a t i o n a l   C a l c u l u s   ! '   T R C  
D o m a i n   R e l a t i o n a l   C a l c u l u s   ! '   D R C

Your syllabus explicitly mentions **tuple calculus**, so TRC is our main focus.

## 4. WHAT DOES “TUPLE” MEAN HERE?

Remember:

TEXT

Tuple = Row

Consider:

## STUDENT

| Sid | Name  | Dept |
|-----|-------|------|
| 1   | Ravi  | CSE  |
| 2   | Anu   | AIML |
| 3   | Kiran | CSE  |

A tuple variable represents a row of a relation.

For example:

TEXT

t

may represent one tuple from STUDENT.

If:

TEXT

t = (1, Ravi, CSE)

then:

TEXT

t.Sid = 1  
t.Name = Ravi  
t.Dept = CSE

## 5. GENERAL FORM OF TRC

A common TRC expression looks like:

$\{ t \mid P(t) \}$

Read it as:

**The set of all tuples t such that predicate P(t) is true.**

Where:

TEXT

t = tuple variable  
P(t) = condition/predicate

## MEMORY TRICK

Think:

TEXT

{ WHAT I WANT | CONDITION }

The vertical bar `|` means:

**such that**

So:

$\{ t \mid P(t) \}$

means:

**all tuples  $t$  such that condition  $P(t)$  holds.**

## 6. SIMPLE EXAMPLE

Suppose:

TEXT

STUDENT(Sid, Name, Dept)

Question:

**Find all CSE students.**

TRC:

$\{ t \mid \text{STUDENT}(t) \wedge t.\text{Dept} = \text{'CSE'} \}$

Read:

**Find every tuple  $t$  such that  $t$  belongs to STUDENT and its department is CSE.**

Result:

TEXT

(1, Ravi, CSE)  
(3, Kiran, CSE)

## 7. WHAT IS A PREDICATE?

A **predicate** is simply a condition that can evaluate to true or false.

Examples:

TEXT

`t.Age > 20`

`t.Dept = 'CSE'`

`t.Salary >= 50000`

These can be combined using logical operators.

## 8. LOGICAL CONNECTIVES

TRC frequently uses:

TEXT

`" "      A N D`

`"(      O R`

`¬      NOT`

`! '      I M P L I E S`

Example:

`\[ t.Dept='CSE' \land t.Age>20 \]`

means:

**Department is CSE AND age is above 20.**

## 9. EXISTENTIAL QUANTIFIER — "

Symbol:

`\[ \exists \]`

Means:

**There exists at least one**

This is one of the most important TRC symbols.

## Example

Relations:

TEXT

`STUDENT(Sid, Name)`

`ENROLLMENT(Sid, CourseID)`

Question:



Find students who are enrolled in at least one course.

Conceptually:

$$\neg [\exists s \text{ mid STUDENT}(s) \wedge \text{exists } e( \text{ ENROLLMENT}(e) \wedge e.\text{Sid}=s.\text{Sid} ) ]$$

Read:

Find student tuples  $s$  for which there exists some enrollment tuple  $e$  having the same Sid.

## MEMORY TRICK

 $\exists$  [ \exists exists ]

= At least one exists

Think:

" = SOME

## 10. EXISTENTIAL EXAMPLE

## ENROLLMENT

| Sid | Course |
|-----|--------|
| 1   | DBMS   |
| 1   | OS     |
| 3   | CN     |

Students:

```
TEXT
1 Ravi
2 Anu
3 Kiran
```

```
TEXT
1 Ravi
2 Anu
3 Kiran
```

Question:

Which students have at least one enrollment?

```
TEXT
exists DBMS enrollment '
```

```
TEXT
exists DBMS enrollment '
```

TEXT

n o e n r o l l m e n t '

Kiran:

TEXT

e x i s t s C N e n r o l l m e n t '

Answer:

TEXT

Ravi  
Kiran

The word:

**some / at least one / exists**

strongly suggests " .

## 11. UNIVERSAL QUANTIFIER — "

Symbol:

$\forall$  [forall]

Means:

**For every / for all**

Example:

**Find students who completed all required courses.**

The word:

TEXT

ALL

suggests:

$\forall$  [forall]

This is similar to how the word ALL suggested **division** in relational algebra.

## MEMORY TRICK

TEXT

" !' S O M E  
" !' A L L

This is one of the most useful TRC memories.

## 12. UNIVERSAL QUANTIFIER EXAMPLE

Suppose:

TEXT

ENROLLMENT(Student, Course)  
REQUIRED(Course)

Question:

**Find students enrolled in every required course.**

Conceptually:

$\forall \{s \mid \text{STUDENT}(s) \wedge \forall c (\text{REQUIRED}(c) \rightarrow \exists e (\text{ENROLLMENT}(e) \wedge e.\text{Student}=s.\text{Sid} \wedge e.\text{Course}=c.\text{Course}))\}$

This looks difficult.

But read it slowly:

**For every required course  $c$ , there must exist an enrollment  $e$  connecting student  $s$  to course  $c$ .**

That's all it means.

## 13. WHY IMPLICATION IS USED WITH "

This pattern is important:

$\forall \{x \mid (P(x) \rightarrow Q(x))\}$

means:

**For every  $x$ , if it belongs to the required set, then condition  $Q$  must hold.**

Example:

$\forall c (\text{REQUIRED}(c) \rightarrow \text{Completed}(s,c))$

means:

**For every course, if it is required, the student completed it.**

## 14. VERY IMPORTANT LOGIC EQUIVALENCE

A universal condition can often be rewritten using NOT EXISTS.

$\forall \{x \mid (P(x) \rightarrow Q(x))\}$

is logically equivalent to:

$\forall \neg \exists x(P(x) \wedge \neg Q(x))$

In simple English:

**“Everyone satisfies Q”**

is equivalent to:

**“There does not exist anyone who fails Q.”**

This is extremely useful in:

- TRC
- SQL NOT EXISTS
- division-type questions

## MEMORY TRICK

For ALL:

**No exception exists.**

So:

TEXT

ALL required courses completed

=

There does NOT exist  
a required course  
that was NOT completed

This pattern will return again in SQL.

## 15. FREE VARIABLE

A variable is **free** if it is not controlled by a quantifier.

Example:

$\forall t \mid \text{STUDENT}(t) \wedge t.\text{Dept} = \text{'CSE'}$

Here:

TEXT

t

is free because it is the output variable and is not inside " " or " " . t

The values of free variables determine the result.

## 16. BOUND VARIABLE

A variable is **bound** when it is controlled by a quantifier.

Example:

$\lceil \exists e (\text{ENROLLMENT}(e) \wedge e.\text{Sid} = s.\text{Sid}) \rceil$

Here:

TEXT

e

is bound by:

$\lceil \exists e \rceil$

## MEMORY TRICK

TEXT

Free variable

! ' a p p e a r s i n f i n a l a n s w e r

Bound variable

! ' h e l p e r v a r i a b l e c o n t r o l l e d b y " o r "

This isn't an absolute linguistic definition, but it is a very useful exam intuition.

## 17. EXAMPLE — FREE AND BOUND VARIABLES

Expression:

$\lceil \{s \mid \text{STUDENT}(s) \wedge \exists e (\text{ENROLLMENT}(e) \wedge e.\text{Sid} = s.\text{Sid})\} \rceil$

Variables:

TEXT

s ! ' f r e e

e ! ' b o u n d

Why?

**s** determines which students appear in the result.

**e** is only used internally to check whether an enrollment exists.

## 18. ATOMIC FORMULAS / BASIC CONDITIONS

TRC predicates can include conditions such as:

### Membership

TEXT

STUDENT(t)

means tuple **t** belongs to STUDENT.

### Attribute comparison

TEXT

t.Age > 20

### Comparison between variables

TEXT

s.Sid = e.Sid

### Constant comparison

TEXT

s.Dept = 'CSE'

These are combined using:

TEXT

" " ( " " "

## 19. SIMPLE TRC QUERY 1

Question:

**Find all students older than 20.**

$\{s \mid \text{STUDENT}(s) \wedge s.\text{Age} > 20\}$

## 20. SIMPLE TRC QUERY 2

Question:

**Find CSE students older than 20.**

$\{s \mid \text{STUDENT}(s) \wedge s.\text{Dept} = \text{'CSE'} \wedge s.\text{Age} > 20\}$

## 21. RETURNING ONLY SOME ATTRIBUTES

Sometimes we want:

**Only student names**

instead of the complete STUDENT tuple.

Different texts use slightly different notation.

A common conceptual form is:

$\{t \mid \exists s ( \text{STUDENT}(s) \wedge s.\text{Dept} = \text{'CSE'} \wedge t.\text{Name} = s.\text{Name} ) \}$

The important GATE idea is not notation memorization but:

**The output tuple contains the desired attributes while quantified variables verify the conditions.**

## 22. TRC QUERY WITH TWO RELATIONS

Relations:

TEXT

STUDENT(Sid, Name, DeptID)

DEPARTMENT(DeptID, DeptName)

Question:

**Find students belonging to CSE department.**

Conceptually:

$\{s \mid \text{STUDENT}(s) \wedge \exists d ( \text{DEPARTMENT}(d) \wedge s.\text{DeptID} = d.\text{DeptID} \wedge d.\text{DeptName} = \text{'CSE'} ) \}$

Read:

**Find student  $s$  for which there exists a department  $d$  having matching DeptID and DeptName CSE.**

## 23. TRC QUERY WITH “NOT”

Question:

**Find students who are not in CSE.**

$\{s \mid \text{STUDENT}(s) \wedge \neg (s.\text{Dept} = \text{'CSE'}) \}$

or equivalently:

$\{s \mid \text{STUDENT}(s) \wedge s.\text{Dept} \neq \text{'CSE'} \}$

## 24. TRC QUERY WITH OR

Question:

**Find CSE or AIML students.**

$\lceil \{s \mid \text{STUDENT}(s) \wedge (s.\text{Dept} = \text{'CSE'} \vee s.\text{Dept} = \text{'AIML'}) \} \rceil$

## 25. TRC QUERY — “AT LEAST ONE”

Suppose:

TEXT  
WORKS(Eid, ProjectID)

Question:

**Find employees working on at least one project.**

Immediately think:

TEXT  
AT LEAST ONE  
! ' "

Expression conceptually:

$\lceil \{e \mid \text{EMPLOYEE}(e) \wedge \exists w ( \text{WORKS}(w) \wedge w.\text{Eid} = e.\text{Eid} ) \} \rceil$

## 26. TRC QUERY — “NO”

Question:

**Find employees who work on no projects.**

Translate:

**There does NOT exist a WORKS tuple for that employee.**

$\lceil \{e \mid \text{EMPLOYEE}(e) \wedge \neg \exists w ( \text{WORKS}(w) \wedge w.\text{Eid} = e.\text{Eid} ) \} \rceil$

## MEMORY TRICK

TEXT  
A t l e a s t o n e ! ' "  
N o n e ! ' ¬"

## 27. TRC QUERY — “ALL”

Question:



**Find employees who work on all projects.**

Conceptual form:

$$\forall \{e \mid \text{EMPLOYEE}(e) \wedge \text{forall } p( \text{PROJECT}(p) \rightarrow \text{exists } w( \text{WORKS}(w) \wedge w.\text{Eid}=e.\text{Eid} \wedge w.\text{ProjectID}=p.\text{ProjectID} ) ) \}$$

Again:

**For every project, there exists a matching WORKS tuple.**

## 28. THE DOUBLE NOT-EXISTS METHOD

The previous ALL query can also be thought of as:

**Find employees for whom there does not exist a project that they do not work on.**

Conceptually:

$$\forall \{ \neg \text{exists } p( \text{PROJECT}(p) \wedge \neg \text{exists } w(\dots) ) \}$$

This “double NOT EXISTS” pattern is very important because SQL often expresses “all” this way.

## 29. SAFE vs UNSAFE TRC EXPRESSIONS

This is one of the most important theoretical GATE concepts.

A relational-calculus query should return a **finite result based on values from the database/domain relevant to the query**.

Such expressions are called **safe** or **range-restricted**.

## 30. WHY UNSAFE QUERIES ARE A PROBLEM

Suppose we write:

$$\forall \{t \mid \neg \text{STUDENT}(t)\}$$

This means:

**Give every tuple in the entire universe that is NOT a STUDENT tuple.**

But how large is the universe?

Potentially infinite.

For example:

TEXT

random names  
random numbers  
random values

...

So the result may be infinite or depend on values outside the database.

That is unsafe.

## SAFE QUERY

Example:

$\{ t \mid \text{STUDENT}(t) \wedge t.\text{Dept} = \text{'CSE'} \}$

Every result tuple comes from STUDENT.

Finite database  $\Rightarrow$  finite result.

Therefore safe.

## 31. SAFE EXPRESSION INTUITION

A safe query should only produce values that come from:

- relations in the database
- constants appearing in the query
- values derived from those bounded inputs

It should not generate arbitrary values from an infinite universe.

## MEMORY TRICK

**SAFE = Search only inside the database world.**

**UNSAFE = May wander into the infinite outside universe.**

## 32. EXAMPLE — SAFE OR UNSAFE?

### Expression A

$\{ t \mid \text{STUDENT}(t) \}$

Safe.

Why?

Result comes directly from STUDENT.

### Expression B

$\{ t \mid \neg \text{STUDENT}(t) \}$

Unsafe.

Why?

It asks for everything not in STUDENT.

Potentially infinite.

### 33. RELATIONAL COMPLETENESS

A major theoretical fact:

**Safe relational calculus and relational algebra have equivalent expressive power for the classical relational query language.**

This is related to **Codd's theorem**.

In simplified GATE terms:

TEXT

Relational Algebra  
"H  
Safe Relational Calculus

in expressive capability.

They express queries differently:

TEXT

R A ! ' p r o c e d u r a l  
T R C ! ' d e c l a r a t i v e

but safe expressions can represent equivalent relational queries.

### & pGATE TRAP

Do not say:

**TRC is more powerful because it looks more mathematical.**

For the classical safe relational query setting:

**RA and safe relational calculus are equivalent in expressive power.**

### 34. RELATIONAL ALGEBRA vs TRC

| Relational Algebra | Tuple Relational Calculus  |
|--------------------|----------------------------|
| Procedural         | Non-procedural/declarative |
| Specifies HOW      | Specifies WHAT             |

| Relational Algebra                      | Tuple Relational Calculus                                      |
|-----------------------------------------|----------------------------------------------------------------|
| Uses operators                          | Uses predicates/logic                                          |
| $\sigma$ , $\pi$ , $\bowtie$ etc.       | $\wedge$ , $\vee$ , $\neg$ , $\rightarrow$ , $\leftrightarrow$ |
| Operates on relations                   | Uses tuple variables                                           |
| Equivalent expressive power to safe TRC | Equivalent to RA when safe                                     |

## 35. TRC vs SQL

SQL is largely **declarative**.

Example:

SQL

```
SELECT Name
FROM STUDENT
WHERE Dept = 'CSE';
```

You say:

**I want names where Dept is CSE.**

You don't specify the exact physical execution steps.

So conceptually SQL is closer to:

TEXT

Relational Calculus

than to the procedural nature of relational algebra.

However, SQL is a practical language with many features beyond pure TRC.

## 36. QUANTIFIER NEGATION RULES

Very useful logic rules:

$$\neg [\neg \forall x P(x)] \equiv \exists x \neg P(x)$$

Meaning:

**Not everyone has property P**

means:

**Someone does not have P.**

And:

$$\neg [\neg \exists x P(x)] \equiv \forall x P(x)$$

Meaning:

**Nobody has property P**

means:

**Everyone fails P.**

These are called quantifier-negation rules.

## MEMORY TRICK

When NOT crosses a quantifier:

TEXT

" c h a n g e s t o "  
" c h a n g e s t o "

and the predicate is negated.

So:

TEXT

N O T A L L ! ' S O M E N O T  
N O T S O M E ! ' A L L N O T

Very useful.

## 37. IMPLICATION ELIMINATION

Another logical identity:

$\lceil P \rightarrow Q \rceil$

is equivalent to:

$\lceil \neg P \vee Q \rceil$

This can help simplify TRC expressions.

## 38. UNIVERSAL QUANTIFIER USING EXISTS

Suppose:

$\lceil \forall x (P(x) \rightarrow Q(x)) \rceil$

Equivalent:

$\lceil \neg \exists x (P(x) \wedge \neg Q(x)) \rceil$

This is worth remembering more than the implication rule itself because it directly helps with “ALL” queries.

## 39. GATE-STYLE EXAMPLE 1

Relation:

TEXT

STUDENT(Sid, Name, Dept)

Which TRC idea represents:

**Find all CSE students?**

Correct conceptual expression:

$\{s \mid \text{STUDENT}(s) \wedge s.\text{Dept} = \text{'CSE'}\}$

Why?

s must:

1. belong to STUDENT
2. satisfy Dept = CSE

## 40. GATE-STYLE EXAMPLE 2

Question:

**Find students enrolled in at least one course.**

Keyword:

TEXT

at least one

Think:

$\{ \exists \}$

So we need:

**There exists an enrollment tuple matching that student.**

## 41. GATE-STYLE EXAMPLE 3

Question:

**Find students enrolled in every required course.**

Keyword:

TEXT

every

Think:

$\forall \text{forall}$

or:

TEXT

NOT EXISTS a required course  
that the student did NOT take

This is an extremely important recognition pattern.

## 42. GATE-STYLE EXAMPLE 4 — FREE/BOUND

Expression:

$\forall s \mid \text{STUDENT}(s) \wedge \exists e( \text{ENROLLMENT}(e) \wedge e.\text{Sid} = s.\text{Sid} )$

Which variables are free/bound?

TEXT

s !' f r e e

e !' b o u n d

## 43. GATE-STYLE EXAMPLE 5 — SAFE/UNSAFE

Consider:

$\forall t \mid \text{STUDENT}(t)$

Safe.

Now:

$\forall t \mid \neg \text{STUDENT}(t)$

Unsafe.

Why?

The second could return infinitely many tuples not in STUDENT.

## 44. HOW TO SOLVE TRC QUESTIONS IN GATE

When you see a TRC expression, use this procedure.

### STEP 1 — Find the output variable

Look before the vertical bar:

$\forall t \mid \dots$

**t** usually indicates what the answer contains.

## STEP 2 — Find relation membership

Look for:

TEXT  
STUDENT(t)  
COURSE(c)  
EMPLOYEE(e)

This tells you which relation each tuple variable belongs to.

## STEP 3 — Mark quantifiers

TEXT  
" !' a t l e a s t o n e  
" !' e v e r y / a l l

## STEP 4 — Read conditions like English

Example:

TEXT  
e.Sid = s.Sid

means:

**Enrollment belongs to the same student.**

## STEP 5 — Watch NOT carefully

Especially:

TEXT  
¬"

means:

**there does not exist**

and:

TEXT  
¬" ( . . . ¬" . . . )



often represents an ALL query.

## 45. ENGLISH !' TRC KEYWORDS

| English phrase | TRC idea                 |
|----------------|--------------------------|
| some           | "                        |
| at least one   | "                        |
| exists         | "                        |
| all            | "                        |
| every          | "                        |
| none           | $\neg$ "                 |
| not all        | " with negation          |
| no exception   | $\neg$ " of failing case |
| and            | "                        |
| or             | "(                       |
| not            | $\neg$                   |

## 46. RELATIONAL ALGEBRA !' TRC INTUITION

RA:

$\sigma_{\text{Dept}='CSE'}(\text{STUDENT})$

TRC idea:

$\{s \mid \text{STUDENT}(s) \wedge s.\text{Dept}='CSE'\}$

RA join:

$\text{STUDENT} \bowtie \text{ENROLLMENT}$

TRC idea:

TEXT

Student tuple exists  
AND  
Enrollment tuple exists  
AND  
their common IDs match

RA division:

TEXT

ENROLLMENT  $\div$  REQUIRED

TRC idea:

TEXT

For every required course,  
there exists a corresponding enrollment.

This connection is highly useful.

## 47. GATE TRAPS

### & pTrap 1 — RA vs TRC

RA:

TEXT

Procedural

TRC:

TEXT

Declarative / non-procedural

### & pTrap 2 — "

$\exists$

does NOT mean exactly one.

It means:

**at least one.**

There may be many.

### & pTrap 3 — "

$\forall$

means every value satisfying the range/antecedent condition.

### & pTrap 4 — Free variable

Free variable contributes to/query result determination.

Bound variables are quantified.

---

## & pTrap 5 — Unsafe expression

---

Not every syntactically valid logical expression is a useful safe database query.

Example:

$\exists t \mid \neg R(t)$

can be unsafe.

---

## & pTrap 6 — ALL questions

---

Do not always expect explicit "all".

"All" can also be represented using:

TEXT

NOT EXISTS an exception

---

## & pTrap 7 — " " " " " "

---

TEXT

" = one or more

" = every

---

## 48. NOTEBOOK NOTES

---

### TUPLE RELATIONAL CALCULUS

---

TRC = declarative/non-procedural query language

General form:

$\exists t \mid P(t)$

Meaning:

Set of tuples  $t$  satisfying predicate  $P$ .

---

### QUANTIFIERS

---

TEXT

" = exists / some / at least one

" = every / all

---

## LOGICAL OPERATORS

### TEXT

"' = A N D  
"( = O R  
¬ = NOT  
!' = i m p l i c a t i o n

## VARIABLES

### TEXT

Free variable  
!' n o t q u a n t i f i e d ; d e t e r m i n e s r e s u l t  
  
Bound variable  
!' c o n t r o l l e d b y " o r "

## SAFE TRC

Safe query:

### TEXT

returns finite values based on database/query domain

Unsafe example:

$\forall \{t \mid \neg \text{STUDENT}(t)\}$

## IMPORTANT EQUIVALENCE

$\forall \{ \text{forall } x(P(x) \rightarrow Q(x)) \}$

equivalent to:

$\forall \{ \neg \text{exists } x(P(x) \wedge \neg Q(x)) \}$

Meaning:

### TEXT

ALL satisfy Q  
=  
No violating example exists

## RA vs TRC

### TEXT

RA = HOW  
  
TRC = WHAT

Safe TRC and relational algebra have equivalent classical expressive power.

## MEMORY PAGE

Remember:

TEXT

TRC = Conditions, not operations

$\{t \mid P(t)\}$

= tuples  $t$  such that  $P$  is true

" = S O M E

" = A L L

$\neg$ " = N O N E

ALL

= NO EXCEPTION EXISTS

Free = output-related

Bound = quantified/helper

Safe = finite/database-bounded

## 30-SECOND REVISION

TEXT

TRC = declarative

General form:

$\{t \mid P(t)\}$

" = a t l e a s t o n e

" = a l l

Free variable = not quantified

Bound variable = quantified

ALL can be written:

$\neg$ " f a i l i n g c a s e

Safe query:

finite/database-bounded result

RA = procedural/HOW

TRC = declarative/WHAT

S a f e T R C " H R A i n e x p r e s s i v e p o w e r

## CHECK YOUR UNDERSTANDING

### Q1

Which is procedural?

A. Tuple Relational Calculus B. Relational Algebra C. Both D. Neither

## Q2

What does:

$\forall [ \exists x ]$

mean?

A. Exactly one x B. At least one x C. Every x D. No x

## Q3

In:

$\forall [ s \mid \text{STUDENT}(s) \wedge \exists e(\text{ENROLLMENT}(e) \wedge e.\text{Sid} = s.\text{Sid}) ]$

identify:

1. Free variable
2. Bound variable

## Q4

Which quantifier is naturally associated with:

**“every required course”?**

A. " B. "

## Q5

Which expression is more likely unsafe?

A.

$\forall [ t \mid \text{STUDENT}(t) \wedge t.\text{Age} > 20 ]$

B.

$\forall [ t \mid \neg \text{STUDENT}(t) ]$

## Q6 — Important logic

Complete the statement:

**“Student has completed all required courses”**

can be rewritten as:

**“There does not exist a \_\_\_\_\_ that the student has \_\_\_\_\_.”**

Next comes one of the **most important and highest-scoring DBMS chapters for GATE:**

## TOPIC 6 — SQL

We will go carefully through:

**SELECT !' WHERE !' DISTINCT !' ORDER BY !' aggregate functions !' GROUP BY !' HAVING !' joins !' nested queries !' correlated subqueries !' IN !' EXISTS !' ANY !' ALL !' NULL !' three-valued logic !' set operations !' views !' constraints !' output-prediction traps.**

This will require multiple sections because SQL deserves much deeper coverage than one short lesson.

Type ``NEXT``.

USER

next

ASSISTANT

## TOPIC 6 — SQL

SQL is one of the **highest-value topics** in GATE DBMS because questions are often direct, logical, and scoring if you understand execution order, grouping, NULL behavior, joins, and subqueries properly. Your syllabus explicitly includes SQL under the relational model.

We will cover SQL in multiple parts so it stays understandable.

## SQL — PART 1

Today:

**SELECT !' FROM !' WHERE !' DISTINCT !' ORDER BY !' expressions !' aliases !' aggregate functions !' GROUP BY !' HAVING !' execution order !' GATE traps**

Later SQL parts will cover:

**joins !' nested queries !' correlated subqueries !' IN !' EXISTS !' ANY !' ALL !' NULL !' three-valued logic !' set operations !' views !' constraints.**

## 1. WHY SQL EXISTS

Relational Algebra lets us express queries mathematically.

Example:

$\pi_{\text{Name}}(\sigma_{\text{Dept}=\text{'CSE'}}(\text{STUDENT}))$

But in practical relational database systems, we use a query language such as **SQL**.

Equivalent SQL:

SQL

```
SELECT Name
FROM STUDENT
WHERE Dept = 'CSE';
```

SQL stands for:

**Structured Query Language**

## 2. SQL IS DECLARATIVE

Remember:

TEXT

Relational Algebra  
!' P r o c e d u r a l  
!' H O W

SQL

!' D e c l a r a t i v e  
!' W H A T

When you write:

SQL

```
SELECT Name
FROM STUDENT
WHERE Dept = 'CSE';
```

you tell the DBMS what result you need.

The optimizer decides how to obtain it physically.

## 3. OUR SAMPLE TABLE

We will use this table repeatedly.

### STUDENT

| Sid | Name  | Dept | Marks | Age |
|-----|-------|------|-------|-----|
| 1   | Ravi  | CSE  | 85    | 20  |
| 2   | Anu   | AIML | 92    | 21  |
| 3   | Kiran | CSE  | 75    | 20  |
| 4   | Riya  | ECE  | 88    | 22  |
| 5   | Raj   | CSE  | 92    | 21  |
| 6   | Neha  | AIML | 75    | 20  |

## 4. BASIC SELECT QUERY

General syntax:



SQL

```
SELECT column_list
FROM table_name
WHERE condition;
```

Example:

SQL

```
SELECT Name
FROM STUDENT;
```

Result:

TEXT

```
Ravi
Anu
Kiran
Riya
Raj
Neha
```

## 5. SELECT \*

\* means all columns.

SQL

```
SELECT *
FROM STUDENT;
```

Returns the complete table.

## MEMORY

TEXT

\* = all attributes

## 6. SELECT MULTIPLE COLUMNS

SQL

```
SELECT Name, Marks
FROM STUDENT;
```

Result:

| Name  | Marks |
|-------|-------|
| Ravi  | 85    |
| Anu   | 92    |
| Kiran | 75    |

| Name | Marks |
|------|-------|
| Riya | 88    |
| Raj  | 92    |
| Neha | 75    |

## 7. SELECT vs RELATIONAL ALGEBRA PROJECTION

SQL:

```
SQL
SELECT Name
FROM STUDENT;
```

looks similar to:

$\pi_{\text{Name}}(\text{STUDENT})$

But there is a major difference.

### Relational Algebra projection

Removes duplicates.

### SQL SELECT

By default, duplicates may remain.

This is a huge GATE trap.

## 8. DISTINCT

Use:

```
SQL
DISTINCT
```

to remove duplicate rows from the result.

Query:

```
SQL
SELECT Dept
FROM STUDENT;
```

Result may be:

```
TEXT
CSE
AIML
CSE
ECE
```

```
CSE
AIML
```

Now:

```
SQL
SELECT DISTINCT Dept
FROM STUDENT;
```

Result:

```
TEXT
CSE
AIML
ECE
```

---

## MEMORY TRICK

```
TEXT
RA projection
!' duplicate removal automatic

SQL SELECT
!' duplicates remain

SQL SELECT DISTINCT
!' duplicates removed
```

---

## 9. DISTINCT APPLIES TO THE WHOLE SELECTED TUPLE

Suppose:

```
SQL
SELECT DISTINCT Dept, Marks
FROM STUDENT;
```

SQL removes duplicate **pairs**:

```
TEXT
(Dept, Marks)
```

It does not independently make each column distinct.

Very important.

---

## 10. WHERE

**WHERE** filters rows.

Equivalent concept to selection in relational algebra.

SQL

```
SELECT *  
FROM STUDENT  
WHERE Dept = 'CSE';
```

Result:

| Sid | Name  | Dept | Marks | Age |
|-----|-------|------|-------|-----|
| 1   | Ravi  | CSE  | 85    | 20  |
| 3   | Kiran | CSE  | 75    | 20  |
| 5   | Raj   | CSE  | 92    | 21  |

## MEMORY CONNECTION

TEXT

```
SQL WHERE  
"H  
R e l a t i o n a l   A l g e b r a   ã
```

and:

TEXT

```
SQL SELECT column list  
"H  
R e l a t i o n a l   A l g e b r a   Æ
```

## 11. COMPARISON OPERATORS

SQL uses:

TEXT

```
=  
<>  
!=  
<  
>  
<=  
>=
```

Example:

SQL

```
SELECT Name  
FROM STUDENT  
WHERE Marks >= 85;
```

Result:

TEXT

Ravi  
Anu  
Riya  
Raj

## 12. LOGICAL OPERATORS

Important:

TEXT

AND  
OR  
NOT

Example:

SQL

```
SELECT Name  
FROM STUDENT  
WHERE Dept = 'CSE'  
AND Marks > 80;
```

Result:

TEXT

Ravi  
Raj

## 13. AND

Both conditions must be true.

SQL

```
WHERE Dept = 'CSE'  
AND Marks > 80
```

means:

TEXT

```
C S E '
AND
M a r k s   >   8 0   '
```

## 14. OR

At least one condition true.

SQL

```
SELECT Name
FROM STUDENT
WHERE Dept = 'CSE'
OR Dept = 'ECE';
```

returns:

TEXT

```
Ravi
Kiran
Riya
Raj
```

---

## 15. NOT

Negates a condition.

SQL

```
SELECT Name
FROM STUDENT
WHERE NOT Dept = 'CSE';
```

returns non-CSE students.

---

## 16. OPERATOR PRECEDENCE

Important in tricky queries.

Typically:

TEXT

```
NOT
before
AND
before
OR
```

So:

SQL

```
A OR B AND C
```

is interpreted like:

SQL

```
A OR (B AND C)
```

not:

SQL

```
( A OR B ) AND C
```

## GATE RULE

If conditions are complex:

**Explicitly insert parentheses while solving.**

Never rely on visual reading.

## 17. BETWEEN

Instead of:

SQL

```
Marks >= 75 AND Marks <= 90
```

you can write:

SQL

```
Marks BETWEEN 75 AND 90
```

**BETWEEN** is generally **inclusive**.

So:

SQL

```
BETWEEN 75 AND 90
```

includes both:

TEXT

```
75  
90
```

## & pGATE TRAP

**BETWEEN a AND b** normally includes endpoints.

## 18. IN

For multiple possible values:

SQL

```
SELECT *  
FROM STUDENT  
WHERE Dept IN ('CSE', 'AIML');
```

Equivalent conceptually to:

SQL

```
WHERE Dept = 'CSE'  
OR Dept = 'AIML'
```

We'll study **IN** more deeply with subqueries later.

## 19. NOT IN

SQL

```
SELECT *  
FROM STUDENT  
WHERE Dept NOT IN ('CSE', 'ECE');
```

Conceptually returns students outside those departments.

But **NOT IN** becomes tricky when NULL appears in the list/subquery.

We will handle that carefully in the NULL section.

## 20. LIKE

Used for pattern matching in strings.

Important wildcard symbols:

TEXT

```
% !' zero or more characters  
_ !' exactly one character
```

## 21. LIKE EXAMPLES

Names beginning with **R**:

SQL

```
SELECT Name  
FROM STUDENT  
WHERE Name LIKE 'R%';
```

May return:

TEXT

```
Ravi  
Riya  
Raj
```

Names ending with **a**:



SQL

```
WHERE Name LIKE '%a'
```

Names containing **vi**:

SQL

```
WHERE Name LIKE '%vi%'
```

Exactly four-character names starting with R:

SQL

```
WHERE Name LIKE 'R____'
```

Why?

TEXT

R

+

3 characters

=

4 total

## MEMORY TRICK

TEXT

% = MANY characters

\_ = ONE character

## 22. EXPRESSIONS IN SELECT

SQL can calculate values.

Suppose:

TEXT

```
EMPLOYEE(Name, Salary)
```

Query:

SQL

```
SELECT Name, Salary * 12  
FROM EMPLOYEE;
```

returns annual salary.

You can also use arithmetic:

TEXT

+  
-  
\*  
/

subject to DBMS numeric semantics.

## 23. ALIAS — AS

Alias gives a temporary name to a result column/table.

Example:

SQL

```
SELECT Name, Marks AS Score  
FROM STUDENT;
```

Output heading:

TEXT

Name | Score

AS does not permanently rename the database column.

It only labels the query result.

## GATE TRAP

Alias:

TEXT

temporary query-level name

not:

TEXT

permanent schema modification

## 24. ORDER BY

Used to sort output.

SQL

```
SELECT Name, Marks  
FROM STUDENT  
ORDER BY Marks;
```

Default is usually:

TEXT

ASC

ascending.

Equivalent:

SQL

```
ORDER BY Marks ASC;
```

## 25. DESC

For descending order:

SQL

```
SELECT Name, Marks  
FROM STUDENT  
ORDER BY Marks DESC;
```

Result starts with highest Marks.

## 26. MULTIPLE SORT KEYS

SQL

```
SELECT Name, Dept, Marks  
FROM STUDENT  
ORDER BY Dept ASC, Marks DESC;
```

Meaning:

First:

TEXT

sort by Dept

Then within equal Dept values:

TEXT

sort by Marks descending

## MEMORY

TEXT

```
ORDER BY A, B
```

means:

A is primary sorting key.

B breaks ties in A.

## 27. AGGREGATE FUNCTIONS

This is very important for GATE.

Major aggregate functions:

TEXT  
COUNT  
SUM  
AVG  
MIN  
MAX

They summarize multiple rows.

## 28. COUNT(\*)

SQL  
SELECT COUNT ( \* )  
FROM STUDENT ;

Returns:

TEXT  
6

because six rows exist.

Important:

**COUNT ( \* ) counts rows.**

Even rows containing NULL values in some attributes are still counted.

## 29. COUNT(column)

Suppose:

| Sid | Phone |
|-----|-------|
| 1   | 999   |
| 2   | NULL  |

| Sid | Phone |
|-----|-------|
| 3   | 888   |
| 4   | NULL  |

Query:

```
SQL
SELECT COUNT(Phone)
FROM STUDENT;
```

returns:

```
TEXT
2
```

because aggregate `COUNT(column)` ignores NULL values.

## VERY IMPORTANT GATE RULE

```
TEXT
COUNT(*)
!' counts rows

COUNT(attribute)
!' counts non - NULL values

COUNT(DISTINCT attribute)
!' counts distinct non - NULL values
```

Write this in your notes.

## 30. COUNT(DISTINCT ...)

Using our sample STUDENT:

Departments:

```
TEXT
CSE
AIML
CSE
ECE
CSE
AIML
```

Query:

```
SQL
SELECT COUNT(DISTINCT Dept)
FROM STUDENT;
```

returns:

TEXT  
3

## 31. SUM

SQL  
SELECT SUM(Marks)  
FROM STUDENT;

Using:

TEXT  
85 + 92 + 75 + 88 + 92 + 75

= 507.

## 32. AVG

SQL  
SELECT AVG(Marks)  
FROM STUDENT;

\[ 507/6=84.5 \]

So:

TEXT  
84.5

## 33. MIN / MAX

SQL  
SELECT MIN(Marks), MAX(Marks)  
FROM STUDENT;

Result:

TEXT  
75  
92

## 34. NULL AND AGGREGATES — PREVIEW

Most common SQL aggregates ignore NULL values:

```
TEXT
SUM
AVG
MIN
MAX
COUNT(column)
```

But:

```
TEXT
COUNT(*)
```

counts rows regardless of NULLs in columns.

We'll study this again in the NULL section because GATE frequently exploits it.

---

## 35. IMPORTANT AVG TRAP

Suppose values are:

```
TEXT
10
20
NULL
```

Then:

```
SQL
AVG(X)
```

is usually:

$\sqrt{(10+20)/2 = 15}$

not:

$\sqrt{(10+20+0)/3}$

NULL is **ignored**, not treated as zero.

Very important.

---

## 36. AGGREGATE WITHOUT GROUP BY

```
SQL
SELECT AVG(Marks)
FROM STUDENT;
```

All rows belong conceptually to one overall group.

Result:

TEXT

84.5

## 37. GROUP BY — WHY IT EXISTS

Suppose instead of overall average, we want:

**Average marks department-wise.**

We need to divide rows into groups:

TEXT

CSE group  
AIML group  
ECE group

Then calculate AVG separately.

SQL:

SQL

```
SELECT Dept, AVG(Marks)
FROM STUDENT
GROUP BY Dept;
```

## 38. GROUP BY EXAMPLE

Our table:

TEXT

CSE :  
85  
75  
92

AIML :  
92  
75

ECE :  
88

Averages:

**CSE**

$\lfloor (85+75+92)/3=84 \rfloor$

**AIML**

$\lfloor (92+75)/2=83.5 \rfloor$

**ECE**



[ 88 ]

Result:

| Dept | AVG(Marks) |
|------|------------|
| CSE  | 84         |
| AIML | 83.5       |
| ECE  | 88         |

## MEMORY TRICK

**GROUP BY creates buckets.**

TEXT

Rows

!"

Groups/Buckets

!"

Aggregate each bucket

## 39. HOW MANY OUTPUT ROWS FROM GROUP BY?

Usually:

**one result row per group**

Example:

SQL

GROUP BY Dept

Distinct departments:

TEXT

CSE

AIML

ECE

So there are:

TEXT

3 groups

and normally 3 result rows.

## GATE SHORTCUT

If asked:

**How many groups?**

Think:

**Number of distinct combinations of GROUP BY attributes.**

Example:

SQL

```
GROUP BY Dept, Age
```

Each distinct:

TEXT

```
(Dept, Age)
```

pair forms a group.

## 40. MULTIPLE GROUP BY ATTRIBUTES

SQL

```
SELECT Dept, Age, COUNT(*)  
FROM STUDENT  
GROUP BY Dept, Age;
```

Each unique combination forms one group.

Using our table:

TEXT

```
( C S E , 2 0 )    !'   R a v i ,   K i r a n  
( C S E , 2 1 )    !'   R a j  
( A I M L , 2 1 ) !'   A n u  
( A I M L , 2 0 ) !'   N e h a  
( E C E , 2 2 )    !'   R i y a
```

So:

TEXT

5 groups

## 41. GROUP BY RULE

In standard SQL reasoning, if a SELECT list contains:

- aggregate expressions

- ordinary attributes

then ordinary attributes generally must be included in `GROUP BY`.

Correct:

SQL

```
SELECT Dept, AVG(Marks)
FROM STUDENT
GROUP BY Dept;
```

Potentially invalid in strict standard SQL:

SQL

```
SELECT Name, Dept, AVG(Marks)
FROM STUDENT
GROUP BY Dept;
```

Why?

Within CSE:

TEXT

```
Ravi
Kiran
Raj
```

Which `Name` should SQL output for the CSE group?

There is no single well-defined Name.

## & pGATE TRAP

For grouping queries:

**Every selected non-aggregate attribute should generally be a grouping attribute.**

Some DBMS products have extensions/functional-dependency exceptions, but for GATE use standard relational SQL reasoning unless the question states otherwise.

## 42. WHERE + GROUP BY

Suppose:

**Department-wise average marks, but consider only students aged at least 21.**

Query:

SQL

```
SELECT Dept, AVG(Marks)
FROM STUDENT
WHERE Age >= 21
GROUP BY Dept;
```

Important:

`WHERE` filters rows **before grouping**.

### 43. HAVING — WHY IT EXISTS

Suppose we want:

**Show only departments whose average marks exceed 84.**

We cannot conceptually use:

```
SQL
WHERE AVG(Marks) > 84
```

because `WHERE` operates before groups and aggregate results are formed.

Instead:

```
SQL
SELECT Dept, AVG(Marks)
FROM STUDENT
GROUP BY Dept
HAVING AVG(Marks) > 84;
```

### MEMORY TRICK

```
TEXT
WHERE  !' filters ROWS
HAVING !' filters GROUPS
```

This is one of the most important SQL distinctions.

### 44. WHERE vs HAVING

| WHERE                                       | HAVING                           |
|---------------------------------------------|----------------------------------|
| Filters individual rows                     | Filters groups                   |
| Before GROUP BY                             | After GROUP BY                   |
| Usually doesn't use aggregate group results | Commonly uses aggregates         |
| Reduces input to grouping                   | Reduces groups after aggregation |

### Example

SQL

```
SELECT Dept, AVG(Marks)
FROM STUDENT
WHERE Age >= 20
GROUP BY Dept
HAVING AVG(Marks) >= 85;
```

Read:

### Step 1

Keep rows:

TEXT

Age >= 20

### Step 2

Group remaining rows by Dept.

### Step 3

Compute AVG(Marks).

### Step 4

Keep groups where:

TEXT

AVG >= 85

## 45. CRITICAL SQL LOGICAL EXECUTION ORDER

This is one of the most important things to memorize for GATE.

Although we **write**:

SQL

```
SELECT ...
FROM ...
WHERE ...
GROUP BY ...
HAVING ...
ORDER BY ...
```

logical processing is approximately:

TEXT

```
FROM
! "
WHERE
! "
GROUP BY
! "
HAVING
! "
SELECT
```

```
! "  
DISTINCT  
! "  
ORDER BY
```

For queries with joins/subqueries there are further details, but this order is the key exam model.

## MEMORY TRICK

Remember:

**FWGHSDO**

```
TEXT  
FROM  
WHERE  
GROUP BY  
HAVING  
SELECT  
DISTINCT  
ORDER BY
```

Mnemonic:

**Friendly Wizards Group Heroes, Select Distinct Orders**

The sentence is silly, but that makes it easier to recall.

Or simply memorize:

```
TEXT  
FROM !' WHERE !' GROUP !' HAVING !' SELECT !' DISTINCT !' ORD
```

## 46. WHY EXECUTION ORDER MATTERS

Consider:

```
SQL  
SELECT Dept, AVG(Marks)  
FROM STUDENT  
WHERE Age >= 20  
GROUP BY Dept  
HAVING AVG(Marks) > 84  
ORDER BY Dept;
```

You should solve it as:

```
TEXT  
STUDENT  
! "  
WHERE Age >= 20
```

```

! "
GROUP BY Dept
! "
calculate averages
! "
HAVING AVG > 84
! "
SELECT requested columns
! "
ORDER BY Dept

```

This prevents most SQL output-prediction mistakes.

## 47. SOLVED GATE-STYLE EXAMPLE 1

Given STUDENT:

| Sid | Name  | Dept | Marks |
|-----|-------|------|-------|
| 1   | Ravi  | CSE  | 85    |
| 2   | Anu   | AIML | 92    |
| 3   | Kiran | CSE  | 75    |
| 4   | Riya  | ECE  | 88    |
| 5   | Raj   | CSE  | 92    |
| 6   | Neha  | AIML | 75    |

Evaluate:

```

SQL
SELECT DISTINCT Dept
FROM STUDENT
WHERE Marks >= 85;

```

### Step 1 — WHERE

Marks "e85:

```

TEXT
Ravi  CSE   85
Anu   AIML  92
Riya  ECE   88
Raj   CSE   92

```

### Step 2 — SELECT Dept

```

TEXT
CSE
AIML
ECE
CSE

```

### Step 3 — DISTINCT

TEXT

CSE  
AIML  
ECE

#### Answer

3 rows.

## 48. SOLVED EXAMPLE 2

Evaluate:

SQL

```
SELECT Dept, COUNT(*)  
FROM STUDENT  
GROUP BY Dept;
```

Groups:

TEXT

```
C S E   !'   3  
A I M L !'   2  
E C E   !'   1
```

Result:

| Dept | COUNT(*) |
|------|----------|
| CSE  | 3        |
| AIML | 2        |
| ECE  | 1        |

## 49. SOLVED EXAMPLE 3

Evaluate:

SQL

```
SELECT Dept, MAX(Marks)  
FROM STUDENT  
GROUP BY Dept;
```

CSE:

TEXT

```
8 5 ,   7 5 ,   9 2 !'   9 2
```

AIML:



TEXT

9 2 , 7 5 ! ' 9 2

ECE:

TEXT

8 8 ! ' 8 8

Result:

| Dept | MAX |
|------|-----|
| CSE  | 92  |
| AIML | 92  |
| ECE  | 88  |

## 50. SOLVED EXAMPLE 4 — HAVING

Query:

SQL

```
SELECT Dept, COUNT(*)
FROM STUDENT
GROUP BY Dept
HAVING COUNT(*) >= 2;
```

Groups:

TEXT

C S E ! ' 3 '  
A I M L ! ' 2 '  
E C E ! ' 1 '

Result:

TEXT

CSE 3  
AIML 2

## 51. SOLVED EXAMPLE 5 — WHERE + HAVING

Query:

SQL

```
SELECT Dept, AVG(Marks)
FROM STUDENT
WHERE Marks >= 80
GROUP BY Dept
HAVING COUNT(*) >= 2;
```

### Step 1 — WHERE Marks >= 80

Remaining:

TEXT

```
Ravi CSE 85
Anu  AIME 92
Riya ECE 88
Raj  CSE 92
```

### Step 2 — Groups

TEXT

```
C S E !'   R a v i ,   R a j
A I M L !'   A n u
E C E !'   R i y a
```

### Step 3 — HAVING COUNT(\*) >= 2

Only:

TEXT

CSE

survives.

AVG:

\[ (85+92)/2 = 88.5 \]

### Answer

TEXT

CSE 88.5

## 52. GATE-STYLE COUNT TRICK

Suppose relation contains:

| A    |
|------|
| 10   |
| 20   |
| NULL |
| 20   |
| NULL |

Then:

SQL

```
SELECT COUNT(*)
```

returns:

TEXT

5

because there are 5 rows.

---

SQL

```
SELECT COUNT(A)
```

returns:

TEXT

3

because 3 non-NULL values.

---

SQL

```
SELECT COUNT(DISTINCT A)
```

returns:

TEXT

2

because distinct non-NULL values:

---

TEXT

10

20

---

## WRITE THIS IN A BOX

---

TEXT

COUNT(\*) = all rows

COUNT(A) = non-NULL A

COUNT(DISTINCT A)  
= distinct non-NULL A

This is extremely exam-worthy.

---

## 53. SUM/AVG NULL TRICK

Values:

```
TEXT
10
20
NULL
30
```

Then:

```
SQL
SUM(A)
```

=

```
TEXT
60
```

And:

```
SQL
AVG(A)
```

=

\[ 60/3=20 \]

not:

\[ 60/4=15 \]

because NULL is ignored.

## 54. DISTINCT + NULL PREVIEW

`DISTINCT` typically groups multiple NULL appearances into one distinct result representation for `SELECT DISTINCT`.

But SQL's NULL comparison semantics are not the same as ordinary equality.

This sounds contradictory initially, so we'll handle it properly later instead of oversimplifying now.

## 55. ORDER BY DOES NOT CHANGE WHICH ROWS EXIST

Important.

`ORDER BY` changes only:

```
TEXT
presentation order
```

not:

TEXT

number of rows

unless combined with DBMS-specific limiting features, which aren't our current concern.

## 56. SQL CLAUSE PURPOSE TABLE

| Clause   | Purpose                           |
|----------|-----------------------------------|
| SELECT   | Choose output columns/expressions |
| FROM     | Choose source relation(s)         |
| WHERE    | Filter rows                       |
| DISTINCT | Remove duplicate result rows      |
| GROUP BY | Form groups                       |
| HAVING   | Filter groups                     |
| ORDER BY | Sort final output                 |

This is a must-write table.

## 57. GATE TRAPS

### & pTrap 1 — SQL SELECT duplicates

SQL

```
SELECT Dept
```

does **not** automatically remove duplicates.

Need:

SQL

```
SELECT DISTINCT Dept
```

### & pTrap 2 — WHERE vs HAVING

TEXT

```
WHERE !' rows
```

```
HAVING !' groups
```

### & pTrap 3 — COUNT(\*)

Counts rows even if some fields contain NULL.

---

### & pTrap 4 — COUNT(column)

---

Ignores NULL values.

---

### & pTrap 5 — AVG

---

Ignores NULL rather than treating NULL as 0.

---

### & pTrap 6 — DISTINCT

---

Applied to the complete selected tuple.

SQL

```
SELECT DISTINCT A,B
```

means unique:

TEXT

(A,B) combinations

---

### & pTrap 7 — GROUP BY

---

For exam-standard SQL, non-aggregate SELECT columns should generally appear in GROUP BY.

---

### & pTrap 8 — HAVING

---

HAVING is evaluated after grouping.

---

### & pTrap 9 — BETWEEN

---

Endpoints are included.

---

### & pTrap 10 — LIKE

---

TEXT

% = zero or more characters

\_ = exactly one

---

### & pTrap 11 — logical precedence

---

TEXT

NOT > AND > OR

Use parentheses while solving.

## 58. PYQ-ORIENTED SQL SOLVING METHOD

Whenever GATE gives an SQL query and asks output/count, do this:

### STEP 1 — Ignore SELECT temporarily

Start from:

TEXT

FROM

Identify the source table(s).

### STEP 2 — Apply WHERE

Cross out rows that fail.

### STEP 3 — Apply GROUP BY

Physically create groups on rough paper.

Example:

TEXT

CSE: rows 1,3,5

AIML: rows 2,6

ECE: row 4

### STEP 4 — Calculate aggregates

TEXT

COUNT

AVG

MAX

...

for each group.

### STEP 5 — Apply HAVING

Remove failing groups.

## STEP 6 — Build SELECT output

Only now write requested columns.

## STEP 7 — DISTINCT

Remove duplicate result tuples if requested.

## STEP 8 — ORDER BY

Sort last.

This procedure solves a large percentage of SQL output questions.

## 59. NOTEBOOK NOTES

### SQL

Declarative relational query language.

#### Basic query

```
SQL
SELECT columns
FROM relation
WHERE condition;
```

### DISTINCT

```
SQL
SELECT DISTINCT A
```

removes duplicate output tuples.

SQL SELECT alone may retain duplicates.

### WHERE

Filters rows.

```
TEXT
WHERE "H R A selection" Æ
```

### LIKE



TEXT

```
% !' 0 or more chars
_ !' exactly 1 char
```

## AGGREGATES

TEXT

```
COUNT
SUM
AVG
MIN
MAX
```

## COUNT RULE

TEXT

```
COUNT ( * ) !' all rows

COUNT ( A ) !' non - NULL A values

COUNT ( DISTINCT A )
!' distinct non - NULL A values
```

## GROUP BY

Forms groups based on distinct values/combinations.

One result row per surviving group.

## WHERE vs HAVING

TEXT

```
WHERE !' filter rows

HAVING !' filter groups
```

## LOGICAL EXECUTION ORDER

TEXT

```
FROM
!"
WHERE
!"
GROUP BY
!"
HAVING
!"
```

```
SELECT  
!"  
DISTINCT  
!"  
ORDER BY
```

Mnemonic:

```
TEXT  
F W G H S D O
```

---

## MEMORY PAGE

Remember:

```
TEXT  
S E L E C T   ! '   W H A T   o u t p u t  
F R O M   ! '   W H E R E   f r o m  
W H E R E   ! '   W H I C H   r o w s  
G R O U P   B Y   ! '   m a k e   b u c k e t s  
H A V I N G   ! '   W H I C H   b u c k e t s  
D I S T I N C T   ! '   r e m o v e   d u p l i c a t e   o u t p u t s  
O R D E R   B Y   ! '   a r r a n g e   r e s u l t
```

And:

```
TEXT  
WHERE before groups  
HAVING after groups
```

---

## 30-SECOND REVISION

```
TEXT  
SQL = declarative  
  
SELECT does not automatically  
remove duplicates  
  
DISTINCT removes duplicates  
  
WHERE = row filter  
  
GROUP BY = groups/buckets  
  
HAVING = group filter  
  
COUNT(*) = rows
```

```
COUNT(A) = non-NULL A
COUNT(DISTINCT A)
= unique non-NULL A
```

AVG ignores NULL

LIKE:  
% many  
\_ one

Execution:  
FROM !' WHERE !' GROUP !' HAVING  
!' SELECT !' DISTINCT !' ORDER

## CHECK YOUR UNDERSTANDING

### Q1

Values of column A are:

```
TEXT
10
10
20
NULL
NULL
```

Find:

1. COUNT(\*)
2. COUNT(A)
3. COUNT(DISTINCT A)

### Q2

What does this return?

```
SQL
SELECT DISTINCT Dept
FROM STUDENT;
```

A. Every department occurrence B. One row for each distinct department C. Only CSE D. Number of departments

### Q3

Which clause filters groups?

A. WHERE B. SELECT C. HAVING D. ORDER BY

### Q4

What is the correct logical processing sequence among these?

```
TEXT
SELECT
WHERE
FROM
GROUP BY
HAVING
```

## Q5

Suppose:

```
TEXT
A = 10, 20, NULL
```

What is:

```
SQL
AVG(A)
```

A. 10 B. 15 C. 20 D. 30

## Q6

What does:

```
SQL
WHERE Name LIKE 'A__'
```

mean?

Next is:

## SQL — PART 2: JOINS

We will cover:

**cross join !' inner join !' equi join !' natural join !' self join !' left/right/full outer joins !' ON vs WHERE !' join output cardinality !' key-foreign key joins !' GATE output-prediction traps.**

Type ``NEXT``.

```
USER
NEXT
```

ASSISTANT

## SQL — PART 2: JOINS

Joins are one of the most important SQL areas for GATE because questions often combine **multiple tables, foreign**

**keys, NULL values, and output prediction.** Your DBMS syllabus explicitly includes SQL, and joins are essential to using relational data across multiple relations.

We already learned the relational-algebra idea:

$\bowtie$  [Join \approx Cartesian\ Product + Matching\ Condition]

Now we will learn the practical SQL version.

## 1. WHY DO WE NEED JOINS?

Suppose data is divided into two tables.

### STUDENT

| Sid | Name  | DeptID |
|-----|-------|--------|
| 1   | Ravi  | 10     |
| 2   | Anu   | 20     |
| 3   | Kiran | 10     |
| 4   | Riya  | 30     |
| 5   | Raj   | 40     |

### DEPARTMENT

| DeptID | DeptName |
|--------|----------|
| 10     | CSE      |
| 20     | AIML     |
| 30     | ECE      |
| 50     | CIVIL    |

Notice:

- STUDENT has department IDs.
- DEPARTMENT has department names.

If we want:

TEXT

Ravi CSE  
Anu AIML  
Kiran CSE  
Riya ECE

we must combine information from both relations.

That is what a **join** does.

---

## 2. MAIN SQL JOIN TYPES

---

We need to understand:

TEXT

CROSS JOIN

INNER JOIN

%% % Equi Join  
%% % Theta Join

NATURAL JOIN

LEFT OUTER JOIN

RIGHT OUTER JOIN

FULL OUTER JOIN

SELF JOIN

The concepts overlap, so we'll build them carefully.

---

## 3. CROSS JOIN

---

A `CROSS JOIN` gives the Cartesian product.

SQL

```
SELECT *  
FROM STUDENT  
CROSS JOIN DEPARTMENT;
```

If:

TEXT

STUDENT has 5 rows  
DEPARTMENT has 4 rows

then result has:

$5 \times 4 = 20$

rows.

---

## CROSS JOIN FORMULA

---

If:

$|R|=m$

and:

$|S|=n$

then:

$|R \times S| = mn$

Exactly the same as relational algebra Cartesian product.

## MEMORY TRICK

**CROSS = every row crosses with every row.**

No matching condition is required.

## 4. OLD-STYLE CARTESIAN PRODUCT

This SQL:

```
SQL
SELECT *
FROM STUDENT, DEPARTMENT;
```

without a **WHERE** condition also produces a Cartesian product.

So:

```
SQL
FROM R, S
```

can conceptually behave like:

```
TEXT
R × S
```

unless a condition filters combinations.

## & pGATE TRAP

If two tables contain:

```
TEXT
100 rows
200 rows
```

and query has:

```
SQL
FROM R, S
```

with no filtering condition,

result cardinality:

$|100 \times 200| = 20,000$

Do not add them.

## 5. INNER JOIN

An **inner join** returns only rows satisfying the join condition.

Example:

SQL

```
SELECT S.Name, D.DeptName
FROM STUDENT S
INNER JOIN DEPARTMENT D
ON S.DeptID = D.DeptID;
```

We compare:

TEXT

```
STUDENT.DeptID
with
DEPARTMENT.DeptID
```

## 6. RESULT OF INNER JOIN

Given our tables:

STUDENT:

TEXT

```
Ravi  10
Anu    20
Kiran  10
Riya   30
Raj    40
```

DEPARTMENT:

TEXT

```
10 CSE
20 AIML
30 ECE
50 CIVIL
```

Matches:

TEXT

```
1 0 !" 1 0 '
2 0 !" 2 0 '
1 0 !" 1 0 '
3 0 !" 3 0 '
4 0 !" n o t h i n g  '
```

Result:



| Name  | DeptName |
|-------|----------|
| Ravi  | CSE      |
| Anu   | AIML     |
| Kiran | CSE      |
| Riya  | ECE      |

Raj disappears because DeptID 40 has no matching department.

CIVIL also disappears because no student has DeptID 50.

## MEMORY TRICK

**INNER = Intersection-like matching region**

Only matched combinations survive.

## 7. JOIN USING WHERE

The same equi-join idea can also be written using older SQL syntax:

SQL

```
SELECT S.Name, D.DeptName
FROM STUDENT S, DEPARTMENT D
WHERE S.DeptID = D.DeptID;
```

Conceptually:

TEXT

```
Cartesian Product
      !
WHERE matching condition
      !
Join result
```

This directly connects to relational algebra:

$\sigma_{S.DeptID=D.DeptID}(STUDENT \times DEPARTMENT)$

## 8. ON CLAUSE

In explicit joins:

SQL

```
FROM STUDENT S
JOIN DEPARTMENT D
ON S.DeptID = D.DeptID
```

**ON** specifies the condition used to match rows.

Think:

TEXT

ON = How the tables are related

## 9. TABLE ALIASES

Instead of repeatedly writing:

TEXT

STUDENT.DeptID  
DEPARTMENT.DeptID

we write:

SQL

FROM STUDENT S  
JOIN DEPARTMENT D

Then:

TEXT

S.DeptID  
D.DeptID

Here:

TEXT

S = STUDENT alias  
D = DEPARTMENT alias

Aliases make joins much easier to read.

## 10. WHY QUALIFIED COLUMN NAMES MATTER

Both relations contain:

TEXT

DeptID

So:

SQL

SELECT DeptID

may be ambiguous.

Better:

SQL

```
SELECT S.DeptID
```

or:

SQL

```
SELECT D.DeptID
```

## & pGATE TRAP

When the same attribute name occurs in multiple input relations, an unqualified reference may be ambiguous.

## 11. EQUI JOIN

An **equi join** is a join whose condition uses equality.

Example:

SQL

```
SELECT *  
FROM STUDENT S  
JOIN DEPARTMENT D  
ON S.DeptID = D.DeptID;
```

Condition:

TEXT

=

Therefore this is an equi join.

## 12. THETA JOIN

A theta join uses any comparison operator:

TEXT

=

<

>

<=

>=

<>

Example:

SQL

```
SELECT *  
FROM EMPLOYEE E  
JOIN GRADE G  
ON E.Salary >= G.MinSalary
```

```
AND E.Salary <= G.MaxSalary;
```

This uses comparison predicates beyond simple equality.

## MEMORY

TEXT

Theta Join  
= any comparison

Equi Join  
= equality only

Therefore:

**Every equi join is a theta join.**

But every theta join is not necessarily an equi join.

## 13. NATURAL JOIN

A **NATURAL JOIN** automatically matches columns with the **same names**.

Suppose:

TEXT

STUDENT(Sid, Name, DeptID)

DEPARTMENT(DeptID, DeptName)

Then:

SQL

```
SELECT *  
FROM STUDENT  
NATURAL JOIN DEPARTMENT;
```

automatically joins using:

TEXT

DeptID

## 14. NATURAL JOIN OUTPUT

Result schema:

TEXT

Sid  
Name

```
DeptID
DeptName
```

Notice:

`DeptID` appears only once.

This resembles relational algebra natural join.

## NATURAL JOIN vs EQUI JOIN

### Equi join

SQL

```
SELECT *
FROM STUDENT S
JOIN DEPARTMENT D
ON S.DeptID = D.DeptID;
```

may conceptually contain both:

TEXT

```
S.DeptID
D.DeptID
```

depending on selected columns.

### Natural join

Automatically:

1. Finds common-name columns.
2. Checks equality.
3. Keeps one copy of the common column.

## 15. HUGE NATURAL JOIN TRAP

Suppose:

TEXT

```
R(A, B, C)
```

```
S(B, C, D)
```

Natural join is on:

TEXT

```
B AND C
```

not just B.

Why?

Because both:

TEXT

B  
C

have the same names.

## GATE RULE

**NATURAL JOIN** uses all identically named attributes.

This is a frequent source of mistakes.

## 16. WHAT IF THERE ARE NO COMMON ATTRIBUTE NAMES?

Suppose:

TEXT

R(A, B)  
S(C, D)

There are no common names.

Then a natural join behaves conceptually like Cartesian product because there are no equality conditions to enforce.

So:

$R \bowtie S = R \times S$

in that situation.

This is a useful theoretical GATE fact.

## 17. WHAT IF COMMON COLUMNS HAVE UNINTENDED MEANINGS?

Suppose:

TEXT

EMPLOYEE(ID, Name, City)

COMPANY(ID, Name, City)

A natural join may match on:

TEXT

ID  
Name  
City

because all are common names.

That may not be what you intended.

This is why explicit `JOIN ... ON ...` is often safer in real SQL.

---

## 18. KEY–FOREIGN KEY JOIN

This is extremely important for cardinality questions.

Suppose:

```
TEXT
DEPARTMENT.DeptID
```

is primary key.

And:

```
TEXT
STUDENT.DeptID
```

is a valid foreign key.

If every student's DeptID is non-NULL and references an existing department, then:

```
SQL
SELECT *
FROM STUDENT S
JOIN DEPARTMENT D
ON S.DeptID = D.DeptID;
```

returns exactly one result row for every STUDENT row.

Therefore:

\[

|      |   |         |
|------|---|---------|
| Join | = | STUDENT |
|------|---|---------|

\]

---

## WHY?

One student has:

```
TEXT
one DeptID
```

The referenced relation has:

```
TEXT
at most one department with that DeptID
```

because it is a key.

So each student matches exactly once.

## GATE SHORTCUT

For:

TEXT

Foreign Key !' Referenced Primary / Candidate Key

If every FK is valid and non-NULL:

\[ Join\ cardinality=number\ of\ referencing\ rows \]

## 19. IF FOREIGN KEY CAN BE NULL?

Suppose STUDENT:

| Sid | Name  | DeptID |
|-----|-------|--------|
| 1   | Ravi  | 10     |
| 2   | Anu   | NULL   |
| 3   | Kiran | 20     |

For an inner join:

SQL

ON S.DeptID = D.DeptID

Anu does not match because:

TEXT

NULL = something

is not TRUE.

So inner join produces only Ravi and Kiran.

We'll study SQL's NULL logic deeply soon.

## 20. OUTER JOINS — WHY THEY EXIST

Inner join loses nonmatching rows.

Sometimes we want to preserve them.

That gives us:



TEXT

LEFT OUTER JOIN

RIGHT OUTER JOIN

FULL OUTER JOIN

## 21. LEFT OUTER JOIN

A left join keeps:

**ALL rows from the left table**

plus matching data from the right table.

If there is no match:

TEXT

r i g h t - s i d e   c o l u m n s   ! '   N U L L

Query:

SQL

```
SELECT S.Name, D.DeptName
FROM STUDENT S
LEFT JOIN DEPARTMENT D
ON S.DeptID = D.DeptID;
```

## 22. LEFT JOIN RESULT

Recall:

STUDENT includes:

TEXT

R a j   ! '   D e p t I D   4 0

But DEPARTMENT has no 40.

Result:

| Name  | DeptName |
|-------|----------|
| Ravi  | CSE      |
| Anu   | AIML     |
| Kiran | CSE      |
| Riya  | ECE      |
| Raj   | NULL     |

Raj is preserved because STUDENT is the **left** relation.

## MEMORY TRICK

**LEFT JOIN = Left never gets lost.**

Very easy to remember.

## 23. RIGHT OUTER JOIN

A right join keeps:

**ALL rows from the right relation**

Example:

SQL

```
SELECT S.Name, D.DeptName
FROM STUDENT S
RIGHT JOIN DEPARTMENT D
ON S.DeptID = D.DeptID;
```

DEPARTMENT contains:

TEXT

50 CIVIL

No student belongs to CIVIL.

But right join retains CIVIL:

| Name  | DeptName |
|-------|----------|
| Ravi  | CSE      |
| Kiran | CSE      |
| Anu   | AIML     |
| Riya  | ECE      |
| NULL  | CIVIL    |

## MEMORY TRICK

**RIGHT JOIN = Right never gets lost.**

## 24. FULL OUTER JOIN

A full outer join preserves:

- all matching rows
- unmatched left rows
- unmatched right rows

Example:

SQL

```
SELECT S.Name, D.DeptName
FROM STUDENT S
FULL OUTER JOIN DEPARTMENT D
ON S.DeptID = D.DeptID;
```

Conceptually:

| Name  | DeptName |
|-------|----------|
| Ravi  | CSE      |
| Kiran | CSE      |
| Anu   | AIML     |
| Riya  | ECE      |
| Raj   | NULL     |
| NULL  | CIVIL    |

Both unmatched cases survive.

## INNER vs LEFT vs RIGHT vs FULL

Memorize this:

TEXT

INNER

! ' m a t c h e d o n l y

LEFT

! ' m a t c h e d + u n m a t c h e d l e f t

RIGHT

! ' m a t c h e d + u n m a t c h e d r i g h t

FULL

! ' m a t c h e d + u n m a t c h e d b o t h s i d e s

## 25. OUTER JOIN NULLS

Outer joins introduce NULL placeholders where a matching tuple is absent.

Example:

```
TEXT
Raj | NULL
```

does **not** mean Raj's department name is actually stored as NULL in DEPARTMENT.

It means:

**No matching DEPARTMENT tuple was found.**

This distinction matters.

## 26. LEFT JOIN CARDINALITY

A left outer join must produce **at least one output row for every left row**.

Therefore generally:

\[

|                                   |                  |     |
|-----------------------------------|------------------|-----|
| $R \setminus \text{LEFT JOIN } S$ | $\setminus_{ge}$ | $R$ |
|-----------------------------------|------------------|-----|

\]

But can it exceed  $|R|$  ?

Yes.

If one left row matches several right rows, it appears several times.

### Example

R:

```
TEXT
A
1
```

S:

```
TEXT
A | B
1 | X
1 | Y
1 | Z
```

Left join on A:

```
TEXT
1 X
1 Y
1 Z
```

One left row produced 3 output rows.

Therefore:

**left join cardinality is not always exactly the number of left rows.**

## GATE TRAP

Do not blindly say:

TEXT

LEFT JOIN result size = left-table size

Correct:

**It is at least left-table size, unless filtering is later applied.**

Exactly equal only under conditions such as at-most-one right match per left row.

## 27. LEFT JOIN WITH KEY MATCH

If right-side join attribute is unique/key:

TEXT

$R.FK = S.PK$

then each left row matches at most one S row.

A left join will then produce exactly:

$\setminus$

R

$\setminus$

rows.

Because:

- match !' 1 output
- no match !' 1 NULL-extended output

So every R row appears exactly once.

## 28. RIGHT JOIN CARDINALITY

Similarly, if each right row can match at most one left tuple:

$\setminus$

|           |   |       |
|-----------|---|-------|
| RightJoin | = | Right |
|-----------|---|-------|

$\setminus$

Otherwise multiple matches can produce more rows.

## 29. FULL OUTER JOIN CARDINALITY

If the join is one-to-one with unique join attributes:

TEXT  
matching rows = m

then:

$\setminus$

FULL  $\setminus$  OUTER  $\setminus$  JOIN

$= |R| + |S| - m \setminus$

because matched pairs appear once.

But with duplicate matching values, cardinality calculation can become more complex.

## 30. SELF JOIN

A self join joins a relation with itself.

Example:

### EMPLOYEE

| Eid | Name | ManagerID |
|-----|------|-----------|
| 1   | Ravi | NULL      |
| 2   | Anu  | 1         |
| 3   | Raj  | 1         |
| 4   | Riya | 2         |

We want:

Employee name and manager name.

## 31. SELF JOIN QUERY

#### SQL

```
SELECT E.Name AS Employee,  
       M.Name AS Manager  
FROM EMPLOYEE E  
LEFT JOIN EMPLOYEE M  
ON E.ManagerID = M.Eid;
```

Here:

#### TEXT

E = employee role

M = manager role

Same physical table, two logical aliases.

## Result

| Employee | Manager |
|----------|---------|
| Ravi     | NULL    |
| Anu      | Ravi    |
| Raj      | Ravi    |
| Riya     | Anu     |

## MEMORY TRICK

**Self join = Same table, different roles.**

Aliases are essential.

## 32. ON vs WHERE — BASIC DIFFERENCE

For an **inner join**, these often produce the same result:

#### SQL

```
SELECT *  
FROM R  
JOIN S  
ON R.A = S.A  
WHERE R.X > 10;
```

and logically:

SQL

```
SELECT *  
FROM R, S  
WHERE R.A = S.A  
AND R.X > 10;
```

But with **outer joins**, moving conditions between **ON** and **WHERE** can dramatically change the result.

This is a major GATE trap.

### 33. LEFT JOIN — CONDITION IN ON

Suppose:

STUDENT:

| Sid | Name | DeptID |
|-----|------|--------|
| 1   | Ravi | 10     |
| 2   | Anu  | 20     |
| 3   | Raj  | 40     |

DEPARTMENT:

| DeptID | DeptName |
|--------|----------|
| 10     | CSE      |
| 20     | AIML     |

Query:

SQL

```
SELECT S.Name, D.DeptName  
FROM STUDENT S  
LEFT JOIN DEPARTMENT D  
ON S.DeptID = D.DeptID  
AND D.DeptName = 'CSE';
```

Matching happens only when department is CSE.

Result:

TEXT

```
R a v i   !'   C S E  
A n u     !'   N U L L  
R a j     !'   N U L L
```

Why is Anu preserved?

Because this is still a LEFT JOIN.

The extra condition affects matching but does not eliminate left rows.



## 34. SAME CONDITION IN WHERE

Now:

SQL

```
SELECT S.Name, D.DeptName
FROM STUDENT S
LEFT JOIN DEPARTMENT D
ON S.DeptID = D.DeptID
WHERE D.DeptName = 'CSE';
```

First left join produces:

TEXT

```
Ravi CSE
Anu AIML
Raj NULL
```

Then WHERE checks:

TEXT

```
DeptName = CSE
```

Only Ravi satisfies TRUE.

So result:

TEXT

```
Ravi CSE
```

## HUGE GATE TRAP

With outer joins:

TEXT

```
ON condition
!' controls MATCHING

WHERE condition
!' filters the resulting rows
```

A **WHERE** predicate on a NULL-extended right-side column can effectively remove unmatched rows and make the result behave more like an inner join.

## MEMORY TRICK

**ON** decides partners.

**WHERE** decides survivors.

This is a very useful mental model.

---

## 35. OLD STYLE JOIN AND ACCIDENTAL CROSS JOIN

---

Suppose someone writes:

```
SQL
SELECT *
FROM STUDENT S, DEPARTMENT D;
```

and forgets:

```
SQL
WHERE S.DeptID = D.DeptID;
```

Then instead of a join:

```
TEXT
5 students × 4 departments
= 20 rows
```

This is an accidental Cartesian product.

A common SQL mistake.

---

## 36. NATURAL JOIN vs JOIN USING

---

Some SQL dialects support:

```
SQL
SELECT *
FROM STUDENT
JOIN DEPARTMENT
USING (DeptID);
```

`USING(DeptID)` explicitly says the shared column to use.

Compared with:

```
SQL
NATURAL JOIN
```

which automatically uses all same-named columns.

Conceptual exam distinction:

```
TEXT
NATURAL
!' automatically all common names

USING
!' specified common column(s)
```

```
ON
!'  a r b i t r a r y   j o i n   c o n d i t i o n
```

## 37. MULTIPLE TABLE JOIN

Suppose:

```
TEXT
STUDENT(Sid, Name, DeptID)

DEPARTMENT(DeptID, DeptName)

ENROLLMENT(Sid, CourseID)

COURSE(CourseID, Title)
```

Question:

**Find student names with course titles.**

We need:

```
TEXT
STUDENT
    !"  S i d
ENROLLMENT
    !"  C o u r s e I D
COURSE
```

Query:

```
SQL
SELECT S.Name, C.Title
FROM STUDENT S
JOIN ENROLLMENT E
ON S.Sid = E.Sid
JOIN COURSE C
ON E.CourseID = C.CourseID;
```

## HOW TO THINK ABOUT MULTIPLE JOINS

Build a path:

```
TEXT
STUDENT
|
Sid
|
ENROLLMENT
|
CourseID
|
COURSE
```

This is easier than trying to write the query directly.

## 38. JOIN CARDINALITY — GENERAL IDEA

Suppose:

```
TEXT
R has m rows
S has n rows
```

For an arbitrary inner join:

minimum:

$\max\{0, \dots\}$

if no rows match.

Maximum:

$\min\{m, n\}$

if every R tuple matches every S tuple.

So:

$\max\{0, \dots\}$

in a general theta/equi join scenario.

## 39. KEY-KEY JOIN

Suppose join attribute is unique in both relations.

Then every R tuple can match at most one S tuple, and vice versa.

Therefore:

$\min\{|R|, |S|\}$

Example:

```
TEXT
R has 100 rows
S has 80 rows
```

both join columns are keys.

Maximum matching rows:

```
TEXT
80
```

## 40. FOREIGN KEY !' KEY JOIN

Suppose:

TEXT

R has FK referencing S's PK

If all FK values are valid and non-NULL:

\[

|            |   |   |
|------------|---|---|
| R\bowtie S | = | R |
|------------|---|---|

\]

This is a very important GATE shortcut.

## 41. NON-KEY JOIN

Suppose:

R:

TEXT

Dept  
CSE  
CSE

S:

TEXT

Dept  
CSE  
CSE  
CSE

Join on Dept.

Every R CSE matches every S CSE.

Result:

\[ 2\times 3=6 \]

rows.

This demonstrates why duplicate matching values can dramatically increase join cardinality.

## 42. GENERAL MATCHING-FREQUENCY FORMULA

For equi-join on attribute A:

If a value  $v$  appears:

TEXT

$r_v$  times in R

$s_v$  times in S

then value  $v$  contributes:

$$|r_v \times s_v|$$

rows to the join.

Therefore:

$$|$$

R\bowtie S

$$= \sum_v |r_v \times s_v|$$

over values occurring in both relations.

This is an excellent GATE cardinality technique.

## EXAMPLE

R.A:

TEXT

1  
1  
2  
3

S.A:

TEXT

1  
1  
1  
2  
4

Frequency:

For value 1:

TEXT

|   |   |   |
|---|---|---|
| R | 1 | 2 |
| S | 1 | 3 |

Contribution:

$$|2 \times 3| = 6$$

For value 2:

TEXT

1 × 1 = 1

3 doesn't match.

4 doesn't match.

Therefore total:

$\backslash 6+1=7 \backslash$

join rows.

## GATE SHORTCUT

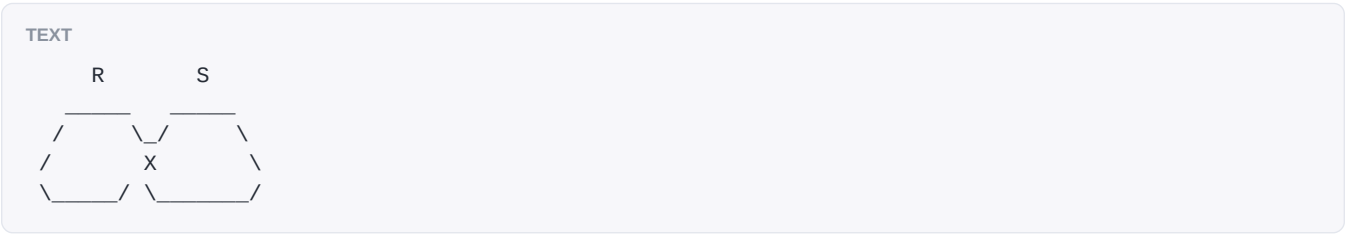
For equality joins:

Frequency in left × frequency in right for each common value, then add.

This is extremely useful.

## 43. INNER JOIN vs OUTER JOIN VISUAL MEMORY

Think of two sets:



### INNER

Only matching overlap.

### LEFT

All of R + matching S.

### RIGHT

All of S + matching R.

### FULL

Everything from both.

This analogy is only conceptual; SQL joins operate on tuple matching rather than pure set membership.

## 44. SQL JOIN COMPARISON TABLE

| Join  | Keeps matches? | Keeps unmatched left? | Keeps unmatched right? |
|-------|----------------|-----------------------|------------------------|
| INNER | Yes            | No                    | No                     |
| LEFT  | Yes            | Yes                   | No                     |
| RIGHT | Yes            | No                    | Yes                    |
| FULL  | Yes            | Yes                   | Yes                    |

| Join  | Keeps matches?   | Keeps unmatched left? | Keeps unmatched right? |
|-------|------------------|-----------------------|------------------------|
| CROSS | All combinations | —                     | —                      |

Must-write table.

## 45. JOIN vs SUBQUERY

Some queries can be written using either:

TEXT

JOIN

or:

TEXT

subquery

Example:

**Find students belonging to CSE.**

Join:

SQL

```
SELECT S.Name
FROM STUDENT S
JOIN DEPARTMENT D
ON S.DeptID = D.DeptID
WHERE D.DeptName = 'CSE';
```

Could also be expressed with a subquery.

We'll learn that in the next SQL section.

For GATE:

**Understand semantics first; don't assume one style always produces identical duplicate behavior without checking.**

## 46. GATE-STYLE EXAMPLE 1 — CROSS JOIN

Suppose:

TEXT

R has 8 rows.  
S has 12 rows.

Query:



SQL

```
SELECT *  
FROM R CROSS JOIN S;
```

Rows:

$\sqrt{8 \times 12 = 96}$

## 47. GATE-STYLE EXAMPLE 2 — INNER KEY/FK JOIN

Suppose:

TEXT

EMPLOYEE has 500 rows.

DEPARTMENT has 10 rows.

Every EMPLOYEE.DeptID  
is non-NULL FK referencing  
DEPARTMENT.DeptID PK.

Query:

SQL

```
SELECT *  
FROM EMPLOYEE E  
JOIN DEPARTMENT D  
ON E.DeptID = D.DeptID;
```

Each employee matches exactly one department.

Answer:

$\sqrt{500}$

Not:

TEXT

5000

## 48. GATE-STYLE EXAMPLE 3 — LEFT JOIN

R:

| A |  |
|---|--|
| 1 |  |
| 2 |  |
| 3 |  |

S:

| A |
|---|
| 1 |
| 1 |
| 2 |

Query:

```
SQL
SELECT *
FROM R
LEFT JOIN S
ON R.A = S.A;
```

For A=1 :

```
TEXT
1 in R
matches 2 rows in S
!' 2 o u t p u t s
```

For A=2 :

```
TEXT
1 match
!' 1 o u t p u t
```

For A=3 :

```
TEXT
no match
!' o n e N U L L - e x t e n d e d o u t p u t
```

Total:

\[ 2+1+1=4 \]

49. GATE-STYLE EXAMPLE 4 — FREQUENCY METHOD

R.A:

```
TEXT
1
1
1
2
2
```

S.A:

TEXT

1  
1  
2  
2  
2  
3

Join:

SQL

ON R.A = S.A

For value 1:

$\sqrt[3]{3 \times 2 = 6}$

For value 2:

$\sqrt[2]{2 \times 3 = 6}$

Total:

$\sqrt[12]{12}$

**Answer**

12 tuples.

## 50. GATE-STYLE EXAMPLE 5 — NATURAL JOIN DEGREE

Relations:

TEXT

R(A, B, C, D)

S(B, C, E, F)

Common attributes:

TEXT

B, C

Natural join output attributes:

TEXT

A, B, C, D, E, F

Degree:

$\sqrt[4+4-2]{4 \times 4} = 6$

## 51. GATE-STYLE EXAMPLE 6 — SELF JOIN

EMPLOYEE:

| Eid | Name | ManagerID |
|-----|------|-----------|
| 1   | A    | NULL      |
| 2   | B    | 1         |
| 3   | C    | 1         |
| 4   | D    | 2         |

Query:

SQL

```
SELECT E.Name, M.Name
FROM EMPLOYEE E
JOIN EMPLOYEE M
ON E.ManagerID = M.Eid;
```

Results:

TEXT

```
B A
C A
D B
```

Why isn't A present?

Because A's:

TEXT

```
ManagerID = NULL
```

so there is no inner-join match.

If it were `LEFT JOIN`, A would appear with manager NULL.

## 52. GATE-STYLE EXAMPLE 7 — ON vs WHERE

Query A:

SQL

```
SELECT *
FROM R
LEFT JOIN S
ON R.A = S.A
AND S.B = 10;
```

Meaning:

**Preserve every R row, but only match S rows having B=10.**

Query B:

SQL

```
SELECT *  
FROM R  
LEFT JOIN S  
ON R.A = S.A  
WHERE S.B = 10;
```

Meaning:

**Perform left join first, then keep only rows where S.B=10.**

Unmatched rows usually have:

TEXT

S.B = NULL

and are removed by WHERE.

Therefore A and B are **not equivalent**.

## 53. GATE TRAPS

### & pTrap 1 — CROSS vs INNER

TEXT

CROSS = every combination

INNER = only matching combinations

### & pTrap 2 — Natural join

Uses **every same-named attribute**.

### & pTrap 3 — Equi join vs natural join

Natural join removes duplicate common attributes from conceptual output.

Equi join doesn't automatically require that.

### & pTrap 4 — LEFT JOIN

Does not mean exactly one output per left row.

One left tuple may match multiple right tuples.

Correct minimum:

**Left tuples are preserved.**

---

## & pTrap 5 — ON vs WHERE

---

Especially for outer joins:

TEXT

```
ON !'   m a t c h i n g
WHERE !'   f i n a l   f i l t e r i n g
```

---

## & pTrap 6 — NULL in inner join

---

NULL does not ordinarily match using:

SQL

=

---

## & pTrap 7 — Foreign key join

---

When FK references a key and all FKs are valid/non-NULL:

TEXT

join size = referencing relation size

---

## & pTrap 8 — Duplicate join values

---

Matching multiplicity multiplies.

TEXT

2 copies × 3 copies = 6 joined rows

---

## & pTrap 9 — Self join

---

Use aliases to distinguish logical roles.

---

## & pTrap 10 — Forgotten join condition

---

SQL

FROM R, S

without proper matching condition can generate Cartesian product.

---

## 54. PYQ-ORIENTED JOIN SOLVING METHOD

---

When you see a join question, do this:

### STEP 1 — Identify join type

TEXT

CROSS?  
INNER?  
LEFT?  
RIGHT?  
FULL?  
NATURAL?

### STEP 2 — Identify join attributes

Example:

TEXT

R.A = S.B

### STEP 3 — Check keys

Ask:

TEXT

Is one side PK/candidate key?  
  
Is other side FK?  
  
Are NULLs allowed?

This may immediately determine result cardinality.

### STEP 4 — If duplicates exist, build frequency table

Example:

| Value | Count in R | Count in S | Contribution |
|-------|------------|------------|--------------|
| 1     | 3          | 2          | 6            |
| 2     | 2          | 3          | 6            |

Add contributions.

### STEP 5 — For outer join, add unmatched preserved tuples

For LEFT JOIN:

TEXT

matching contributions  
+  
left rows having no match

For RIGHT:

TEXT

+  
unmatched right rows

For FULL:

TEXT

+  
unmatched both sides

**STEP 6 — Apply WHERE afterward**

This is especially important for outer joins.

**55. NOTEBOOK NOTES**

**SQL JOINS**

**CROSS JOIN**

TEXT

Every tuple of R with every tuple of S

$\mathcal{V}$

|              |   |   |  |   |
|--------------|---|---|--|---|
| $R \times S$ | = | R |  | S |
|--------------|---|---|--|---|

$\mathcal{V}$

**INNER JOIN**

Only matching tuples.

SQL

R JOIN S  
ON condition

**EQUI JOIN**

Join condition uses =.

**THETA JOIN**

Join condition can use:



TEXT

=, <, >, <=, >=, <>

## NATURAL JOIN

TEXT

Automatically matches  
ALL same-name attributes

Duplicate common columns  
appear once

If no common attributes:

TEXT

N a t u r a l   j o i n "H   C a r t e s i a n   p r o d u c t

## OUTER JOINS

TEXT

I N N E R ! '   m a t c h e d   o n l y

L E F T ! '   a l l   l e f t

R I G H T ! '   a l l   r i g h t

F U L L ! '   a l l   b o t h

Memory:

TEXT

Left never gets lost.  
Right never gets lost.

## SELF JOIN

Same relation joined with itself using aliases.

## JOIN CARDINALITY

General:

$\mathcal{V} [ 0 | e | R | \text{bowtie } S | \setminus e | R | | S | \setminus ]$

Equality join frequency:

$\mathcal{V} [$

R\bowtie S

$$= \sum_v f_R(v) f_S(v)$$

FK !' referenced key, valid/non-NULL:

$$\left[ \begin{array}{ccc} R \bowtie S & = & R \end{array} \right]$$

where R is the referencing relation.

## NATURAL JOIN DEGREE

If there are  $k$  common attributes:

$$\left[ \deg(R \bowtie S) = \deg(R) + \deg(S) - k \right]$$

## ON vs WHERE

TEXT

ON !' decides matches

WHERE !' filters resulting rows

Critical for outer joins.

## MEMORY PAGE

TEXT

CROSS !' ALL combinations

INNER !' MATCHES

LEFT !' Left survives

RIGHT !' Right survives

FULL !' Both survive

SELF !' Same table, two roles

NATURAL !' all same-name columns

ON !' Partners

WHERE !' Survivors

And the GATE cardinality trick:

**Count-left(value) × Count-right(value), then add.**

## 30-SECOND REVISION

### TEXT

#### CROSS:

$|R \times S| = |R| |S|$

#### INNER:

matching rows only

#### NATURAL:

match all common attribute names

remove duplicate common columns

#### LEFT:

preserve all left rows

#### RIGHT:

preserve all right rows

#### FULL:

preserve both sides

#### SELF:

same relation + aliases

**F K ! ' P K i n n e r j o i n :**

if every FK valid and non-NULL,

result rows = referencing rows

Equi-join duplicate values:

frequency multiplication

ON = matching

WHERE = filtering

## CHECK YOUR UNDERSTANDING

### Q1

**R** contains 20 tuples and **S** contains 5 tuples.

How many tuples are returned by:

### SQL

```
SELECT *  
FROM R CROSS JOIN S;
```

### Q2

Suppose:

### TEXT

**EMPLOYEE** = 1000 tuples

**DEPARTMENT** = 20 tuples

Every **EMPLOYEE.DeptID** is a valid non-NULL FK referencing the primary key **DEPARTMENT.DeptID**.

How many tuples are returned by their inner equi-join on DeptID?

### Q3

R.A contains:

TEXT

1  
1  
2  
2  
2

S.A contains:

TEXT

1  
1  
1  
2

How many tuples does:

SQL

R JOIN S ON R.A = S.A

produce?

### Q4

Which join guarantees that every left-side tuple appears at least once before any later `WHERE` filtering?

A. INNER JOIN B. LEFT OUTER JOIN C. CROSS JOIN D. NATURAL JOIN

### Q5

Relations:

TEXT

R(A, B, C)  
S(B, C, D, E)

What is the degree of:

SQL

R NATURAL JOIN S

?

### Q6 — Very Important

Explain the difference between:

SQL

```
LEFT JOIN S
ON R.A = S.A
AND S.B = 10
```

and:

SQL

```
LEFT JOIN S
ON R.A = S.A
WHERE S.B = 10
```

in one or two sentences.

Next:

## SQL — PART 3: SUBQUERIES

We will cover the most GATE-tricky SQL ideas:

**nested queries !' scalar subqueries !' IN !' NOT IN !' EXISTS !' NOT EXISTS !' correlated subqueries !' ANY/SOME !' ALL !' highest/second-highest type queries !' NULL traps !' double NOT EXISTS for “all.”**

Type `NEXT`.

USER

NEXT

ASSISTANT

## SQL — PART 3: SUBQUERIES, IN, EXISTS, ANY, ALL

This is one of the **most important SQL reasoning sections for GATE**.

Questions here often look complicated because one SQL query appears inside another. But nearly all of them reduce to a few patterns:

TEXT

|            |                                             |
|------------|---------------------------------------------|
| IN         | !' belongs to a set                         |
| NOT IN     | !' not in a set                             |
| EXISTS     | !' at least one matching row exists         |
| NOT EXISTS | !' no matching row exists                   |
| ANY        | !' comparison true for at least one value   |
| ALL        | !' comparison true for every value          |
| Correlated | !' inner query depends on current outer row |

If you learn these patterns properly, many difficult-looking SQL questions become mechanical.

### 1. WHAT IS A SUBQUERY?

A **subquery** is a query written inside another query.

Example:

```
SQL
SELECT Name
FROM STUDENT
WHERE DeptID IN
(
    SELECT DeptID
    FROM DEPARTMENT
    WHERE Location = 'Hyderabad'
);
```

There are two queries:

### Outer query

```
SQL
SELECT Name
FROM STUDENT
WHERE DeptID IN (...)
```

### Inner query / subquery

```
SQL
SELECT DeptID
FROM DEPARTMENT
WHERE Location = 'Hyderabad'
```

## 2. HOW TO SOLVE A SIMPLE SUBQUERY

Usually:

**Solve the innermost independent query first.**

Then substitute its result into the outer query.

### Example

#### STUDENT

| Sid | Name | DeptID |
|-----|------|--------|
| 1   | Ravi | 10     |
| 2   | Anu  | 20     |
| 3   | Raj  | 10     |
| 4   | Riya | 30     |

## DEPARTMENT

| DeptID | DeptName | Location  |
|--------|----------|-----------|
| 10     | CSE      | Hyderabad |
| 20     | AIML     | Hyderabad |
| 30     | ECE      | Chennai   |

Query:

SQL

```
SELECT Name
FROM STUDENT
WHERE DeptID IN
(
    SELECT DeptID
    FROM DEPARTMENT
    WHERE Location = 'Hyderabad'
);
```

### Step 1 — inner query

SQL

```
SELECT DeptID
FROM DEPARTMENT
WHERE Location = 'Hyderabad';
```

Result:

TEXT

```
10
20
```

### Step 2 — outer query becomes

SQL

```
SELECT Name
FROM STUDENT
WHERE DeptID IN (10,20);
```

Result:

TEXT

```
Ravi
Anu
Raj
```

## 3. TYPES OF SUBQUERIES

For GATE, understand these:

### 1. Scalar/single-value subquery

2. Multi-row subquery
  3. Correlated subquery
  4. Nested/multiple-level subquery
- 

## 4. SCALAR SUBQUERY

---

A scalar subquery returns **one value**.

Example:

```
SQL
SELECT Name
FROM STUDENT
WHERE Marks >
(
    SELECT AVG(Marks)
    FROM STUDENT
);
```

Inner query:

```
SQL
SELECT AVG(Marks)
FROM STUDENT;
```

returns one number.

Suppose:

```
TEXT
AVG = 80
```

Outer query becomes:

```
SQL
SELECT Name
FROM STUDENT
WHERE Marks > 80;
```

---

## GATE RECOGNITION

---

If inner query produces exactly one value, ordinary operators can be used:

```
TEXT
=
>
<
>=
<=
<>
```

Example:



SQL

```
Salary > (SELECT AVG(Salary) ...)
```

## 5. MULTI-ROW SUBQUERY

Suppose inner query returns:

TEXT

```
10
20
30
```

Then writing:

SQL

```
WHERE DeptID =
(
    SELECT DeptID ...
)
```

is problematic because `=` expects one scalar value.

Instead we commonly need:

TEXT

```
IN
ANY
ALL
EXISTS
```

depending on the meaning.

## 6. IN

`IN` asks whether a value belongs to a set.

SQL

```
WHERE DeptID IN (10,20,30)
```

means:

TEXT

```
DeptID = 10
OR
DeptID = 20
OR
DeptID = 30
```

## MEMORY TRICK

**IN = Membership**

Ask:

**“Is my value inside this result set?”**

## 7. IN WITH SUBQUERY

SQL

```
SELECT Name
FROM STUDENT
WHERE DeptID IN
(
    SELECT DeptID
    FROM DEPARTMENT
    WHERE Location='Hyderabad'
);
```

Meaning:

**Find students whose department belongs to the set of departments located in Hyderabad.**

## 8. NOT IN

SQL

```
WHERE DeptID NOT IN (10,20)
```

means conceptually:

TEXT

```
DeptID <> 10
AND
DeptID <> 20
```

Notice:

**IN** expands using **OR**.

**NOT IN** effectively expands using **AND**.

## MEMORY

TEXT

```
IN:
= A OR = B OR = C
```

```
NOT IN:
<> A AND <> B AND <> C
```

This becomes extremely important when NULL is involved.

## 9. EXISTS

EXISTS asks:

**Does the subquery return at least one row?**

Syntax:

```
SQL
WHERE EXISTS
(
    SELECT ...
)
```

If subquery returns at least one tuple:

```
TEXT
EXISTS = TRUE
```

If subquery returns zero tuples:

```
TEXT
EXISTS = FALSE
```

## MEMORY TRICK

```
TEXT
EXISTS = At least one row?
```

It does **not care about the actual selected value**.

It cares whether a row exists.

## 10. EXISTS DOES NOT CARE ABOUT SELECT LIST

These are typically equivalent for existence testing:

```
SQL
EXISTS
(
    SELECT *
    FROM ENROLLMENT E
    WHERE ...
)
```

and:

```
SQL
EXISTS
(
    SELECT 1
    FROM ENROLLMENT E
    WHERE ...
)
```

Why?

Because EXISTS asks only:

**Did the subquery produce a row?**

Not:

**What value did it produce?**

## & pGATE TRAP

Inside EXISTS:

```
SQL
SELECT 1
```

does **not** mean that the database checks whether value 1 exists.

It is simply a conventional placeholder.

## 11. SIMPLE EXISTS EXAMPLE

Suppose:

### STUDENT

| Sid | Name |
|-----|------|
| 1   | Ravi |
| 2   | Anu  |
| 3   | Raj  |

### ENROLLMENT

| Sid | Course |
|-----|--------|
| 1   | DBMS   |
| 1   | OS     |

| Sid | Course |
|-----|--------|
| 3   | CN     |

Query:

```
SQL
SELECT S.Name
FROM STUDENT S
WHERE EXISTS
(
    SELECT 1
    FROM ENROLLMENT E
    WHERE E.Sid = S.Sid
);
```

Evaluate each student.

### Ravi

Is there E where:

```
TEXT
E.Sid = 1?
```

Yes.

Return Ravi.

### Anu

Any E where:

```
TEXT
E.Sid = 2?
```

No.

Do not return.

### Raj

Any E where:

```
TEXT
E.Sid = 3?
```

Yes.

Return Raj.

Result:

```
TEXT
Ravi
Raj
```

## 12. THIS IS A CORRELATED SUBQUERY

Look carefully:

```
SQL
WHERE E.Sid = S.Sid
```

`S.Sid` belongs to the **outer query**.

Therefore the inner query depends on the current outer row.

This is called a:

**Correlated subquery**

## 13. CORRELATED SUBQUERY

A correlated subquery cannot generally be evaluated once independently.

Instead conceptually:

```
TEXT
Take one outer row
      !"
Run inner query using that row
      !"
TRUE/FALSE
      !"
Take next outer row
      !"
Repeat
```

## MEMORY TRICK

**Correlated = Connected to outer row**

If the inner query contains an attribute from an outer-query alias:

```
SQL
E.Sid = S.Sid
```

then think:

```
TEXT
CORRELATED
```

## 14. NON-CORRELATED vs CORRELATED

## Non-correlated

SQL

```
SELECT Name
FROM STUDENT
WHERE Marks >
(
    SELECT AVG(Marks)
    FROM STUDENT
);
```

Inner query can run independently.

## Correlated

SQL

```
SELECT S.Name
FROM STUDENT S
WHERE EXISTS
(
    SELECT *
    FROM ENROLLMENT E
    WHERE E.Sid = S.Sid
);
```

Inner query needs current `S.Sid`.

## COMPARISON

| Non-correlated              | Correlated                      |
|-----------------------------|---------------------------------|
| Independent inner query     | Depends on outer query          |
| Often conceptually run once | Conceptually runs per outer row |
| No outer alias inside       | References outer alias          |

## 15. NOT EXISTS

`NOT EXISTS` means:

**The subquery returns zero rows.**

Example:

SQL

```
SELECT S.Name
FROM STUDENT S
WHERE NOT EXISTS
(
    SELECT 1
```

```
FROM ENROLLMENT E
WHERE E.Sid = S.Sid
);
```

Question:

**Find students having no enrollment.**

Using previous data:

TEXT

```
R a v i   !'   e n r o l l m e n t   e x i s t s   !'   r e j e c t
A n u     !'   n o   e n r o l l m e n t   !'   a c c e p t
R a j     !'   e n r o l l m e n t   e x i s t s   !'   r e j e c t
```

Result:

TEXT

Anu

## MEMORY

TEXT

EXISTS

```
!'   a t   l e a s t   o n e
```

NOT EXISTS

```
!'   n o n e
```

This exactly matches our TRC understanding.

## 16. IN vs EXISTS

Conceptually:

**IN**

Tests:

**Is this value a member of the returned set?**

**EXISTS**

Tests:

**Does at least one matching row exist?**

They can sometimes express equivalent queries, but they are **not universally interchangeable**, especially once NULL semantics and duplicate behavior enter the picture.

For GATE:



Solve according to the exact SQL semantics in the question.

## 17. ANY / SOME

ANY means:

**The comparison must be true for at least one value returned by the subquery.**

SOME is generally synonymous with ANY.

Example:

```
SQL
Salary > ANY
(
    SELECT Salary
    FROM EMPLOYEE
    WHERE Dept='CSE'
)
```

Means:

**Salary is greater than at least one CSE salary.**

## Example values

Subquery returns:

```
TEXT
20
40
60
```

Condition:

```
SQL
X > ANY (20, 40, 60)
```

We ask:

**Is X > at least one of these?**

For:

```
TEXT
X = 30
```

Check:

TEXT

30 > 20 !' TRUE

Enough.

So overall condition TRUE.

## 18. > ANY SHORTCUT

For non-empty non-NULL comparison sets:

$\forall [X > \text{ANY}(S)]$

means essentially:

$\forall [X > \text{MIN}(S)]$

because to be greater than **at least one**, you only need to beat the smallest.

## Example

TEXT

$S = \{20, 40, 60\}$

Then:

TEXT

$X > \text{ANY}(S)$

equivalent intuitively to:

TEXT

$X > 20$

## 19. < ANY SHORTCUT

$\forall [X < \text{ANY}(S)]$

For at least one value, it is enough to be smaller than the largest.

So:

$\forall [X < \text{MAX}(S)]$

for ordinary non-empty non-NULL sets.

## 20. ALL

ALL means:

**Comparison must be true for every value returned.**

Example:

```
SQL
Salary > ALL
(
    SELECT Salary
    FROM EMPLOYEE
    WHERE Dept='CSE'
);
```

Means:

**Salary is greater than every CSE salary.**

## Example

Subquery:

```
TEXT
20
40
60
```

Check:

```
TEXT
X > ALL (20,40,60)
```

For X = 70:

```
TEXT
70 > 20 '
70 > 40 '
70 > 60 '
```

TRUE.

For X = 50:

```
TEXT
50 > 20 '
50 > 40 '
50 > 60 '
```

FALSE.

## 21. > ALL SHORTCUT

For ordinary non-empty non-NULL sets:

$\forall X > \text{ALL}(S) \setminus$

means:

$\forall X > \text{MAX}(S) \setminus$

## 22. < ALL SHORTCUT

$\forall X < \text{ALL}(S) \setminus$

means:

$\forall X < \text{MIN}(S) \setminus$

## MASTER ANY / ALL SHORTCUT TABLE

For a non-empty comparison set without troublesome NULLs:

| Expression          | Shortcut            |
|---------------------|---------------------|
| $X > \text{ANY}(S)$ | $X > \text{MIN}(S)$ |
| $X < \text{ANY}(S)$ | $X < \text{MAX}(S)$ |
| $X > \text{ALL}(S)$ | $X > \text{MAX}(S)$ |
| $X < \text{ALL}(S)$ | $X < \text{MIN}(S)$ |

## MEMORY TRICK

TEXT

A N Y ! ' e a s i e s t v a l u e t o b e a t

A L L ! ' h a r d e s t v a l u e t o b e a t

For >:

TEXT

A N Y ! ' m i n i m u m

A L L ! ' m a x i m u m

## 23. = ANY

SQL

$X = \text{ANY}(S)$

means:

**X equals at least one value in S.**

That's essentially like:

SQL

`X IN (S)`

under ordinary non-NULL conditions.

## 24. `<>` ALL

SQL

`X <> ALL(S)`

means:

**X is different from every value in S.**

This resembles:

SQL

`X NOT IN (S)`

again with important NULL caveats.

## 25. ANY vs ALL

Suppose:

TEXT

`S = {10, 20, 30}`

For:

TEXT

`X = 25`

**`X > ANY(S)`**

TEXT

`25 > 10 !' TRUE`

So TRUE.

**`X > ALL(S)`**

Need:

TEXT

`25 > 10 '  
25 > 20 '  
25 > 30 '`

So FALSE.

## MEMORY

### TEXT

ANY = one success is enough

ALL = every comparison must succeed

## 26. FIND EMPLOYEE WITH HIGHEST SALARY

One way:

### SQL

```
SELECT *
FROM EMPLOYEE
WHERE Salary =
(
    SELECT MAX(Salary)
    FROM EMPLOYEE
);
```

Important:

This returns **all employees tied for maximum salary**.

## & PGATE TRAP

Suppose:

### TEXT

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| A | ! | ' | 1 | 0 | 0 |
| B | ! | ' | 1 | 0 | 0 |
| C | ! | ' | 9 | 0 |   |

Query:

### SQL

```
WHERE Salary = (SELECT MAX(Salary) ...)
```

returns:

### TEXT

A  
B

not only one arbitrary employee.

## 27. HIGHEST SALARY USING ALL

Could also express conceptually:

SQL

```
SELECT *
FROM EMPLOYEE E
WHERE E.Salary >= ALL
(
    SELECT Salary
    FROM EMPLOYEE
);
```

This says:

**E's salary is at least every salary.**

So maximum salary employees survive.

## 28. EMPLOYEES ABOVE AVERAGE

Very common pattern.

SQL

```
SELECT Name
FROM EMPLOYEE
WHERE Salary >
(
    SELECT AVG(Salary)
    FROM EMPLOYEE
);
```

Procedure:

TEXT

1. Calculate AVG
2. Compare each employee against AVG

## 29. ABOVE DEPARTMENT AVERAGE — CORRELATED

Question:

**Find employees whose salary exceeds the average salary of their own department.**

Query:

SQL

```
SELECT E1.Name
FROM EMPLOYEE E1
WHERE E1.Salary >
(
    SELECT AVG(E2.Salary)
    FROM EMPLOYEE E2
    WHERE E2.DeptID = E1.DeptID
);
```

This is correlated because inner query references:

```
TEXT
E1.DeptID
```

## HOW TO SOLVE

For each E1 employee:

```
TEXT
find their DeptID
! "
calculate AVG salary for that Dept
! "
compare their salary
```

## 30. SECOND-HIGHEST DISTINCT SALARY — CONCEPT

A classic database question.

One elegant formulation:

```
SQL
SELECT MAX(Salary)
FROM EMPLOYEE
WHERE Salary <
(
    SELECT MAX(Salary)
    FROM EMPLOYEE
);
```

Why?

Inner query:

```
TEXT
highest salary
```

Outer WHERE:

```
TEXT
remove highest salary
```

Outer MAX:

```
TEXT
largest remaining salary
```

= second-highest **distinct** salary.



## Example

Salaries:

TEXT

100  
100  
90  
80

Inner MAX:

TEXT

100

Rows <100:

TEXT

90  
80

Outer MAX:

TEXT

90

Correct second-highest distinct salary.

## 31. EXISTS WITH MULTIPLE MATCHES

Suppose one student has 10 enrollments.

In:

SQL

```
WHERE EXISTS  
(  
  SELECT 1  
  FROM ENROLLMENT E  
  WHERE E.Sid=S.Sid  
)
```

The outer student still appears only once, assuming the outer relation itself has one row for that student.

Why?

EXISTS produces only:

TEXT

TRUE / FALSE

It does not replicate the outer row for each inner match.

## & IMPORTANT JOIN vs EXISTS DIFFERENCE

Suppose Ravi has three enrollments.

### JOIN

SQL

```
SELECT S.Name  
FROM STUDENT S  
JOIN ENROLLMENT E  
ON S.Sid=E.Sid;
```

May produce:

TEXT

```
Ravi  
Ravi  
Ravi
```

because there are three joined tuples.

### EXISTS

SQL

```
SELECT S.Name  
FROM STUDENT S  
WHERE EXISTS (...)
```

returns Ravi once from the outer STUDENT row.

This difference is very important.

## 32. “ALL” USING DOUBLE NOT EXISTS

This is one of the most important patterns in the whole SQL chapter.

Suppose:

TEXT

```
STUDENT(Sid, Name)  
  
REQUIRED(CourseID)  
  
ENROLLMENT(Sid, CourseID)
```

Question:

**Find students who have taken all required courses.**

Think logically:

**There must NOT EXIST a required course that this student has NOT taken.**

SQL:

SQL

```
SELECT S.Name
FROM STUDENT S
WHERE NOT EXISTS
(
    SELECT 1
    FROM REQUIRED R
    WHERE NOT EXISTS
    (
        SELECT 1
        FROM ENROLLMENT E
        WHERE E.Sid = S.Sid
        AND E.CourseID = R.CourseID
    )
);
```

Looks frightening.

But the logic is simple.

---

### 33. READ DOUBLE NOT EXISTS LIKE ENGLISH

---

Inner:

SQL

```
NOT EXISTS
(
    matching enrollment
)
```

means:

**Student has NOT taken this required course.**

Outer:

SQL

```
NOT EXISTS
(
    a required course student hasn't taken
)
```

means:

**There is no missing required course.**

Therefore:

**Student completed ALL required courses.**

---

### MEMORY TRICK

---

**ALL = No missing item exists**

or:

**ALL = NOT EXISTS an exception**

This is the same logic we learned in TRC.

## 34. SQL DIVISION CONNECTION

Recall Relational Algebra:

```
TEXT
ENROLLMENT ÷ REQUIRED
```

finds students enrolled in **all** required courses.

In SQL, we often express the same idea using:

```
TEXT
NOT EXISTS
+
NOT EXISTS
```

So:

```
TEXT
RA Division
      !"
"ALL"      !"
SQL Double NOT EXISTS
```

This connection is worth remembering.

## 35. NULL — NOW THE IMPORTANT PART

SQL NULL causes some of the most difficult GATE traps.

Remember:

```
TEXT
NULL " ` 0
NULL " ` empty string
NULL " ` ordinary known value
```

It represents missing/unknown/not-applicable information.

## 36. NEVER TEST NULL WITH = NULL

Wrong:

```
SQL
WHERE Salary = NULL
```

Correct:

```
SQL
WHERE Salary IS NULL
```

Similarly:

```
SQL
WHERE Salary IS NOT NULL
```

## MEMORY

```
TEXT
NULL uses IS
```

not ordinary equality.

## 37. WHY `NULL = NULL` IS NOT TRUE

SQL uses **three-valued logic**:

```
TEXT
TRUE
FALSE
UNKNOWN
```

If we compare:

```
SQL
NULL = 5
```

we don't know.

Result:

```
TEXT
UNKNOWN
```

Likewise:

```
SQL
NULL = NULL
```

does not evaluate to ordinary TRUE.

It is:

```
TEXT
UNKNOWN
```

because each NULL represents an unknown value, not one universal constant.

## 38. THREE-VALUED LOGIC

SQL predicates can evaluate to:

```
TEXT
T = TRUE
F = FALSE
U = UNKNOWN
```

Important because `WHERE` keeps rows only when the condition evaluates to:

```
TEXT
TRUE
```

Rows where condition is:

```
TEXT
FALSE
or
UNKNOWN
```

are rejected.

## 39. WHERE AND UNKNOWN

Suppose:

| Name | Salary |
|------|--------|
| A    | 100    |
| B    | NULL   |
| C    | 50     |

Query:

```
SQL
SELECT Name
FROM EMPLOYEE
WHERE Salary > 60;
```

A:

TEXT

100 > 60 = TRUE

Keep.

B:

TEXT

NULL > 60 = UNKNOWN

Reject.

C:

TEXT

50 > 60 = FALSE

Reject.

Result:

TEXT

A

---

## 40. NOT UNKNOWN

Important:

TEXT

NOT UNKNOWN = UNKNOWN

So:

SQL

WHERE NOT (Salary > 60)

does not necessarily retrieve rows with NULL salary.

For B:

TEXT

Salary > 60 = UNKNOWN

NOT UNKNOWN = UNKNOWN

Still rejected.

---

## 41. THREE-VALUED AND

Important table:

| A | B | A AND B |
|---|---|---------|
| T | T | T       |
| T | F | F       |
| T | U | U       |
| F | T | F       |
| F | F | F       |
| F | U | F       |
| U | T | U       |
| U | F | F       |
| U | U | U       |

The shortcut:

**In AND, FALSE dominates.**

TEXT

FALSE AND anything = FALSE

Otherwise UNKNOWN may survive.

## 42. THREE-VALUED OR

| A | B | A OR B |
|---|---|--------|
| T | T | T      |
| T | F | T      |
| T | U | T      |
| F | T | T      |
| F | F | F      |
| F | U | U      |
| U | T | T      |
| U | F | U      |
| U | U | U      |

Shortcut:

**In OR, TRUE dominates.**



TEXT

TRUE OR anything = TRUE

## MEMORY

TEXT

A N D ! ' F A L S E d o m i n a t e s

O R ! ' T R U E d o m i n a t e s

N O T U N K N O W N ! ' U N K N O W N

Very useful for GATE.

## 43. THE NOT IN + NULL DISASTER

This is one of the most important SQL GATE traps.

Suppose subquery returns:

TEXT

10

20

NULL

Now:

SQL

WHERE DeptID NOT IN (10,20,NULL)

Conceptually:

TEXT

DeptID <> 10

AND

DeptID <> 20

AND

DeptID <> NULL

But:

TEXT

DeptID <> NULL

is:

TEXT

UNKNOWN

## Example DeptID = 30

TEXT

```
3 0 < > 1 0 !' T R U E
3 0 < > 2 0 !' T R U E
3 0 < > N U L L !' U N K N O W N
```

Then:

TEXT

```
TRUE AND TRUE AND UNKNOWN
=
UNKNOWN
```

WHERE rejects UNKNOWN.

So even 30 is not returned.

## HUGE GATE RULE

If a **NOT IN** subquery/list contains NULL, the result may unexpectedly become empty for ordinary non-NULL outer values.

Always inspect NULLs.

## 44. IN WITH NULL

Suppose:

TEXT

```
X IN (10,20,NULL)
```

For:

TEXT

```
X=10
```

One comparison is TRUE.

Therefore:

TEXT

```
TRUE
```

For:

TEXT

```
X=30
```

comparisons:

TEXT

```
3 0 = 1 0  !'  F A L S E
3 0 = 2 0  !'  F A L S E
3 0 = N U L L  !'  U N K N O W N
```

Then:

TEXT

```
FALSE OR FALSE OR UNKNOWN
=
UNKNOWN
```

Rejected by WHERE.

## 45. WHY NOT EXISTS IS OFTEN SAFER WITH NULL

NOT EXISTS tests whether matching rows exist based on explicit predicates.

It does not have the same direct NOT IN (...NULL...) semantics.

Therefore in real SQL and exam reasoning:

**Pay special attention when comparing NOT IN and NOT EXISTS in the presence of NULL.**

They are not always equivalent.

## 46. COUNT + NULL RECALL

Remember:

TEXT

```
COUNT(*)
!'  r o w s
```

```
COUNT(A)
!'  n o n - N U L L  A  v a l u e s
```

```
COUNT(DISTINCT A)
!'  d i s t i n c t  n o n - N U L L  A  v a l u e s
```

This often gets combined with subqueries.

## 47. AGGREGATE SUBQUERY WITH NULL

Values:

TEXT

```
10
20
NULL
```

Then:

SQL

```
SELECT AVG(A)
```

returns:

[ 15 ]

because NULL is ignored.

So:

SQL

```
WHERE X > (SELECT AVG(A) ...)
```

compares against 15.

## 48. ANY/ALL WITH NULL

NULL makes **ANY** and **ALL** more subtle because individual comparisons may become UNKNOWN.

For GATE, if NULL is explicitly present:

**Do not blindly replace ANY/ALL with MIN/MAX.**

Instead evaluate three-valued logic carefully.

Our MIN/MAX shortcuts are safe as an exam shortcut only when:

TEXT

comparison set non-empty  
and  
no problematic NULL semantics

## 49. EMPTY SUBQUERY — VERY IMPORTANT

Suppose subquery returns **no rows**.

Then:

SQL

```
EXISTS(empty)
```

=

TEXT

FALSE

and:

SQL

```
NOT EXISTS(empty)
```

=

TEXT  
TRUE

Simple.

But **ANY** and **ALL** have an interesting logical result.

## 50. **ANY** ON EMPTY SET

Condition:

TEXT  
 $X > \text{ANY}(\text{empty set})$

asks:

**Is there at least one value for which  $X > \text{value}$ ?**

There are no values.

Therefore:

TEXT  
FALSE

Generally:

**comparison ANY empty-set = FALSE.**

## 51. **ALL** ON EMPTY SET

Condition:

TEXT  
 $X > \text{ALL}(\text{empty set})$

asks:

**Is it true that for every value in the empty set,  $X$  is greater?**

There is no counterexample.

In classical SQL logical semantics, this is:

TEXT  
TRUE

This is an example of **vacuous truth**.

## GATE MEMORY

TEXT

A N Y   e m p t y   ! '   F A L S E

A L L   e m p t y   ! '   T R U E

This is a favorite theoretical trap.

## 52. WHY `ALL(empty)` IS TRUE

Suppose statement:

**Every dragon in this classroom is wearing a blue hat.**

If there are zero dragons, can you find a dragon violating the statement?

No.

Therefore universal statement is logically true.

Same principle.

## 53. SCALAR SUBQUERY RETURNING MULTIPLE ROWS

Suppose:

SQL

```
WHERE Salary =  
(  
  SELECT Salary  
  FROM EMPLOYEE  
  WHERE Dept='CSE'  
)
```

If subquery returns:

TEXT

50

60

70

ordinary scalar equality is invalid because more than one value is returned.

You may need:

SQL

IN

ANY

ALL

depending on query meaning.

---

## 54. SCALAR SUBQUERY RETURNING ZERO ROWS

In SQL implementations, a scalar subquery returning no rows is generally treated as NULL.

Then comparisons can produce UNKNOWN.

This can appear in tricky theoretical questions.

---

## 55. IN vs = ANY

Under ordinary non-NULL set semantics:

```
SQL
X IN (subquery)
```

is equivalent in intent to:

```
SQL
X = ANY (subquery)
```

because both ask:

**Does X equal at least one returned value?**

---

## 56. NOT IN vs <> ALL

Similarly:

```
SQL
X NOT IN (subquery)
```

corresponds conceptually to:

```
SQL
X <> ALL (subquery)
```

Again:

**NULL makes this dangerous.**

---

## 57. SOLVED GATE-STYLE EXAMPLE 1 — ANY

Subquery returns:

TEXT

10  
20  
30  
40

Evaluate:

TEXT

25 > ANY(...)

Since:

TEXT

2 5 > 1 0 !' T R U E

overall TRUE.

Shortcut:

\[ 25>MIN=10 \]

Yes.

## 58. SOLVED EXAMPLE 2 — ALL

Same set:

TEXT

10  
20  
30  
40

Evaluate:

TEXT

25 > ALL(...)

Need:

TEXT

25>40?

No.

Therefore FALSE.

Shortcut:

\[ 25>MAX=40? \]

No.



## 59. SOLVED EXAMPLE 3

Evaluate:

TEXT

$25 < \text{ANY}(10, 20, 30, 40)$

Need one value larger than 25.

TEXT

2 5 < 3 0 '

TRUE.

Shortcut:

$\neg [25 < \text{MAX}(40)]$

TRUE.

## 60. SOLVED EXAMPLE 4

Evaluate:

TEXT

$25 < \text{ALL}(10, 20, 30, 40)$

Must be less than every value.

But:

TEXT

2 5 < 1 0 '

FALSE.

Shortcut:

$\neg [25 < \text{MIN}(10)?]$

FALSE.

## 61. SOLVED EXAMPLE 5 — EXISTS

STUDENT:

TEXT

1 Ravi  
2 Anu  
3 Raj

ENROLLMENT:

TEXT

1 DBMS  
1 OS  
3 DBMS

Query:

SQL

```
SELECT S.Name
FROM STUDENT S
WHERE EXISTS
(
    SELECT 1
    FROM ENROLLMENT E
    WHERE E.Sid=S.Sid
);
```

Result:

TEXT

Ravi  
Raj

## 62. SOLVED EXAMPLE 6 — NOT EXISTS

Same tables:

SQL

```
SELECT S.Name
FROM STUDENT S
WHERE NOT EXISTS
(
    SELECT 1
    FROM ENROLLMENT E
    WHERE E.Sid=S.Sid
);
```

Result:

TEXT

Anu

## 63. SOLVED EXAMPLE 7 — NULL + NOT IN

Query:

SQL

```
SELECT X
FROM R
WHERE X NOT IN (1,2,NULL);
```

Suppose  $X = 3$ .

TEXT

```
3 < > 1 !' TRUE
3 < > 2 !' TRUE
3 < > NULL !' UNKNOWN
```

AND:

TEXT

```
TRUE AND TRUE AND UNKNOWN
= UNKNOWN
```

WHERE keeps only TRUE.

Therefore X=3 is **not returned**.

This is exactly the kind of SQL question GATE can use to trap you.

## 64. SOLVED EXAMPLE 8 — CORRELATED AVG

EMPLOYEE:

| Name | Dept | Salary |
|------|------|--------|
| A    | CSE  | 100    |
| B    | CSE  | 50     |
| C    | AIML | 90     |
| D    | AIML | 70     |

Query:

SQL

```
SELECT E1.Name
FROM EMPLOYEE E1
WHERE E1.Salary >
(
    SELECT AVG(E2.Salary)
    FROM EMPLOYEE E2
    WHERE E2.Dept = E1.Dept
);
```

**CSE average**

\[ (100+50)/2=75 \]

A:

TEXT

```
1 0 0 > 7 5 '
```

B:

TEXT

5 0 > 7 5 '

### AIML average

\[ (90+70)/2=80 \]

C:

TEXT

9 0 > 8 0 '

D:

TEXT

7 0 > 8 0 '

Result:

TEXT

A  
C

---

## 65. GATE SHORTCUT — HOW TO RECOGNIZE EACH

---

If question says:

**“belongs to one of these”**

Think:

TEXT

IN

**“at least one related record”**

Think:

TEXT

EXISTS

**“no related record”**

Think:

TEXT

NOT EXISTS

**“greater than at least one”**

Think:

TEXT

> ANY

**“greater than everyone”**

Think:

TEXT

> ALL

**“all required items”**

Think:

TEXT

double NOT EXISTS

**inner query uses outer alias**

Think:

TEXT

correlated subquery

---

## 66. GATE TRAPS

### & pTrap 1 — EXISTS

EXISTS asks whether **a row exists**.

The actual SELECT expression inside usually doesn't matter.

### & pTrap 2 — ANY

ANY means **at least one**, not all.

### & pTrap 3 — ALL

Every comparison must hold.

### & pTrap 4 — > ANY

Usually think:

TEXT

> minimum

not maximum.

---

## & pTrap 5 — > ALL

---

Think:

TEXT

> maximum

---

## & pTrap 6 — correlated subquery

---

If inner query references outer alias, do not evaluate inner query only once.

---

---

## & pTrap 7 — NULL

---

SQL

A = NULL

is wrong logic.

Use:

SQL

A IS NULL

---

## & pTrap 8 — WHERE

---

WHERE keeps only:

TEXT

TRUE

Not:

TEXT

UNKNOWN

---

## & pTrap 9 — NOT IN with NULL

---

Very dangerous.

A NULL in the returned set can cause ordinary candidates to evaluate to UNKNOWN.

---

---

## & pTrap 10 — EXISTS vs JOIN

---

EXISTS doesn't multiply outer rows due to multiple matches.

---

JOIN can.

## & pTrap 11 — empty sets

TEXT

```
A N Y ( e m p t y ) ! '   F A L S E
```

```
A L L ( e m p t y ) ! '   T R U E
```

## & pTrap 12 — aggregate ties

SQL

```
Salary = MAX(Salary)
```

can return several employees.

## 67. NOTEBOOK NOTES

### SUBQUERY

Query inside another query.

#### Non-correlated

Independent inner query.

#### Correlated

Inner query depends on outer row.

Detection:

TEXT

```
inner query references outer alias
```

### IN

TEXT

```
Membership in result set
```

Equivalent intuition:

TEXT

```
= a OR = b OR ...
```

## EXISTS

TEXT

```
TRUE if subquery returns "e1 row"
```

## NOT EXISTS

TEXT

```
TRUE if subquery returns 0 rows
```

## ANY / SOME

At least one comparison must be TRUE.

For non-empty non-NULL S:

TEXT

```
X > ANY ( S ) !' X > MIN ( S )
```

```
X < ANY ( S ) !' X < MAX ( S )
```

## ALL

All comparisons must be TRUE.

TEXT

```
X > ALL ( S ) !' X > MAX ( S )
```

```
X < ALL ( S ) !' X < MIN ( S )
```

## EMPTY-SET RULE

TEXT

```
ANY(empty) = FALSE
```

```
ALL(empty) = TRUE
```

## NULL

Use:

SQL

```
IS NULL
```

```
IS NOT NULL
```

not:



```
SQL
= NULL
<> NULL
```

SQL logic:

```
TEXT
TRUE
FALSE
UNKNOWN
```

WHERE retains only TRUE.

---

## LOGIC SHORTCUT

```
TEXT
FALSE dominates AND
TRUE dominates OR
NOT UNKNOWN = UNKNOWN
```

---

## NOT IN + NULL

```
TEXT
NOT IN containing NULL
!' possible UNKNOWN
!' major trap
```

---

## “ALL” QUERY

Think:

```
TEXT
ALL
=
NO missing item
=
NOT EXISTS an exception
```

Often implemented with:

```
TEXT
NOT EXISTS
NOT EXISTS
```

---

## MEMORY PAGE

Memorize:

TEXT

IN = MEMBERSHIP

EXISTS = SOME ROW EXISTS

NOT EXISTS = NONE

ANY = SOME

ALL = EVERY

CORRELATED = INNER NEEDS OUTER

NULL = UNKNOWN

WHERE = TRUE ONLY

ALL required  
= NO missing item exists

And the four comparison tricks:

TEXT

> ANY !' > MIN  
< ANY !' < MAX

> ALL !' > MAX  
< ALL !' < MIN

## 30-SECOND REVISION

TEXT

Subquery = query inside query

Correlated:  
inner references outer row

IN = membership

EXISTS = at least one row

NOT EXISTS = zero rows

ANY = at least one comparison

ALL = every comparison

> ANY !' > MIN  
> ALL !' > MAX

< ANY !' < MAX  
< ALL !' < MIN

ANY(empty) = FALSE

ALL(empty) = TRUE

NULL comparisons !' UNKNOWN

Use IS NULL

WHERE keeps TRUE only

NOT IN + NULL = dangerous

ALL/every query:

NOT EXISTS an exception

---

## CHECK YOUR UNDERSTANDING

---

### Q1

Subquery returns:

TEXT

10, 20, 30, 40

Evaluate:

TEXT

25 > ANY(...)

TRUE or FALSE?

---

### Q2

Evaluate:

TEXT

25 > ALL(10, 20, 30, 40)

---

### Q3

What is the shortcut for:

TEXT

X < ALL(S)

assuming S is non-empty and has no problematic NULLs?

A. X < MAX(S) B. X < MIN(S) C. X > MIN(S) D. X > MAX(S)

---

### Q4

What does:

SQL

WHERE EXISTS

(

SELECT 1

FROM ENROLLMENT E

```
WHERE E.Sid=S.Sid
)
```

test?

A. Exactly one enrollment exists B. At least one matching enrollment exists C. Every enrollment matches D. Enrollment count equals 1

## Q5

What is wrong with:

```
SQL
WHERE Phone = NULL
```

and what should replace it?

## Q6 — Important GATE trap

Evaluate conceptually:

```
TEXT
3 NOT IN (1, 2, NULL)
```

Does it evaluate to TRUE, FALSE, or UNKNOWN?

## Q7

If a subquery returns **zero tuples**, what are:

```
TEXT
X > ANY(subquery)
```

and

```
TEXT
X > ALL(subquery)
```

?

## Q8 — Most important

Complete this sentence:

A student has completed all required courses if there does not exist a \_\_\_\_\_ course for which there does not exist a matching \_\_\_\_\_ record.

Next we finish the remaining SQL core:

# SQL — PART 4

Set operations ( `UNION`, `INTERSECT`, `EXCEPT` ) ! `UNION ALL` ! views ! DDL/DML !  
CREATE/INSERT/UPDATE/DELETE ! constraints ! PRIMARY KEY/FOREIGN KEY/UNIQUE/CHECK ! referential  
actions ! DELETE vs DROP vs TRUNCATE ! final SQL master revision.

Type `NEXT`.

```
USER
NEXT
```

ASSISTANT

## SQL — PART 4: SET OPERATIONS, VIEWS, DDL/DML, CONSTRAINTS

This section completes the main SQL coverage from your GATE DBMS syllabus.

We will cover:

UNION ! UNION ALL ! INTERSECT ! EXCEPT ! CREATE ! INSERT ! UPDATE ! DELETE ! PRIMARY KEY !  
FOREIGN KEY ! UNIQUE ! CHECK ! referential actions ! views ! DELETE vs TRUNCATE vs DROP ! SQL  
master revision

### 1. SQL SET OPERATIONS

Suppose we have:

R

| A |  |
|---|--|
| 1 |  |
| 2 |  |
| 3 |  |

S

| A |  |
|---|--|
| 3 |  |
| 4 |  |
| 5 |  |

SQL provides set-style operations.

Main ones:

TEXT

UNION  
UNION ALL  
INTERSECT  
EXCEPT

## 2. UNION

**UNION** combines rows from two query results and removes duplicates.

SQL

```
SELECT A  
FROM R
```

UNION

```
SELECT A  
FROM S;
```

Result:

TEXT

1  
2  
3  
4  
5

Notice:

TEXT

3

appears only once.

## MEMORY TRICK

**UNION = Combine + remove duplicates**

This corresponds closely to mathematical set union.

## 3. UNION ALL

**UNION ALL** combines the results **without removing duplicates**.

SQL

```
SELECT A  
FROM R
```

UNION ALL

```
SELECT A
FROM S;
```

Result:

TEXT

```
1
2
3
3
4
5
```

## UNION vs UNION ALL

| UNION                             | UNION ALL                   |
|-----------------------------------|-----------------------------|
| Removes duplicates                | Keeps duplicates            |
| Set-like semantics                | Bag/multiset-like           |
| May require duplicate elimination | Usually simpler             |
| Cardinality can decrease          | Cardinalities generally add |

## IMPORTANT CARDINALITY

If:

TEXT

```
|R| = m
|S| = n
```

then:

### UNION ALL

$|R \cup_{ALL} S| = m + n$

because every result row is retained.

### UNION

$|$

$R \cup S$

$= m + n - \text{duplicates across results}$

More exactly under set semantics:

$|$

$R \setminus S$

=

R

+

S

-

$R \setminus S$

$\setminus$

if each input itself represents sets.

## GATE TRAP

Do not assume:

SQL  
UNION

keeps duplicates.

It removes them.

For duplicates retained, use:

SQL  
UNION ALL

## 4. INTERSECT

**INTERSECT** returns rows appearing in both result sets.

SQL  
SELECT A  
FROM R  
  
INTERSECT  
  
SELECT A  
FROM S;

Result:

TEXT  
3

Conceptually:

$\setminus R \setminus S \setminus$

## 5. EXCEPT

**EXCEPT** returns rows in the first query but not in the second.



SQL

```
SELECT A
FROM R
```

EXCEPT

```
SELECT A
FROM S;
```

Result:

TEXT

```
1
2
```

Conceptually:

$\setminus R - S \setminus$

Some DBMS products historically use `MINUS` for similar functionality, but for GATE focus on the concept rather than vendor syntax unless specified.

## 6. SET OPERATION COMPATIBILITY

For set operations, the query results must be structurally compatible.

Typically:

1. Same number of output columns.
2. Corresponding columns should have compatible data types/domains.

Example:

Valid conceptually:

SQL

```
SELECT Sid
FROM STUDENT
```

UNION

```
SELECT Sid
FROM ALUMNI;
```

Both return one compatible column.

Invalid conceptually:

SQL

```
SELECT Sid, Name
FROM STUDENT
```

UNION

```
SELECT CourseID
FROM COURSE;
```

because:

TEXT

2 columns vs 1 column

## 7. SET OPERATIONS MEMORY

TEXT

```
U N I O N      !'   e i t h e r
I N T E R S E C T  !'   b o t h
E X C E P T      !'   f i r s t   b u t   n o t   s e c o n d
U N I O N   A L L  !'   c o m b i n e   e v e r y t h i n g
```

## 8. DDL vs DML

SQL commands are often classified into categories.

For GATE, understand the distinction.

### DDL

#### Data Definition Language

Used for database/schema structure.

Examples:

TEXT

```
CREATE
ALTER
DROP
TRUNCATE
```

### DML

#### Data Manipulation Language

Used to manipulate table data.

Examples:

TEXT

```
INSERT
UPDATE
DELETE
SELECT
```

Classification of `SELECT` is sometimes separately called DQL in teaching material, but for GATE focus on what each statement does rather than getting trapped by naming conventions.

## MEMORY

### TEXT

D D L ! ' D e s i g n / D e f i n i t i o n

D M L ! ' D a t a m a n i p u l a t i o n

## 9. CREATE TABLE

Used to create a relation/table.

Example:

### SQL

```
CREATE TABLE STUDENT
(
    Sid INT,
    Name VARCHAR(50),
    Marks INT
);
```

This defines the schema.

It does not automatically insert rows.

## 10. DATATYPES

Common conceptual SQL data types:

### TEXT

INT  
CHAR(n)  
VARCHAR(n)  
DECIMAL  
DATE

GATE generally focuses more on query semantics than vendor-specific data types.

## 11. INSERT

Adds rows.

Example:

### SQL

```
INSERT INTO STUDENT
VALUES (1, 'Ravi', 85);
```

Or:

SQL

```
INSERT INTO STUDENT(Sid, Name, Marks)
VALUES (2, 'Anu', 92);
```

## 12. UPDATE

Modifies existing rows.

SQL

```
UPDATE STUDENT
SET Marks = 90
WHERE Sid = 1;
```

Very important:

TEXT

WHERE

determines which rows are modified.

Without WHERE:

SQL

```
UPDATE STUDENT
SET Marks = 90;
```

all rows are updated.

## & SQL TRAP

Forget `WHERE` in UPDATE:

**all rows can be affected.**

## 13. DELETE

Deletes rows.

SQL

```
DELETE FROM STUDENT
WHERE Sid = 1;
```

Only matching rows removed.

Without WHERE:

SQL

```
DELETE FROM STUDENT;
```

all rows are removed, but the table structure remains.

---

## 14. CONSTRAINTS

---

Your GATE syllabus explicitly includes integrity constraints.

SQL lets us define constraints such as:

```
TEXT
PRIMARY KEY
FOREIGN KEY
UNIQUE
NOT NULL
CHECK
```

These enforce correctness rules.

---

## 15. PRIMARY KEY

---

Example:

```
SQL
CREATE TABLE STUDENT
(
    Sid INT PRIMARY KEY,
    Name VARCHAR(50)
);
```

A primary key must:

```
TEXT
uniquely identify each row
```

and:

```
TEXT
cannot be NULL
```

This follows entity integrity.

---

## 16. COMPOSITE PRIMARY KEY

---

Example:

```
SQL
CREATE TABLE ENROLLMENT
(
    Sid INT,
    CourseID INT,
    Grade CHAR(2),

    PRIMARY KEY (Sid, CourseID)
```

```
);
```

Here:

```
TEXT
(Sid, CourseID)
```

together uniquely identify a row.

## 17. UNIQUE

Example:

```
SQL
CREATE TABLE STUDENT
(
    Sid INT PRIMARY KEY,
    Email VARCHAR(100) UNIQUE
);
```

Email values must be unique among applicable stored values, subject to DBMS-specific NULL handling.

For GATE:

**UNIQUE enforces uniqueness, but it is not exactly the same as PRIMARY KEY.**

## PRIMARY KEY vs UNIQUE

| PRIMARY KEY                          | UNIQUE                                                      |
|--------------------------------------|-------------------------------------------------------------|
| Identifies rows                      | Enforces uniqueness                                         |
| Cannot be NULL                       | NULL handling varies by SQL implementation/standard context |
| One primary key constraint per table | Multiple UNIQUE constraints possible                        |
| Entity identity                      | Alternate uniqueness constraint                             |

For exam safety:

**Do not say UNIQUE and PRIMARY KEY are identical.**

## 18. NOT NULL

Example:

SQL

```
Name VARCHAR(50) NOT NULL
```

Means:

**Name must have a non-NULL value.**

## 19. CHECK

Used to enforce a condition.

Example:

SQL

```
CREATE TABLE STUDENT
(
    Sid INT PRIMARY KEY,
    Marks INT CHECK (Marks >= 0 AND Marks <= 100)
);
```

Invalid:

TEXT

```
Marks = 150
```

because CHECK fails.

## MEMORY

TEXT

```
CHECK = value must satisfy condition
```

## 20. FOREIGN KEY

Example:

SQL

```
CREATE TABLE DEPARTMENT
(
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(50)
);
```

SQL

```
CREATE TABLE STUDENT
(
    Sid INT PRIMARY KEY,
    Name VARCHAR(50),
```

```
DeptID INT,  
  
FOREIGN KEY (DeptID)  
REFERENCES DEPARTMENT(DeptID)  
);
```

Now:

```
TEXT  
STUDENT.DeptID
```

references:

```
TEXT  
DEPARTMENT.DeptID
```

---

## 21. REFERENTIAL INTEGRITY

---

Suppose DEPARTMENT contains:

```
TEXT  
10  
20  
30
```

Then inserting:

```
TEXT  
STUDENT DeptID = 20
```

is valid.

But:

```
TEXT  
DeptID = 99
```

is invalid if 99 does not exist in the referenced relation.

Unless:

```
TEXT  
DeptID = NULL
```

and NULL is permitted.

---

## MEMORY

---

**Foreign key must point to an existing referenced key value, or be NULL if allowed.**



---

## 22. WHAT HAPPENS IF REFERENCED ROW IS DELETED?

---

Suppose:

```
TEXT
DEPARTMENT
DeptID = 10
```

is referenced by many STUDENT rows.

Now someone tries:

```
SQL
DELETE FROM DEPARTMENT
WHERE DeptID = 10;
```

What should happen to students referencing 10?

SQL provides referential actions.

Important concepts:

```
TEXT
RESTRICT / NO ACTION
CASCADE
SET NULL
SET DEFAULT
```

---

## 23. ON DELETE CASCADE

---

Example:

```
SQL
FOREIGN KEY (DeptID)
REFERENCES DEPARTMENT(DeptID)
ON DELETE CASCADE
```

If department 10 is deleted:

**referencing STUDENT rows are also deleted.**

---

## MEMORY

---

**CASCADE = deletion flows down.**

---

## 24. ON DELETE SET NULL

---

SQL

```
ON DELETE SET NULL
```

If referenced department disappears:

TEXT

```
S T U D E N T . D e p t I D ! ' N U L L
```

provided the foreign key column allows NULL.

## 25. RESTRICT / NO ACTION

Conceptually:

**Prevent deletion if rows still reference the key.**

So parent deletion is rejected unless references are handled.

## 26. ON UPDATE

Similar actions may be defined for updates to referenced key values.

Example conceptually:

SQL

```
ON UPDATE CASCADE
```

If referenced key changes:

TEXT

```
o l d   D e p t I D ! '   n e w   D e p t I D
```

foreign-key references may change correspondingly.

## REFERENTIAL ACTION MEMORY

TEXT

CASCADE

```
! '   c h a n g e / d e l e t e   c h i l d   r e f e r e n c e s   t o o
```

SET NULL

```
! '   c h i l d   F K   b e c o m e s   N U L L
```

RESTRICT / NO ACTION

```
! '   p r e v e n t   o p e r a t i o n
```

## 27. ENTITY INTEGRITY vs REFERENTIAL INTEGRITY

## Entity Integrity

Primary key:

```
TEXT
cannot be NULL
```

## Referential Integrity

Foreign key:

```
TEXT
must match referenced key
or be NULL if permitted
```

## MEMORY TRICK

```
TEXT
Entity Integrity
!' i d e n t i t y   o f   O W N   r o w

Referential Integrity
!' r e l a t i o n s h i p   t o   O T H E R   r o w
```

## 28. DOMAIN CONSTRAINT

Controls allowed values.

Example:

```
SQL
Marks INT CHECK (Marks BETWEEN 0 AND 100)
```

This constrains the domain.

## 29. KEY CONSTRAINT

Ensures uniqueness of candidate/key attributes.

Examples:

```
TEXT
PRIMARY KEY
UNIQUE
```

## 30. COMPLETE INTEGRITY CONSTRAINT SUMMARY

#### TEXT

Domain Constraint

! ' v a l i d v a l u e r a n g e / t y p e

Key Constraint

! ' u n i q u e n e s s

Entity Integrity

! ' P K c a n n o t b e N U L L

Referential Integrity

! ' F K m u s t r e f e r e n c e e x i s t i n g k e y o r a l l o w e d N U L L

This is a must-write summary.

## 31. VIEW

A **view** is a virtual relation defined by a query.

Example:

#### SQL

```
CREATE VIEW CSE_STUDENTS AS
SELECT Sid, Name, Marks
FROM STUDENT
WHERE Dept = 'CSE';
```

Then:

#### SQL

```
SELECT *
FROM CSE_STUDENTS;
```

returns the query-defined result.

## 32. WHY VIEWS ARE USEFUL

Views can provide:

1. Simplification
2. Security
3. Customized user representation
4. Logical abstraction

Example:

Students should not see:

#### TEXT

Salary

Personal data

Administrative fields

A view can expose only permitted columns.

---

## 33. VIEW vs BASE TABLE

---

### Base table

Actually stores data.

### View

Generally defined through a query over base tables.

```
TEXT
Base relation
! "
Query
! "
View
```

---

## 34. IS A VIEW ALWAYS PHYSICALLY STORED?

---

A normal logical SQL view generally stores its **definition**, not necessarily a separate physical copy of all result rows.

When queried, its result can be derived from base relations.

There are also materialized-view concepts in some systems, but those are beyond what we need unless explicitly asked.

---

## 35. VIEW UPDATE PROBLEM

---

Not every view is easily updatable.

Simple view:

```
SQL
CREATE VIEW V AS
SELECT Sid, Name
FROM STUDENT;
```

may be straightforward to map back.

But consider:

```
SQL
CREATE VIEW V AS
SELECT Dept, AVG(Marks)
FROM STUDENT
GROUP BY Dept;
```

If you try to change:

TEXT

AVG(Marks) from 80 to 90

which underlying students' Marks should change?

Ambiguous.

Therefore aggregate/grouped views are generally not directly updatable in the simple sense.

## GATE TRAP

Views involving:

TEXT

GROUP BY  
aggregate functions  
DISTINCT  
complex joins

may not be straightforwardly updatable.

Do not assume every view supports arbitrary updates.

## 36. DELETE vs TRUNCATE vs DROP

This is a common comparison question.

## DELETE

SQL

DELETE FROM STUDENT;

Removes rows.

Table structure remains.

Can use:

TEXT

WHERE

to remove selected rows.

## TRUNCATE

SQL

TRUNCATE TABLE STUDENT;

Removes all rows from the table.

Typically:

- no row-selection `WHERE`
- table structure remains
- treated as DDL in many standard classifications

## DROP

SQL

```
DROP TABLE STUDENT;
```

Removes the table itself.

That means:

TEXT

data + table definition

are removed.

## MASTER COMPARISON

| DELETE        | TRUNCATE                   | DROP                    |
|---------------|----------------------------|-------------------------|
| Removes rows  | Removes all rows           | Removes table           |
| Can use WHERE | No WHERE for selected rows | No row filtering        |
| Table remains | Table remains              | Table structure removed |
| DML           | Usually DDL                | DDL                     |

## MEMORY TRICK

**DELETE = Delete data**

**TRUNCATE = Empty table**

**DROP = Destroy table**

## 37. IMPORTANT CAVEAT

Transactional/rollback behavior of `TRUNCATE` can depend on the DBMS.

For GATE, unless the question explicitly gives a DBMS-specific behavior, focus on:

TEXT

```
DELETE !' rows
TRUNCATE !' all rows, structure retained
DROP !' structure + data removed
```

Avoid memorizing oversimplified vendor-specific claims such as “TRUNCATE can never rollback.”

## 38. ALTER TABLE

Changes table structure.

Examples conceptually:

SQL

```
ALTER TABLE STUDENT
ADD Email VARCHAR(100);
```

This changes the schema.

Remember our fundamentals:

**Adding an attribute changes schema, not merely the instance.**

## 39. SQL SET-OPERATIONEXAMPLE

R:

TEXT

```
1
1
2
3
```

S:

TEXT

```
2
3
3
4
```

Suppose queries preserve duplicate rows before the set operator.

## UNION

Distinct result:

TEXT

```
1
2
3
4
```



## UNION ALL

TEXT

1  
1  
2  
3  
2  
3  
3  
4

8 rows.

## INTERSECT

Under ordinary set form:

TEXT

2  
3

## EXCEPT

First result not in second:

TEXT

1

## 40. GATE-STYLE EXAMPLE — REFERENTIAL INTEGRITY

DEPARTMENT:

| DeptID |
|--------|
| 10     |
| 20     |

STUDENT:

| Sid | DeptID |
|-----|--------|
| 1   | 10     |
| 2   | 20     |

Suppose foreign key:

TEXT

STUDENT.DeptID  
!' DEPARTMENT . DeptID

Can we insert:

TEXT

Sid = 3  
DeptID = 30

No, unless 30 first exists in DEPARTMENT.

Why?

Referential integrity violation.

---

## 41. GATE-STYLE EXAMPLE — ENTITY INTEGRITY

Suppose Sid is PRIMARY KEY.

Can we insert:

TEXT

Sid = NULL  
Name = Ravi

No.

Because:

**Primary key cannot be NULL.**

---

## 42. GATE-STYLE EXAMPLE — UNIQUE

Suppose:

TEXT

Email UNIQUE

and existing row has:

TEXT

a@gmail.com

Can another non-NULL row store:

TEXT

a@gmail.com

No.

It violates uniqueness.

---

## 43. GATE-STYLE EXAMPLE — CHECK

Constraint:

SQL

```
CHECK (Age >= 18)
```

Attempt:

TEXT

```
Age = 15
```

Fails CHECK.

---

## 44. GATE-STYLE EXAMPLE — CASCADE

Suppose:

TEXT

```
DEPARTMENT 10
```

has 100 referencing STUDENT rows.

Foreign key includes:

SQL

```
ON DELETE CASCADE
```

Deleting department 10 causes those referencing rows to be deleted as part of cascading referential action.

---

## 45. SQL MASTER LOGICAL EXECUTION ORDER

Let's now consolidate SQL.

For a typical query:

SQL

```
SELECT DISTINCT ...  
FROM ...  
JOIN ...  
ON ...  
WHERE ...  
GROUP BY ...  
HAVING ...  
ORDER BY ...
```

A useful logical model is:

TEXT

```
FROM / JOIN      !"  
ON matching      !"  
WHERE            !"  
GROUP BY         !"
```

HAVING  
SELECT  
DISTINCT  
ORDER BY

This is the core output-prediction order.

## 46. MASTER SQL CONNECTION TO RELATIONAL ALGEBRA

| SQL            | Relational Algebra idea |
|----------------|-------------------------|
| FROM           | Source relation         |
| WHERE          | Selection $\sigma$      |
| SELECT columns | Projection $\pi$        |
| JOIN           | Join $\bowtie$          |
| UNION          | Union $\cup$            |
| INTERSECT      | Intersection $\cap$     |
| EXCEPT         | Difference $-$          |

Important difference:

TEXT  
RA projection !' duplicates removed  
SQL SELECT !' duplicates may remain

Need:

SQL  
DISTINCT

to remove duplicates.

## 47. COMPLETE SQL GATE TRAP SHEET

### & Trap 1

SQL  
SELECT

does not automatically remove duplicates.

## & ⚠Trap 2

SQL

```
DISTINCT A, B
```

means distinct **pairs**, not independent distinct values per column.

## & ⚠Trap 3

TEXT

```
WHERE !' rows
HAVING !' groups
```

## & ⚠Trap 4

TEXT

```
COUNT ( * ) !' all rows
COUNT ( A ) !' non - NULL A
COUNT(DISTINCT A)
!' unique non - NULL A
```

## & ⚠Trap 5

Most aggregate functions ignore NULL.

NULL is not zero.

## & ⚠Trap 6

SQL

```
= NULL
```

does not test NULL.

Use:

SQL

```
IS NULL
```

## & ⚠Trap 7

WHERE retains only TRUE.

FALSE and UNKNOWN are rejected.

---

## & ⚠Trap 8

TEXT

NOT UNKNOWN = UNKNOWN

---

## & ⚠Trap 9

NOT IN with NULL is dangerous.

---

## & ⚠Trap 10

EXISTS checks whether at least one row exists, not its selected value.

---

## & ⚠Trap 11

Correlated subquery references the outer query.

---

## & ⚠Trap 12

TEXT

ANY = at least one

ALL = every

---

## & ⚠Trap 13

For clean non-empty sets:

TEXT

> ANY !' > MIN

> ALL !' > MAX

< ANY !' < MAX

< ALL !' < MIN

---

## & ⚠Trap 14

TEXT

ANY(empty) = FALSE

ALL(empty) = TRUE

---

## & ⚠Trap 15

Natural join uses **all same-named columns**.

## & pTrap 16

LEFT JOIN preserves unmatched left rows, but a later WHERE condition may remove them.

## & pTrap 17

Multiple matches in joins multiply rows.

## & pTrap 18

TEXT

```
UNION !' removes duplicates
UNION ALL !' keeps duplicates
```

## & pTrap 19

Primary key cannot be NULL.

## & pTrap 20

Foreign key can repeat.

## 48. SQL QUESTION-TYPE RECOGNITION

When GATE says:

**“unique output”**

think:

SQL

DISTINCT

When it says:

**“department-wise”**

think:

SQL

GROUP BY

When it says:

**“groups whose count...”**

think:

SQL  
HAVING

When it says:

**“at least one related tuple”**

think:

SQL  
EXISTS

When it says:

**“none”**

think:

SQL  
NOT EXISTS

When it says:

**“all required”**

think:

TEXT  
double NOT EXISTS

When it says:

**“greater than every value”**

think:

SQL  
> ALL

When it says:

**“greater than at least one”**

think:



SQL  
> ANY

When it says:

**“combine query outputs without duplicates”**

think:

SQL  
UNION

When it says:

**“combine and preserve duplicates”**

think:

SQL  
UNION ALL

---

## SQL CHAPTER REVISION SHEET

---

### 1. Query Core

---

TEXT  
SELECT  
FROM  
WHERE  
GROUP BY  
HAVING  
DISTINCT  
ORDER BY

Logical order:

TEXT  
FROM/JOIN  
! ' W H E R E  
! ' G R O U P  
! ' H A V I N G  
! ' S E L E C T  
! ' D I S T I N C T  
! ' O R D E R

---

### 2. Aggregates

---

TEXT

COUNT

SUM

AVG

MIN

MAX

Remember:

TEXT

COUNT(\*) = rows

COUNT(A) = non-NULL

AVG = ignores NULL

---

### 3. Joins

TEXT

CROSS !' all combinations

INNER !' matches

LEFT !' keep left

RIGHT !' keep right

FULL !' keep both

SELF !' same table, aliases

NATURAL !' all same-name attributes

---

### 4. Subqueries

TEXT

IN = membership

EXISTS = at least one

NOT EXISTS = none

ANY = some

ALL = every

Correlated = inner uses outer alias

---

### 5. NULL

TEXT

NULL = unknown/missing

Use IS NULL

WHERE keeps TRUE only

Three-valued logic:

TEXT

TRUE

FALSE

UNKNOWN

## 6. Set Operations

TEXT

UNION

INTERSECT

EXCEPT

UNION ALL

## 7. Constraints

TEXT

PRIMARY KEY

FOREIGN KEY

UNIQUE

NOT NULL

CHECK

## 8. Integrity

TEXT

Domain !' allowed values

Key !' uniqueness

Entity !' PK not NULL

Referential !' valid FK reference

## SQL 5-MINUTE REVISION

Memorize this compact chain:

#### TEXT

SELECT alone keeps duplicates  
DISTINCT removes them

WHERE filters rows  
HAVING filters groups

GROUP BY creates buckets

COUNT(\*) counts rows  
COUNT(A) ignores NULL

JOIN duplicates multiply

LEFT keeps left  
RIGHT keeps right  
FULL keeps both

EXISTS = some  
NOT EXISTS = none  
ANY = some comparison  
ALL = every comparison

NULL gives UNKNOWN  
WHERE keeps TRUE only

NOT IN + NULL = danger

UNION removes duplicates  
UNION ALL keeps duplicates

PK = unique + not NULL  
FK = reference

DELETE = rows  
TRUNCATE = empty table  
DROP = remove table

## 30-SECOND REVISION

#### TEXT

UNION !' combine, remove duplicates  
UNION ALL !' combine, keep duplicates  
INTERSECT !' common  
EXCEPT !' first minus second

DDL !' schema  
DML !' data

PK !' unique + non-NULL  
FK !' reference  
CHECK !' condition  
UNIQUE !' uniqueness

Entity integrity:  
PK cannot NULL

Referential integrity:  
FK must match or allowed NULL

View = query-defined virtual relation

```
DELETE !' selected / all rows
TRUNCATE !' all rows, structure remains
DROP !' table removed
```

## CHECK YOUR UNDERSTANDING

### Q1

R returns 5 rows and S returns 7 rows.

How many rows does:

SQL

R-query  
UNION ALL  
S-query

return?

Assume the inputs themselves contain those row counts.

### Q2

Which removes duplicates?

A. UNION ALL B. UNION C. Both D. Neither

### Q3

A primary key can:

A. contain duplicates B. contain NULL C. uniquely identify tuples and cannot be NULL D. always be a foreign key

### Q4

Suppose:

TEXT

STUDENT.DeptID

references:

TEXT

DEPARTMENT.DeptID

and an inserted DeptID does not exist in DEPARTMENT.

Which integrity constraint is violated?

### Q5

What is the difference between:

```
SQL
DELETE FROM STUDENT;
```

and:

```
SQL
DROP TABLE STUDENT;
```

## Q6

What does:

```
SQL
ON DELETE CASCADE
```

do conceptually?

## Q7

Which one generally represents a virtual/query-defined relation?

A. View B. Primary key C. Foreign key D. Domain

We have now completed the **core SQL block**.

The next major chapter is one of the most important in all of GATE DBMS:

## TOPIC 7 — FUNCTIONAL DEPENDENCIES + KEYS FOR NORMALIZATION

We will start from the absolute beginning:

**Functional Dependency !' determinant !' trivial/non-trivial/completely non-trivial FD !' Armstrong axioms !' inference rules !' attribute closure !' candidate-key detection !' prime/non-prime attributes !' FD closure !' GATE shortcuts.**

This chapter will form the foundation for **1NF, 2NF, 3NF, BCNF, minimal cover, lossless decomposition and dependency preservation.**

Type **`NEXT`**.

```
USER
NEXT
```

ASSISTANT

## TOPIC 7 — FUNCTIONAL DEPENDENCIES + ATTRIBUTE CLOSURE + CANDIDATE KEYS

We are now entering one of the **most important GATE DBMS chapters**. Functional dependencies are the foundation

for **candidate keys, minimal cover, 2NF, 3NF, BCNF, lossless decomposition, and dependency preservation**, all needed to handle the “normal forms” portion of your syllabus.

The learning chain is:

TEXT

Functional Dependency

!"

Armstrong's Axioms

!"

Attribute Closure

!"

Superkeys / Candidate Keys

!"

Prime / Non-prime Attributes

!"

Minimal Cover

!"

Normal Forms

!"

Decomposition

So this topic must be strong.

## 1. WHY FUNCTIONAL DEPENDENCY EXISTS

Consider:

### STUDENT

| RollNo | Name | Dept | HOD   |
|--------|------|------|-------|
| 101    | Ravi | CSE  | Rao   |
| 102    | Anu  | AIML | Kumar |
| 103    | Raj  | CSE  | Rao   |

Look at RollNo .

If we know:

TEXT

RollNo = 101

we can uniquely determine:

TEXT

Name = Ravi

Dept = CSE

So we can say:

$\text{RollNo} \rightarrow \text{Name}$

and:

$\forall \text{ RollNo } \rightarrow \text{Dept}$

Read:

**RollNo functionally determines Name.**

Similarly, suppose every department has exactly one HOD.

Then:

$\forall \text{ Dept } \rightarrow \text{HOD}$

Functional dependencies capture these **relationships between attributes**.

## 2. FUNCTIONAL DEPENDENCY — FD

For relation:

$\forall R$

an FD:

$\forall X \rightarrow Y$

means:

**Whenever two tuples have the same values of X, they must also have the same values of Y.**

Here:

TEXT

X = determinant

Y = dependent attributes

## 3. FORMAL MEANING

Suppose:

$\forall X \rightarrow Y$

Then for every pair of tuples  $(t_1, t_2)$ :

If:

$\forall t_1[X] = t_2[X]$

then:

$\forall t_1[Y] = t_2[Y]$

must also hold.

That is the correct theoretical definition.

## MEMORY TRICK



For:

$\{ X \rightarrow Y \}$

read:

Same X  $\Rightarrow$  Same Y

This is the best simple memory.

## 4. EXAMPLE

Relation:

| StudentID | Name | Dept |
|-----------|------|------|
| 1         | Ravi | CSE  |
| 2         | Anu  | AIML |
| 3         | Raj  | CSE  |

Suppose StudentID is unique.

Then:

$\{ \text{StudentID} \rightarrow \text{Name} \}$

is valid.

Why?

No two tuples having the same StudentID can have different Names.

Similarly:

$\{ \text{StudentID} \rightarrow \text{Dept} \}$

## 5. DOES $\text{Dept} \rightarrow \text{StudentID}$ HOLD?

No.

Because:

TEXT

Dept = CSE

appears for:

TEXT

StudentID = 1

StudentID = 3

Same Dept produces different StudentIDs.

Therefore:

$\nexists \text{ Dept } \rightarrow \text{ StudentID }$

## GATE SHORTCUT

To **disprove** an FD from a given instance:

Find two rows where:

TEXT

Left side same  
Right side different

Then the FD definitely does not hold.

Example:

TEXT

Dept = CSE, StudentID = 1  
Dept = CSE, StudentID = 3

Therefore:

$\nexists \text{ Dept } \rightarrow \text{ StudentID }$

## & IMPORTANT GATE TRAP

If a sample table happens to satisfy:

$A \rightarrow B$

you cannot always conclude that the FD is part of the schema semantics.

A relation instance can accidentally satisfy an FD.

But:

**One counterexample is enough to prove that an FD does not hold on that instance.**

When GATE gives FDs explicitly, treat those as the intended constraints.

## 6. DETERMINANT

In:

$X \rightarrow Y$

**X** is called the **determinant**.

Example:

$\text{RollNo} \rightarrow \text{Name}$

Then:

TEXT

RollNo = determinant

## 7. FD WITH MULTIPLE ATTRIBUTES

Example:

$\{ (StudentID, CourseID) \rightarrow Grade \}$

Meaning:

**Student + Course together determine Grade.**

Neither StudentID nor CourseID individually may determine Grade.

This is very important for normalization.

## 8. TRIVIAL FUNCTIONAL DEPENDENCY

An FD:

$\{ X \rightarrow Y \}$

is **trivial** if:

$\{ Y \subseteq X \}$

Example:

$\{ AB \rightarrow A \}$

because:

TEXT

A  $\rightarrow$  A B

Another:

$\{ ABC \rightarrow BC \}$

also trivial.

## MEMORY TRICK

**RHS already inside LHS ! Trivial.**

## 9. NON-TRIVIAL FD

An FD:

$\{ X \rightarrow Y \}$

is non-trivial if:

$\forall Y \subseteq X$

Example:

$A \rightarrow B$

if A and B are different attributes.

Another:

$AB \rightarrow C$

## 10. COMPLETELY NON-TRIVIAL FD

An FD is completely non-trivial when:

$X \cap Y = \emptyset$

Example:

$AB \rightarrow CD$

if:

TEXT

A, B, C, D

are all distinct.

## 11. TRIVIAL vs NON-TRIVIAL vs COMPLETELY NON-TRIVIAL

| Type                   | Condition              |
|------------------------|------------------------|
| Trivial                | $Y \subseteq X$        |
| Non-trivial            | $Y \not\subseteq X$    |
| Completely non-trivial | $X \cap Y = \emptyset$ |

Example:

$AB \rightarrow A$

Trivial.

$AB \rightarrow BC$

Non-trivial, but not completely non-trivial because B appears on both sides.

$AB \rightarrow CD$

Completely non-trivial.

## 12. WHY TRIVIAL FDs ALWAYS HOLD

Consider:

$\{ AB \rightarrow A \}$

If two tuples have equal values for AB, they obviously have equal A values.

So trivial FDs are guaranteed by reflexivity.

This leads us to Armstrong's axioms.

---

## 13. ARMSTRONG'S AXIOMS

Armstrong's axioms are fundamental inference rules for deriving new functional dependencies.

Three basic axioms:

1. Reflexivity
2. Augmentation
3. Transitivity

These are:

**sound and complete for FD inference.**

Meaning:

- Anything derived using them is valid.
- Every logically implied FD can be derived using them.

For GATE, know both the rules and how to use them.

---

## 14. REFLEXIVITY

If:

$\{ Y \subseteq X \}$

then:

$\{ X \rightarrow Y \}$

Example:

$\{ ABC \rightarrow AB \}$

because:

$\{ AB \subseteq ABC \}$

---

## MEMORY

**Subset on RHS !' Reflexivity**

---

## 15. AUGMENTATION

If:

$\lceil X \rightarrow Y \rceil$

then for any Z:

$\lceil XZ \rightarrow YZ \rceil$

Example:

Given:

$\lceil A \rightarrow B \rceil$

we can infer:

$\lceil AC \rightarrow BC \rceil$

We added C to both sides.

---

## MEMORY

Add same thing to both sides.

---

## 16. TRANSITIVITY

If:

$\lceil X \rightarrow Y \rceil$

and:

$\lceil Y \rightarrow Z \rceil$

then:

$\lceil X \rightarrow Z \rceil$

Example:

$\lceil A \rightarrow B \rceil$

$\lceil B \rightarrow C \rceil$

Therefore:

$\lceil A \rightarrow C \rceil$

---

## MEMORY

TEXT

|       |   |   |
|-------|---|---|
| A     | ! | B |
| B     | ! | C |
| ----- |   |   |
| A     | ! | C |

Same as a dependency chain.

## 17. ARMSTRONG MEMORY

Remember:

**RAT**

TEXT

R !' R e f l e x i v i t y  
A !' A u g m e n t a t i o n  
T !' T r a n s i t i v i t y

Easy mnemonic:

**RAT derives FDs.**

## 18. DERIVED RULE — UNION

If:

$X \rightarrow Y$

and:

$X \rightarrow Z$

then:

$X \rightarrow YZ$

Example:

$A \rightarrow B$

and:

$A \rightarrow C$

then:

$A \rightarrow BC$

## MEMORY

**Same LHS !' combine RHS.**

## 19. DERIVED RULE — DECOMPOSITION

If:

$X \rightarrow YZ$

then:

$X \rightarrow Y$

and:

$\{ X \rightarrow Z \}$

Example:

$\{ A \rightarrow BC \}$

implies:

$\{ A \rightarrow B \}$

and:

$\{ A \rightarrow C \}$

## MEMORY

One RHS can split.

This rule becomes very important for canonical/minimal cover.

## 20. DERIVED RULE — PSEUDOTRANSITIVITY

If:

$\{ X \rightarrow Y \}$

and:

$\{ WY \rightarrow Z \}$

then:

$\{ WX \rightarrow Z \}$

Example:

$\{ A \rightarrow B \}$

and:

$\{ BC \rightarrow D \}$

then:

$\{ AC \rightarrow D \}$

## 21. ARMSTRONG + DERIVED RULES TABLE

| Rule         | Given                          | Infer                |
|--------------|--------------------------------|----------------------|
| Reflexivity  | $\{ Y \vdash X \}$             | $\{ X \vdash Y \}$   |
| Augmentation | $\{ X \vdash Y \}$             | $\{ XZ \vdash YZ \}$ |
| Transitivity | $\{ X \vdash Y, Y \vdash Z \}$ | $\{ X \vdash Z \}$   |



| Rule               | Given                                   | Infer                                  |
|--------------------|-----------------------------------------|----------------------------------------|
| Union              | $\{X \rightarrow Y, X \rightarrow Z\}$  | $\{X \rightarrow YZ\}$                 |
| Decomposition      | $\{X \rightarrow YZ\}$                  | $\{X \rightarrow Y, X \rightarrow Z\}$ |
| Pseudotransitivity | $\{X \rightarrow Y, WY \rightarrow Z\}$ | $\{WX \rightarrow Z\}$                 |

This is worth writing in your notebook.

## 22. FD CLOSURE — $\{F^+\}$

Suppose a set of FDs is:

$F = \{A \rightarrow B, B \rightarrow C\}$

The **closure of F**, written:

$\{F^+\}$

is the set of **all FDs logically implied by F**.

For example, F includes:

$\{A \rightarrow B\}$

$\{B \rightarrow C\}$

$\{A \rightarrow C\}$

plus trivial and augmented dependencies.

## & DO NOT CONFUSE TWO CLOSURES

There are two different ideas:

### Functional dependency closure

$\{F^+\}$

= all FDs implied by F.

### Attribute closure

$\{X^+\}$

= all attributes functionally determined by X under F.

GATE asks attribute closure much more often computationally.

## 23. ATTRIBUTE CLOSURE — $\{X^+\}$

Given:

- relation R
- set of FDs F
- attribute set X

the closure:

$X^+$

means:

**All attributes that can be determined from X using the FDs.**

This is one of the **most important GATE techniques**.

## 24. WHY ATTRIBUTE CLOSURE MATTERS

Attribute closure helps us determine:

1. Is X a superkey?
2. Is X a candidate key?
3. Does FD  $(X \rightarrow Y)$  follow from F?
4. Which attributes are implied by X?

So learn the algorithm properly.

## 25. ATTRIBUTE CLOSURE ALGORITHM

To find:

$X^+$

### Step 1

Start with:

$X^+ = X$

### Step 2

Look at every FD:

$Y \rightarrow Z$

If:

$Y \subseteq X^+$

add Z to  $X^+$ .

### Step 3

Repeat until no new attribute can be added.

## MEMORY

**Start !' Scan !' Add !' Repeat**

I'll abbreviate:

But the actual logic is more important.

## 26. SOLVED EXAMPLE 1 — ATTRIBUTE CLOSURE

Relation:

$\{ R(A,B,C,D) \}$

FDs:

$\{ A \rightarrow B \}$

$\{ B \rightarrow C \}$

$\{ C \rightarrow D \}$

Find:

$\{ A^+ \}$

**Start**

$\{ A^+ = \{A\} \}$

Using:

$\{ A \rightarrow B \}$

add B:

$\{ A^+ = \{A,B\} \}$

Using:

$\{ B \rightarrow C \}$

add C:

$\{ A^+ = \{A,B,C\} \}$

Using:

$\{ C \rightarrow D \}$

add D:

$\{ A^+ = \{A,B,C,D\} \}$

Therefore:

$\{ \boxed{A^+ = ABCD} \}$

Since closure contains all attributes:

**A is a superkey.**

## 27. SUPERKEY TEST USING CLOSURE

For relation:

$$\forall R(A_1, A_2, \dots, A_n)$$

X is a superkey iff:

$$\forall X^+ = R$$

meaning X contains every attribute.

This is extremely important.

---

## MEMORY TRICK

Closure gets everything ! Superkey

---

## 28. CANDIDATE KEY TEST

For X to be a candidate key:

### Condition 1

X must be a superkey:

$$\forall X^+ = R$$

### Condition 2

X must be minimal:

No proper subset of X should also determine all attributes.

So:

Candidate Key = Superkey + Minimality

---

## 29. SOLVED EXAMPLE 2

Relation:

$$\forall R(A, B, C, D)$$

FDs:

$$\forall A \rightarrow B$$

$$\forall B \rightarrow C$$

$$\forall C \rightarrow D$$

We found:

$$\forall A^+ = ABCD$$

So A is a superkey.

Since A has only one attribute, it is automatically minimal.

Therefore:

$\boxed{A \text{ is a candidate key}}$

---

### 30. SOLVED EXAMPLE 3 — COMPOSITE CANDIDATE KEY

---

Relation:

$R(A,B,C,D)$

FDs:

$AB \rightarrow C$

$C \rightarrow D$

Find candidate key.

Start with AB:

$(AB)^+ = AB$

Using:

$AB \rightarrow C$

add C:

$ABC$

Using:

$C \rightarrow D$

add D:

$ABCD$

So AB is a superkey.

Now check minimality.

#### **A z**

No FD starts with A alone.

$A^+ = A$

Not superkey.

#### **B z**

$B^+ = B$

Not superkey.

Therefore AB is minimal.

$\boxed{AB \text{ is a candidate key}}$

---

### 31. IMPORTANT GATE SHORTCUT — ATTRIBUTES NEVER ON RHS

---

Suppose an attribute never appears on the RHS of any non-trivial FD.

Can it be generated from other attributes?

Usually no.

Therefore:

**Such an attribute must be present in every candidate key, unless it is itself derivable via some FD not apparent after decomposition—but under the given FD set/closure logic, attributes absent from all RHSs cannot be produced and must be initially included.**

This is an extremely powerful candidate-key shortcut.

## Example

Relation:

$\{ R(A,B,C,D,E) \}$

FDs:

$\{ A \rightarrow B \}$

$\{ B \rightarrow C \}$

$\{ CD \rightarrow E \}$

RHS attributes:

TEXT

B, C, E

Attributes not appearing on RHS:

TEXT

A, D

Therefore A and D must be included in every candidate key.

Try:

$\{ AD^+ \}$

Start:

TEXT

AD

A! B :

TEXT

ABD

B! C :

TEXT

ABCD

C D! E :

TEXT

ABCDE

So:

$\{ AD^+ = ABCDE \}$

Hence AD is candidate key.

## GATE SHORTCUT

First calculate:

TEXT

Attributes not present on any RHS

Call these **must-have attributes**.

Start candidate-key testing with them.

This often saves a huge amount of time.

## 32. ATTRIBUTE CATEGORIES FOR KEY SEARCH

A useful exam strategy:

### Category 1 — Never on RHS

These usually **must be in every candidate key**.

### Category 2 — Never on LHS

These cannot help determine additional attributes and usually do not need to be voluntarily added unless unavoidable for identity/must-have reasons.

### Category 3 — On both sides

These are often the attributes you experiment with while constructing candidate keys.

This is a heuristic, not a substitute for closure verification.

## 33. SOLVED GATE-STYLE KEY EXAMPLE

Relation:

$\{ R(A,B,C,D,E) \}$

FDs:

$\{ A \rightarrow B \}$

$\{BC \rightarrow D\}$

$\{D \rightarrow E\}$

### Step 1 — RHS attributes

TEXT

B, D, E

Not on RHS:

TEXT

A, C

Therefore candidate key must include:

TEXT

AC

### Step 2 — Compute $\{ (AC)^+ \}$

Start:

$\{AC\}$

A! B :

$\{ABC\}$

B C! D :

$\{ABCD\}$

D! E :

$\{ABCDE\}$

Therefore:

$\{ (AC)^+ = ABCDE \}$

So AC is superkey.

Neither A nor C alone can determine all attributes.

Therefore:

$\{\boxed{AC}\}$

is a candidate key.

---

## 34. MULTIPLE CANDIDATE KEYS

Consider:

$\{R(A,B,C)\}$

FDs:

$\{A \rightarrow B\}$



$\lceil B \rightarrow A \rceil$

No FD determines C.

C never appears on RHS.

So C must be in every candidate key.

Now:

**A C z**

A! B :

$\lceil AC^+=ABC \rceil$

Thus AC is candidate key.

**B C z**

B! A :

$\lceil BC^+=ABC \rceil$

Thus BC is also candidate key.

Candidate keys:

$\lceil \boxed{AC, BC} \rceil$

---

## 35. PRIME ATTRIBUTES

Recall:

**Prime attribute = attribute that belongs to at least one candidate key.**

If candidate keys:

$\lceil AC, BC \rceil$

then:

TEXT

A is prime

B is prime

C is prime

All three occur in some candidate key.

---

## 36. NON-PRIME ATTRIBUTE

An attribute that belongs to no candidate key.

Suppose relation:

$\lceil R(A,B,C,D) \rceil$

candidate keys:

$\lceil AB, AC \rceil$

Prime attributes:

TEXT

A, B, C

Non-prime:

TEXT

D

## & ~~p~~HUGE GATE TRAP

Prime does **not** mean:

**part of the primary key.**

Correct:

**part of at least one candidate key.**

This distinction becomes crucial in 3NF.

## 37. FD IMPLICATION USING ATTRIBUTE CLOSURE

Suppose F contains:

$\{ A \rightarrow B \}$

$\{ B \rightarrow C \}$

Question:

Does F imply:

$\{ A \rightarrow C \}$

Compute:

$\{ A^+ \}$

We get:

TEXT

A !' B !' C

So:

$\{ C \in A^+ \}$

Therefore:

$\{ F \models A \rightarrow C \}$

Yes.

---

## GATE RULE

---

To test whether:

$\{ X \rightarrow Y \}$

is implied by F:

1. Compute  $X^+$ .
2. Check whether:

$\{ Y \subseteq X^+ \}$

If yes  $F$  implied.

---

## 38. SOLVED EXAMPLE

---

FDs:

$\{ A \rightarrow BC \}$

$\{ C \rightarrow D \}$

$\{ D \rightarrow E \}$

Does:

$\{ A \rightarrow E \}$

hold?

Compute  $A^+$  :

Start:

TEXT

A

$A^+ B C$  :

TEXT

ABC

$C^+ D$  :

TEXT

ABCD

$D^+ E$  :

TEXT

ABCDE

E belongs to  $A^+$  .

Therefore:

---

$\boxed{A \rightarrow E}$

is implied.

---

### 39. DOES THE REVERSE FD HOLD?

From:

$A \rightarrow B$

can we infer:

$B \rightarrow A$

?

No.

FDs are **not symmetric**.

Example:

StudentID !' Dept may hold.

But Dept !' StudentID normally does not.

---

## MEMORY

**Arrow direction matters.**

Never reverse an FD unless another dependency proves it.

---

### 40. FD AND KEY RELATIONSHIP

If X is a superkey of R:

$X \rightarrow R$

meaning X functionally determines every attribute of relation R.

That's exactly what closure verifies:

$X^+ = R$

---

### 41. NUMBER OF SUPERKEYS — RECALL

If a relation has:

TEXT

n attributes

and exactly one candidate key contains:

TEXT

$k$  attributes

then:

$$2^{n-k}$$

superkeys exist.

Why?

Every superkey must include the candidate key, while each of the remaining  $(n-k)$  attributes can independently be included or excluded.

## Example

$$R(A,B,C,D,E)$$

only candidate key:

$$AB$$

Then:

$$2^{5-2}=8$$

superkeys.

But with multiple candidate keys, count carefully to avoid double counting.

## 42. EXTRANEOUS ATTRIBUTE — PREVIEW

Suppose:

$$AB \rightarrow C$$

If:

$$A \rightarrow C$$

already follows from  $F$ ,

then  $B$  is unnecessary in the LHS of  $AB \rightarrow C$ .

$B$  is called an **extraneous attribute** in that FD.

This becomes important in **minimal/canonical cover**, which comes next.

## 43. PARTIAL DEPENDENCY — PREVIEW

Suppose candidate key:

$$AB$$

and:

$$A \rightarrow C$$

where  $C$  is non-prime.

Then C depends on only **part of the composite candidate key**.

This is called:

### Partial dependency

It is the central issue in **2NF**.

## 44. FULL FUNCTIONAL DEPENDENCY

Suppose:

$[AB \rightarrow C]$

and neither:

$[A \rightarrow C]$

nor:

$[B \rightarrow C]$

holds.

Then C is **fully functionally dependent** on AB.

Meaning:

**Both A and B are genuinely required.**

## 45. PARTIAL vs FULL DEPENDENCY

Suppose AB is a candidate key.

### Partial dependency

TEXT

A B !' C  
but  
A !' C

or B!' C .

C depends on only part of the key.

### Full dependency

TEXT

A B !' C

but neither A nor B alone determines C.

## MEMORY

**Partial = Part of key is enough.**

**Full = Full key is needed.**

## 46. TRANSITIVE DEPENDENCY — PREVIEW

Suppose:

$A \rightarrow B$

and:

$B \rightarrow C$

Then:

$A \rightarrow C$

transitively.

For normalization, a problematic pattern may be:

TEXT

Key ! ' non - key ! ' non - key

Example:

$StudentID \rightarrow DeptID$

$DeptID \rightarrow DeptName$

Then:

$StudentID \rightarrow DeptName$

through DeptID.

This becomes central in **3NF**.

## MEMORY

TEXT

Key ! ' Middle ! ' Attribute

Think:

**“dependency through another attribute.”**

## 47. ATTRIBUTE CLOSURE — FAST EXAM METHOD

Suppose:

$\{ R(A,B,C,D,E,F) \}$

FDs:

TEXT

A !' B  
B !' C  
C D !' E  
E !' F

Find candidate key.

### Step 1 — RHS attributes

TEXT

B, C, E, F

Missing from RHS:

TEXT

A, D

So:

TEXT

AD must be in every candidate key

### Step 2 — A D z

Start:

TEXT

AD

A!' B :

TEXT

ABD

B!' C :

TEXT

ABCD

C D!' E :

TEXT

ABCDE

E!' F :



TEXT  
ABCDEF

Everything obtained.

Thus:

$\boxed{AD}$

is candidate key.

This is much faster than blindly trying combinations.

## 48. ANOTHER GATE-STYLE CANDIDATE KEY PROBLEM

Relation:

$R(A,B,C,D)$

FDs:

$A \rightarrow B$

$B \rightarrow A$

$C \rightarrow D$

Find candidate keys.

RHS:

TEXT  
B, A, D

Only C is absent from RHS.

Therefore every candidate key must contain C.

Try AC:

TEXT  
AC  
A! ' B  
C! ' D

gives:

TEXT  
ABCD

So AC is candidate key.

Try BC:

TEXT  
BC  
B! ' A  
C! ' D

gives:

TEXT  
ABCD

So BC also candidate key.

Thus:

$\boxed{AC, BC}$

Prime:

TEXT  
A, B, C

Non-prime:

TEXT  
D

## 49. GATE-STYLE FD CHECK FROM INSTANCE

Suppose relation instance:

| A | B |
|---|---|
| 1 | X |
| 1 | X |
| 2 | Y |
| 3 | Y |

Does:

$A \rightarrow B$

hold on this instance?

For every same A:

TEXT  
 $A = 1 \rightarrow B = X$

same B.

Other A values occur once.

So yes, this instance satisfies  $A \rightarrow B$ .

Does:

$B \rightarrow A$

hold?

For:

TEXT  
B=Y

we have:

TEXT  
A=2  
A=3

different.

Therefore:

$\neg (B \rightarrow A)$

---

## GATE SHORTCUT FOR INSTANCE FD QUESTIONS

To test:

$\neg (X \rightarrow Y)$

group rows by X.

If any group has multiple different Y values:

TEXT  
FD fails

Otherwise:

TEXT  
FD holds on that instance

---

## 50. COMMON ATTRIBUTE-CLOSURE MISTAKE

FDs:

$A \rightarrow B$

$BC \rightarrow D$

Start with A:

TEXT  
A z = A B

Can we use B C! D ?

No.

Because:

TEXT

C is missing

You need **all attributes on the LHS** before applying an FD.

## & pGATE TRAP

For:

$\{ BC \rightarrow D \}$

having B alone is insufficient.

You need:

TEXT

B AND C

in closure.

## 51. ORDER OF FD APPLICATION

The closure result does not depend on the order if you repeat until no new attributes can be added.

But during calculation, an FD unusable initially may become usable later.

Example:

TEXT

|   |   |   |     |
|---|---|---|-----|
| A | ! | ' | B   |
| B | ! | ' | C   |
| C | D | ! | ' E |

Starting with AD:

At first:

TEXT

|   |   |   |     |
|---|---|---|-----|
| C | D | ! | ' E |
|---|---|---|-----|

cannot be used.

After:

TEXT

|   |   |   |   |
|---|---|---|---|
| A | ! | ' | B |
| B | ! | ' | C |

we obtain C.

Now C D! becomes usable.

So always **rescan FDs**.

## MEMORY

Closure = keep looping until nothing changes.

Do not scan FDs only once.

## 52. MUST-HAVE ATTRIBUTE SHORTCUT — LIMITATION

The “not on RHS” method gives attributes that must be present in every candidate key.

But it does **not always immediately give the whole candidate key**.

Example:

$\text{[ R(A,B,C,D) ]}$

FDs:

$\text{[ A} \rightarrow \text{B ]}$

$\text{[ C} \rightarrow \text{D ]}$

Attributes not RHS:

TEXT

A, C

A C  $\neq$  ABCD.

Great.

But in more complex cases the must-have attributes may not determine everything, so you may have to add one or more attributes from the remaining candidate pool.

Always verify using closure.

## 53. CANDIDATE KEY SEARCH ALGORITHM

When GATE gives FDs:

### STEP 1

Write all relation attributes.

### STEP 2

Collect RHS attributes.

### STEP 3

Attributes not on RHS are **mandatory starting attributes**.

### STEP 4

Find closure of those mandatory attributes.

### STEP 5

If closure = all attributes:

Candidate key found.

#### STEP 6

If not, add minimal combinations of remaining useful attributes.

#### STEP 7

For every superkey found, test minimality.

#### STEP 8

Find all candidate keys if asked.

---

## 54. GATE SHORTCUT — LHS/RHS CLASSIFICATION

You can organize attributes like this:

### TEXT

Only LHS  
Only RHS  
Both LHS & RHS  
Neither

Important insight:

#### Only LHS / Neither

cannot be generated if never appearing RHS !' usually mandatory in candidate keys.

#### Only RHS

usually won't need to be added voluntarily because they are meant to be derived, though always rely on closure rather than this heuristic alone.

#### Both

often form the variable part of alternative candidate keys.

---

## 55. GATE-STYLE MULTIPLE KEY EXAMPLE

Relation:

$\{ R(A,B,C,D,E) \}$

FDs:

$\{ A \rightarrow B \}$

$\{ B \rightarrow A \}$

$\{ C \rightarrow D \}$

$\{ D \rightarrow C \}$

No FD determines E.

Therefore E is mandatory.

We need one attribute from:

TEXT

A/B pair

and one from:

TEXT

C/D pair

plus E.

Candidate keys:

$\{ACE\}$

$\{ADE\}$

$\{BCE\}$

$\{BDE\}$

Verify one:

$\{ (ACE)^+ \}$

A! B :

TEXT

ABCE

C! D :

TEXT

ABCDE

Yes.

Same logic for others.

---

## GATE PATTERN RECOGNITION

When attributes mutually determine each other:

$A \rightarrow B$

they may create alternative choices in candidate keys.

Likewise:

$C \rightarrow D$

If another mandatory attribute E exists, combinations emerge.

This is a common candidate-key pattern.

---

## 56. NUMBER OF CANDIDATE KEYS IN SUCH PATTERNS

Suppose independent alternatives:

TEXT

choose A or B  
choose C or D  
must include E

Number candidate keys:

$\{ 2 \times 2 = 4 \}$

Candidate keys:

TEXT

ACE  
ADE  
BCE  
BDE

This product-rule shortcut can be useful, but closure must support the independence assumption.

## 57. FD EQUIVALENCE

Two FD sets:

$\{ F \}$

and:

$\{ G \}$

are equivalent if:

$\{ F^+ = G^+ \}$

That means they imply exactly the same functional dependencies.

This idea is fundamental for minimal cover.

## 58. HOW TO TEST FD SET EQUIVALENCE

Conceptually:

1. Every FD in F must be implied by G.
2. Every FD in G must be implied by F.

Use attribute closures.

If both directions hold:

$\{ F^+ = G^+ \}$

Therefore equivalent.

## 59. GATE TRAPS

### & pTrap 1 — FD direction



$\lceil A \rightarrow B \rceil$

does **not** imply:

$\lceil B \rightarrow A \rceil$

---

## & pTrap 2 — Trivial FD

---

$\lceil AB \rightarrow A \rceil$

always holds.

---

## & pTrap 3 — Closure

---

Don't stop after one pass through FDs.

Repeat until no attribute is added.

---

## & pTrap 4 — Composite LHS

---

For:

$\lceil AB \rightarrow C \rceil$

you need both A and B.

---

## & pTrap 5 — Candidate key

---

Finding:

$\lceil X^+ = R \rceil$

only proves X is a **superkey**.

You still must check minimality.

---

## & pTrap 6 — Prime attribute

---

Prime means part of any candidate key, not only primary key.

---

## & pTrap 7 — Not-on-RHS shortcut

---

Those attributes are mandatory starting attributes, but closure still must be computed.

---

## & pTrap 8 — FD from sample data

---

A sample relation satisfying an FD doesn't necessarily prove it is a semantic database constraint.

---

## & pTrap 9 — Superkey vs candidate key

---

Extra attributes are allowed in superkeys.

Not in candidate keys.

---

## & ⚠Trap 10 — Partial dependency

Only relevant when a candidate key has multiple attributes; a single-attribute key has no proper non-empty subset that can cause partial dependency in the usual 2NF sense.

---

## 60. PYQ-ORIENTED QUESTION TYPES

GATE commonly asks things like:

### Type 1

Find:

$\bigcup X^+$

### Type 2

Determine candidate keys.

### Type 3

Count candidate keys/superkeys.

### Type 4

Identify prime and non-prime attributes.

### Type 5

Determine whether an FD is implied.

### Type 6

Choose the correct Armstrong inference.

### Type 7

Determine whether an FD holds on a relation instance.

### Type 8

Use all this inside normalization questions.

---

## 61. FASTEST RELIABLE METHOD FOR CANDIDATE KEYS

Write this in your notebook:

#### TEXT

1. Find attributes not on RHS.
2. Put them in key candidate.
3. Compute closure.
4. If all attributes obtained !' check minimality.
5. If not !' add smallest useful combinations.
6. Repeat to find alternatives.

This is much faster than brute-force checking all subsets.

## 62. SOLVED GATE-STYLE PROBLEM

Relation:

$R(A, B, C, D, E, F)$

FDs:

$A \rightarrow B$

$B \rightarrow C$

$CD \rightarrow E$

$E \rightarrow F$

Find candidate key.

### STEP 1 — RHS

TEXT

B, C, E, F

Not on RHS:

TEXT

A, D

Thus AD mandatory.

### STEP 2 — A D z

Start:

$AD$

$A \rightarrow B$  :

$ABD$

$B \rightarrow C$  :

$ABCD$

$CD \rightarrow E$  :

$ABCDE$

$E \rightarrow F$  :

$ABCDEF$

So AD is superkey.

### STEP 3 — minimality

A cannot derive D.

D cannot derive A.

So both needed.

Therefore:

$\boxed{AD}$

is candidate key.

Prime attributes:

TEXT

A, D

Non-prime:

TEXT

B, C, E, F

---

## 63. SOLVED GATE-STYLE PROBLEM — MULTIPLE KEYS

---

Relation:

$R(A, B, C, D)$

FDs:

$A \rightarrow B$

$B \rightarrow A$

$C \rightarrow D$

RHS:

TEXT

B, A, D

C not RHS !' mandatory.

Need A or B.

Candidate keys:

$AC$

and:

$BC$

Prime:

TEXT

A, B, C

Non-prime:

TEXT

D

## 64. NOTEBOOK NOTES

### FUNCTIONAL DEPENDENCY

$[X \rightarrow Y]$

means:

TEXT

Same X

! 0

Same Y

X = determinant.

### TYPES

#### Trivial

$[Y \subseteq X]$

#### Non-trivial

$[Y \not\subseteq X]$

#### Completely non-trivial

$[X \cap Y = \emptyset]$

### ARMSTRONG AXIOMS — RAT

#### Reflexivity

$[Y \subseteq X \rightarrow X \rightarrow Y]$

#### Augmentation

$[X \rightarrow Y \rightarrow XZ \rightarrow YZ]$

#### Transitivity

$[X \rightarrow Y, Y \rightarrow Z \rightarrow X \rightarrow Z]$

Derived:

TEXT

Union

Decomposition

Pseudotransitivity

### ATTRIBUTE CLOSURE

$[X^+]$

= all attributes determined by X.

Algorithm:

TEXT

Start with X  
Apply usable FDs  
Add RHS attributes  
Repeat until no change

## SUPERKEY TEST

$\forall X^+ = R \setminus Y$

$X^+$  is superkey.

## CANDIDATE KEY

TEXT

Superkey  
+  
Minimality

## CANDIDATE KEY SHORTCUT

TEXT

Attributes not on RHS  
! must be included  
in every candidate key

Then compute closure.

## PRIME ATTRIBUTE

Appears in at least one candidate key.

## NON-PRIME

Appears in none.

## FD IMPLICATION

To test:

$\forall X \rightarrow Y$

compute  $X^+$ .

If:

$Y \subseteq X^+$

FD is implied.

## MEMORY PAGE

TEXT

FD:

S a m e   L H S   ! '   S a m e   R H S

RAT:

Reflexivity

Augmentation

Transitivity

Closure:

S t a r t   ! '   A d d   ! '   R e p e a t

Everything in closure?

! '   S u p e r k e y

Superkey + minimal?

! '   C a n d i d a t e   k e y

Never on RHS?

! '   M u s t   b e   i n   e v e r y   c a n d i d a t e   k e y

Prime?

! '   P a r t   o f   S O M E   c a n d i d a t e   k e y

And normalization preview:

TEXT

Partial dependency

= part of composite key determines attribute

Transitive dependency

= key determines attribute through another attribute

## 30-SECOND REVISION

TEXT

X   ! '   Y :

same X means same Y

Trivial:

Y   "†   X

Armstrong:

RAT

Reflexivity

Augmentation

Transitivity

X+ =

everything determined by X

X+ = all relation attributes

! '   X   i s   s u p e r k e y

Candidate key  
= minimal superkey

Attributes never on RHS  
!' must appear in every candidate key

Prime  
= part of at least one candidate key

To test  $X \twoheadrightarrow Y$  :  
compute  $X^+$   
Y "†  $X^+$  !' y e s

Partial dependency:  
part of composite key sufficient

Transitive:  
 $X \twoheadrightarrow Y$  and  $Y \twoheadrightarrow Z$  gives  $X \twoheadrightarrow Z$

---

## CHECK YOUR UNDERSTANDING

### Q1

Given:

$F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D\}$

find:

$A^+$

---

### Q2

Relation:

$R(A, B, C, D, E)$

FDs:

$A \rightarrow B$

$BC \rightarrow D$

$D \rightarrow E$

Which attributes must definitely occur in every candidate key?

---

### Q3

For Q2, determine the candidate key.

---

### Q4

Classify:

$ABC \rightarrow AB$

as:



A. Trivial B. Non-trivial C. Completely non-trivial

---

### Q5

Given:

$$A \rightarrow B$$

and:

$$A \rightarrow C$$

which rule gives:

$$A \rightarrow BC$$

?

---

### Q6

Candidate keys are:

$$AB, AC$$

for relation:

$$R(A, B, C, D, E)$$

Identify:

1. Prime attributes
  2. Non-prime attributes
- 

### Q7 — GATE shortcut

Relation:

$$R(A, B, C, D)$$

FDs:

$$A \rightarrow B$$

$$B \rightarrow A$$

$$C \rightarrow D$$

Find **all candidate keys**.

---

### Q8 — Important

If:

$$(ABC)^+ = ABCDE$$

can we immediately say ABC is a candidate key?

Explain why or why not.

---

Next is the direct continuation:

---

## TOPIC 8 — MINIMAL/CANONICAL COVER + NORMALIZATION FOUNDATION

We will learn:

**extraneous attributes !' redundant FDs !' minimal cover algorithm !' full/partial/transitive dependency !' 1NF !' 2NF !' exact GATE decision method.**

Then after that:

**3NF !' BCNF !' decomposition !' lossless join !' dependency preservation.**

Type `NEXT`.

USER  
NEXT

ASSISTANT

## TOPIC 8 — MINIMAL / CANONICAL COVER + NORMALIZATION FOUNDATION

Now we move from **functional dependencies** into the actual machinery used

USER  
NEXT

ASSISTANT

## TOPIC 8 — MINIMAL / CANONICAL COVER + 1NF + 2NF

We are continuing directly from functional dependencies into normalization. This is part of the **normal forms** portion of your GATE DBMS syllabus.

The flow is:

```
TEXT
Functional Dependencies
! "
Extraneous Attributes
! "
Redundant FDs
! "
Minimal / Canonical Cover
! "
Partial Dependency
! "
1NF
! "
2NF
! "
3NF
! "
BCNF
```

---

## PART A — MINIMAL / CANONICAL COVER

---

### 1. WHY DO WE NEED A MINIMAL COVER?

---

Suppose we have:

$F = \{ A \rightarrow BC, B \rightarrow C, A \rightarrow B \}$

There may be:

- combined RHS attributes
- unnecessary attributes
- FDs that are already implied by other FDs

We want a simpler FD set that has **exactly the same meaning**.

That simplified equivalent set is called a:

**Minimal Cover / Canonical Cover**

Different books sometimes use the terms with slight procedural differences, but for GATE the essential idea is:

**Remove unnecessary parts while preserving  $(F^+)$ .**

---

### 2. WHAT MUST A MINIMAL COVER SATISFY?

---

A minimal cover generally satisfies:

#### Condition 1

Every FD has a **single attribute on RHS**.

Example:

$A \rightarrow BC$

becomes:

$A \rightarrow B$

$A \rightarrow C$

---

#### Condition 2

No attribute on the LHS is extraneous.

---

#### Condition 3

No FD itself is redundant.

---

### MEMORY TRICK

---

Remember:

### S-E-R

#### TEXT

S !' S i n g l e R H S  
E !' N o E x t r a n e o u s L H S a t t r i b u t e  
R !' N o R e d u n d a n t F D

## 3. STEP 1 — SPLIT RHS

Suppose:

$A \rightarrow BCD$

Using decomposition:

$A \rightarrow B$

$A \rightarrow C$

$A \rightarrow D$

### GATE RULE

For minimal cover:

**First make every RHS contain exactly one attribute.**

## 4. EXTRANEOUS ATTRIBUTE

Consider:

$AB \rightarrow C$

Maybe B is unnecessary.

If:

$A \rightarrow C$

can already be inferred using the FD set, then B contributes nothing.

Then B is **extraneous** in the LHS.

## MEMORY

**Extraneous = Extra**

If removing an attribute does not change what the FD can determine, it is unnecessary.

## 5. HOW TO TEST AN EXTRANEOUS LHS ATTRIBUTE

Suppose we want to test whether B is extraneous in:

$\{ AB \rightarrow C \}$

Compute:

$\{ A^+ \}$

using the relevant FD set.

If:

$\{ C \in A^+ \}$

then B is extraneous.

Therefore replace:

$\{ AB \rightarrow C \}$

with:

$\{ A \rightarrow C \}$

## 6. SOLVED EXAMPLE — EXTRANEOUS ATTRIBUTE

FDs:

$F = \{ AB \rightarrow C, A \rightarrow B \}$

Is B extraneous in:

$\{ AB \rightarrow C \}$

?

Compute:

$\{ A^+ \}$

Start:

TEXT

A

Using:

$\{ A \rightarrow B \}$

we get:

TEXT

AB

Now:

$\{ AB \rightarrow C \}$

gives:

TEXT

ABC

So:

$\{ C \mid A^+ \}$

Therefore B is extraneous.

We can simplify:

$\{ AB \rightarrow C \}$

to:

$\{ A \rightarrow C \}$

## 7. IMPORTANT CAUTION

When testing an extraneous attribute, you are not asking:

**Is A itself already written as determining C?**

You ask:

**Can A determine C using all available dependencies?**

Closure is the reliable method.

## 8. REDUNDANT FUNCTIONAL DEPENDENCY

An FD is redundant if it can be derived from the remaining FDs.

Example:

$F = \{ A \rightarrow B, B \rightarrow C, A \rightarrow C \}$

Is:

$\{ A \rightarrow C \}$

necessary?

Remove it temporarily.

Remaining:

$\{ A \rightarrow B \}$

$\{ B \rightarrow C \}$

By transitivity:

$\{ A \rightarrow C \}$

still follows.

Therefore:

$\{ A \rightarrow C \}$

is redundant.

---

## 9. HOW TO TEST REDUNDANT FD

Suppose testing:

$\{ X \rightarrow Y \}$

### Step 1

Temporarily remove it.

### Step 2

Using remaining FDs, calculate:

$\{ X^+ \}$

### Step 3

If:

$\{ Y \subseteq X^+ \}$

then the FD is redundant.

---

## MEMORY TRICK

Remove !' Closure !' Check

---

## 10. FULL MINIMAL-COVER ALGORITHM

Use this GATE-friendly order:

TEXT

STEP 1

Split RHS

STEP 2

Remove extraneous LHS attributes

STEP 3

Remove redundant FDs

STEP 4

Optionally combine same LHS FDs

For example:

TEXT

$A \rightarrow B$

$A \rightarrow C$

can be written as:

$$\{ A \rightarrow BC \}$$

in a canonical presentation, if desired.

For GATE problems, always pay attention to the convention used in the options.

---

## 11. SOLVED MINIMAL-COVER EXAMPLE

---

Given:

$$F = \{ A \rightarrow BC, B \rightarrow C, A \rightarrow B \}$$

Find a minimal cover.

### STEP 1 — Split RHS

$$\{ A \rightarrow B \}$$

$$\{ A \rightarrow C \}$$

$$\{ B \rightarrow C \}$$

$$\{ A \rightarrow B \}$$

Remove duplicate identical FD:

$$\{ A \rightarrow B \}$$

$$\{ A \rightarrow C \}$$

$$\{ B \rightarrow C \}$$

---

### STEP 2 — Check redundancy

Is:

$$\{ A \rightarrow C \}$$

redundant?

Remove it temporarily.

Remaining:

$$\{ A \rightarrow B \}$$

$$\{ B \rightarrow C \}$$

Then:

$$\{ A \rightarrow C \}$$

follows by transitivity.

Therefore remove:

$$\{ A \rightarrow C \}$$

Minimal cover:

$$\{ \boxed{A \rightarrow B, B \rightarrow C} \}$$

---



## 12. ANOTHER MINIMAL-COVER EXAMPLE

Given:

$F = \{ A \rightarrow C, A \rightarrow B, C \rightarrow D \}$

**Test B in  $A \rightarrow B$  C**

Compute  $A \rightarrow B$  :

TEXT

```
A
! ' B
! ' n o w A B
! ' C
! ' D
```

Thus:

$F^+ = ABCD$

So B is extraneous.

Replace:

$A \rightarrow C$

with:

$A \rightarrow C$

Now F becomes:

$A \rightarrow C$

$A \rightarrow B$

$C \rightarrow D$

No FD is redundant.

Minimal cover:

$\boxed{A \rightarrow B, A \rightarrow C, C \rightarrow D}$

## & pGATE TRAP — MINIMAL COVER IS NOT ALWAYS UNIQUE

Different valid minimal covers can sometimes exist depending on the transformations.

What matters is:

$F^+ = G^+$

and G is minimal under the required rules.

So don't assume there is always only one textual answer.

## PART B — WHY NORMALIZATION EXISTS

## 13. THE PROBLEM WITH BAD RELATIONS

Consider:

## STUDENT\_DEPT

| Sid | StudentName | DeptID | DeptName | HOD   |
|-----|-------------|--------|----------|-------|
| 1   | Ravi        | 10     | CSE      | Rao   |
| 2   | Anu         | 20     | AIML     | Kumar |
| 3   | Raj         | 10     | CSE      | Rao   |

Observe:

TEXT

DeptID = 10  
DeptName = CSE  
HOD = Rao

is repeated.

This creates:

1. Update anomaly
2. Insertion anomaly
3. Deletion anomaly

Normalization reduces such problems.

## 14. UPDATE ANOMALY

Suppose CSE HOD changes:

TEXT

R a o ! ' S h a r m a

We may need to update many rows.

If one row is missed:

TEXT

C S E ! ' R a o  
C S E ! ' S h a r m a

Database becomes inconsistent.

## 15. INSERTION ANOMALY

Suppose a new department exists:

TEXT

Cyber Security

but no student has joined it yet.

If department information is stored only inside STUDENT\_DEPT, we may be unable to represent the department without inventing a student.

That's an insertion anomaly.

## 16. DELETION ANOMALY

Suppose only one student belongs to AIML.

If we delete that student's row, we may accidentally lose:

TEXT

AIML department information  
HOD information

That's deletion anomaly.

## MEMORY TRICK

Normalization attacks:

UID

TEXT

U !' U p d a t e a n o m a l y  
I !' I n s e r t i o n a n o m a l y  
D !' D e l e t i o n a n o m a l y

## 17. NORMALIZATION

Normalization is the process of organizing relations to:

- reduce undesirable redundancy
- avoid anomalies
- preserve meaningful dependencies

The main normal forms in your GATE path:

TEXT

1NF  
2NF  
3NF  
BCNF

## IMPORTANT HIERARCHY

$[ \text{BCNF} \rightarrow 3\text{NF} \rightarrow 2\text{NF} \rightarrow 1\text{NF} ]$

Meaning:

Every BCNF relation is in 3NF.

Every 3NF relation is in 2NF.

Every 2NF relation is in 1NF.

But reverse directions do not necessarily hold.

## MEMORY

TEXT

BCNF

! "

3NF

! "

2NF

! "

1NF

Higher '!' satisfies lower.

## PART C — 1NF

### 18. FIRST NORMAL FORM — 1NF

At the classical GATE level:

A relation is in 1NF if attribute values are atomic and there are no repeating groups/multivalued entries stored in one cell.

Example violating 1NF:

| Sid | Name | Phone    |
|-----|------|----------|
| 1   | Ravi | 999, 888 |

Phone contains multiple values.

### 19. CONVERT TO 1NF

Possible representation:

| Sid | Name | Phone |
|-----|------|-------|
| 1   | Ravi | 999   |
| 1   | Ravi | 888   |

or preferably a separate relation:

TEXT

STUDENT(Sid, Name)

STUDENT\_PHONE(Sid, Phone)

depending on design.

The key concept:

**one cell != one atomic value.**

## MEMORY TRICK

**1NF = One value per cell**

Not the full formal theory, but excellent for GATE recall.

## 20. WHAT DOES “ATOMIC” MEAN?

Atomic means the DBMS treats the value as indivisible for the relational schema.

Example:

TEXT

Age = 20

atomic.

A list such as:

TEXT

20, 21, 22

stored in one field as multiple logical values violates the standard 1NF interpretation.

## & pGATE TRAP

A composite real-world object does not automatically violate 1NF if stored as a single atomic domain value from the DBMS viewpoint.

But in standard exam examples:

TEXT

multiple phone numbers in one field  
multiple courses in one field

signal a 1NF violation.

## 21. DOES 1NF REMOVE REDUNDANCY?

Not necessarily.

A relation can be in 1NF and still contain huge redundancy.

1NF only gets us to proper atomic relational form.

The deeper anomalies are addressed by 2NF, 3NF, BCNF.

## PART D — FULL vs PARTIAL FUNCTIONAL DEPENDENCY

## 22. WHY THIS MATTERS FOR 2NF

Suppose candidate key is:

$\{ (StudentID, CourseID) \}$

Relation:

TEXT

```
ENROLLMENT(  
  StudentID,  
  CourseID,  
  StudentName,  
  CourseName,  
  Grade  
)
```

FDs:

$\{ StudentID \rightarrow StudentName \}$

$\{ CourseID \rightarrow CourseName \}$

$\{ StudentID, CourseID \rightarrow Grade \}$

Observe:

**StudentName** depends only on:

TEXT

StudentID

which is only **part of the composite key**.

Similarly CourseName depends only on CourseID.

These are partial dependencies.

## 23. FULL FUNCTIONAL DEPENDENCY

Y is fully functionally dependent on X if:

$\forall [ X \rightarrow Y ]$

and no proper subset of X determines Y.

Example:

$\forall [ (StudentID, CourseID) \rightarrow Grade ]$

assuming:

TEXT

```
S t u d e n t I D  ! >   G r a d e
C o u r s e I D  ! >   G r a d e
```

Then Grade is fully dependent on the complete key.

## 24. PARTIAL DEPENDENCY

Suppose:

$\forall [ AB ]$

is candidate key.

If:

$\forall [ AB \rightarrow C ]$

but also:

$\forall [ A \rightarrow C ]$

then C is only partially dependent on AB.

Because a proper subset:

TEXT

A

is sufficient.

## MEMORY TRICK

TEXT

PARTIAL

! ' P A R T o f c o m p o s i t e k e y i s e n o u g h

FULL

! ' F U L L k e y r e q u i r e d

## 25. CRITICAL POINT — PARTIAL DEPENDENCY REQUIRES A COMPOSITE

## KEY

Suppose candidate key is just:

$\{A\}$

Can there be partial dependency on A?

No meaningful non-empty proper subset exists.

So:

**Relations with only single-attribute candidate keys are automatically free of partial dependencies of the type prohibited by 2NF.**

This gives us an important shortcut.

## PART E — SECOND NORMAL FORM — 2NF

### 26. DEFINITION OF 2NF

A relation is in **2NF** if:

1. It is in 1NF.
2. Every **non-prime attribute** is fully functionally dependent on every candidate key.

In simpler GATE language:

**No non-prime attribute should depend on a proper subset of a candidate key.**

## MEMORY TRICK

**2NF removes PARTIAL dependency.**

Think:

TEXT

1 N F ! ' A t o m i c  
2 N F ! ' N o P a r t i a l d e p e n d e n c y

### 27. VERY IMPORTANT WORDING

2NF cares particularly about:

TEXT

NON-PRIME ATTRIBUTE

Remember:



**Prime attribute = belongs to at least one candidate key.**

So a partial dependency involving a **non-prime attribute** causes 2NF violation.

## 28. SIMPLE 2NF EXAMPLE

Relation:

$\{ R(A,B,C,D) \}$

Candidate key:

$\{ AB \}$

FDs:

$\{ AB \rightarrow C \}$

$\{ A \rightarrow D \}$

Suppose:

TEXT

C, D = non-prime

Since:

$\{ A \rightarrow D \}$

and A is only part of candidate key AB:

TEXT

Partial dependency exists.

Therefore relation is:

TEXT

NOT in 2NF

## 29. 2NF EXAMPLE THAT PASSES

Relation:

$\{ R(A,B,C) \}$

Candidate key:

$\{ AB \}$

Only non-trivial FD:

$\{ AB \rightarrow C \}$

Neither:

$\{ A \rightarrow C \}$

nor:

$\{ B \rightarrow C \}$

holds.

Then C fully depends on AB.

Therefore:

**Assuming relation is in 1NF, it is in 2NF.**

### 30. SINGLE-ATTRIBUTE KEY SHORTCUT

Relation:

$\{ R(A,B,C,D) \}$

Candidate key:

$\{ A \}$

Then 2NF cannot be violated by partial dependency because A has no proper non-empty subset.

Therefore:

**If every candidate key is single-attribute, relation is automatically in 2NF once it is in 1NF.**

### & IMPORTANT NUANCE

If there are **multiple candidate keys**, inspect all relevant candidate-key structures.

Do not check only the selected primary key.

Normal forms are based on **candidate keys**, not merely the chosen primary key.

### 31. GATE-STYLE 2NF PROBLEM

Relation:

$\{ R(A,B,C,D,E) \}$

FDs:

$\{ AB \rightarrow C \}$

$\{ A \rightarrow D \}$

$\{ B \rightarrow E \}$

Assume AB is the only candidate key.

Prime:

TEXT

A, B

Non-prime:

TEXT

C, D, E

Check:

$\text{A} \rightarrow \text{D}$

A is proper subset of AB and D is non-prime.

Violation.

Also:

$\text{B} \rightarrow \text{E}$

another partial dependency.

Therefore:

$\boxed{\text{R} \text{ is not in 2NF}}$

## 32. HOW TO FIX THIS USING DECOMPOSITION

From:

TEXT

|   |   |   |   |
|---|---|---|---|
| A | B | ! | C |
| A | ! | D |   |
| B | ! | E |   |

decompose conceptually:

TEXT

R1(A, D)

R2(B, E)

R3(A, B, C)

Now:

- D belongs with A
- E belongs with B
- C remains with full key AB

This removes partial-dependency redundancy.

## 33. REAL-WORLD 2NF EXAMPLE

Relation:

TEXT

```
ENROLLMENT(  
  StudentID,  
  CourseID,  
  StudentName,  
  CourseName,  
  Grade  
)
```

Candidate key:

$\{ (StudentID, CourseID) \}$

FDs:

$\{ StudentID \rightarrow StudentName \}$

$\{ CourseID \rightarrow CourseName \}$

$\{ StudentID, CourseID \rightarrow Grade \}$

Partial dependencies:

TEXT

```
S t u d e n t I D ! '   S t u d e n t N a m e  
  
C o u r s e I D ! '   C o u r s e N a m e
```

Hence not 2NF.

Decompose:

TEXT

```
STUDENT(  
  StudentID,  
  StudentName  
)
```

```
COURSE(  
  CourseID,  
  CourseName  
)
```

```
ENROLLMENT(  
  StudentID,  
  CourseID,  
  Grade  
)
```

Now the information is much cleaner.

---

## 34. 2NF DOES NOT REMOVE TRANSITIVE DEPENDENCY

---

This is crucial.

Suppose:

$\{ StudentID \rightarrow DeptID \}$

$\{ DeptID \rightarrow DeptName \}$

Candidate key:

TEXT

StudentID

Because key is single attribute, no partial dependency exists.

Therefore relation can be in **2NF**.

But:

TEXT

S t u d e n t I D ! ' D e p t I D ! ' D e p t N a m e

contains transitive dependency.

That is handled by:

**3NF**

## MEMORY CHAIN

TEXT

1NF

! ' A t o m i c

2NF

! ' R e m o v e P a r t i a l d e p e n d e n c y

3NF

! ' R e m o v e p r o b l e m a t i c T r a n s i t i v e d e p e n d e n c y

BCNF

! ' E v e r y d e t e r m i n a n t m u s t b e a s u p e r k e y

We will make the last two precise next.

## 35. TRANSITIVE DEPENDENCY — FORMAL INTUITION

Suppose:

$X \rightarrow Y$

and:

$Y \rightarrow Z$

then:

$X \rightarrow Z$

If:

- X is a candidate key
- Y is not a superkey

- Z is non-prime

then this pattern often signals a 3NF violation.

We'll derive the exact 3NF test in the next topic.

---

## 36. NORMAL-FORM EXAM DECISION METHOD — SO FAR

Given a relation:

### STEP 1 — Find candidate keys

Without candidate keys, don't try normalization.

### STEP 2 — Identify:

TEXT

Prime attributes

Non-prime attributes

### STEP 3 — Check 1NF

Atomic values?

### STEP 4 — Check 2NF

Ask:

**Does a proper subset of any candidate key determine a non-prime attribute?**

If YES:

TEXT

Not 2NF

If NO:

TEXT

At least 2NF

---

## 37. GATE SHORTCUT — HIGHEST NORMAL FORM

If relation fails 2NF but is in 1NF:

TEXT

Highest NF = 1NF

If relation passes 2NF but later fails 3NF:

TEXT

Highest NF = 2NF

If passes 3NF but fails BCNF:

TEXT

Highest NF = 3NF

This is a very common GATE question style.

### 38. SOLVED GATE-STYLE PROBLEM 1

Relation:

$R(A, B, C)$

FDs:

$AB \rightarrow C$

Assume AB candidate key.

Prime:

TEXT

A, B

Non-prime:

TEXT

C

Check partial dependency:

TEXT

|   |   |   |   |    |
|---|---|---|---|----|
| A | ! | C | ? | No |
| B | ! | C | ? | No |

Thus C fully depends on AB.

Assuming 1NF:

$R$  is in 2NF

### 39. SOLVED GATE-STYLE PROBLEM 2

Relation:

$R(A, B, C, D)$

FDs:

$AB \rightarrow C$

$A \rightarrow D$

Candidate key:

$AB$

D non-prime.

Since:

$\{A\} \twoheadrightarrow D$

and:

$\{A\} \subset AB$

partial dependency exists.

Therefore:

$\boxed{\text{Not in 2NF}}$

Highest possible among first two:

TEXT

1NF

assuming atomicity.

---

## 40. SOLVED GATE-STYLE PROBLEM 3 — SINGLE KEY

---

Relation:

$R(A,B,C,D)$

FDs:

$A \rightarrow B$

$B \rightarrow C$

$C \rightarrow D$

Candidate key:

$\{A\}$

Could 2NF be violated?

No.

Why?

Candidate key is single attribute.

There cannot be dependency on a proper non-empty subset of A.

Therefore:

TEXT

At least 2NF

But 3NF may fail because:

$B \rightarrow C$

and B is not a superkey.

We'll evaluate that next.



## 41. SOLVED GATE-STYLE MINIMAL COVER

Given:

$F = \{ A \rightarrow BC, B \rightarrow C, A \rightarrow B, AB \rightarrow D \}$

### STEP 1 — Split RHS

$A \rightarrow B$

$A \rightarrow C$

$B \rightarrow C$

$A \rightarrow B$

$AB \rightarrow D$

Remove duplicate  $A \rightarrow B$ .

Remaining:

$A \rightarrow B$

$A \rightarrow C$

$B \rightarrow C$

$AB \rightarrow D$

### STEP 2 — Remove redundant $A \rightarrow C$

Because:

$A \rightarrow B$

and:

$B \rightarrow C$

imply:

$A \rightarrow C$

Remove it.

Now:

$A \rightarrow B$

$B \rightarrow C$

$AB \rightarrow D$

### STEP 3 — Check $B \rightarrow C$ extraneous in $AB \rightarrow D$

Can  $A$  alone determine  $D$  using current FDs?

Answer:

TEXT

A  
! B  
! C  
! now AB available

! ' D

So:

$\backslash [ D \text{ in } A^+ \backslash ]$

Therefore B is extraneous.

Replace:

$\backslash [ AB \rightarrow D \backslash ]$

with:

$\backslash [ A \rightarrow D \backslash ]$

Final minimal cover:

$\backslash [ \boxed{A \rightarrow B, B \rightarrow C, A \rightarrow D} \backslash ]$

## 42. & pGATE TRAPS — MINIMAL COVER

### Trap 1

Do not forget to split RHS:

$\backslash [ A \rightarrow BC \backslash ]$

into two FDs.

### Trap 2

Extraneous attribute testing requires closure.

Do not remove an LHS attribute merely because it “looks unnecessary.”

### Trap 3

Redundant FD:

Remove it first, then test whether remaining FDs imply it.

### Trap 4

Minimal cover may not be textually unique.

### Trap 5

Don't confuse:

TEXT

extraneous attribute

with:

TEXT

redundant FD

One removes part of an FD.

The other removes the entire FD.

## 43. & pGATE TRAPS — 1NF / 2NF

### Trap 1

1NF does not imply no redundancy.

It mainly requires relational atomicity.

### Trap 2

2NF is about **partial dependency of non-prime attributes**.

### Trap 3

Do not check only the primary key.

Normal forms use candidate keys.

### Trap 4

If candidate key is one attribute:

TEXT

No partial dependency

so 2NF is automatically satisfied if 1NF holds.

### Trap 5

A relation can be in 2NF and still have transitive dependencies.

That is why 3NF exists.

## 44. COMPARISON TABLE

| Normal Form | Main issue addressed              |
|-------------|-----------------------------------|
| 1NF         | Non-atomic/repeating values       |
| 2NF         | Partial dependency                |
| 3NF         | Problematic transitive dependency |
| BCNF        | Non-superkey determinants         |

This table is extremely important.

## 45. NOTEBOOK NOTES

### MINIMAL / CANONICAL COVER

Equivalent simplified FD set.

Conditions:

TEXT

1. Single RHS attribute
2. No extraneous LHS attribute
3. No redundant FD

Algorithm:

TEXT

```
Split RHS
!' Remove extraneous attributes
!' Remove redundant FDs
```

### Extraneous LHS attribute

For:

$[ A \rightarrow C ]$

B extraneous if:

$[ C \in A^+ ]$

### Redundant FD

For:

$[ X \rightarrow Y ]$

remove it temporarily.

If:

$[ Y \subseteq X^+ ]$

using remaining FDs:

TEXT

FD redundant

## NORMALIZATION

Purpose:

#### TEXT

Reduce redundancy  
Avoid UID anomalies

#### TEXT

U = Update  
I = Insert  
D = Delete

Hierarchy:

$[BCNF \rightarrow 3NF \rightarrow 2NF \rightarrow 1NF]$

## 1NF

#### TEXT

Atomic values  
No repeating/multivalued groups in one field

Memory:

#### TEXT

1NF = One value per cell

## PARTIAL DEPENDENCY

Candidate key = AB.

If:

$[A \rightarrow C]$

where C non-prime:

#### TEXT

partial dependency

## FULL DEPENDENCY

$[AB \rightarrow C]$

but:

#### TEXT

|   |    |   |
|---|----|---|
| A | !> | C |
| B | !> | C |

Then C fully depends on AB.

## 2NF

TEXT

1NF

+

No non-prime attribute  
partially dependent  
on any candidate key

Shortcut:

**Single-attribute candidate key !' no partial dependency.**

## MEMORY PAGE

TEXT

Minimal Cover = SER

Single RHS

No Extraneous

No Redundant FD

Normalization:

TEXT

1 N F !' A t o m i c

2 N F !' N o P a r t i a l

3 N F !' N o p r o b l e m a t i c T r a n s i t i v e

B C N F !' D e t e r m i n a n t m u s t b e S u p e r k e y

And:

TEXT

Partial = PART of key enough

Full = FULL key needed

## 30-SECOND REVISION

TEXT

Minimal cover:

Split RHS

Remove extraneous attributes

Remove redundant FDs

Extraneous:

remove attribute and use closure

Redundant FD:

remove whole FD and test implication

1NF:  
atomic values

2NF:  
1NF + no partial dependency  
of non-prime attributes

Partial:  
proper subset of candidate key  
determines non-prime attribute

Single-attribute key  
! ' a u t o m a t i c a l l y   n o   p a r t i a l   d e p e n d e n c y

B C N F ! '   3 N F ! '   2 N F ! '   1 N F

---

## CHECK YOUR UNDERSTANDING

### Q1

Given:

$\{A \rightarrow BC\}$

what should be done first when constructing a minimal cover?

### Q2

Given:

$F = \{A \rightarrow B, B \rightarrow C, A \rightarrow C\}$

Which FD is redundant?

### Q3

Relation:

$R(A, B, C, D)$

candidate key:

$\{AB\}$

FD:

$\{A \rightarrow C\}$

and C is non-prime.

What type of dependency is this?

### Q4

If the only candidate key is:

$\{A\}$

can a usual 2NF partial dependency exist?

Why?

### Q5

Relation:

$\{ R(A,B,C,D,E) \}$

candidate key:

$\{ AB \}$

FDs:

$\{ AB \rightarrow C \}$

$\{ A \rightarrow D \}$

$\{ B \rightarrow E \}$

What is the highest normal form if the relation is atomic?

### Q6

What three anomalies does normalization try to reduce?

### Q7 — Minimal cover

Given:

$F = \{ AB \rightarrow C, A \rightarrow B \}$

Is B extraneous in  $\{ A \rightarrow B \}$ ?

Next is the **most important normalization lesson**:

## TOPIC 9 — 3NF + BCNF + HIGHEST NORMAL FORM

We will learn:

**transitive dependency !' exact 3NF condition !' exact BCNF condition !' 3NF vs BCNF !' prime-attribute exception !' multiple candidate keys !' fastest GATE decision algorithm !' difficult solved questions.**

Type **NEXT**.

USER

NEXT

ASSISTANT

## TOPIC 9 — 3NF + BCNF + HIGHEST NORMAL FORM

We now move to the most important normalization decision section for GATE. This directly continues the **normal**



**forms** portion of your DBMS syllabus.

The core progression is:

TEXT

1 N F ! ' A t o m i c   v a l u e s

2 N F ! ' N o   p a r t i a l   d e p e n d e n c y

3 N F ! ' N o   p r o b l e m a t i c   n o n - k e y   d e t e r m i n a n t   t o   n o n - p r i m e   a

B C N F ! ' E v e r y   d e t e r m i n a n t   m u s t   b e   a   s u p e r k e y

The real exam challenge is usually:

**Given relation + FDs, find the highest normal form.**

So we will focus heavily on a reliable decision method.

## 1. WHY DO WE NEED 3NF?

Consider:

$\{ R(\text{StudentID}, \text{DeptID}, \text{DeptName}) \}$

Suppose:

$\{ \text{StudentID} \rightarrow \text{DeptID} \}$

and:

$\{ \text{DeptID} \rightarrow \text{DeptName} \}$

StudentID is the candidate key.

Therefore:

$\{ \text{StudentID} \rightarrow \text{DeptName} \}$

through DeptID.

Diagram:

TEXT

StudentID

!"

DeptID

!"

DeptName

This creates repetition.

For example:

| StudentID | DeptID | DeptName |
|-----------|--------|----------|
| 1         | 10     | CSE      |
| 2         | 10     | CSE      |

| StudentID | DeptID | DeptName |
|-----------|--------|----------|
| 3         | 20     | AIML     |
| 4         | 10     | CSE      |

DeptName = CSE repeats every time DeptID 10 appears.

This creates update anomalies.

## 2. TRANSITIVE DEPENDENCY

Intuitively:

If:

$A \rightarrow B$

and:

$B \rightarrow C$

then:

$A \rightarrow C$

transitively.

In normalization, the problematic pattern is commonly:

TEXT

Candidate Key

!"

Non-key attribute

!"

Non-prime attribute

Example:

TEXT

StudentID

!"

DeptID

!"

DeptName

## MEMORY TRICK

**Transitive = Through another attribute**

TEXT

K e y   ! '   M i d d l e   ! '   F i n a l

### 3. THIRD NORMAL FORM — EXACT GATE CONDITION

A relation is in **3NF** if for every non-trivial FD:

$\{X\} \rightarrow A$

at least one of the following is true:

#### Condition 1

$\{X\}$

is a **superkey**.

OR

#### Condition 2

$A$

is a **prime attribute**.

This is the exact GATE test.

### WRITE THIS IN A BOX

For every non-trivial:

$\{X\} \rightarrow A$

3NF requires:

TEXT

X is Superkey  
OR  
A is Prime

### MEMORY TRICK

**3NF = Superkey OR Prime**

This is one of the most important DBMS shortcuts.

### 4. WHY PRIME ATTRIBUTE MATTERS

Suppose:

$\{X\} \rightarrow A$

and X is not a superkey.

Normally that looks suspicious.

But 3NF allows it when:

TEXT

A is prime

This is the famous **prime-attribute exception**.

BCNF does not allow this exception.

That is the main difference between 3NF and BCNF.

## 5. BCNF — BOYCE-CODD NORMAL FORM

A relation is in BCNF if for every non-trivial FD:

$X \rightarrow Y$

we have:

$X \rightarrow Y$

is a **superkey**.

That's it.

No prime-attribute exception.

## WRITE THIS IN A BOX

### 3NF

TEXT

LHS Superkey  
OR  
RHS Prime

### BCNF

TEXT

LHS MUST be Superkey

## MEMORY TRICK

**BCNF = Boss on Left**

The determinant/LHS must be powerful enough to determine the whole relation, i.e. be a superkey.

A better formal memory:

**BCNF = Every determinant is a superkey.**

## 6. 3NF vs BCNF

| 3NF                                       | BCNF                                  |
|-------------------------------------------|---------------------------------------|
| LHS superkey OR RHS prime                 | LHS must be superkey                  |
| Allows certain non-superkey determinants  | Does not                              |
| Less strict                               | More strict                           |
| Dependency preservation easier to achieve | May sacrifice dependency preservation |
| Every BCNF relation is 3NF                | Not every 3NF relation is BCNF        |

## IMPORTANT HIERARCHY

$BCNF \rightarrow 3NF$

But:

$3NF \nrightarrow BCNF$

This is extremely important.

## 7. SIMPLE 3NF + BCNF EXAMPLE

Relation:

$R(A,B,C)$

FDs:

$A \rightarrow B$

$A \rightarrow C$

Candidate key:

$\{A\}$

For:

$A \rightarrow B$

A is a superkey.

For:

$A \rightarrow C$

A is a superkey.

Therefore:

TEXT

BCNF

And automatically:

TEXT

3 N F '  
2 N F '  
1 N F '

Highest normal form:

$\boxed{\text{BCNF}}$

## 8. EXAMPLE — 2NF BUT NOT 3NF

Relation:

$R(A, B, C)$

FDs:

$A \rightarrow B$

$B \rightarrow C$

Candidate key:

$A$

Prime:

TEXT

A

Non-prime:

TEXT

B, C

Because candidate key has one attribute:

TEXT

2NF automatically satisfied

Now check 3NF.

### FD 1

$A \rightarrow B$

A is superkey.

Pass.

### FD 2

$B \rightarrow C$

Is B a superkey?

No.

Is C prime?

No.

So:

TEXT

3NF fails.

Highest normal form:

$\boxed{2NF}$

## MEMORY

TEXT

A !' B !' C

with A as key often gives:

TEXT

2NF yes

3NF no

if B is not a key and C is non-prime.

## 9. EXAMPLE — 1NF BUT NOT 2NF

Relation:

$R(A,B,C)$

Candidate key:

$AB$

FD:

$A \rightarrow C$

C is non-prime.

A is proper subset of AB.

Therefore partial dependency exists.

So:

TEXT

2NF

Assuming atomicity:

$\boxed{\text{Highest NF}=1NF}$

## 10. THE MOST IMPORTANT CASE — 3NF BUT NOT BCNF

This is a favorite GATE pattern.

Consider:

$R(A,B,C)$

FDs:

$AB \rightarrow C$

$C \rightarrow B$

Find candidate keys.

---

## 11. FIND CANDIDATE KEYS

---

**A B z**

Using:

$AB \rightarrow C$

we get:

$ABC$

Therefore AB is candidate key.

Now A C z :

Start:

TEXT

AC

C! B :

TEXT

ABC

So AC is also candidate key.

Thus:

$\boxed{AB, AC}$

Candidate keys:

TEXT

AB

AC

Prime attributes:

TEXT

A, B, C

All attributes are prime.



## 12. CHECK 3NF

FD:

$\{ AB \rightarrow C \}$

AB is candidate key !' superkey.

Pass.

FD:

$\{ C \rightarrow B \}$

C is NOT a superkey.

But RHS:

TEXT

B

is prime because B belongs to candidate key AB.

Therefore 3NF condition:

TEXT

LHS superkey OR RHS prime

passes.

So:

TEXT

3 N F ' '

## 13. CHECK BCNF

For:

$\{ C \rightarrow B \}$

C is not a superkey.

BCNF requires:

TEXT

every determinant = superkey

Therefore:

TEXT

B C N F ' '

Highest normal form:

$\boxed{3NF}$

---

## THIS IS THE KEY DIFFERENCE

---

3NF says:

TEXT  
C not superkey  
BUT  
B prime  
  
!' a c c e p t a b l e

BCNF says:

TEXT  
C not superkey  
  
!' v i o l a t i o n

Regardless of RHS being prime.

---

## MEMORY TRICK

**3NF forgives a weak determinant if RHS is prime.**

**BCNF forgives nothing: determinant must be superkey.**

---

## 14. EXACT 3NF TEST — WHY SINGLE RHS HELPS

---

Suppose:

$X \rightarrow YZ$

First decompose conceptually:

$X \rightarrow Y$

$X \rightarrow Z$

Then test each RHS attribute separately.

This makes prime/non-prime checking easy.

---

## 15. TRIVIAL FDs DO NOT CAUSE NORMAL-FORM VIOLATIONS

---

Example:

$AB \rightarrow A$

is trivial because:

$A \subseteq AB$

Normal-form tests generally care about **non-trivial** FDs.

So don't waste time testing trivial ones as violations.

---

## 16. GATE HIGHEST NORMAL FORM ALGORITHM

---

This is the most important algorithm of this topic.

Given:

```
TEXT
Relation R
+
FD set F
```

do this:

### STEP 1 — Find all candidate keys

Use closure.

### STEP 2 — Mark prime attributes

Union of attributes appearing in any candidate key.

### STEP 3 — Check 2NF

Ask:

Does a proper subset of any candidate key determine a non-prime attribute?

If YES:

```
TEXT
Highest 2NF
```

If NO:

```
TEXT
At least 2NF
```

### STEP 4 — Check 3NF

For every non-trivial:

$X \rightarrow A$

check:

```
TEXT
Is X superkey?
OR
Is A prime?
```

If every FD passes:

TEXT

3 N F ' '

If any fails:

TEXT

3 N F ' '

## STEP 5 — Check BCNF

For every non-trivial FD:

TEXT

Is X superkey?

If all yes:

TEXT

BCNF

If 3NF passed but this fails:

TEXT

Highest = 3NF

## MASTER DECISION FLOW

TEXT

Find Candidate Keys

!"

Find Prime Attributes

!"

Partial Dependency?

Y E S !' 1 N F

NO

!"

F o r e v e r y X!' A :

X superkey OR A prime?

N O !' 2 N F

YES

!"

F o r e v e r y X!' A :

X superkey?

N O !' 3 N F

Y E S !' B C N F

This is one of the best things to write in your notebook.

## 17. SOLVED GATE PROBLEM 1

Relation:

$R(A, B, C, D)$

FDs:

$A \rightarrow B$

$B \rightarrow C$

$C \rightarrow D$

Find highest NF.

## Step 1 — Candidate key

A z :

TEXT

A  
! ' B  
! ' C  
! ' D

So:

$A^+ = ABCD$

Candidate key:

$A$

Prime:

TEXT

A

Non-prime:

TEXT

B, C, D

## Step 2 — 2NF

Candidate key is single attribute.

No partial dependency.

TEXT

2 N F

## Step 3 — 3NF

Check:

$A \rightarrow B$

A superkey !' pass.

$B \rightarrow C$

B superkey?

No.

C prime?

No.

Fail.

Therefore:

$\boxed{\text{Highest NF=2NF}}$

No need to check BCNF.

---

## 18. SOLVED GATE PROBLEM 2

---

Relation:

$R(A,B,C)$

FDs:

$AB \rightarrow C$

$C \rightarrow B$

We already found:

Candidate keys:

$AB, AC$

All A,B,C prime.

2NF?

Any non-prime attributes?

No.

So 2NF automatically passes.

3NF:

TEXT

A B ! ' C :  
A B s u p e r k e y '

TEXT

C ! ' B :  
C n o t s u p e r k e y  
b u t B p r i m e '

Therefore:

TEXT

3NF

BCNF:

TEXT

C! B :  
C not super key

Therefore:

$\boxed{\text{Highest NF}=3\text{NF}}$

## 19. SOLVED GATE PROBLEM 3

Relation:

$R(A,B,C,D)$

FDs:

$AB \rightarrow C$

$C \rightarrow D$

Candidate key?

Attributes not RHS:

TEXT

A, B

Compute:

$(AB)^+$

A B! C :

TEXT

ABC

C! D :

TEXT

ABCD

So AB candidate key.

Prime:

TEXT

A, B

Non-prime:

TEXT

C, D

## 2NF

Any proper subset:

TEXT

A

B

determines C or D?

Given FDs, no.

Therefore:

TEXT

2 N F ' '

## 3NF

A B! C :

TEXT

A B s u p e r k e y ' '

C! D :

TEXT

C superkey? No

D prime? No

Fail.

Highest:

$\boxed{2NF}$

## 20. SOLVED GATE PROBLEM 4

Relation:

$R(A, B, C, D)$

FDs:

$A \rightarrow B$

$A \rightarrow C$

$A \rightarrow D$



Candidate key:

TEXT

A

Every non-trivial FD has determinant A.

A is superkey.

Therefore:

$\boxed{\text{BCNF}}$

## 21. SOLVED GATE PROBLEM 5 — MULTIPLE CANDIDATE KEYS

Relation:

$R(A,B,C,D)$

FDs:

$A \rightarrow B$

$B \rightarrow A$

$C \rightarrow D$

Candidate keys from previous topic:

$AC, BC$

Prime:

TEXT

A, B, C

Non-prime:

TEXT

D

Check 2NF.

Candidate key AC:

Proper subset C determines:

$C \rightarrow D$

D is non-prime.

Therefore partial dependency exists.

So:

TEXT

2NF

Highest:

$\boxed{1NF}$

This is a good reminder:

**Multiple candidate keys can make 2NF analysis less obvious.**

## 22. PRIME ATTRIBUTE EXCEPTION IN 3NF

Consider:

$R(A,B,C)$

Candidate keys:

TEXT

AB

AC

FD:

$C \rightarrow B$

C is not superkey.

B is prime.

Therefore:

TEXT

3NF'

This exact pattern distinguishes 3NF from BCNF.

## 23. WHAT DOES “DETERMINANT” MEAN AGAIN?

In:

$X \rightarrow Y$

X is determinant.

BCNF rule:

**Every non-trivial determinant must be a superkey.**

So if you see:

TEXT

non-superkey! something

immediately suspect BCNF violation.

## 24. QUICK BCNF TEST

For each given/minimal-cover non-trivial FD:

TEXT

Compute LHS closure.

If:

$\forall [X^+ = R]$

then X is superkey !' okay.

If not:

TEXT

BCNF violation.

This is all you need.

## 25. QUICK 3NF TEST

For each:

$\forall [X \rightarrow A]$

if  $X \neq A$ :

TEXT

pass

Otherwise check:

TEXT

Is A prime?

If yes:

TEXT

pass

Otherwise:

TEXT

3NF violation

## 26. IMPORTANT — USE IMPLIED FDs TOO

Normal-form definitions apply to **all dependencies in  $\forall(F^+)$** , not just the FDs visibly written.

In many exam questions, checking a canonical/minimal cover is sufficient because violations can be identified from an equivalent basis.

But conceptually:

**You cannot ignore an implied dependency merely because it wasn't explicitly listed.**

For GATE problem-solving, candidate keys + proper FD implication reasoning protect you from this trap.

## 27. EXAMPLE OF IMPLIED FD

Given:

$\{ A \rightarrow B \}$

$\{ B \rightarrow C \}$

we know:

$\{ A \rightarrow C \}$

But if A is superkey, A! causes no issue.

The visible FD:

$\{ B \rightarrow C \}$

already exposes the 3NF violation.

So usually a minimal cover is enough to identify practical violations.

## 28. 3NF TRANSITIVE-DEPENDENCY INTUITION vs EXACT TEST

You may hear:

**3NF removes transitive dependencies.**

Useful intuition, but not sufficiently precise.

Exact GATE test:

For every non-trivial:

$\{ X \rightarrow A \}$

either:

TEXT

X superkey

OR

A prime

Use the exact test in questions.

## & pGATE TRAP

Do not blindly say:

**“Any transitive dependency means not 3NF.”**

There are situations involving prime attributes where the exact 3NF condition still holds.

Always use:

TEXT

Superkey OR Prime

## 29. 2NF EXACT INTUITION

Similarly, don't say:

**2NF means every attribute fully depends on primary key.**

Too imprecise.

Correct GATE version:

**Every non-prime attribute must be fully functionally dependent on every candidate key.**

Candidate keys, not only primary key.

## 30. BCNF IS STRONGER THAN 3NF

Why was BCNF introduced?

Because 3NF still permits some redundancy when:

TEXT

n o n - s u p e r k e y ! ' p r i m e a t t r i b u t e

BCNF removes that case.

Therefore:

TEXT

3NF may still allow some anomalies.  
BCNF is stricter.

## 31. BUT WHY NOT ALWAYS DECOMPOSE INTO BCNF?

Because BCNF decomposition may fail to preserve all functional dependencies directly.

3NF decomposition has a major advantage:

**We can always obtain a lossless, dependency-preserving 3NF decomposition using the synthesis algorithm.**

BCNF decomposition:

can guarantee lossless join, but may lose dependency preservation.

This becomes extremely important in the next topic.

## 32. NORMAL FORM COMPARISON

| NF   | Main requirement                                              |
|------|---------------------------------------------------------------|
| 1NF  | Atomic values                                                 |
| 2NF  | No partial dependency of non-prime attribute on candidate key |
| 3NF  | For each $(X \rightarrow A)$ , $X$ superkey OR $A$ prime      |
| BCNF | For each $(X \rightarrow A)$ , $X$ superkey                   |

## MEMORY LADDER

TEXT

1NF  
Atomic

2NF  
No Partial

3NF  
Superkey OR Prime

BCNF  
Superkey ONLY

This is the normalization ladder you should memorize.

## 33. GATE SHORTCUT — FIND HIGHEST NF FAST

### Case 1

Composite candidate key and a proper subset determines non-prime attribute:

TEXT

! ' h i g h e s t 1 N F

### Case 2

No partial dependency, but:

TEXT

n o n - s u p e r k e y ! ' n o n - p r i m e

then:

TEXT

```
!' 3 N F f a i l s
!' h i g h e s t 2 N F
```

### Case 3

Every violating non-superkey FD has prime RHS:

TEXT

```
!' 3 N F p a s s e s
!' B C N F f a i l s
!' h i g h e s t 3 N F
```

### Case 4

Every determinant is superkey:

TEXT

```
!' B C N F
```

This four-case shortcut is extremely useful.

## 34. SOLVED GATE-STYLE PROBLEM 6

Relation:

$\{ R(A,B,C,D,E) \}$

FDs:

$\{ A \rightarrow BC \}$

$\{ C \rightarrow D \}$

$\{ D \rightarrow E \}$

Find highest NF.

### Candidate key

A z :

TEXT

```
A
!' B , C
!' D
!' E
```

So:

$\{ A \}$

is candidate key.

2NF automatically passes.

Now 3NF:

$\{ A \rightarrow B \}$

A superkey '

$\{A\} \rightarrow C$

A superkey '

$\{C\} \rightarrow D$

C not superkey.

D non-prime.

Fail.

Thus:

$\boxed{\text{Highest}=2\text{NF}}$

---

## 35. SOLVED GATE-STYLE PROBLEM 7

---

Relation:

$R(A,B,C)$

FDs:

$A \rightarrow B$

$B \rightarrow A$

$AB \rightarrow C$

Candidate keys?

A z :

TEXT

A! ' B

Now A B! ' C :

TEXT

ABC

So A itself is candidate key.

Similarly B z :

TEXT

B! ' A! ' C

So B is also candidate key.

Candidate keys:

TEXT

A  
B

Prime:



TEXT

A, B

Non-prime:

TEXT

C

Check BCNF:

$A \nrightarrow B$  :

TEXT

A super key '

$B \nrightarrow A$  :

TEXT

B super key '

$AB \nrightarrow C$  :

TEXT

AB super key '

Therefore:

$\boxed{\text{BCNF}}$

## 36. SOLVED GATE-STYLE PROBLEM 8 — TRICKY 3NF

Relation:

$R(A, B, C)$

FDs:

$A \rightarrow B$

$B \rightarrow A$

Candidate keys?

No FD generates C, so C must be present.

Candidate keys:

$\{AC, BC\}$

Prime:

TEXT

A, B, C

Now:

$\nexists A \rightarrow B$

A is not superkey because:

TEXT

A Z = A B

doesn't contain C.

But B is prime.

So 3NF passes.

Similarly:

$\nexists B \rightarrow A$

B not superkey, but A prime.

3NF passes.

BCNF fails because A and B are not superkeys.

Therefore:

$\boxed{\text{Highest}=3\text{NF}}$

Classic exam pattern.

---

## 37. GATE TRAPS

### & pTrap 1

Do not check only primary key.

Use all candidate keys.

### & pTrap 2

3NF is NOT simply:

TEXT

No transitive dependency

Use exact condition:

TEXT

LHS superkey OR RHS prime

### & pTrap 3

BCNF has no RHS-prime exception.

## & ⚠Trap 4

Prime attribute:

TEXT

member of any candidate key

not only the chosen primary key.

## & ⚠Trap 5

If relation has all attributes prime, it does **not automatically mean BCNF**.

It may be 3NF but not BCNF.

Example:

TEXT

C !' B

with C non-superkey and B prime.

## & ⚠Trap 6

Single-attribute candidate key means 2NF automatically, but not 3NF automatically.

## & ⚠Trap 7

Failing 2NF means no need to check 3NF/BCNF for highest NF.

Highest is 1NF, assuming 1NF.

## & ⚠Trap 8

Passing 3NF does not imply BCNF.

## & ⚠Trap 9

Every BCNF relation is automatically:

TEXT

3NF

2NF

1NF

## 38. NOTEBOOK NOTES

## TRANSITIVE DEPENDENCY

Conceptually:

TEXT

K e y ! ' Y ! ' Z

where dependency reaches Z through Y.

## 3NF

For every non-trivial:

$\forall X \rightarrow A$

at least one:

TEXT

1. X is superkey

OR

2. A is prime

Memory:

TEXT

3NF = Superkey OR Prime

## BCNF

For every non-trivial:

$\forall X \rightarrow A$

TEXT

X must be superkey

Memory:

TEXT

BCNF = Determinant must be superkey

## HIERARCHY

$\forall \text{BCNF} \rightarrow 3\text{NF} \rightarrow 2\text{NF} \rightarrow 1\text{NF}$

## HIGHEST NF METHOD

#### TEXT

1. Candidate keys
2. Prime attributes
3. Check partial dependency
4. Check 3NF
5. Check BCNF

Decision:

#### TEXT

Partial non-prime dependency?

!' 1 N F

Non - superkey !' non - prime ?

!' 2 N F

Non - superkey !' prime only ?

!' 3 N F

All determinants superkeys?

!' B C N F

## MEMORY PAGE

#### TEXT

1NF

ONE atomic value

2NF

NO PARTIAL dependency

3NF

SUPERKEY OR PRIME

BCNF

SUPERKEY ONLY

And:

**3NF is forgiving. BCNF is strict.**

## 30-SECOND REVISION

#### TEXT

3NF:

For every X!' A ,  
X superkey OR A prime

BCNF:

For every X!' A ,  
X must be superkey

Prime =

part of any candidate key

BCNF != 3NF  
but  
3NF != BCNF

Single-attribute key:  
2NF automatic  
but 3NF not automatic

Highest NF shortcut:

partial dependency  
!= 1NF

non-superkey != non-prime  
!= 2NF

non-superkey != prime  
!= can be 3NF, not BCNF

all determinants superkeys  
!= BCNF

---

## CHECK YOUR UNDERSTANDING

### Q1

Relation:

$\{ R(A,B,C) \}$

FDs:

$\{ A \rightarrow B \}$

$\{ B \rightarrow C \}$

Candidate key = A.

What is the highest normal form?

---

### Q2

For:

$\{ X \rightarrow A \}$

what are the two conditions that can make the FD satisfy 3NF?

---

### Q3

What condition must hold for BCNF?

---

### Q4

Relation:

$\{ R(A,B,C) \}$

FDs:

---

$\{ AB \rightarrow C \}$

$\{ C \rightarrow B \}$

Find:

1. Candidate keys
  2. Prime attributes
  3. Highest normal form
- 

### Q5

Is every 3NF relation also BCNF?

Why or why not?

---

### Q6

If every candidate key has one attribute, what can we immediately conclude about 2NF?

---

### Q7 — Important

Relation:

$\{ R(A,B,C) \}$

FDs:

$\{ A \rightarrow B \}$

$\{ B \rightarrow A \}$

No FD determines C.

Find the highest normal form.

---

Next is the final major normalization section:

## TOPIC 10 — DECOMPOSITION

---

We will cover:

**lossless join !' lossy decomposition !' binary lossless condition !' dependency preservation !' lossless vs dependency-preserving !' BCNF decomposition algorithm !' 3NF synthesis !' GATE tableau intuition !' tricky solved problems.**

Type ``NEXT``.

---

USER

NEXT

ASSISTANT

## TOPIC 10 — DECOMPOSITION: LOSSLESS JOIN + DEPENDENCY PRESERVATION

This is the final major part of the **normalization** block in your GATE DBMS syllabus. It builds directly on functional dependencies, 3NF, and BCNF.

The key ideas are:

TEXT

```
Decomposition
! "
Lossless Join
! "
Dependency Preservation
! "
BCNF Decomposition
! "
3NF Synthesis
```

For GATE, the most important questions are usually:

- Is a decomposition lossless?
- Is it dependency preserving?
- Can it be both?
- How do we decompose into BCNF?
- Why can BCNF lose dependency preservation?
- Why is 3NF sometimes preferred?

### 1. WHY DO WE DECOMPOSE A RELATION?

Suppose:

$\{ R(A,B,C) \}$

with FDs:

$\{ A \rightarrow B \}$

$\{ B \rightarrow C \}$

Candidate key:

$\{ A \}$

This relation is not in 3NF because:

$\{ B \rightarrow C \}$

where:

TEXT

```
B = not superkey
C = non-prime
```

So we may split the relation into:

$\{ R_1(A,B) \}$

and:



$\forall R_2(B, C) \setminus$

This process is called:

**Decomposition**

## 2. WHAT IS DECOMPOSITION?

A relation:

$\forall R \setminus$

is decomposed into smaller relations:

$\forall R_1, R_2, \dots, R_n \setminus$

such that:

$\forall R_1 \cup R_2 \cup \dots \cup R_n = R \setminus$

in terms of attributes.

Example:

TEXT

$R(A, B, C, D)$

can be decomposed into:

TEXT

$R_1(A, B)$

$R_2(B, C, D)$

Their attributes together cover:

TEXT

$A, B, C, D$

## 3. IS ANY DECOMPOSITION GOOD?

No.

A good decomposition should ideally satisfy:

1. Lossless join
2. Dependency preservation

These are different properties.

## MEMORY TRICK

A good decomposition should preserve:

TEXT  
DATA  
+  
DEPENDENCIES

Meaning:

TEXT  
Lossless Join  
!' p r e s e r v e o r i g i n a l i n f o r m a t i o n  
Dependency Preservation  
!' p r e s e r v e c o n s t r a i n t s l o c a l l y

## PART A — LOSSLESS JOIN

### 4. WHAT IS LOSSLESS DECOMPOSITION?

Suppose:

$\{ R \}$

is decomposed into:

$\{ R_1 \}$

and:

$\{ R_2 \}$

If joining the decomposed relations reconstructs **exactly the original relation**, the decomposition is **lossless**.

Symbolically:

$\{ R = \pi_{R_1}(R) \bowtie \pi_{R_2}(R) \}$

for every legal relation instance satisfying the FDs.

## MEMORY

**Lossless = No information is lost and no fake information appears when we join back.**

### 5. WHY THE WORD “LOSSLESS” CAN BE CONFUSING

The major danger of a bad decomposition is not only losing tuples.

It can create:

**spurious tuples**

These are fake tuples that were not in the original relation.

So lossless really means:

TEXT

Join back  
!' exactly original relation

No missing information.

No extra spurious information.

## 6. LOSSY DECOMPOSITION EXAMPLE

Suppose original:

**R(A,B,C)**

| A | B | C |
|---|---|---|
| 1 | X | P |
| 2 | X | Q |

Decompose:

$\bowtie R_1(A,B)$

Result:

| A | B |
|---|---|
| 1 | X |
| 2 | X |

and:

$\bowtie R_2(B,C)$

Result:

| B | C |
|---|---|
| X | P |
| X | Q |

Now natural join:

$\bowtie R_1 \bowtie R_2$

produces:

| A | B | C |
|---|---|---|
| 1 | X | P |
| 1 | X | Q |

| A | B | C |
|---|---|---|
| 2 | X | P |
| 2 | X | Q |

But original had only:

TEXT

(1, X, P)  
(2, X, Q)

The tuples:

TEXT

(1, X, Q)  
(2, X, P)

are **spurious**.

Therefore decomposition is lossy.

## MEMORY TRICK

**Lossy !' Fake rows appear after joining.**

## 7. BINARY LOSSLESS-JOIN CONDITION

This is one of the most important formulas in normalization.

If:

$R \rightarrow R_1, R_2$

then decomposition is lossless with respect to F if at least one holds:

$(R_1 \cap R_2) \rightarrow R_1$

or:

$(R_1 \cap R_2) \rightarrow R_2$

in  $(F^+)$ .

## WRITE THIS IN A BOX

For decomposition:

$R \rightarrow R_1, R_2$

let:

$X = R_1 \cap R_2$

Then lossless iff:

$\forall X \rightarrow R_1$

OR

$\forall X \rightarrow R_2$

under the given FDs.

Equivalent intuition:

**The common attributes must functionally determine all attributes of at least one decomposed relation.**

## MEMORY TRICK

**Common must control one side.**

This is the easiest way to remember binary lossless join.

## 8. SOLVED LOSSLESS EXAMPLE

Relation:

$\forall R(A,B,C)$

FD:

$\forall A \rightarrow B$

Decomposition:

$\forall R_1(A,B)$

$\forall R_2(A,C)$

Intersection:

$\forall R_1 \cap R_2 = \{A\}$

Check:

$\forall A \rightarrow R_1?$

R1 attributes:

TEXT

A, B

Given:

$\forall A \rightarrow B$

and trivially:

$\forall A \rightarrow A$

therefore:

$\lceil A \rightarrow AB \rceil$

Yes.

Therefore:

$\lceil \boxed{\text{Lossless}} \rceil$

---

## 9. ANOTHER LOSSLESS EXAMPLE

---

Relation:

$\lceil R(A,B,C,D) \rceil$

FD:

$\lceil B \rightarrow C \rceil$

Decompose:

$\lceil R_1(B,C) \rceil$

$\lceil R_2(A,B,D) \rceil$

Intersection:

TEXT

B

Does B determine R1?

$\lceil B \rightarrow BC \rceil$

Yes.

Therefore decomposition is lossless.

---

## 10. LOSSY EXAMPLE USING CONDITION

---

Relation:

$\lceil R(A,B,C) \rceil$

FD:

$\lceil A \rightarrow B \rceil$

Decompose:

$\lceil R_1(A,C) \rceil$

$\lceil R_2(B,C) \rceil$

Common attributes:

$\lceil C \rceil$

Does:

$\lceil C \rightarrow AC? \rceil$

No.

---

Does:

$\lceil C \rightarrow BC \rceil$

No.

Therefore decomposition is:

$\lceil \boxed{\text{Lossy}} \rceil$

---

## GATE SHORTCUT

For binary decomposition:

TEXT

1. Find common attributes.
2. Compute their closure.
3. If closure contains all attributes of R1 or R2  
    ! ' l o s s l e s s .
4. Otherwise  
    ! ' l o s s y .

Very fast.

---

## 11. WHY COMMON ATTRIBUTE MATTERS

If two relations share an attribute that uniquely identifies one side, joining on that attribute cannot incorrectly pair tuples from that side.

Example:

TEXT

R1(A, B)  
R2(B, C)

if:

$\lceil B \rightarrow C \rceil$

then B identifies the R2 tuple's C value.

So joining does not create multiple false C possibilities for one B.

That is the intuition.

---

## PART B — DEPENDENCY PRESERVATION

---

### 12. WHAT IS DEPENDENCY PRESERVATION?

Suppose original FDs are:

$\lceil F \rceil$

After decomposition into:

$\lceil R_1, R_2, \dots, R_n \rceil$

we would like to enforce all original FDs by checking individual relations separately.

If that is possible, the decomposition is:

### Dependency Preserving

## SIMPLE INTUITION

Suppose original FD:

$A \rightarrow B$

If A and B remain together inside one decomposed relation:

TEXT

$R_1(A, B)$

then enforcing  $A \rightarrow B$  is easy.

But if A and B get separated:

TEXT

$R_1(A, C)$

$R_2(B, C)$

we may need to join relations just to verify  $A \rightarrow B$ .

That is undesirable.

## MEMORY TRICK

**Dependency preserving = Constraints can be checked without joining tables.**

Excellent GATE intuition.

## 13. FORMAL IDEA

Let:

$\{F_i\}$

be the projection of original FDs onto  $\{R_i\}$ .

A decomposition is dependency preserving if:

$(F_1 \cup F_2 \cup \dots \cup F_n)^+ = F^+$

In simple terms:

**The dependencies enforceable inside the decomposed relations together imply all original FDs.**



## 14. SIMPLE DEPENDENCY-PRESERVING EXAMPLE

---

Original:

$R(A,B,C)$

FDs:

$A \rightarrow B$

$B \rightarrow C$

Decompose:

$R_1(A,B)$

$R_2(B,C)$

Then:

TEXT

$A \neq B$

can be enforced in  $R_1$ .

And:

TEXT

$B \neq C$

can be enforced in  $R_2$ .

All original FDs remain directly enforceable.

Therefore:

$\boxed{\text{Dependency preserving}}$

---

## 15. NON-DEPENDENCY-PRESERVING EXAMPLE

---

Original:

$R(A,B,C)$

FD:

$A \rightarrow C$

Decompose:

$R_1(A,B)$

$R_2(B,C)$

No relation contains both A and C.

Neither local relation alone can directly enforce:

$A \rightarrow C$

Unless other dependencies imply it through B.

If not, the decomposition is not dependency preserving.

---

## 16. IMPORTANT — “FD NOT IN ONE RELATION” DOES NOT ALWAYS MEAN NOT PRESERVED

---

This is a big trap.

Suppose original FDs:

$A \rightarrow B$

$B \rightarrow C$

and implied FD:

$A \rightarrow C$

Decompose:

TEXT

$R_1(A, B)$

$R_2(B, C)$

There is no table containing A and C together.

But:

$A \rightarrow B$

plus:

$B \rightarrow C$

still implies:

$A \rightarrow C$

Therefore the dependency is preserved **indirectly**.

So dependency preservation is about **implication**, not merely physical co-location.

---

## GATE TRAP

---

Do not ask only:

Are X and Y present together?

Instead ask:

Can the projected dependencies imply  $X \rightarrow Y$ ?

---

## 17. LOSSLESS vs DEPENDENCY PRESERVING

---

These properties are independent.

A decomposition can be:

TEXT

Lossless + Dependency preserving

or:

TEXT

Lossless + Not dependency preserving

or potentially:

TEXT

Lossy + Dependency preserving

etc.

In good database design we aim primarily for:

TEXT

Lossless

and preferably:

TEXT

Dependency preserving

## MASTER COMPARISON

| Lossless Join                 | Dependency Preservation    |
|-------------------------------|----------------------------|
| Preserves data/information    | Preserves constraints      |
| No spurious tuples            | FDs enforceable locally    |
| Concerned with reconstruction | Concerned with FD checking |
| Essential                     | Highly desirable           |

## MEMORY

TEXT

L o s s l e s s ! ' D A T A

D e p e n d e n c y   p r e s e r v a t i o n ! ' R U L E S

## PART C — BCNF DECOMPOSITION

## 18. WHY DECOMPOSE INTO BCNF?

Suppose relation violates BCNF because:

$$\{ X \rightarrow Y \}$$

where X is not a superkey.

We can decompose using that violating FD.

## 19. BCNF DECOMPOSITION RULE

Given relation:

$$\{ R \}$$

and violating FD:

$$\{ X \rightarrow Y \}$$

a common BCNF decomposition is:

$$\{ R_1 = X \cup Y \}$$

and:

$$\{ R_2 = R - (Y - X) \}$$

Equivalent common expression:

$$\{ R_2 = R - Y + X \}$$

because X must remain as common attributes.

## EASY MEMORY

For violating:

$$\{ X \rightarrow Y \}$$

create:

TEXT

$$R_1 = XY$$

and:

TEXT

$$R_2 = \text{everything else} + X$$

## 20. BCNF DECOMPOSITION EXAMPLE

Relation:

$$\{ R(A,B,C) \}$$

FDs:

$\{ A \rightarrow B \}$

$\{ B \rightarrow C \}$

Candidate key:

$\{ A \}$

BCNF violation:

$\{ B \rightarrow C \}$

because B is not superkey.

Decompose using  $B \twoheadrightarrow C$ .

**R1**

$\{ R_1(B,C) \}$

**R2**

Keep remaining attributes + B:

$\{ R_2(A,B) \}$

So:

$\{ \boxed{R_1(B,C), R_2(A,B)} \}$

---

## 21. IS THIS LOSSLESS?

---

Common attribute:

$\{ B \}$

And:

$\{ B \rightarrow BC \}$

So B determines R1.

Hence binary lossless condition holds.

Therefore BCNF decomposition algorithm produces a lossless decomposition.

---

## 22. ARE BOTH RELATIONS BCNF?

---

**R1(B,C)**

FD:

$\{ B \rightarrow C \}$

B is key of R1.

So BCNF.

**R2(A,B)**

FD:

$\{ A \rightarrow B \}$

---

A is key of R2.

So BCNF.

Thus decomposition is complete.

---

## 23. BCNF DECOMPOSITION ALGORITHM

---

### TEXT

1. Find a BCNF - violating FD  $X \twoheadrightarrow Y$ .
2. Create  $R_1 = X \rightarrow * Y$ .
3. Create  $R_2 = \text{remaining attributes} + X$ .
4. Check each new relation for BCNF.
5. Repeat until all are BCNF.

---

## 24. BCNF DECOMPOSITION GUARANTEE

---

BCNF decomposition can be made:

### Lossless

But it may fail:

### Dependency preservation

This is extremely important for GATE.

---

## 25. CLASSIC 3NF BUT NOT BCNF EXAMPLE

---

Consider:

$\{ R(A,B,C) \}$

FDs:

$\{ AB \rightarrow C \}$

$\{ C \rightarrow B \}$

Candidate keys:

$\{ AB, AC \}$

So relation is 3NF but not BCNF because:

$\{ C \rightarrow B \}$

C is not a superkey.

If we decompose based on  $C \rightarrow B$ :

$\{ R_1(B,C) \}$

$\{ R_2(A,C) \}$

This is lossless because common attribute C determines R1.

But what about:

$\{A \rightarrow B\}$

?

R1 does not contain A.

R2 does not contain B.

This FD is not obviously enforceable in either relation.

In fact, this classic decomposition can lose dependency preservation.

---

## HUGE GATE FACT

**BCNF decomposition is always achievable losslessly, but dependency preservation is not guaranteed.**

Memorize this.

---

## PART D — 3NF SYNTHESIS

---

### 26. WHY 3NF SYNTHESIS EXISTS

Suppose we want:

1. Lossless decomposition
2. Dependency preservation

3NF synthesis gives us a systematic method to obtain both.

That's a major practical advantage of 3NF.

---

### 27. 3NF SYNTHESIS ALGORITHM

Use a minimal/canonical cover.

Suppose minimal cover:

$\{G\}$

#### Step 1

For each FD:

$\{X \rightarrow A\}$

create relation:

$\{XA\}$

or combine FDs with same LHS.

#### Step 2

If no created relation contains a candidate key of original R, add one relation containing a candidate key.

### Step 3

Remove redundant relations that are subsets of others.

Result is generally:

TEXT  
3NF  
Lossless  
Dependency preserving

## MEMORY TRICK

3NF synthesis:

**FDs !' Relations !' Ensure Key**

## 28. SIMPLE 3NF SYNTHESIS EXAMPLE

Relation:

$\{ R(A,B,C,D) \}$

FDs:

$\{ A \rightarrow B \}$

$\{ B \rightarrow C \}$

$\{ C \rightarrow D \}$

Minimal cover is the same.

Create:

$\{ R_1(A,B) \}$

$\{ R_2(B,C) \}$

$\{ R_3(C,D) \}$

Candidate key of original:

$\{ A \}$

R1 contains A.

So no extra key relation needed.

This decomposition preserves:

TEXT  
 $A \not\rightarrow B$   
 $B \not\rightarrow C$   
 $C \not\rightarrow D$

directly.



## 29. WHY ADD A CANDIDATE KEY RELATION?

Suppose synthesis relations created from FDs do not contain any full candidate key of original R.

Then joining them may not necessarily preserve the full original relation losslessly.

Adding a candidate-key relation helps ensure losslessness.

## 30. 3NF SYNTHESIS GUARANTEE

A standard 3NF synthesis algorithm gives:

TEXT

Dependency preservation !' YES

Lossless join !' YES

and each relation is in:

TEXT

3NF

This is why 3NF remains useful even though BCNF is stricter.

## 31. 3NF vs BCNF DECOMPOSITION

| Property                    | 3NF Synthesis         | BCNF Decomposition   |
|-----------------------------|-----------------------|----------------------|
| Normal form                 | 3NF                   | BCNF                 |
| Lossless                    | Yes                   | Yes                  |
| Dependency preserving       | Yes                   | Not guaranteed       |
| Stronger redundancy removal | Less                  | More                 |
| Main advantage              | Preserve dependencies | Stronger normal form |

## MEMORY

- 3NF protects dependencies.
- BCNF protects stronger normalization.

## PART E — LOSSLESS BINARY TEST IN DEPTH

## 32. EXAMPLE

Relation:

$\{ R(A,B,C,D,E) \}$

Decomposition:

$\{ R_1(A,B,C) \}$

$\{ R_2(C,D,E) \}$

FDs:

$\{ C \rightarrow DE \}$

Intersection:

$\{ C \}$

Does:

$\{ C \rightarrow R_2? \}$

R2 attributes:

TEXT

C, D, E

Given:

$\{ C \rightarrow DE \}$

plus trivial  $C \rightarrow C$  :

$\{ C \rightarrow CDE \}$

Yes.

Hence:

$\boxed{\text{Lossless}}$

## 33. EXAMPLE

Same decomposition:

$\{ R_1(A,B,C) \}$

$\{ R_2(C,D,E) \}$

but FDs:

$\{ A \rightarrow B \}$

$\{ D \rightarrow E \}$

Intersection:

$\{ C \}$

$C \rightarrow$  :

TEXT

C

Doesn't determine R1.

Doesn't determine R2.

Therefore:

$\boxed{\text{Lossy}}$

## 34. WHAT IF INTERSECTION IS EMPTY?

Suppose:

$R_1(A,B)$

$R_2(C,D)$

Intersection:

$\varnothing$

Natural join behaves as Cartesian product.

Unless highly unusual constraints imply something from empty set, this is generally lossy for a nontrivial decomposition.

GATE intuition:

**No common attribute !' serious warning sign.**

## PART F — MULTIWAY DECOMPOSITION

### 35. BINARY TEST IS ONLY DIRECT FOR TWO RELATIONS

For more than two relations:

TEXT

R1

R2

R3

...

you cannot always just check pairwise intersections independently and conclude losslessness.

For multiway decompositions, one may use:

- chase/tableau method
- repeated application if decomposition produced step-by-step by known lossless binary decompositions

For most GATE problems, the latter reasoning often suffices.

## 36. TABLEAU / CHASE — INTUITION

The chase method checks whether joining decomposed projections can reconstruct an original symbolic tuple without generating ambiguity.

Detailed symbolic tableaux can become lengthy, so for GATE:

**First use binary lossless conditions whenever the decomposition structure permits.**

If a true multiway chase question appears, we'll solve it systematically.

## PART G — DEPENDENCY PRESERVATION TEST

### 37. EASY METHOD FOR SMALL GATE PROBLEMS

Suppose original FDs:

$\{ F \}$

and decomposition:

$\{ R_1, R_2 \}$

#### Step 1

Find FDs that hold entirely within each relation.

These form:

TEXT

F1

F2

#### Step 2

Union:

$G = F_1 \cup F_2$

#### Step 3

For every original FD:

$X \rightarrow Y$

compute  $X$  using  $G$ .

If every original FD can be inferred:

TEXT

Dependency preserving

Otherwise:

TEXT

Not dependency preserving

---

## 38. SOLVED DEPENDENCY-PRESERVATIONEXAMPLE

---

Original:

$\{ R(A,B,C) \}$

FDs:

$\{ A \rightarrow B \}$

$\{ B \rightarrow C \}$

Decompose:

$\{ R_1(A,B) \}$

$\{ R_2(B,C) \}$

Projected:

TEXT

F 1 : A! ' B  
F 2 : B! ' C

Together:

$\{ A \rightarrow B \rightarrow C \}$

So every original dependency is preserved.

Yes.

---

## 39. EXAMPLE — NOT PRESERVED

---

Original:

$\{ R(A,B,C) \}$

FDs:

$\{ AB \rightarrow C \}$

Decompose:

$\{ R_1(A,B) \}$

$\{ R_2(B,C) \}$

Neither local relation can establish:

$\{ AB \rightarrow C \}$

because:

- R1 has AB but not C
- R2 has BC but not A

Projected dependencies cannot imply the original FD.

Therefore:

$\{ \boxed{\text{Not dependency preserving}} \}$

---

---

## 40. IMPORTANT DEPENDENCY-PRESERVATIONALGORITHM — ATTRIBUTE PROPAGATION

---

For larger cases, a formal test exists using repeated closure propagation across decomposed schemas.

But GATE usually gives manageable FD sets where projected-FD reasoning and closure suffice.

The core exam question remains:

**Can the original FD be checked without computing a join?**

---

## PART H — DECOMPOSITION ANOMALY REASONING

---

### 41. WHY LOSSLESS IS ESSENTIAL

---

If decomposition is lossy:

TEXT

Store correct R1  
Store correct R2

but joining creates fake combinations.

Then the decomposition no longer faithfully represents the original data semantics.

So:

**Lossless join is non-negotiable.**

---

### 42. WHY DEPENDENCY PRESERVATIONIS DESIRABLE

---

Suppose original:

$[A \rightarrow C]$

but decomposed relations separate A and C.

To verify whether  $A \rightarrow C$  remains true, DBMS might need:

TEXT

Join multiple relations  
! ' t h e n c h e c k F D

This is expensive and inconvenient.

Dependency preservation lets constraints be checked locally.

---

### 43. LOSSLESS DOES NOT IMPLY DEPENDENCY PRESERVING

---

A decomposition can reconstruct original data perfectly but still require joins to enforce some original FD.

This is exactly what may happen with BCNF decompositions.

---

## 44. DEPENDENCY PRESERVING DOES NOT AUTOMATICALLY MEAN LOSSLESS

---

Constraints may be preserved while the decomposition can still generate spurious tuples.

Therefore check both properties separately.

---

## PART I — GATE-STYLE SOLVED PROBLEMS

---

### 45. PROBLEM 1 — LOSSLESS?

---

Relation:

$\{ R(A,B,C) \}$

FD:

$\{ B \rightarrow C \}$

Decomposition:

$\{ R_1(A,B) \}$

$\{ R_2(B,C) \}$

Intersection:

$\{ B \}$

Check:

$\{ B \rightarrow BC \}$

Yes.

Therefore:

$\boxed{\text{Lossless}}$

---

### 46. PROBLEM 2 — LOSSLESS?

---

Relation:

$\{ R(A,B,C,D) \}$

FDs:

$\{ A \rightarrow B \}$

$\{ C \rightarrow D \}$

Decompose:

$\{ R_1(A,B) \}$

$\{ R_2(B,C,D) \}$

Intersection:

$\{B\}$

Compute:

$\{B^+\}$

No FD starts from B.

So:

TEXT

B z = B

Does not determine AB.

Does not determine BCD.

Therefore:

$\boxed{\text{Lossy}}$

## 47. PROBLEM 3 — LOSSLESS + DEPENDENCY PRESERVING

Relation:

$R(A,B,C)$

FDs:

$A \rightarrow B$

$B \rightarrow C$

Decomposition:

$R_1(A,B)$

$R_2(B,C)$

Lossless?

Intersection B.

$B \rightarrow BC$

Yes.

Dependency preserving?

TEXT

A! ' B i n R 1  
B! ' C i n R 2

Yes.

Therefore:

$\boxed{\text{Both}}$



## 48. PROBLEM 4 — 3NF BUT NOT BCNF

Relation:

$\{ R(A,B,C) \}$

FDs:

$\{ AB \rightarrow C \}$

$\{ C \rightarrow B \}$

Candidate keys:

$\{ AB, AC \}$

3NF yes.

BCNF violation:

$\{ C \rightarrow B \}$

Decompose:

$\{ R_1(B,C) \}$

$\{ R_2(A,C) \}$

Lossless?

Intersection:

$\{ C \}$

$C \rightarrow B \mid C \rightarrow C$

Yes.

Dependency preserving?

Need original:

$\{ AB \rightarrow C \}$

Projected relations don't retain a way to enforce it locally.

Therefore:

TEXT

Lossless '   
 Dependency preserving '

Classic BCNF trade-off.

## 49. PROBLEM 5 — LOSSLESS TEST

Relation:

$\{ R(A,B,C,D) \}$

Decomposition:

$\{ R_1(A,B,C) \}$

$\{ R_2(C,D) \}$

FD:

$\{C \rightarrow D\}$

Common:

TEXT  
C

C determines:

TEXT  
CD

= all of R2.

Therefore:

$\boxed{\text{Lossless}}$

---

## 50. PROBLEM 6 — HIGHEST NF + DECOMPOSITION

---

Relation:

$R(A, B, C, D)$

FDs:

$A \rightarrow B$

$B \rightarrow C$

$C \rightarrow D$

Candidate key:

$A$

Highest NF:

TEXT  
2NF

because  $B \nrightarrow C$  violates 3NF.

BCNF decomposition can proceed:

Using  $C \rightarrow D$ :

TEXT  
 $R_1(C, D)$   
 $R_2(A, B, C)$

Then R2 still has violation  $B \nrightarrow C$ :

TEXT  
 $R_3(B, C)$   
 $R_4(A, B)$

Final:

TEXT

R1(C, D)  
R3(B, C)  
R4(A, B)

Each is BCNF.

## 51. ARE THESE DEPENDENCIES PRESERVED?

Original:

TEXT

A! ' B  
B! ' C  
C! ' D

Now:

TEXT

A! ' B   i n   A B  
B! ' C   i n   B C  
C! ' D   i n   C D

Yes.

So in this example BCNF decomposition happens to also preserve dependencies.

Important:

**BCNF may lose dependency preservation, but it doesn't always lose it.**

## 52. GATE TRAPS

### & pTrap 1

Lossless does not mean “no tuples deleted during decomposition.”

It means:

TEXT

join projections back  
=  
exact original relation

### & pTrap 2

Spurious tuples indicate lossy decomposition.

### & pTrap 3

For binary decomposition:

$[R_1 \cap R_2]$

must determine all of R1 or all of R2.

### & pTrap 4

Dependency preservation is not the same as losslessness.

### & pTrap 5

An original FD need not appear literally in one relation to be preserved.

It may be implied by local dependencies.

### & pTrap 6

BCNF decomposition:

TEXT

lossless guaranteed

dependency preservation NOT guaranteed

### & pTrap 7

3NF synthesis:

TEXT

lossless

+

dependency preserving

### & pTrap 8

Do not simply choose BCNF as “always best.”

There can be a trade-off with dependency preservation.

### & pTrap 9

Pairwise binary lossless test cannot blindly prove arbitrary multiway decompositions lossless.

### & pTrap 10

If common attributes determine one decomposed relation, binary decomposition is lossless—even if they do not

determine the entire original R.

---

## 53. PYQ-ORIENTED DECISION PROCEDURE

---

When GATE gives decomposition:

### STEP 1 — LOSSLESS

---

For binary:

TEXT

$X = R_1 \cup R_2$

Calculate  $X \bowtie Z$ .

If:

TEXT

$R_1 \cap R_2 = X \cap Z$

or:

TEXT

$R_2 \cap R_1 = X \cap Z$

then lossless.

---

### STEP 2 — DEPENDENCY PRESERVATION

---

Project FDs to each relation.

Combine them.

Check whether every original FD can be inferred.

---

### STEP 3 — NORMAL FORM

---

For each decomposed relation:

- find projected FDs
- determine keys
- test BCNF/3NF separately

---

## 54. NOTEBOOK NOTES

---

### DECOMPOSITION

---

Split:

$R \rightarrow R_1, R_2, \dots$

Goals:

TEXT

Lossless join  
Dependency preservation

## LOSSLESS JOIN

Joining decomposed relations gives exactly original relation.

No spurious tuples.

For binary decomposition:

$\forall R \rightarrow R_1, R_2$

let:

$\forall X = R_1 \cap R_2$

Lossless if:

$\forall X \rightarrow R_1$

or:

$\forall X \rightarrow R_2$

under  $(F^+)$ .

Memory:

TEXT

Common attributes  
must determine one side.

## LOSSY

Join creates:

TEXT

spurious tuples

## DEPENDENCY PRESERVATION

Original FDs can be enforced using decomposed relations without joining them.

Formally:

$\forall (\cup F_i)^+ = F^+$

## BCNF DECOMPOSITION

For violation:

[  $X \rightarrow Y$  ]

create:

TEXT

R 1 = X "\*" Y

R2 = remaining attributes + X

Repeat.

Guarantee:

TEXT

Lossless '  
Dependency preservation !' not guaranteed

## 3NF SYNTHESIS

TEXT

1. Find minimal cover.
2. Create relation for each FD / same LHS group.
3. Ensure some relation contains candidate key.
4. Remove redundant subset relations.

Guarantee:

TEXT

3NF  
Lossless  
Dependency preserving

## MEMORY PAGE

TEXT

LOSSLESS  
= DATA survives exactly

DEPENDENCY PRESERVING  
= RULES survive locally

Binary lossless:

TEXT

COMMON !' ONE SIDE

BCNF:

TEXT

Lossless yes  
Dependency preservation maybe

3NF synthesis:

TEXT

Lossless yes  
Dependency preservation yes

## NORMALIZATION MASTER SHEET

You can now connect the whole chapter:

TEXT

FDs  
!"  
Closure  
!"  
Candidate Keys  
!"  
Prime / Non-prime  
!"  
Minimal Cover  
!"  
1NF  
Atomic  
!"  
2NF  
No partial dependency  
!"  
3NF  
LHS superkey OR RHS prime  
!"  
BCNF  
LHS superkey only  
!"  
Decomposition  
!"  
Lossless + Dependency Preservation

## 5-MINUTE NORMALIZATION REVISION

### Candidate Key

TEXT

Minimal superkey

### Prime Attribute

TEXT

Part of any candidate key

### 1NF



TEXT  
Atomic

## 2NF

TEXT  
No partial dependency of non-prime attributes

## 3NF

For every non-trivial:

$[X \rightarrow A]$

TEXT  
X superkey OR A prime

## BCNF

TEXT  
X must be superkey

## Minimal Cover

TEXT  
Single RHS  
No extraneous LHS  
No redundant FD

## Binary Lossless

TEXT  
Common attributes determine one relation

## Dependency Preservation

TEXT  
All original FDs implied by local projected FDs

## 30-SECOND REVISION

TEXT  
Decomposition:  
split relation

Lossless:  
join back = exact original

Lossy:  
spurious tuples appear

Binary lossless:  
 $(R_1 \bowtie R_2) \neq R_1$   
OR  
 $(R_1 \bowtie R_2) \neq R_2$

Dependency preserving:  
all original FDs can be checked  
without joining decomposed relations

BCNF decomposition:  
lossless guaranteed  
dependency preservation not guaranteed

3NF synthesis:  
lossless + dependency preserving

## CHECK YOUR UNDERSTANDING

### Q1

Relation:

$R(A,B,C)$

FD:

$B \rightarrow C$

Decomposition:

$R_1(A,B)$

$R_2(B,C)$

Is it lossless?

### Q2

For binary decomposition, complete:

$R_1 \cap R_2 = X$

The decomposition is lossless if:

$X \rightarrow R_1 \cup R_2$

or:

$X \rightarrow R_1 \cup R_2$

### Q3

What are **spurious tuples**?

#### Q4

What is the difference between:

TEXT

Lossless join

and:

TEXT

Dependency preservation

?

#### Q5

Does BCNF decomposition always preserve dependencies?

A. Yes B. No

#### Q6

What does standard 3NF synthesis guarantee?

A. BCNF only B. Dependency preservation only C. Lossless only D. Lossless + dependency preservation + 3NF

#### Q7 — Important

Relation:

$\{ R(A,B,C) \}$

FDs:

$\{ A \rightarrow B \}$

$\{ B \rightarrow C \}$

Decomposition:

$\{ R_1(A,B), R_2(B,C) \}$

Determine:

1. Lossless or lossy?
2. Dependency preserving or not?

We have now completed the entire **Functional Dependencies + Keys + Minimal Cover + Normalization + Decomposition** block.

Next we move to a completely new part of your syllabus:

## TOPIC 11 — FILE ORGANIZATION

We will cover:

**disk/block basics !' records !' fixed/variable-length records !' blocking factor !' spanned vs unspanned !' number of blocks !' heap files !' sequential files !' hashing basics !' record access cost !' GATE numerical**

shortcuts.

Type `NEXT`.

USER

NEXT

ASSISTANT

## TOPIC 11 — FILE ORGANIZATION

This section covers the **File Organization** part of your GATE DBMS syllabus.

This topic is numerical and concept-heavy, but the questions usually come from a small number of patterns:

**records !' blocks/pages !' record size !' blocking factor !' spanned/unspanned records !' number of blocks !' heap file !' sequential file !' hashing basics !' access cost**

If you understand the formulas, this becomes a scoring topic.

### 1. WHY FILE ORGANIZATION EXISTS

So far we treated a table logically:

TEXT

STUDENT

| RollNo | Name | Dept |
|--------|------|------|
| 101    | Ravi | CSE  |
| 102    | Anu  | AIML |

But a database ultimately has to store these records on a physical device.

The DBMS does not normally fetch one tiny field directly from disk.

Storage is transferred in units called:

#### Blocks / Pages

So physical database access looks roughly like:

TEXT

Database Table  
! "

Records  
! "

Blocks / Pages  
! "

Disk / Storage

The number of block accesses is a major part of database performance.

## 2. RECORD

A **record** is a collection of related fields representing one entity/tuple in storage.

Example:

TEXT

RollNo = 101  
Name = Ravi  
Dept = CSE  
Marks = 85

Together:

TEXT

(101, Ravi, CSE, 85)

form one record.

Logical-model vocabulary:

TEXT

Tuple

Storage-level vocabulary:

TEXT

Record

They are closely related.

## 3. FIELD

A field stores one attribute value inside a record.

Example:

TEXT

Record:  
101 | Ravi | CSE | 85

Fields:

TEXT

101  
Ravi  
CSE  
85

## 4. FIXED-LENGTH RECORD

Every record occupies the same amount of space.

Example:

TEXT

|          |         |
|----------|---------|
| RollNo   | 4 bytes |
| Age      | 4 bytes |
| Marks    | 4 bytes |
| DeptCode | 4 bytes |

Record size:

$[4+4+4+4=16\text{ bytes}]$

Every record occupies:

TEXT

16 bytes

## 5. VARIABLE-LENGTH RECORD

Records may have different sizes.

Example:

TEXT

|             |
|-------------|
| Name        |
| Address     |
| Description |
| VARCHAR     |

One student's name:

TEXT

Raj

another:

TEXT

Venkata Sai Krishna

They don't need equal storage.

Variable-length records may require:

- offset tables
- delimiters
- pointers
- length information

For GATE, the most important numericals usually use **fixed-length records**.

## 6. BLOCK / PAGE

A block/page is the basic unit of data transfer between secondary storage and memory.

Suppose:

TEXT

Block size = 4096 bytes

The DBMS might read 4096 bytes together even if it needs only one small record.

Therefore:

**Fewer block accesses usually mean faster retrieval.**

## MEMORY TRICK

TEXT

Record = one row-like stored item

Block = box containing several records

## 7. BLOCKING FACTOR — BFR

The number of records that fit inside one block is called the:

**Blocking Factor**

Notation often:

$\lfloor \text{bfr} \rfloor$

For **unspanned fixed-length records**:

$\lfloor \text{bfr} = \left\lfloor \frac{B}{R} \right\rfloor \rfloor$

where:

TEXT

B = block size

R = record size

The floor is extremely important.

## 8. WHY FLOOR?

Suppose:

TEXT

Block size = 1000 bytes

Record size = 300 bytes

Then:

$\lfloor 1000/300 = 3.333 \rfloor$

Can we fit 3.333 records?

No.

Only:

TEXT

3 complete records

So:

$\lfloor \text{bfr} = \lfloor \text{floor} 3.333 \rfloor = 3 \rfloor$

## MEMORY

**Blocking factor always counts complete records.**

So use:

TEXT

FLOOR

for unspanned organization.

## 9. SOLVED EXAMPLE 1

Given:

TEXT

Block size = 4096 bytes

Record size = 100 bytes

Find blocking factor.

$\lfloor \text{bfr} = \left\lfloor \text{floor} \frac{4096}{100} \right\rfloor \rfloor$

$\lfloor = 40 \rfloor$

because:

TEXT

$40 \times 100 = 4000$  bytes

41 records need:

TEXT

4100 bytes

which exceeds the block size.



## Answer

$\boxed{40}$

## 10. UNUSED SPACE / BLOCK WASTAGE

In previous example:

TEXT

Block size = 4096  
Used =  $40 \times 100 = 4000$

Unused space:

$4096 - 4000 = 96$

bytes.

Formula:

$\boxed{\text{Waste} = B - (bfr \times R)}$

for fixed-length unspanned blocking.

## 11. SPANNED vs UNSPANNED RECORDS

Very important distinction.

### Unspanned

A record must fit completely inside one block.

A record cannot cross a block boundary.

TEXT

BLOCK 1  
| R1 | R2 | R3 | unused |

BLOCK 2  
| R4 | R5 | R6 | unused |

### Spanned

A record may start in one block and continue in the next.

TEXT

BLOCK 1  
| R1 | R2 | part of R3 |

BLOCK 2  
| remaining R3 | R4 | ...

## MEMORY TRICK

### TEXT

Unspanned = Unbroken record

Spanned = Split record allowed

## 12. ADVANTAGE OF UNSPANNED

Simple record access.

A record is entirely inside one block.

But:

### TEXT

some block space may be wasted

## 13. ADVANTAGE OF SPANNED

Better storage utilization.

Less unused block space.

But:

### TEXT

one record may require two block accesses

and record management is more complex.

## 14. SPANNED vs UNSPANNED

| Unspanned                     | Spanned                                   |
|-------------------------------|-------------------------------------------|
| Record cannot cross block     | Record may cross blocks                   |
| Simpler access                | More complex                              |
| More wasted space possible    | Better storage use                        |
| Record typically in one block | One record may occupy parts of two blocks |

## 15. NUMBER OF BLOCKS — UNSPANNED

Suppose:

TEXT

$r$  = number of records  
 $bfr$  = records per block

Then number of blocks needed:

$$\lceil b = \left\lceil \frac{r}{bfr} \right\rceil \rceil$$

Notice:

Here we use CEILING.

## WHY CEILING?

Suppose:

TEXT

100 records  
30 records per block

$$100/30=3.333$$

3 blocks store only:

TEXT

90 records

We still need another block.

So:

$$b=4$$

## CRITICAL MEMORY

TEXT

Records per block  
!' F L O O R

Blocks required  
!' C E I L I N G

This is one of the most important numerical shortcuts.

## 16. SOLVED EXAMPLE 2

Given:

TEXT

Block size = 1024 bytes  
Record size = 100 bytes

Number of records = 1000

### Step 1 — Blocking factor

$\lceil bfr = \left\lfloor \frac{1024}{100} \right\rfloor = 10 \rceil$

### Step 2 — Number of blocks

$\lceil b = \left\lceil \frac{1000}{10} \right\rceil = 100 \rceil$

### Answer

TEXT

Blocking factor = 10  
Blocks = 100

## 17. SOLVED EXAMPLE 3

Given:

TEXT

B = 4096 bytes  
R = 150 bytes  
r = 10,000 records

### BFR

$\lceil \left\lfloor \frac{4096}{150} \right\rfloor \rceil$

$150 \times 27:$

TEXT

4050

$150 \times 28:$

TEXT

$4200 > 4096$

Therefore:

$\lceil bfr = 27 \rceil$

### Blocks

$\lceil b = \left\lceil \frac{10000}{27} \right\rceil \rceil$

$\lceil = 371 \rceil$

because:

TEXT

$27 \times 370 = 9990$

10 records remain.

So one additional block.

### Answer

$\lceil \frac{371}{10000} \rceil$

## GATE SHORTCUT

Don't waste time doing long decimal division.

For:

$\lceil \frac{10000}{27} \rceil$

notice:

TEXT

$27 \times 370 = 9990$

so instantly:

TEXT

371 blocks

## 18. NUMBER OF BLOCKS — SPANNED

When records are allowed to span blocks, a common basic approximation for fixed-size records is based on total bytes:

$\lceil \frac{R}{B} \rceil$

because unused fragments can be used by portions of records.

## 19. SPANNED EXAMPLE

Given:

TEXT

1000 records  
Record size = 100 bytes  
Block = 1024 bytes

Total size:

$1000 \times 100 = 100000$

Then:

$\lceil \frac{100000}{1024} \rceil$

$\lceil 97.65625 \rceil = 98$

Compare:

## Unspanned

TEXT

```
10 records/block
!' 1 0 0   b l o c k s
```

## Spanned

TEXT

```
98 blocks
```

Spanning reduced wasted space.

## & pGATE TRAP

Don't automatically use:

$\lfloor \frac{r}{bfr} \rfloor$

for spanned storage.

For spanned basic byte-level calculation, think:

$\lceil \frac{\text{Total record bytes}}{\text{Block size}} \rceil$

then ceiling.

## 20. BLOCKING FACTOR WHEN RECORD > BLOCK

Suppose:

TEXT

```
Record size = 5000 bytes
Block size = 4096 bytes
```

Unspanned organization cannot store such a record in a single block.

Spanning is necessary or another storage mechanism must be used.

## PART B — FILE ORGANIZATION TYPES

### 21. WHAT IS FILE ORGANIZATION?

File organization determines:

**How records are physically arranged in blocks/files.**

Important GATE-relevant organizations include:

#### TEXT

Heap / Unordered  
Sequential / Ordered  
Hash-based  
Indexed

Indexing will become the next major chapter.

## 22. HEAP FILE ORGANIZATION

Heap means records are stored in **no particular ordering**.

New records can often be placed wherever free space is available.

Conceptually:

#### TEXT

Block 1:  
Raj  
Anu  
Kiran

Block 2:  
Ravi  
Neha

No ordering by RollNo, Name, etc.

## 23. ADVANTAGE OF HEAP

Insertion is usually very fast.

Why?

We don't need to maintain sorted order.

Simply:

#### TEXT

```
find free location  
!' insert
```

## 24. DISADVANTAGE OF HEAP

Searching for a record without an index may require a **linear scan**.

If file has:

[ b ]

blocks:

**Worst case**

[ b ]

block accesses.

### Average successful search

approximately:

$$\lceil \frac{b+1}{2} \rceil$$

if records are uniformly distributed and search scans sequentially.

## HEAP MEMORY

**Heap = Easy Insert, Hard Search**

## 25. HEAP ACCESS COST

For equality search without index:

### Successful average

$$\lceil \frac{b+1}{2} \rceil$$

### Unsuccessful

$$b$$

because all blocks may need to be checked.

## 26. SEQUENTIAL / ORDERED FILE

Records are stored in sorted order based on a search key.

Example sorted by StudentID:

#### TEXT

```
101 Ravi
102 Anu
103 Raj
104 Neha
105 Riya
```

## 27. ADVANTAGE OF ORDERED FILE

Searching based on the ordering field can be much faster.

If blocks are ordered, binary search may be possible.

Cost roughly:

$$\lceil \log_2 b \rceil$$

block accesses for locating the relevant block, depending on implementation assumptions.



## 28. EXAMPLE

Suppose ordered file has:

```
TEXT
1024 blocks
```

Binary search needs approximately:

$\lceil \log_2 1024 \rceil = 10$

block comparisons.

Instead of:

```
TEXT
~512 average blocks
```

for a linear scan of an unordered file.

Huge improvement.

## 29. DISADVANTAGE OF SEQUENTIAL FILE

Insertion becomes more expensive.

If new key:

```
TEXT
103.5
```

must appear between:

```
TEXT
103
104
```

the system may need:

- record movement
- overflow areas
- reorganization

Therefore:

```
TEXT
Heap
!' insertion easy

Ordered
!' searching / range access easier
```

## 30. RANGE QUERIES

Sequential/ordered organization is useful for queries like:

**Find RollNo between 100 and 200.**

Because once the first matching record is located, nearby records are physically ordered.

Hashing, by contrast, is usually better for exact equality searches but poor for range queries.

## MEMORY

TEXT

Ordered file

!' e q u a l i t y o k a y

!' R A N G E v e r y g o o d

## 31. HEAP vs SEQUENTIAL

| Heap                        | Sequential / Ordered       |
|-----------------------------|----------------------------|
| No ordering                 | Sorted on search key       |
| Fast insert                 | Insert more costly         |
| Linear search without index | Binary-search possibility  |
| Range search poor           | Range search good          |
| Simple organization         | Needs ordering maintenance |

## PART C — HASH FILE ORGANIZATION

### 32. WHAT IS HASHING?

Hashing applies a function to the search key to determine the bucket/block.

Example:

$\lfloor h(\text{Key}) = \text{Key} \bmod 10 \rfloor$

For:

TEXT

Key = 123

$\lfloor 123 \bmod 10 = 3 \rfloor$

So record goes to:

TEXT

Bucket 3

### 33. HASH FUNCTION

General idea:

$[ h(K) \rightarrow \text{bucket} ]$

Instead of searching through many blocks:

TEXT

Key

!"

Hash Function

!"

Bucket address

### 34. WHY HASHING IS FAST

For equality search:

TEXT

WHERE RollNo = 123

the DBMS computes:

$[ h(123) ]$

and directly identifies the expected bucket.

Under ideal conditions, access cost can be close to:

TEXT

constant time

in terms of bucket lookup.

### 35. HASHING IS GOOD FOR

TEXT

Equality search

Example:

SQL

WHERE StudentID = 101

## 36. HASHING IS BAD FOR

TEXT

Range search

Example:

SQL

```
WHERE StudentID BETWEEN 100 AND 200
```

because nearby key values may hash to unrelated buckets.

## MEMORY TRICK

Hash = Exact Match

Ordered = Range

Very important comparison.

## 37. COLLISION

A collision occurs when different keys hash to the same bucket.

Example:

$h(K) = K \bmod 10$

Then:

TEXT

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 2 | 3 | ! | ' | 3 |
| 2 | 1 | 3 | ! | ' | 3 |
| 3 | 4 | 3 | ! | ' | 3 |

Different records map to Bucket 3.

This is a collision.

## 38. COLLISION HANDLING

Conceptually can involve:

- overflow buckets
- chaining
- alternative techniques

For GATE DBMS storage, understand that collisions increase access cost.

## 39. STATIC vs DYNAMIC HASHING — BASIC IDEA

### Static hashing

Number of buckets fixed.

As file grows:

TEXT

more collisions / overflow

may occur.

### Dynamic hashing

Bucket structure can expand/contract.

Examples include:

- extendible hashing
- linear hashing

Unless explicitly needed in your syllabus/PYQs, we won't go deeply beyond the required concepts.

## PART D — RECORD ACCESS COST

### 40. WHY BLOCK ACCESSES MATTER

CPU operations are much faster than secondary-storage accesses.

Therefore database algorithms are often analyzed using:

**Number of block I/Os**

not just number of comparisons.

### 41. HEAP FILE SEARCH COST

File contains:

$\lfloor \frac{b}{2} \rfloor$

blocks.

**Equality search, successful average:**

$\lfloor \frac{b}{2} \rfloor$

approximately.

**Equality search, unsuccessful:**

$\lfloor \frac{b}{2} \rfloor$

## 42. ORDERED FILE SEARCH COST

Binary search over blocks:

$\lceil \log_2 \lceil \frac{b}{B} \rceil \rceil$

roughly for locating the block.

Then record search inside memory is cheap relative to another disk access.

## 43. HASH FILE SEARCH COST

Ideal equality search:

TEXT

approximately 1 bucket access

plus overflow accesses if collisions occur.

So conceptual ranking for equality lookup:

TEXT

Hash  
usually very good

Ordered  
good

Heap  
slow without index

## 44. RANGE SEARCH COMPARISON

For range queries:

TEXT

Ordered file  
!' e x c e l l e n t

Hash  
!' p o o r

Heap  
!' g e n e r a l l y p o o r

## 45. FILE ORGANIZATION CHOICE

Use this mental table:

| Need                     | Good choice |
|--------------------------|-------------|
| Very frequent insertions | Heap        |

| Need                     | Good choice         |
|--------------------------|---------------------|
| Equality search          | Hash                |
| Range queries            | Ordered/sequential  |
| Mixed efficient searches | Indexing structures |

Indexing gives us much more flexibility, which we learn next.

## PART E — RECORD SIZE NUMERICALS

### 46. CALCULATING RECORD SIZE

Suppose schema:

TEXT

```

S t u d e n t I D    ! '    4    b y t e s
N a m e             ! '    3 0    b y t e s
A g e               ! '    4    b y t e s
M a r k s           ! '    4    b y t e s

```

Fixed record size:

$R = 4 + 30 + 4 + 4$

$R = 42 \text{ bytes}$

If block = 512 bytes:

$bfr = \lfloor 512 / 42 \rfloor$

$42 \times 12$ :

TEXT

504

Therefore:

$bfr = 12$

Unused:

$512 - 504 = 8$

bytes.

### 47. SOLVED GATE NUMERICAL

A file has:

TEXT

```

30,000 records
Record size = 200 bytes
Block size = 4096 bytes

```

Unspanned

Find blocks.

### Step 1

$\lfloor \frac{4096}{200} \rfloor$

20 records use:

TEXT

4000 bytes

21 need:

TEXT

4200

So:

$\lfloor \frac{4096}{200} \rfloor$

### Step 2

$\lceil \frac{4096}{200} \rceil = 21$

### Answer

$\boxed{21 \text{ blocks}}$

## 48. SOLVED GATE NUMERICAL — WASTAGE

Using same values:

TEXT

Block = 4096  
Record = 200  
20 records/block

Used:

$20 \times 200 = 4000$

Unused:

$4096 - 4000 = 96$

bytes/block.

Total unused if all 1500 blocks were equally filled:

$1500 \times 96 = 144000$

bytes.

But note:

The last block's actual occupancy may need separate treatment if record count isn't an exact multiple.



---

## 49. LAST BLOCK TRAP

Suppose:

```
TEXT
101 records
10 records/block
```

Number of blocks:

```
TEXT
11
```

First 10 blocks:

```
TEXT
10 records each
```

Last block:

```
TEXT
1 record
```

So you cannot assume every allocated block has the same unused space.

This matters in exact storage-wastage questions.

---

## 50. SOLVED LAST-BLOCK EXAMPLE

Given:

```
TEXT
Block = 1000 bytes
Record = 300 bytes
Records = 10
Unspanned
```

BFR:

$\lceil 3 \rceil$

Blocks:

$\lceil \lceil 10/3 \rceil \rceil = 4$

First 3 blocks:

```
TEXT
3 records each
!' 900 bytes used
!' 100 bytes waste each
```

Last block:

TEXT

```
1 record
!' 3 0 0   u s e d
!' 7 0 0   w a s t e
```

Total unused:

$\lfloor 3(100)+700 \rfloor$

$\lfloor =1000 \rfloor$

bytes.

## 51. RECORD ADDRESSING — BASIC IDEA

To locate a record, DBMS needs information such as:

TEXT

```
Block number
+
offset within block
```

Conceptually:

TEXT

```
Record Address
=
(Page ID, Slot)
```

Variable-length pages often use a **slot directory** to track record offsets.

This is storage implementation context; GATE usually focuses more on block calculations and indexing.

## 52. SPANNED RECORD POINTER OVERHEAD

If a record crosses block boundary, the system may require a pointer/continuation indication to locate its remaining portion.

Therefore pure:

$\lfloor \lceil rR/B \rceil \rfloor$

is the simple theoretical calculation when no extra overhead is specified.

If GATE specifies pointer/header overhead:

**include it.**

Never silently ignore explicitly given metadata sizes.

## PART F — IMPORTANT FORMULAS

## 53. UNSPANNED BLOCKING FACTOR

$\boxed{bfr = \left\lfloor \frac{B}{R} \right\rfloor}$

## 54. UNSPANNED NUMBER OF BLOCKS

$\boxed{b = \left\lceil \frac{r}{bfr} \right\rceil}$

## 55. SPANNED BLOCKS — BASIC

$\boxed{b = \left\lceil \frac{rR}{B} \right\rceil}$

assuming no other overhead.

## 56. UNUSED SPACE PER FULL UNSPANNED BLOCK

$\boxed{B - (bfr \times R)}$

## 57. HEAP SEARCH

Successful average:

$\boxed{b/2}$

Unsuccessful:

$\boxed{b}$

roughly.

## 58. ORDERED FILE BINARY SEARCH

$\boxed{\lceil \log_2 b \rceil}$

approximately.

## 59. GATE SHORTCUT FORMULA MEMORY

TEXT

Fit records?

!' F L O O R

Need blocks?

!' C E I L I N G

Search heap?

!' H A L F   b l o c k s   a v e r a g e

Search ordered?

!' L O G ,   b l o c k s

Excellent revision shortcut.

---

## PART G — GATE TRAPS

---

### & pTrap 1 — Floor vs Ceiling

---

Blocking factor:

TEXT  
FLOOR

Blocks required:

TEXT  
CEILING

---

### & pTrap 2 — Unspanned vs Spanned

---

Unspanned wastes leftover space.

Spanned can use leftover space across block boundaries.

---

### & pTrap 3 — Block vs Record

---

A database I/O usually reads a block/page, not just one tiny record field.

---

### & pTrap 4 — Heap search

---

Without an index, equality lookup may require linear block scanning.

---

### & pTrap 5 — Ordered file

---

Good search doesn't mean good insertion.

Maintaining sorted order can be expensive.

---

### & pTrap 6 — Hash

---

Excellent for equality does not imply good for range queries.

---

### & pTrap 7 — Binary search

---

Binary search advantage assumes ordered blocks on the relevant search field.

You cannot binary-search an arbitrary heap file.

## & pTrap 8 — Last block

Don't assume last block is full.

## & pTrap 9 — Record size overhead

If pointer/header/record-marker sizes are explicitly given, include them.

## & pTrap 10 — Spanned record cost

A spanned record itself may require portions from two blocks.

Better packing doesn't automatically mean every individual record access is cheaper.

## PART H — PYQ-ORIENTED QUESTION TYPES

GATE can ask:

### Pattern 1

Given:

TEXT

Block size  
Record size

find:

TEXT

Blocking factor

### Pattern 2

Given:

TEXT

Number of records  
BFR

find blocks.

### Pattern 3

Compare:

TEXT

Spanned vs unspanned

block usage.

#### Pattern 4

Find unused block space.

---

#### Pattern 5

Given number of blocks, calculate average/worst search cost.

---

#### Pattern 6

Choose best organization for:

TEXT

equality search  
range search  
insertion

---

#### Pattern 7

Compare heap, ordered and hash files.

---

## 60. SOLVING PROCEDURE FOR NUMERICALS

---

When you see file-organization numerical:

### STEP 1

Write:

TEXT

B = block size  
R = record size  
r = number of records

### STEP 2

Check:

TEXT

Spanned or unspanned?

### STEP 3

For unspanned:

$\lfloor bfr = \lfloor \text{floor } B/R \rfloor \rfloor$

### STEP 4

Then:

$\lfloor b = \lceil \text{ceil } r/bfr \rceil \rfloor$

### STEP 5

If asked search cost:

Use:

TEXT

```
H e a p ! ' ~ b / 2 a v e r a g e  
O r d e r e d ! ' l o g , b
```

subject to problem assumptions.

---

## NOTEBOOK NOTES

---

### FILE ORGANIZATION

---

Physical arrangement of database records in blocks/pages.

TEXT

```
T u p l e ! ' l o g i c a l  
R e c o r d ! ' p h y s i c a l / s t o r a g e r e p r e s e n t a t i o n  
B l o c k / P a g e ! ' I / O u n i t
```

---

### FIXED vs VARIABLE RECORD

---

TEXT

```
F i x e d ! ' s a m e s i z e  
V a r i a b l e ! ' d i f f e r e n t s i z e s
```

---

### UNSPANNED

---

Record cannot cross block.

$$\lfloor bfr = \left\lfloor \frac{BR}{r} \right\rfloor \rfloor$$

$$\lfloor b = \left\lceil \frac{r}{B} \right\rceil \rfloor$$

---

### SPANNED

---

Record may cross blocks.

Basic:

$$\lfloor b = \left\lceil \frac{rR}{B} \right\rceil \rfloor$$

---

### WASTAGE

---

$$\lfloor \text{Waste}_{\text{fullblock}} = B - bfrR \rfloor$$

---

### HEAP

---

#### TEXT

Unordered

I n s e r t ! ' f a s t  
S e a r c h ! ' l i n e a r

Average successful search:

$\lfloor b/2 \rfloor$

Worst/unsuccessful:

$\lfloor b \rfloor$

## ORDERED / SEQUENTIAL

#### TEXT

Sorted records

E q u a l i t y ! ' g o o d  
R a n g e ! ' v e r y g o o d  
I n s e r t i o n ! ' c o s t l y

Binary search:

$\lfloor \log_2 b \rfloor$

approximately.

## HASH

#### TEXT

H a s h k e y ! ' b u c k e t  
  
E q u a l i t y ! ' e x c e l l e n t  
R a n g e ! ' p o o r  
C o l l i s i o n ! ' m u l t i p l e k e y s s a m e b u c k e t

## MEMORY PAGE

#### TEXT

F L O O R ! ' R e c o r d s F I T i n b l o c k  
  
C E I L I N G ! ' B l o c k s N E E D E D  
  
H e a p ! ' I n s e r t  
  
H a s h ! ' E x a c t e q u a l i t y  
  
O r d e r e d ! ' R a n g e

And:



#### TEXT

Unspanned = record unbroken

Spanned = record can split

## 30-SECOND REVISION

#### TEXT

Record = stored tuple

Block/Page = disk I/O unit

Unspanned:  
record cannot cross block

Spanned:  
record may cross block

BFR =  
 $\text{floor}(\text{BlockSize} / \text{RecordSize})$

Blocks unspanned =  
 $\text{ceil}(\text{records} / \text{BFR})$

Blocks spanned " $H$ "  
 $\text{ceil}(\text{total record bytes} / \text{block size})$

Heap:  
fast insertion, linear search

Ordered:  
binary search + good ranges,  
but insert costly

Hash:  
excellent equality search,  
poor range search

FLOOR = fit  
CEILING = need

## CHECK YOUR UNDERSTANDING

### Q1

Block size:

#### TEXT

4096 bytes

Record size:

#### TEXT

250 bytes

For unspanned records, calculate:

1. Blocking factor
2. Unused bytes in each full block

## Q2

A file contains:

```
TEXT
10,000 records
```

and:

```
TEXT
bfr = 25
```

How many blocks are needed?

## Q3

Which organization is usually best suited for an equality search such as:

```
SQL
WHERE StudentID = 1001
```

A. Heap B. Hash C. Ordered only D. None

## Q4

Which is usually better for:

```
SQL
WHERE Salary BETWEEN 50000 AND 80000
```

A. Hash file B. Ordered file

## Q5

A heap file contains:

```
TEXT
1000 blocks
```

Approximately how many block accesses are needed for an average successful linear search without an index?

## Q6

What is the major difference between:

TEXT

Spanned

and:

TEXT

Unspanned

records?

## Q7 — Numerical

Given:

TEXT

Block size = 1000 bytes  
Record size = 300 bytes  
Number of records = 100

For **unspanned** organization, find:

1. BFR
2. Number of blocks

Next comes one of the **highest-scoring numerical areas**:

## TOPIC 12 — INDEXING

We will cover:

**search key !' primary index !' clustering index !' secondary index !' dense vs sparse index !' index-entry size !' fan-out !' multilevel indexes !' disk-access calculations !' GATE numerical tricks**, and then move directly into **B-Tree and B+ Tree**.

Type ``NEXT``.

USER

NEXT

ASSISTANT

## TOPIC 12 — INDEXING

This section covers the **Indexing** part of your GATE DBMS syllabus.

Indexing is one of the most important numerical topics because GATE can ask:

- primary index
- clustering index
- secondary index

- dense vs sparse index
- ordered index
- index blocking factor
- number of index blocks
- multilevel index
- number of levels
- disk accesses
- search-key vs primary-key confusion

The central idea is simple:

**An index is a smaller structure that helps us find records without scanning the whole data file.**

## 1. WHY DO WE NEED AN INDEX?

Suppose a file contains:

TEXT

1,000,000 student records

Without an index, finding:

SQL

WHERE RollNo = 573421

may require searching many data blocks.

Instead, we create a smaller structure:

TEXT

| Search Key  | Pointer         |
|-------------|-----------------|
| 1 0 0 0 0 1 | ! ' B l o c k 1 |
| 1 0 0 1 0 1 | ! ' B l o c k 2 |
| 1 0 0 2 0 1 | ! ' B l o c k 3 |
| ...         |                 |

Now we search the smaller index first.

Then use its pointer to reach the actual data block.

## MEMORY TRICK

Think of a textbook.

Without an index:

**Search every page.**

With an index:

Find keyword !' page number !' jump directly.

Database indexing works similarly.

## 2. BASIC INDEX ENTRY

An index entry usually contains:

`\[ <SearchKey,\Pointer> \]`

For example:

TEXT

```
(101, P1)
(201, P2)
(301, P3)
```

where:

TEXT

```
101 = search-key value
P1 = pointer to block/record
```

## 3. SEARCH KEY

A **search key** is the attribute or set of attributes used to look up records through the index.

Example:

SQL

```
WHERE Dept = 'CSE'
```

If we have an index on Dept:

TEXT

```
Dept is the search key
```

## & pVERY IMPORTANT GATE TRAP

Search key does NOT necessarily mean candidate key or primary key.

For example:

TEXT

```
Dept
```

may contain duplicates.

Still, it can be used as an index search key.

So:

TEXT

```
Search key " necessarily unique key
```

## 4. ORDERED INDEX

An ordered index stores index entries ordered by the search-key value.

Example:

TEXT

```
1 0 !' pointer
2 0 !' pointer
3 0 !' pointer
4 0 !' pointer
```

This supports efficient lookup.

Major ordered-index types:

TEXT

```
Primary Index
Clustering Index
Secondary Index
```

And separately:

TEXT

```
Dense
Sparse
```

These classifications must not be confused.

## 5. PRIMARY INDEX

A **primary index** is built when the data file itself is ordered on a **key field**, typically the primary key.

Example:

Data file ordered by:

TEXT

```
RollNo
```

and RollNo is unique.

Then an index on RollNo is a primary index.

## 6. PRIMARY INDEX EXAMPLE

Data blocks:

#### TEXT

Block 1:

101  
102  
103

Block 2:

104  
105  
106

Block 3:

107  
108  
109

Primary index may contain:

#### TEXT

|   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 0 | 1 | ! | ' | B | l | o | c | k | 1 |
| 1 | 0 | 4 | ! | ' | B | l | o | c | k | 2 |
| 1 | 0 | 7 | ! | ' | B | l | o | c | k | 3 |

Notice:

**We do not necessarily need an index entry for every record.**

One entry per data block can be enough because data inside the file is ordered.

Therefore a primary index is commonly **sparse**.

## MEMORY TRICK

**Primary index = file ordered on unique key**

## 7. SPARSE PRIMARY INDEX

Suppose each data block contains many sorted records.

We can store one index entry for the first record in each block:

#### TEXT

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 1 | 0 | 1 | ! | ' | B | 1 |
| 2 | 0 | 1 | ! | ' | B | 2 |
| 3 | 0 | 1 | ! | ' | B | 3 |

To find:

#### TEXT

247

index tells us:

TEXT

2 0 1 "d 2 4 7 < 3 0 1

Therefore search:

TEXT

Block B2

Because the data file itself is sorted.

## 8. CLUSTERING INDEX

A clustering index is typically built when:

**The file is physically ordered on a non-key field.**

Example:

Data sorted by:

TEXT

Department

But multiple students may belong to the same department.

TEXT

CSE  
CSE  
CSE  
ECE  
ECE  
AIML  
...

The ordering field is not unique.

Then an index on that ordering field is a **clustering index**.

## 9. CLUSTERING INDEX EXAMPLE

Data file:

TEXT

Block 1:  
AIML Ravi  
AIML Anu

Block 2:  
CSE Raj  
CSE Neha

Block 3:  
CSE Kiran



ECE Riya

A clustering index may contain entries like:

TEXT

```
A I M L !' f i r s t A I M L b l o c k
C S E !' f i r s t C S E b l o c k
E C E !' f i r s t E C E b l o c k
```

Records having the same search-key value are stored together or clustered physically.

## MEMORY TRICK

**Clustering = same values physically grouped together.**

## 10. PRIMARY vs CLUSTERING INDEX

| Primary Index              | Clustering Index              |
|----------------------------|-------------------------------|
| File ordered on key field  | File ordered on non-key field |
| Search key unique          | Search key can repeat         |
| Usually sparse             | Usually sparse/group-based    |
| One logical record per key | Many records may share key    |

## 11. SECONDARY INDEX

A secondary index is built on an attribute that is **not the physical ordering field of the data file**.

Suppose file is physically ordered on:

TEXT

RollNo

but we also want fast searches on:

TEXT

Name

Then an index on Name is a:

**Secondary index**

## 12. WHY SECONDARY INDEX IS DIFFERENT

The data file is not ordered according to the secondary search key.

Therefore an index entry such as:

```
TEXT
R a v i ! ' ?
```

cannot simply point to one nearby block and then rely on ordering.

Records with Name = Ravi may be scattered.

Therefore secondary indexes are usually **dense**.

## MEMORY

**Secondary = index on non-ordering attribute**

## 13. DENSE INDEX

A **dense index** has an index entry for:

**every search-key value occurrence or every record, depending on key uniqueness and implementation.**

For a unique search key:

```
TEXT
1 0 1 ! '   r e c o r d   p o i n t e r
1 0 2 ! '   r e c o r d   p o i n t e r
1 0 3 ! '   r e c o r d   p o i n t e r
...
```

One index entry per data record.

## 14. SPARSE INDEX

A sparse index has entries only for some search-key values.

Example:

```
TEXT
1 0 1 ! '   B l o c k   1
2 0 1 ! '   B l o c k   2
3 0 1 ! '   B l o c k   3
```

while individual records:

TEXT

102, 103, ... 199

may not have direct index entries.

Sparse index works only when the data file is ordered appropriately on the indexed field.

## 15. DENSE vs SPARSE

| Dense                                        | Sparse                               |
|----------------------------------------------|--------------------------------------|
| Entry for every record/search-key occurrence | Entries for only some values         |
| Larger index                                 | Smaller index                        |
| Faster direct lookup                         | May require search within data block |
| Can support secondary index                  | Requires ordered data file           |
| More storage                                 | Less storage                         |

## MEMORY TRICK

TEXT

Dense = many index entries

Sparse = few index entries

## 16. IMPORTANT RELATIONSHIP

Typical GATE-level assumptions:

TEXT

Primary index  
!' sparse

Clustering index  
!' sparse / one per distinct cluster

Secondary index  
!' dense

But these are common design properties, not universal mathematical identities.

For exam questions, use the assumptions explicitly stated or the conventional GATE model.

## 17. WHY SPARSE INDEX REQUIRES ORDERED DATA

Suppose sparse index says:

TEXT

```
2 0 0 !' B l o c k 5
3 0 0 !' B l o c k 8
```

To find:

TEXT

250

we know it should lie between those entries **only because the data file is sorted on that field.**

If data were unordered, this logic would fail.

Therefore:

**Sparse indexes require an ordered data file on the search key.**

## 18. SECONDARY INDEX ON UNIQUE FIELD

Suppose data file ordered by RollNo.

We create secondary index on unique Email.

Then:

TEXT

```
E m a i l !' R e c o r d   p o i n t e r
```

one entry per record.

Dense index.

## 19. SECONDARY INDEX ON NON-UNIQUE FIELD

Suppose secondary index on:

TEXT

Dept

and many students share CSE.

Possible structure:

TEXT

```
C S E !'   b u c k e t / l i s t   o f   p o i n t e r s
A I M L !'  b u c k e t / l i s t   o f   p o i n t e r s
E C E !'   b u c k e t / l i s t   o f   p o i n t e r s
```

The pointer may lead to:

TEXT

list/bucket of record pointers

rather than one record.

Conceptually:

TEXT

CSE

! "

[p1, p7, p9, p20, ...]

## 20. PRIMARY vs SECONDARY INDEX

| Primary                      | Secondary                                  |
|------------------------------|--------------------------------------------|
| Based on ordering key        | Based on non-ordering field                |
| Data file physically ordered | Data not physically ordered by index field |
| Usually one                  | Multiple secondary indexes possible        |
| Usually sparse               | Usually dense                              |
| Smaller                      | Often larger                               |

## & PGATE TRAP

A file can be physically ordered only one main way.

Therefore:

**There can normally be only one primary/clustering ordering index on a file.**

But:

**Many secondary indexes can exist.**

## 21. PRIMARY INDEX vs PRIMARY KEY

These are not identical concepts.

### Primary key

Logical relational constraint:

TEXT

uniquely identifies rows

## Primary index

Physical access structure:

TEXT

index based on the field used to physically order the file

Often that field is the primary key, but don't confuse the logical and physical concepts.

## 22. INDEX RECORD SIZE

An index entry generally contains:

TEXT

Search key  
+  
Pointer

So:

$$R_i = V + P$$

where:

TEXT

$R_i$  = index record size  
 $V$  = search-key size  
 $P$  = pointer size

## 23. SOLVED INDEX ENTRY EXAMPLE

Given:

TEXT

Search key = 12 bytes  
Block pointer = 8 bytes

Index record size:

$$12 + 8 = 20$$

bytes.

## 24. INDEX BLOCKING FACTOR

If block size:

$$B$$

and index-entry size:

$$R_i$$

then index blocking factor:

$\lceil \boxed{\text{bfr}_i = \left\lfloor \frac{B}{R_i} \right\rfloor} \rceil$

This means:

**Number of index entries per index block.**

## 25. SOLVED EXAMPLE

Given:

TEXT

Block size = 4096 bytes  
Key = 12 bytes  
Pointer = 8 bytes

Index entry:

$\lceil R_i = 20 \rceil$

Then:

$\lceil \text{bfr}_i = \left\lfloor \frac{4096}{20} \right\rfloor \rceil$

$\lceil = 204 \rceil$

because:

TEXT

$204 \times 20 = 4080$   
 $205 \times 20 = 4100 > 4096$

So:

$\lceil \boxed{204} \rceil$

entries/block.

## 26. NUMBER OF PRIMARY INDEX ENTRIES

This is important.

If primary index is sparse and has one entry per **data block**:

TEXT

Number of index entries = number of data blocks

Not number of records.

## GATE TRAP

If:

TEXT

Data records = 100,000  
Data blocks = 2,000

then sparse primary index likely has:

TEXT

2,000 entries

not:

TEXT

100,000 entries

---

## 27. NUMBER OF SECONDARY INDEX ENTRIES

For a dense secondary index on a unique field:

TEXT

Index entries = number of data records

So if:

TEXT

$r = 100,000$  records

then:

TEXT

100,000 index entries

typically.

---

## 28. NUMBER OF INDEX BLOCKS

If:

TEXT

$n_i$  = number of index entries  
 $bfr_i$  = entries per index block

then:

$\lceil b_i \rceil = \left\lceil \frac{n_i}{bfr_i} \right\rceil$

Same familiar pattern:

TEXT

$\lceil \frac{n_i}{bfr_i} \rceil$



## 29. COMPLETE PRIMARY INDEX NUMERICAL

Suppose:

TEXT

Data records = 100,000  
 Record size = 100 bytes  
 Block size = 1000 bytes  
 Unspanned  
 Index key = 10 bytes  
 Block pointer = 10 bytes

Find:

1. data blocking factor
2. data blocks
3. primary index entries
4. index entry size
5. index blocking factor
6. index blocks

### Step 1 — Data BFR

$$bfr = \left\lfloor \frac{1000}{100} \right\rfloor = 10$$

### Step 2 — Data blocks

$$b = \left\lceil \frac{100000}{10} \right\rceil = 10000$$

### Step 3 — Primary index entries

Sparse primary index:

TEXT

one entry per data block

So:

$$n_i = 10000$$

### Step 4 — Index entry size

$$R_i = 10 + 10 = 20$$

bytes.

---

## Step 5 — Index BFR

---

$\lfloor \text{bfr\_i} = \left\lfloor \frac{1000}{20} \right\rfloor = 50 \rfloor$

---

## Step 6 — Index blocks

---

$\lceil \text{b\_i} = \left\lceil \frac{10000}{50} \right\rceil = 200 \rceil$

### Final

#### TEXT

```
Data BFR = 10
Data blocks = 10,000
Index entries = 10,000
Index BFR = 50
Index blocks = 200
```

---

## 30. WHY INDEX HELPS

---

Without index:

#### TEXT

search up to thousands of data blocks

With index:

#### TEXT

```
search ~200 index blocks
then 1 data block
```

Even better, we can build an index **on the index itself**.

That gives:

### Multilevel indexing

---

## PART B — MULTILEVEL INDEX

---

### 31. WHY MULTILEVEL INDEX?

---

Suppose first-level index still has:

#### TEXT

200 blocks

Searching those 200 blocks linearly is better than 10,000 data blocks, but we can do even better.

Create another sparse index on the first-level index.

TEXT

Level 2

!"

Level 1

!"

Data blocks

## 32. BASIC MULTILEVEL IDEA

Suppose:

TEXT

Level 1 index = 200 blocks

and one upper-level index entry points to each Level-1 block.

If upper-level index blocking factor is 50:

$\lceil 200/50 \rceil = 4$

blocks at Level 2.

Now build another level:

$\lceil 4/50 \rceil = 1$

block.

So structure:

TEXT

|           |    |               |
|-----------|----|---------------|
| Top level | !" | 1 block       |
| Level 2   | !" | 4 blocks      |
| Level 1   | !" | 200 blocks    |
| Data      | !" | 10,000 blocks |

## 33. SEARCH COST

If the top-level root is on disk:

TEXT

|          |    |           |
|----------|----|-----------|
| 1 access | !" | top index |
| 1 access | !" | level 2   |
| 1 access | !" | level 1   |
| 1 access | !" | data      |

Total:

$\lceil 4 \rceil$

block accesses.

Compare with thousands without indexing.

## 34. GENERAL MULTILEVEL INDEX RULE

Suppose:

TEXT

$b_0$  = number of data blocks  
fan-out / index blocking factor =  $f$

For a sparse index hierarchy:

$$b_1 = \left\lceil \frac{b_0}{f} \right\rceil$$

$$b_2 = \left\lceil \frac{b_1}{f} \right\rceil$$

and so on until:

$$b_k = 1$$

## 35. NUMBER OF INDEX LEVELS

Approximate:

$$L = \lceil \log_f(b_0) \rceil$$

for a sparse multilevel structure with fan-out  $f$ , depending on whether the problem counts root/data levels separately.

This counting convention is a major GATE trap.

## & COUNTING LEVELS CAREFULLY

If we have:

TEXT

Root  
!"  
Index  
!"  
Index  
!"  
Data

Some questions ask:

TEXT

number of index levels

Then data level is not counted.

Some ask:

TEXT

tree height / disk accesses

Then interpretation may include leaf/data access.

Always inspect wording.

## 36. MULTILEVEL INDEX EXAMPLE

Suppose:

TEXT

Data blocks = 1,000,000  
Index fan-out = 100

Level 1:

$\lceil 1,000,000/100 = 10,000 \rceil$

Level 2:

$\lceil 10,000/100 = 100 \rceil$

Level 3:

$\lceil 100/100 = 1 \rceil$

So:

TEXT

3 index levels

If one final data-block access is required:

TEXT

$3 + 1 = 4$  disk accesses

assuming root is not already memory-resident.

## MEMORY TRICK

**Index levels until you reach ONE block.**

Then:

**+1 data block access if asked for record retrieval cost.**

## 37. FAN-OUT

Fan-out roughly means:

**How many lower-level pointers/children one index node/block can reference.**

For a simple index block consisting of:

TEXT

<key, pointer>

entries:

$\lceil \text{fanout} \approx \text{bfr}_i \rceil$

In B/B+ trees the exact fan-out calculation is slightly different because internal node layout matters.

We will handle that next.

## 38. TWO-LEVEL INDEX NUMERICAL

Suppose:

TEXT

Data blocks = 50,000

Index entries/block = 100

Level 1 blocks:

$\lceil \lceil 50000/100 \rceil \rceil = 500$

Level 2:

$\lceil \lceil 500/100 \rceil \rceil = 5$

Level 3:

$\lceil \lceil 5/100 \rceil \rceil = 1$

So actually:

TEXT

3 index levels

Search:

TEXT

Root

! ' n e x t i n d e x

! ' n e x t i n d e x

! ' d a t a b l o c k

"H disk I/Os if all levels on disk.

## 39. ROOT IN MEMORY

Sometimes the root/top index block is kept permanently in main memory.

Then we may not count its disk access.

Example:

TEXT

3 index levels + 1 data

would normally be:

TEXT

4 disk accesses

If root memory-resident:

TEXT

3 disk accesses

Always read the question.

## PART C — INDEX TYPES IN ONE TABLE

### 40. MASTER COMPARISON

| Feature                      | Primary                    | Clustering                        | Secondary     |
|------------------------------|----------------------------|-----------------------------------|---------------|
| File ordered on index field? | Yes                        | Yes                               | No            |
| Search field unique?         | Usually yes                | No                                | Either        |
| Sparse possible?             | Yes                        | Yes                               | Usually no    |
| Dense common?                | Not required               | Not usually                       | Yes           |
| Number per file              | At most one ordering index | At most one ordering organization | Many possible |

### 41. DENSE/SPARSE IS A DIFFERENT CLASSIFICATION

This is important.

Do not think:

TEXT

Primary, secondary, dense, sparse

are four mutually exclusive types.

Instead:

#### Classification A — relation to file ordering

TEXT

Primary  
Clustering  
Secondary

## Classification B — number of index entries

TEXT

Dense  
Sparse

Example:

TEXT

Primary index + sparse

is a combination.

## 42. INDEX ON NON-KEY ATTRIBUTE

Suppose:

TEXT

Dept

repeats.

A secondary index may use:

TEXT

D e p t ! ' p o i n t e r t o b u c k e t / l i s t

where bucket stores pointers to records.

Example:

TEXT

CSE

!"

p 1 !' R a v i  
p 2 !' K i r a n  
p 8 !' R a j

This is necessary because one index value corresponds to multiple records.

## 43. CLUSTERING INDEX AND DISTINCT VALUES

For a clustering index on a non-key ordering field, index entries can often be:

TEXT

one per distinct value

Example:

TEXT

C S E !' f i r s t C S E b l o c k  
E C E !' f i r s t E C E b l o c k



```
A I M L ! ' f i r s t A I M L b l o c k
```

Number of index entries may therefore equal:

TEXT

number of distinct clustering-field values

rather than number of records or blocks, depending on organization.

Always use the precise question description.

---

## 44. SECONDARY INDEX ON NON-KEY FIELD — TWO LEVEL OF POINTERS

---

Conceptually:

TEXT

Index value

!"

Pointer bucket

!"

Record pointers

Example:

TEXT

C S E ! ' B u c k e t 7

Bucket7:

p1

p5

p9

p20

This avoids storing repeated CSE key values for every record in the top-level index.

Again, the exact organization depends on the design described.

---

## PART D — SEARCH COSTS

---

### 45. SEARCH WITHOUT INDEX

---

Heap file of:

TEXT

b blocks

successful average:

$\lceil b/2 \rceil$

Unsuccessful:

$\lceil b \rceil$

---

## 46. SINGLE-LEVEL ORDERED INDEX

---

If index occupies:

TEXT  
b\_i blocks

and it is ordered, binary search cost:

$\lceil \log_2 b_i \rceil$

plus:

TEXT  
1 data block

typically.

---

## 47. MULTILEVEL INDEX

---

If there are:

TEXT  
L index levels

then retrieval cost often:

$\lceil L+1 \rceil$

disk accesses:

TEXT  
L index blocks  
+  
1 data block

assuming all levels accessed from disk.

---

## 48. WHY MULTILEVEL INDEX resembles A TREE

---

Structure:

TEXT

```
graph TD
    Root --> Index1[Index]
    Root --> Index2[Index]
    Index1 --> Index3[Index]
    Index1 --> Index4[Index]
    Index2 --> Index3
    Index2 --> Index4
    Index3 --> DB1[Data Blocks]
    Index4 --> DB2[Data Blocks]
```

This directly leads to:

---

which maintain such multilevel indexes dynamically and balanced.

## PART E — GATE-STYLE NUMERICAL

### 49. PROBLEM 1 — PRIMARY INDEX

A file has:

#### TEXT

100,000 records  
Record size = 100 bytes  
Block size = 1000 bytes  
Key size = 10 bytes  
Block pointer = 10 bytes

Unspanned, ordered by primary key.

Find number of primary-index blocks.

#### Data BFR

$\lfloor 1000/100=10 \rfloor$

#### Data blocks

$\lfloor 100000/10=10000 \rfloor$

#### Primary index entries

One per data block:

$\lfloor 10000 \rfloor$

#### Index entry size

$\lfloor 10+10=20 \rfloor$

#### Index BFR

$\lfloor 1000/20=50 \rfloor$

#### Index blocks

$\lfloor 10000/50=200 \rfloor$

Answer:

$\boxed{200}$

### 50. PROBLEM 2 — SECONDARY INDEX

Same file:

TEXT

100,000 records

Suppose we create dense secondary index on unique Email.

Index entry size:

TEXT

20 bytes

Block:

TEXT

1000 bytes

Index BFR:

$\lceil 50 \rceil$

Dense index entries:

$\lceil 100000 \rceil$

Index blocks:

$\lceil 100000/50=2000 \rceil$

Compare:

TEXT

Primary sparse index = 200 blocks

Secondary dense index = 2000 blocks

This shows why secondary indexes can be much larger.

## 51. PROBLEM 3 — MULTILEVEL INDEX

Suppose:

TEXT

Level-1 index blocks = 2000

fanout = 50

Level 2:

$\lceil \lceil 2000/50 \rceil \rceil = 40$

Level 3:

$\lceil \lceil 40/50 \rceil \rceil = 1$

Thus:

TEXT

3 index levels total

If root on disk:

TEXT

3 index accesses + 1 data block  
= 4 disk accesses

---

## 52. PROBLEM 4 — FIND LEVELS DIRECTLY

Given:

TEXT

Data blocks =  $10^6$   
Fan-out = 100

We need smallest L such that:

$$\lceil 100^L \rceil \geq 10^6$$

Now:

$$\lceil 100^1 \rceil = 10^2$$

$$\lceil 100^2 \rceil = 10^4$$

$$\lceil 100^3 \rceil = 10^6$$

So:

$$\lceil L = 3 \rceil$$

index levels.

Then +1 data access if needed.

---

## 53. PROBLEM 5 — SPARSE INDEX ENTRY COUNT

Data file:

TEXT

20,000 blocks

Sparse primary index has:

TEXT

one entry per block

Therefore:

$$\lceil 20000 \rceil$$

entries.

If index BFR = 200:

$\lceil \frac{20000}{200} \rceil = 100$

index blocks.

---

## PART F — GATE TRAPS

---

### & pTrap 1 — Search key

Search key need not be unique.

---

### & pTrap 2 — Primary key vs primary index

Logical key and physical index are different concepts.

---

### & pTrap 3 — Sparse primary index

Entries correspond to data blocks, not every record.

---

### & pTrap 4 — Secondary index

Usually dense because data file isn't ordered on secondary field.

---

### & pTrap 5 — Dense vs sparse

Not mutually exclusive with primary/secondary classification.

---

### & pTrap 6 — Clustering index

File ordered on non-key search field.

---

### & pTrap 7 — Number of secondary indexes

A file may have many secondary indexes.

---

### & pTrap 8 — Multilevel index levels

Do not accidentally count the data-file level as an index level unless the question's definition does so.

---

### & pTrap 9 — Disk accesses

If root is already in memory, don't count its disk access.

## & pTrap 10 — Pointer size

Different levels may use different pointer types:

### TEXT

record pointer  
block pointer

Use exactly what question specifies.

## & pTrap 11 — Ceiling

Number of index blocks always needs ceiling if not divisible exactly.

## & pTrap 12 — Ordered file requirement

Sparse index cannot generally work over arbitrary unordered data.

## PART G — PYQ RECOGNITION METHOD

When GATE gives indexing numerical:

### STEP 1

Identify file ordering:

### TEXT

Ordered on key?  
Ordered on non-key?  
Not ordered?

This identifies:

### TEXT

Primary / clustering / secondary

### STEP 2

Identify:

### TEXT

Dense or sparse?

### STEP 3

Calculate data blocks if needed:

$$\lfloor bfr = \lfloor \text{floor } B/R \rfloor \rfloor$$

$$\lfloor b = \lceil \text{ceil } r/bfr \rceil \rfloor$$

---

## STEP 4

Find number of index entries:

TEXT

Primary sparse

! ' number of data blocks

Secondary dense unique

! ' number of records

Clustering

! ' often number of distinct clustering values

Use the exact question.

---

## STEP 5

Index entry size:

$\lfloor \text{key size} + \text{pointer size} \rfloor$

---

## STEP 6

Index BFR:

$\lfloor \text{BFR} \rfloor$

---

## STEP 7

Index blocks:

$\lceil \text{entries} / \text{BFR} \rceil$

---

## STEP 8

For multilevel:

Repeat until:

TEXT

1 block

---

## NOTEBOOK NOTES

## INDEX

Auxiliary access structure:

$\langle \text{SearchKey}, \text{Pointer} \rangle$

Purpose:

---



TEXT

Reduce data-block accesses

---

## SEARCH KEY

Attribute(s) used for lookup.

TEXT

Need not be unique.

---

## PRIMARY INDEX

TEXT

Data file ordered on key field  
Usually sparse  
One index entry per data block

---

## CLUSTERING INDEX

TEXT

Data file ordered on non-key field  
Repeated values stored together  
Usually one entry per cluster/distinct value

---

## SECONDARY INDEX

TEXT

Index on non-ordering field  
Usually dense  
Can have many secondary indexes

---

## DENSE

TEXT

Entry for every record/search-key occurrence

---

## SPARSE

TEXT

Entries only for some records/blocks  
Requires ordered data

---

## INDEX ENTRY SIZE

---

$[R_i = \text{KeySize} + \text{PointerSize}]$

---

## INDEX BFR

---

$[bfr_i = \left\lfloor \frac{B}{R_i} \right\rfloor]$

---

## INDEX BLOCKS

---

$[b_i = \left\lceil \frac{\text{entries}}{bfr_i} \right\rceil]$

---

## MULTILEVEL INDEX

---

Repeat:

$[b_{\text{next}} = \left\lceil \frac{b_{\text{current}}}{\text{fanout}} \right\rceil]$

until:

TEXT

1 block

Approx:

$[L = \lceil \log_f b \rceil]$

---

## MEMORY PAGE

---

TEXT

Primary

! ' Ordered UNIQUE field

! ' Sparse

Clustering

! ' Ordered NON - UNIQUE field

! ' Groups

Secondary

! ' NON - ordering field

! ' Dense

And:

TEXT

Dense = record-level entries

Sparse = block-level/fewer entries

Numericals:

#### TEXT

Index entry  
= key + pointer

Entries per block  
= FLOOR

Blocks required  
= CEILING

Multilevel  
= repeat until ONE

---

## 30-SECOND REVISION

#### TEXT

Index = search key + pointer

Search key need not be unique

Primary index:  
file ordered on key  
usually sparse

Clustering:  
file ordered on non-key  
same values clustered

Secondary:  
non-ordering attribute  
usually dense

Dense:  
entry for every record/key occurrence

Sparse:  
some entries only  
requires ordered file

Index entry size:  
key + pointer

Index BFR:  
 $\text{floor}(\text{block}/\text{index-entry-size})$

Index blocks:  
 $\text{ceil}(\text{entries}/\text{index-BFR})$

Multilevel:  
build index on index  
until root = 1 block

Search cost:  
index levels + data access

---

## CHECK YOUR UNDERSTANDING

### Q1

A file is physically ordered on RollNo, and RollNo is unique.

What type of ordered index on RollNo is this?

A. Primary B. Clustering C. Secondary

---

### Q2

A file is physically ordered on Department, where many records can have the same Department.

What index type is appropriate?

---

### Q3

Why is a secondary index usually dense?

---

### Q4

A data file has:

TEXT

10,000 blocks

A sparse primary index has one entry per block.

If each index block stores:

TEXT

100 entries

how many first-level index blocks are needed?

---

### Q5

Continue Q4.

If every higher index block also stores 100 entries, how many additional levels are needed until the root occupies one block?

---

### Q6

Block size:

TEXT

4096 bytes

Search key:

TEXT

12 bytes

Pointer:

---

TEXT

4 bytes

Calculate index blocking factor.

### Q7 — Important trap

Is every search key also a candidate key?

Why or why not?

Next is the most important tree-index topic:

## TOPIC 13 — B-TREE

We will cover:

**order !' maximum children !' maximum keys !' minimum occupancy !' root exception !' height !' minimum/maximum records/keys !' search !' insertion intuition !' GATE numerical questions !' different definitions of "order."**

Then immediately after:

## TOPIC 14 — B+ TREE

which is even more important for database indexing.

Type ``NEXT``.

USER

NEXT

USER

NEXT

ASSISTANT

## TOPIC 13 — B-TREE

Now we enter one of the most important indexing structures for GATE DBMS.

A **B-Tree** is a balanced multiway search tree designed specifically to reduce **disk/page accesses**.

The most important GATE areas are:

**order !' maximum/minimum children !' keys per node !' root exception !' leaf depth !' height !' search !' insertion !' splitting !' minimum/maximum keys !' numerical questions !' definition-of-order trap**

## 1. WHY DO WE NEED B-TREES?

A normal binary search tree has at most:

TEXT

2 children per node

So for a huge database, its height can become relatively large.

But disk access is expensive.

Suppose every node access required a disk read.

We want:

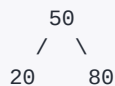
TEXT

fewer levels  
!' fewer disk accesses

B-Trees solve this by allowing each node to contain **many keys and many children**.

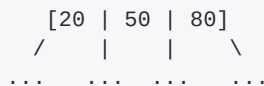
Instead of:

TEXT



we can have something like:

TEXT



One node can guide us toward many subtrees.

## MEMORY TRICK

**B-Tree = Broad Tree**

Many children !' small height.

## 2. BASIC B-TREE PROPERTIES

A B-Tree is:

- balanced
- multiway
- ordered
- height-balanced
- designed for block-oriented storage

Most importantly:

**All leaves are at the same level.**

### 3. ONE NODE CAN STORE MULTIPLE KEYS

Example node:

TEXT

[ 20 | 40 | 70 ]

This divides values into four ranges:

TEXT

Child 1:  $< 20$

Child 2:  $20 < \text{key} < 40$

Child 3:  $40 < \text{key} < 70$

Child 4:  $> 70$

Therefore:

**If a node has  $k$  keys, it can have at most  $k+1$  children.**

This relation is fundamental.

$\boxed{\text{Children} = \text{Keys} + 1}$

for an internal node with all child positions populated.

### 4. THE BIGGEST GATE TRAP — WHAT DOES “ORDER” MEAN?

Different books use the word **order** differently.

This can completely change numerical answers.

There are two common conventions.

#### CONVENTION 1 — ORDER = MAXIMUM CHILDREN

Suppose order is:

$m$

Then:

TEXT

Maximum children =  $m$

Maximum keys =  $m - 1$

This is a very common DBMS convention.

Example:

```
TEXT
Order = 5
```

Then:

```
TEXT
Maximum children = 5
Maximum keys = 4
```

---

## CONVENTION 2 — ORDER = MAXIMUM KEYS

---

Some sources define order differently.

If order  $m$  means:

```
TEXT
maximum keys = m
```

then:

```
TEXT
maximum children = m + 1
```

---

## & pGATE GOLDEN RULE

---

**Never start a B-Tree numerical until you identify how the question defines order.**

If the problem directly says:

**“Each node can have at most 5 children”**

there is no ambiguity.

If it says only:

**“B-tree of order 5”**

check the convention implied by the question/options.

For our lessons, unless otherwise stated:

### We will use:

$\boxed{\text{Order } m = \text{maximum number of children}}$

Therefore:



$$\boxed{\text{Maximum keys} = m - 1}$$

---

## 5. MAXIMUM CHILDREN

---

For order  $(m)$ :

$$\boxed{\text{MaxChildren} = m}$$

---

## 6. MAXIMUM KEYS

---

If maximum children:

$$m$$

then:

$$\boxed{\text{MaxKeys} = m - 1}$$

because:

$$\text{Children} = \text{Keys} + 1$$

---

## EXAMPLE

---

Order:

$$m = 6$$

Then:

TEXT

Maximum children = 6  
Maximum keys = 5

---

## 7. MINIMUM CHILDREN

---

Except for the root, an internal node should be at least half full.

For order  $(m)$ :

$$\boxed{\text{MinChildren} = \left\lceil \frac{m}{2} \right\rceil}$$

for every non-root internal node.

---

## 8. MINIMUM KEYS

---

If minimum children are:

$$\left\lceil \frac{m}{2} \right\rceil$$

then minimum keys are one fewer:

$$\boxed{\text{MinKeys} = \left\lceil \frac{m}{2} \right\rceil - 1}$$

---

for non-root nodes.

---

## EXAMPLE — ORDER 5

---

$\lfloor m=5 \rfloor$

Maximum:

TEXT

Children = 5  
Keys = 4

Minimum non-root:

$\lfloor \lceil 5/2 \rceil = 3 \rfloor$

children.

Therefore:

$\lfloor 3-1=2 \rfloor$

keys.

So:

TEXT

Non-root node:  
2 "d k e y s "d 4

---

## EXAMPLE — ORDER 6

---

$\lfloor m=6 \rfloor$

Maximum:

TEXT

Children = 6  
Keys = 5

Minimum:

$\lfloor \lceil 6/2 \rceil = 3 \rfloor$

children.

So:

TEXT

Minimum keys = 2

Hence:

TEXT

2 "d k e y s "d 5

for non-root nodes.

## 9. ROOT IS SPECIAL

The root has an exception.

If the root is not a leaf, it usually needs at least:

TEXT

2 children

therefore at least:

TEXT

1 key

It does **not** need to satisfy the ordinary half-full minimum.

## MEMORY

**Root gets special treatment.**

For non-root nodes:

$\lceil \text{MinChildren} = \lceil m/2 \rceil \rceil$

For root:

TEXT

minimum children = 2

if it has children.

## 10. ROOT CAN ALSO BE A LEAF

If the tree contains only a few keys:

TEXT

[10 | 20 | 30]

the root itself can be a leaf.

Then there may be no children at all.

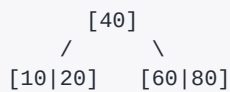
So root rules depend on whether the tree has multiple levels.

## 11. ALL LEAVES AT SAME LEVEL

This is the key balancing property.

Valid:

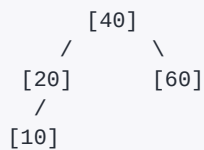
TEXT



Leaves at equal depth.

Invalid:

TEXT



because one leaf is deeper.

## MEMORY

**B-Tree never becomes skewed like an ordinary BST.**

It remains balanced.

## 12. SEARCH IN B-TREE

Suppose node:

TEXT

[20 | 40 | 70]

Search:

TEXT

55

Compare:

TEXT

55 > 20

55 > 40

55 < 70

Therefore follow the child between:

TEXT

40 and 70

Then repeat.

## SEARCH PATH EXAMPLE

TEXT

```
      [30 | 60]
     /   |   \
  [10|20] [40|50] [70|80]
```

Search 50:

### Root

TEXT

30 < 50 < 60

Go middle.

### Middle child

TEXT

[40 | 50]

Found 50.

## 13. WHY SEARCH IS FAST

If each internal node has many children:

TEXT

fan-out large

then tree height becomes small.

Search cost roughly:

$\lceil O(\log_m N) \rceil$

conceptually.

In disk terms:

**roughly one page access per tree level.**

That is what matters for DBMS.

## 14. INSERTION — BASIC IDEA

When inserting a key:

1. Search for correct leaf.
2. Insert key in sorted order.
3. If node still has room !' done.
4. If node overflows !' split it.
5. Promote middle key to parent.
6. Splitting may propagate upward.
7. If root splits !' create a new root.

## MEMORY

TEXT

INSERT

```
!' F i n d   l e a f
!' I n s e r t
!' O v e r f l o w ?
!' S p l i t
!' P r o m o t e
```

## 15. SIMPLE INSERTION WITHOUT SPLIT

Suppose max keys = 4.

Leaf:

TEXT

```
[10 | 20 | 40]
```

Insert:

TEXT

```
30
```

Result:

TEXT

```
[10 | 20 | 30 | 40]
```

No overflow.

Done.

## 16. INSERTION WITH OVERFLOW

Suppose order:

$\lfloor m=4 \rfloor$

So max keys:

$\lfloor 3 \rfloor$

Node:

TEXT

[10 | 20 | 30]

Insert:

TEXT

25

Temporary:

TEXT

[10 | 20 | 25 | 30]

But maximum is only 3 keys.

Overflow.

We split around the median.

A possible conceptual split:

TEXT

```
      [25]
     /   \
  [10|20] [30]
```

The median key is promoted to the parent.

Exact median choice can depend on implementation/convention for even temporary counts, but GATE usually specifies enough information or uses standard splitting behavior.

## 17. ROOT SPLIT

Suppose root itself overflows.

Before:

TEXT

[10 | 20 | 30]

Insert 40:

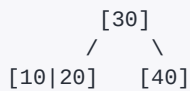
TEXT

[10 | 20 | 30 | 40]

Overflow.

After split, a new root is created:

TEXT



Now height increases by 1.

## KEY FACT

**B-Tree grows upward from the root when root splits.**

Unlike an ordinary BST that grows downward irregularly.

## 18. DELETION — BASIC INTUITION

GATE often focuses more on insertion and occupancy than full deletion procedure.

Conceptually:

1. Delete key.
2. If node still satisfies minimum occupancy !' done.
3. If underflow:
  - borrow/redistribute from sibling, or
  - merge with sibling
4. Parent may need adjustment.
5. Tree can shrink if root becomes unnecessary.

Memory:

TEXT

Underflow

!' Borrow

!' otherwise Merge

## 19. MINIMUM OCCUPANCY

For order  $(m)$ :

Non-root internal node:

$\lceil \text{MinChildren} = \left\lceil \frac{m-1}{2} \right\rceil \rceil$

Therefore percentage occupancy is approximately:

TEXT

at least 50%

This ensures good space utilization and bounded height.



## 20. B-TREE HEIGHT

Now we reach important GATE numericals.

We need to understand maximum and minimum number of keys possible for a given height.

But first:

**Height convention may vary.**

Some define:

TEXT

root at level 0  
height = number of edges root! ' leaf

Others count:

TEXT

number of levels

We will explicitly state which convention we use.

For formulas below:

**Let height  $h$  = number of edges from root to leaf.**

So:

TEXT

root alone !'  $h = 0$

## 21. MAXIMUM NUMBER OF CHILDREN AT EACH LEVEL

Order  $m$ .

Maximum:

TEXT

Level 0:  
1 root

Level 1:  
 $m$  nodes

Level 2:  
 $m^2$  nodes

Level 3:  
 $m^3$  nodes

Thus at level  $i$ :

$m^i$

nodes maximum.

---

## 22. MAXIMUM KEYS AT EACH NODE

---

Each node holds at most:

$$\lfloor m-1 \rfloor$$

keys.

So maximum keys in a B-tree of height  $(h)$ :

$$\lfloor (m-1) \left( 1+m+m^2+\dots+m^h \right) \rfloor$$

Geometric sum:

$$\lfloor 1+m+\dots+m^h = \frac{m^{h+1}-1}{m-1} \rfloor$$

Multiplying:

$$\lfloor N_{\max} = (m-1) \frac{m^{h+1}-1}{m-1} \rfloor$$

Therefore:

$$\boxed{N_{\max} = m^{h+1} - 1}$$

Very useful.

---

## 23. MAXIMUM KEYS EXAMPLE

---

Order:

$$\lfloor m=4 \rfloor$$

Height:

$$\lfloor h=2 \rfloor$$

Maximum keys:

$$\lfloor 4^3 - 1 \rfloor$$

$$\lfloor =64-1 \rfloor$$

$$\boxed{63}$$

Let's verify.

Level 0:

TEXT

$$1 \text{ node} \times 3 \text{ keys} = 3$$

Level 1:

TEXT

$$4 \text{ nodes} \times 3 = 12$$

Level 2:

TEXT

16 nodes  $\times$  3 = 48

Total:

$\lceil 3+12+48=63 \rceil$

Correct.

## 24. MINIMUM KEYS FOR A GIVEN HEIGHT

This is trickier because root has special minimum occupancy.

Let:

$t = \lceil \frac{m}{2} \rceil$

minimum children of a non-root internal node.

If height  $(h \geq 1)$ :

Root minimum:

TEXT

2 children

1 key

At level 1:

TEXT

2 nodes

Each non-root node has minimum:

$t-1$

keys.

If they are internal, each has  $t$  children.

Thus level  $(i \geq 1)$  can have minimum:

$2^{t^{i-1}}$

nodes.

## 25. MINIMUM NUMBER OF KEYS FORMULA

Minimum total keys:

Root contributes:

$1$

Every non-root node contributes at least:

$t-1$

So:

$$N_{\min} = 1 + (t-1) \sum_{i=1}^h 2^{t^{i-1}}$$

The geometric sum simplifies beautifully to:

$$N_{\min} = 2^{t^h - 1}$$

where:

$$t = \left\lceil \frac{m}{2} \right\rceil$$

and  $h$  is edge-height, for  $h \geq 1$ .

## 26. MINIMUM KEYS EXAMPLE

Order:

$$m=5$$

Then:

$$t = \left\lceil \frac{5}{2} \right\rceil = 3$$

Height:

$$h=2$$

Minimum keys:

$$2(3^2) - 1$$

$$18 - 1$$

$$\boxed{17}$$

Let's verify.

Root:

TEXT

- 1 key
- 2 children

Level 1:

TEXT

- 2 nodes

Each minimum keys:

TEXT

- 2

So:

TEXT

- 4 keys

Each has minimum 3 children:

TEXT

6 leaf nodes

Each leaf minimum 2 keys:

TEXT

12 keys

Total:

$$\lceil 1+4+12=17 \rceil$$

Correct.

---

## 27. HEIGHT FROM NUMBER OF KEYS — MAXIMUM CAPACITY

If a tree of order  $m$  and height  $h$  can hold maximum:

$$m^{h+1}-1$$

keys, then to fit  $N$  keys:

$$N \leq m^{h+1}-1$$

Therefore:

$$N+1 \leq m^{h+1}$$

Taking logarithm:

$$h+1 \geq \log_m(N+1)$$

So minimum possible height roughly:

$$\boxed{h_{\min} = \lceil \log_m(N+1) \rceil - 1}$$

under our height convention.

---

## 28. HEIGHT FROM MINIMUM OCCUPANCY

For an existing B-tree of height  $h$ :

$$N \geq 2t^{h-1}$$

So:

$$N+1 \geq 2t^h$$

$$t^h \leq \frac{N+1}{2}$$

Thus:

$$h \leq \log_t \left( \frac{N+1}{2} \right)$$

This gives a maximum-height bound.

GATE may ask such calculations.

## 29. EXAMPLE — MINIMUM POSSIBLE HEIGHT

B-tree order:

$$\lfloor m=10 \rfloor$$

Number of keys:

$$\lfloor N=999 \rfloor$$

Maximum at height  $\lfloor h \rfloor$ :

$$\lfloor 10^{h+1}-1 \rfloor$$

For  $\lfloor h=1 \rfloor$ :

$$\lfloor 10^2-1=99 \rfloor$$

Not enough.

For  $\lfloor h=2 \rfloor$ :

$$\lfloor 10^3-1=999 \rfloor$$

Exactly enough.

Therefore minimum height:

$$\lfloor \boxed{2} \rfloor$$

i.e. 3 levels:

TEXT

root  
level 1  
leaf level

## 30. HEIGHT vs LEVELS

Under our convention:

TEXT

Height = edges root! leaf

Therefore:

$$\lfloor \boxed{\text{Levels}=h+1} \rfloor$$

Example:

TEXT

3 levels  
! height 2

This is a major GATE trap.

## 31. NUMBER OF LEAVES — MAXIMUM

At height  $(h)$ , maximum number of leaves:

$$\boxed{m^h}$$

because each internal node can have  $m$  children.

---

## 32. NUMBER OF LEAVES — MINIMUM

---

For  $(h \geq 1)$ :

Root minimum:

TEXT

2 children

Other internal nodes:

$$t = \lceil m/2 \rceil$$

children.

So minimum leaves:

$$\boxed{2^{t^{h-1}}}$$

where  $(t = \lceil m/2 \rceil)$ .

---

## 33. EXAMPLE

---

Order:

$$m = 6$$

Then:

$$t = 3$$

Height:

$$h = 3$$

Minimum leaves:

$$2 \times 3^2$$

$$= 18$$

Maximum leaves:

$$6^3 = 216$$

---

## 34. CALCULATING ORDER FROM BLOCK SIZE

---

This is important.

Suppose an internal B-tree node contains:

---

TEXT

p child pointers  
p - 1 keys

Let:

TEXT

P = pointer size  
K = key size  
B = block size

Then:

$$\lfloor \frac{pP + (p-1)K}{B} \rfloor$$

We want maximum possible integer  $\lfloor \frac{pP + (p-1)K}{B} \rfloor$ .

That becomes the B-tree order/fan-out under our convention.

---

## 35. SOLVED FAN-OUT EXAMPLE

Given:

TEXT

Block size = 1024 bytes  
Key size = 8 bytes  
Pointer size = 4 bytes

Need maximum  $\lfloor \frac{pP + (p-1)K}{B} \rfloor$ :

$$\lfloor \frac{4p + 8(p-1)}{1024} \rfloor$$

$$\lfloor \frac{4p + 8p - 8}{1024} \rfloor$$

$$\lfloor \frac{12p - 8}{1024} \rfloor$$

$$\lfloor \frac{p}{86} \rfloor$$

So:

$$\lfloor \boxed{m=86} \rfloor$$

Maximum keys:

$$\lfloor 85 \rfloor$$

---

## MEMORY

For internal B-tree node:

$$\lfloor \boxed{\frac{pP + (p-1)K}{B}} \rfloor$$

Solve for largest integer  $\lfloor \frac{pP + (p-1)K}{B} \rfloor$ .

---

## 36. ALTERNATIVE NODE LAYOUT



Some questions specify:

TEXT

each entry contains key + record pointer

or:

TEXT

each node contains extra metadata/pointer

Then formula changes.

Golden rule:

**Build the node byte equation from exactly what the question says.**

Do not blindly memorize one storage equation.

## 37. B-TREE DATA STORAGE

In a B-Tree, search-key values and associated record pointers/data may appear in internal nodes as well as leaf nodes.

This becomes one of the key differences from a **B+ Tree**.

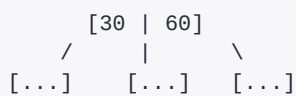
Very important:

**In a B-Tree, a successful search may finish at an internal node.**

## 38. B-TREE SEARCH MAY STOP EARLY

Example:

TEXT



Search:

TEXT

60

It is already in the root.

Search can stop immediately.

In B+ Tree, actual data pointers are normally concentrated at the leaf level, so search generally proceeds to a leaf.

We'll study this difference next.

## 39. MAXIMUM KEYS IN TREE WITH L LEVELS

---

If instead a question gives number of **levels**  $\backslash(L\backslash)$ , where root is level 1:

$$\backslash[ h=L-1 \backslash]$$

Then:

$$\backslash[ N_{\{max\}} = m^{\{h+1\}}-1 \backslash]$$

becomes:

$$\backslash[ \boxed{ N_{\{max\}}=m^L-1 } \backslash]$$

under order = max children.

---

## 40. MINIMUM KEYS WITH L LEVELS

---

Since:

$$\backslash[ h=L-1 \backslash]$$

for  $\backslash(L\geq 2\backslash)$ :

$$\backslash[ \boxed{ N_{\{min\}} = 2^{t^{\{L-1\}}}-1 } \backslash]$$

where:

$$\backslash[ t=\lceil m/2 \rceil \backslash]$$

Again root-only tree is a special case.

---

## PART — GATE-STYLE NUMERICALS

---

### 41. PROBLEM 1

---

A B-tree has order:

$$\backslash[ m=7 \backslash]$$

Find:

1. Maximum children
2. Maximum keys
3. Minimum children of non-root internal node
4. Minimum keys of non-root node

**Maximum children**

$$\backslash[ 7 \backslash]$$

**Maximum keys**

$$\backslash[ 7-1=6 \backslash]$$

**Minimum children**

$$\backslash[ \lceil 7/2 \rceil=4 \backslash]$$

## Minimum keys

$\lfloor 4-1=3 \rfloor$

## Answer

TEXT

```
Max children = 7
Max keys = 6
Min children = 4
Min keys = 3
```

---

## 42. PROBLEM 2

Order:

$\lfloor m=5 \rfloor$

Height:

$\lfloor h=3 \rfloor$

Find maximum number of keys.

$\lfloor N_{\max}=5^4-1 \rfloor$

$\lfloor 625-1 \rfloor$

$\lfloor \boxed{624} \rfloor$

---

## 43. PROBLEM 3

Same tree:

$\lfloor m=5 \rfloor$

$\lfloor h=3 \rfloor$

Find minimum keys.

$\lfloor t=\lceil 5/2 \rceil=3 \rfloor$

$\lfloor N_{\min}=2(3^3)-1 \rfloor$

$\lfloor 54-1 \rfloor$

$\lfloor \boxed{53} \rfloor$

---

## 44. PROBLEM 4

Order:

$\lfloor m=4 \rfloor$

Tree has 3 levels.

Maximum keys?

Levels:

$$\lfloor L=3 \rfloor$$

Use:

$$\lfloor N_{\max}=m^L-1 \rfloor$$

$$\lfloor 4^3-1 \rfloor$$

$$\lfloor 64-1 \rfloor$$

$$\lfloor \boxed{63} \rfloor$$

---

## 45. PROBLEM 5 — FIND HEIGHT

B-tree order:

$$\lfloor m=100 \rfloor$$

Contains:

$$\lfloor 999,999 \rfloor$$

keys.

Minimum possible height?

Check maximum capacity:

For  $(h=1)$ :

$$\lfloor 100^2-1=9,999 \rfloor$$

Not enough.

For  $(h=2)$ :

$$\lfloor 100^3-1=999,999 \rfloor$$

Exactly.

Therefore:

$$\lfloor \boxed{h=2} \rfloor$$

Only 3 levels.

This demonstrates why B-trees are excellent for huge database indexes.

---

## 46. PROBLEM 6 — BLOCK FAN-OUT

Given:

TEXT

Block = 4096 bytes

Key = 16 bytes

Pointer = 8 bytes

Internal node:

TEXT

p pointers  
p-1 keys

Equation:

$$\lfloor 8p+16(p-1) \rfloor \leq 4096$$

$$\lfloor 8p+16p-16 \rfloor \leq 4096$$

$$\lfloor 24p \rfloor \leq 4112$$

$$\lfloor p \rfloor \leq 171.33\dots$$

Maximum integer:

$$\boxed{171}$$

So:

TEXT

Order = 171  
Maximum keys = 170

## 47. PROBLEM 7 — SEARCH COST

Suppose B-tree has:

TEXT

4 levels

and every node access requires one disk I/O.

Worst-case search to leaf:

TEXT

4 node accesses

"H

$$\boxed{4 \text{ disk I/Os}}$$

If root is already memory-resident:

$$\boxed{3 \text{ disk I/Os}}$$

## 48. GATE TRAPS

### & pTrap 1 — Order definition

Most dangerous trap.

Always determine whether order means:

TEXT

max children

or:

TEXT

max keys

## & pTrap 2 — Maximum keys

If order means max children:

$\lfloor \text{MaxKeys} = m - 1 \rfloor$

not  $m$ .

## & pTrap 3 — Minimum children

Use:

$\lceil m/2 \rceil$

for non-root internal nodes.

Not floor.

## & pTrap 4 — Minimum keys

$\lceil m/2 \rceil - 1$

for non-root nodes.

## & pTrap 5 — Root exception

Do not apply normal minimum occupancy rule to root.

## & pTrap 6 — Balanced

All leaves are at same depth.

## & pTrap 7 — Height vs levels

If:

TEXT

height = edges

then:

TEXT

levels = height + 1

### & pTrap 8 — Disk-access count

Root in memory?

If yes, subtract root disk access.

### & pTrap 9 — Insertion split

Overflow may propagate upward.

### & pTrap 10 — Search can terminate at internal B-tree node

Unlike typical B+ Tree data lookup.

## 49. PYQ-ORIENTED B-TREE SOLVING METHOD

Whenever GATE gives B-tree numerical:

### STEP 1

Write definition of order.

TEXT

m = max children?  
or max keys?

Never skip.

### STEP 2

If order = max children:

$\lfloor \text{MaxKeys} = m - 1 \rfloor$

$\lfloor \text{MinChildren} = \lceil m/2 \rceil \rfloor$

$\lfloor \text{MinKeys} = \lceil m/2 \rceil - 1 \rfloor$

### STEP 3

Check root separately.

### STEP 4

Determine whether question gives:

TEXT

height

or:

TEXT

levels

## STEP 5

For maximum capacity:

If edge-height  $\backslash(h\backslash)$ :

$$\backslash[ N_{\{max\}}=m^{\{h+1\}}-1 \backslash]$$

If levels  $\backslash(L\backslash)$ :

$$\backslash[ N_{\{max\}}=m^L-1 \backslash]$$

## STEP 6

For minimum capacity:

Let:

$$\backslash[ t=\backslashceil m/2\backslashrceil \backslash]$$

For edge-height  $\backslash(h\backslashge1\backslash)$ :

$$\backslash[ N_{\{min\}}=2t^{h-1} \backslash]$$

## STEP 7

For block-based order:

Construct byte equation:

$$\backslash[ pP+(p-1)K\backslashle B \backslash]$$

unless question specifies another layout.

# 50. NOTEBOOK NOTES

## B-TREE

Balanced multiway search tree used for disk indexing.

TEXT

All leaves at same level.

## ORDER — OUR CONVENTION



$[m = \text{maximum children}]$

Then:

$[MaxChildren = m]$

$[MaxKeys = m - 1]$

---

## NON-ROOT MINIMUM

$[MinChildren = \lceil \frac{m}{2} \rceil]$

$[MinKeys = \lceil \frac{m}{2} \rceil - 1]$

---

## ROOT

If non-leaf:

TEXT

```
minimum children = 2
minimum keys = 1
```

Root can also be a leaf.

---

## BALANCE

TEXT

```
All leaves same depth
```

---

## INSERT

TEXT

```
Find leaf
!' insert sorted
!' overflow
!' split
!' promote median
```

Root split increases height.

---

## DELETE

TEXT

```
Delete
!' underflow?
!' borrow
!' otherwise merge
```

## MAXIMUM KEYS

Height  $(h)$  measured in edges:

$$\boxed{N_{\max} = m^{h+1} - 1}$$

## MINIMUM KEYS

Let:

$$t = \left\lceil \frac{m}{2} \right\rceil$$

For  $(h \geq 1)$ :

$$\boxed{N_{\min} = 2t^{h-1}}$$

## MAXIMUM LEAVES

$$m^h$$

## MINIMUM LEAVES

$$2t^{h-1}$$

for  $(h \geq 1)$ .

## BLOCK FAN-OUT

If node has:

TEXT

p pointers  
p-1 keys

then:

$$\boxed{p \leq \frac{B}{K}}$$

Take largest integer p.

## MEMORY PAGE

TEXT

B-TREE = BROAD + BALANCED

Order m:

max children = m  
max keys = m-1

Non-root:

min children =  $\lceil m/2 \rceil$   
min keys =  $\lceil m/2 \rceil - 1$

Root:

special

All leaves:  
same level

Height formulas:

TEXT

Maximum keys:  
 $m^{(h+1)} - 1$

Minimum keys:  
 $2 * \text{ceil}(m/2)^h - 1$

Insertion:

TEXT

OVERFLOW  
!' S P L I T  
!' P R O M O T E

Deletion:

TEXT

UNDERFLOW  
!' B O R R O W  
!' M E R G E

---

## 30-SECOND REVISION

---

TEXT

B-tree = balanced multiway search tree

All leaves same depth

If order  $m$  = max children:

Max children =  $m$   
Max keys =  $m-1$

Non-root:  
Min children =  $\text{ceil}(m/2)$   
Min keys =  $\text{ceil}(m/2)-1$

Root is special

Insert:  
find leaf !' insert !' split on overflow  
!' promote median

Height  $h$  in edges:

Max keys =  $m^{(h+1)}-1$

Let  $t=\text{ceil}(m/2)$

Min keys =  $2t^h-1$

Internal block:  
p pointers + (p-1) keys

p P + ( p - 1 ) K "d B l o c k S i z e

Biggest trap:  
ORDER definition

---

## CHECK YOUR UNDERSTANDING

---

### Q1

Assume order means maximum children.

A B-tree has order:

$\lfloor m=8 \rfloor$

Find:

1. Maximum keys/node
2. Minimum children/non-root internal node
3. Minimum keys/non-root node

---

### Q2

Order:

$\lfloor m=4 \rfloor$

Height:

$\lfloor h=2 \rfloor$

What is the maximum possible number of keys?

---

### Q3

For Q2, what is the minimum number of keys?

---

### Q4

A B-tree has:

TEXT

4 levels

What is its height if height means number of edges from root to leaf?

---

### Q5

Why is the root treated differently from other nodes?

---

## Q6 — Numerical

A block is:

TEXT

1024 bytes

Key:

TEXT

8 bytes

Pointer:

TEXT

8 bytes

An internal node contains  $p$  pointers and  $p-1$  keys.

Find the maximum possible  $p$ .

## Q7 — Most important trap

A question says:

“B-tree of order 5”

What should you verify **before doing any numerical calculation**?

Next comes the most important database tree index:

## TOPIC 14 — B+ TREE

We will cover:

**B-Tree vs B+ Tree !' internal vs leaf nodes !' leaf linking !' exact-match search !' range search !' order/fan-out calculations !' leaf capacity !' height !' minimum/maximum entries !' insertion/splitting !' GATE numericals and traps.**

Type `NEXT`.

USER

NEXT

ASSISTANT

## TOPIC 14 — B+ TREE

Now we come to the **most important tree index structure in DBMS for GATE**.

Your syllabus explicitly mentions **B and B+ trees** under indexing.

For GATE, B+ Tree questions usually test:

- B-Tree vs B+ Tree
- internal-node fan-out
- leaf-node capacity
- linked leaves
- search path
- range queries
- minimum/maximum keys
- height
- block-size numericals
- insertion and splitting
- disk-access calculations

The single most important idea is:

**In a B+ Tree, internal nodes guide the search; actual data-record pointers are stored at the leaf level.**

## 1. WHY B+ TREE?

A B-Tree already gives us:

TEXT

balanced tree  
+  
large fan-out  
+  
few disk accesses

So why another structure?

Because databases frequently need:

TEXT

Exact searches  
+  
Range searches  
+  
Sequential access

Example:

SQL

WHERE RollNo = 105

but also:

SQL

WHERE RollNo BETWEEN 100 AND 500

B+ Trees are especially good for both.

## 2. BASIC STRUCTURE

A B+ Tree has two major node types:

TEXT

Internal nodes

! ' s e a r c h   k e y s   +   c h i l d   p o i n t e r s

Leaf nodes

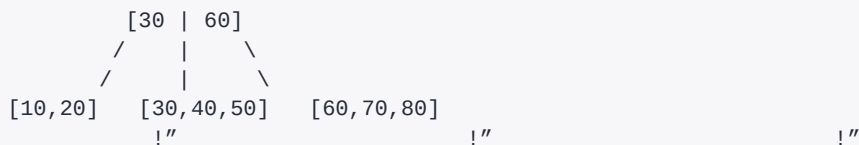
! ' s e a r c h   k e y s   +   r e c o r d / d a t a   p o i n t e r s

And importantly:

**Leaf nodes are linked together.**

Conceptually:

TEXT



The bottom leaf nodes form a linked sequence.

## 3. INTERNAL NODE

Internal node:

TEXT

[ 20 | 40 | 70 ]

contains separator/search-key values and child pointers.

Conceptually:

TEXT

P1   20   P2   40   P3   70   P4

The keys help choose the correct child.

They are primarily used for:

**navigation**

## 4. LEAF NODE

Leaf:

TEXT

[20\* | 25\* | 30\*]

Here \* can represent pointers to actual database records or record locations.

Leaf nodes store:

TEXT

Search key  
+  
Record pointer

and normally:

TEXT

pointer to next leaf

for sequential traversal.

---

## MEMORY TRICK

TEXT

I n t e r n a l ! '   G U I D E  
L e a f ! '   D A T A   P O I N T E R S

---

## 5. B-TREE vs B+ TREE — MAIN DIFFERENCE

---

### B-Tree

Keys/record pointers may occur in:

TEXT

internal nodes  
and  
leaf nodes

A successful search can stop at an internal node.

---

### B+ Tree

Internal nodes mainly contain:

TEXT

search keys + child pointers

Actual record pointers are concentrated in:



TEXT

leaf nodes

Therefore a record lookup normally continues to a leaf.

## 6. VERY IMPORTANT CONSEQUENCE

In B+ Tree, an internal separator key may also appear again in a leaf.

So key values may be duplicated between:

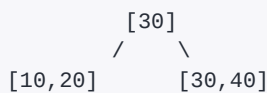
TEXT

internal nodes  
and  
leaf nodes

That is normal.

Example:

TEXT



30 exists:

TEXT

in root as separator  
AND  
in leaf as actual indexed key

## & pGATE TRAP

Do not think duplicated search-key values between internal and leaf levels imply duplicate records.

The internal copy may be only a separator.

## 7. ALL LEAVES AT SAME LEVEL

Just like B-Tree:

**B+ Tree is balanced.**

All leaf nodes are at the same depth.

So:

TEXT

r o o t ! ' l e a f

paths have equal length.

## 8. LINKED LEAVES

This is one of the biggest advantages of B+ Tree.

Leaf nodes contain pointers such as:

TEXT

L e a f 1 ! ' L e a f 2 ! ' L e a f 3 ! ' L e a f 4

Conceptually:

TEXT

[ 1 0 , 2 0 ] ! ' [ 3 0 , 4 0 ] ! ' [ 5 0 , 6 0 ] ! ' [ 7 0 , 8 0 ]

Therefore after locating key 30, we can scan:

TEXT

30, 40, 50, 60, 70...

without repeatedly returning to internal nodes.

This makes **range queries extremely efficient**.

## MEMORY TRICK

**B+ = PLUS linked leaves**

A simple mnemonic:

TEXT

The "+" gives you leaf links.

## 9. RANGE SEARCH

Suppose query:

SQL

WHERE Key BETWEEN 40 AND 80;

Steps:

### Step 1

Use tree to find first relevant leaf containing 40.

### Step 2

Follow leaf links:

TEXT

4 0 !' 5 0 !' 6 0 !' 7 0 !' 8 0

until range ends.

This is much more efficient than performing an independent tree search for every value.

## 10. WHY B+ TREE IS PREFERRED IN DATABASES

Main reasons:

1. High fan-out.
2. Small height.
3. Balanced.
4. Efficient equality search.
5. Excellent range search.
6. Sequential access through linked leaves.
7. Internal nodes can fit more keys because they usually don't store record pointers.

The last point is especially important.

## 11. WHY B+ TREE CAN HAVE HIGHER FAN-OUT THAN B-TREE

Suppose block size is fixed.

In B-Tree internal nodes, we may need:

TEXT

key  
+  
child pointer  
+  
possibly record/data pointer

In B+ internal nodes we generally need only:

TEXT

key  
+  
child pointer

So each internal block can fit more separator keys.

Higher fan-out means:

TEXT

fewer levels  
!' fewer disk accesses

## 12. ORDER / FAN-OUT WARNING

Just like B-Tree:

**Different books define B+ Tree “order” differently.**

Some may define order as:

TEXT  
maximum children

Others:

TEXT  
maximum keys

Some even define separate capacities for:

TEXT  
internal nodes  
and  
leaf nodes

Therefore:

## GOLDEN RULE

Never solve a B+ Tree numerical until you identify exactly what the question means by order.

For our default discussion:

$m = \text{maximum children of an internal node}$

Then:

$\boxed{\text{Maximum internal keys} = m - 1}$

## 13. INTERNAL NODE LAYOUT

Suppose an internal node has:

TEXT  
p child pointers

then it has:

TEXT  
p - 1 search keys

So byte requirement:

$pP + (p-1)K \leq B$

where:

TEXT

P = child-pointer size  
K = key size  
B = block size

Same structural equation we used earlier.

## 14. INTERNAL FAN-OUT FORMULA

We choose the largest integer  $\lfloor p \rfloor$  satisfying:

$$\lfloor pP + (p-1)K \rfloor \leq B$$

Then:

TEXT

Internal fan-out =  $p$   
Maximum internal keys =  $p-1$

## 15. SOLVED INTERNAL FAN-OUT EXAMPLE

Given:

TEXT

Block size = 4096 bytes  
Key size = 12 bytes  
Child pointer = 8 bytes

Find maximum children  $\lfloor p \rfloor$ .

$$\lfloor 8p + 12(p-1) \rfloor \leq 4096$$

$$\lfloor 8p + 12p - 12 \rfloor \leq 4096$$

$$\lfloor 20p \rfloor \leq 4108$$

$$\lfloor p \rfloor \leq 205.4$$

So:

$$\boxed{p=205}$$

Maximum internal keys:

$$\lfloor 204 \rfloor$$

## 16. LEAF NODE LAYOUT

Leaf node usually contains:

TEXT

<Key, RecordPointer>

for each entry plus:

TEXT

one next-leaf pointer

Suppose:

TEXT

q = number of leaf entries  
Pr = record pointer size  
Pn = next-leaf pointer size  
K = key size  
B = block size

Then:

$$\lfloor q(K+P_r)+P_n \rfloor \leq B$$

Therefore:

$$\boxed{q = \left\lfloor \frac{B-P_n}{K+P_r} \right\rfloor}$$

if exactly one next-leaf pointer is stored.

## 17. SOLVED LEAF CAPACITY EXAMPLE

Given:

TEXT

Block = 4096 bytes  
Key = 12 bytes  
Record pointer = 8 bytes  
Next-leaf pointer = 8 bytes

Then:

$$\lfloor q(12+8)+8 \rfloor \leq 4096$$

$$\lfloor 20q \rfloor \leq 4088$$

$$\lfloor q \rfloor \leq 204.4$$

Therefore:

$$\boxed{q=204}$$

leaf entries.

## 18. IMPORTANT — INTERNAL CAPACITY "LEAF CAPACITY

They can be different.

Internal node:

TEXT

keys + child pointers

Leaf node:

TEXT

keys + record pointers + leaf link

Therefore GATE may ask:

**Internal fan-out = ?**

and separately:

**Leaf capacity = ?**

Do not use one formula for both.

## MEMORY

TEXT

Internal:  
children + separator keys

Leaf:  
key + record pointer  
+ next-leaf pointer

## 19. PRIMARY SEARCH FLOW

Suppose:

TEXT

```
      [30 | 60]
     /  |  \
[10,20] [30,40] [60,70]
```

Search:

TEXT

40

**Root**

$\lfloor 30 \leq 40 < 60 \rfloor$

Go middle child.

**Leaf**

TEXT

[30, 40]

Find 40.

## 20. SEARCH COST

If B+ Tree has:

TEXT

L tree levels

and every node access requires disk I/O:

TEXT

"H L d i s k a c c e s s e s

for reaching a leaf.

If the root is memory-resident:

TEXT

"H L - 1 d i s k a c c e s s e s

depending on counting convention.

If record data is stored in a separate heap/data file and leaf has only record pointer:

TEXT

+ one additional data-page access

may be needed.

## & pGATE TRAP

Distinguish:

TEXT

index leaf access

from:

TEXT

actual data-record page access

If the leaves themselves contain the full records, no extra data-page access may be needed.

If leaves contain only pointers, an additional access may be required.

Use the problem statement.

## 21. DENSE INDEX NATURE

A B+ Tree leaf level usually has an entry for every indexed search-key occurrence or appropriate pointer structure.

Therefore a secondary B+ index is often effectively dense at the leaf level.



## 22. DUPLICATE SEARCH-KEY VALUES

Suppose index is on:

TEXT  
Department

Values:

TEXT  
CSE  
CSE  
CSE  
ECE  
...

One approach is:

TEXT  
C S E ! ' l i s t / b u c k e t o f r e c o r d p o i n t e r s

or repeated leaf entries.

Exact layout depends on the index design.

For GATE:

**Understand that B+ Tree search keys need not be unique unless the index is explicitly on a unique field.**

## 23. MINIMUM INTERNAL OCCUPANCY

If internal fan-out/order is  $\lfloor m \rfloor$ , a non-root internal node usually has at least:

$\lceil \left\lceil \frac{m}{2} \right\rceil \rceil$

children.

Thus minimum internal keys:

$\boxed{\lceil \left\lceil \frac{m}{2} \right\rceil - 1 \rceil}$

Root is again special.

## 24. ROOT EXCEPTION

If root is not a leaf, it generally requires at least:

TEXT  
2 children

If the entire tree fits in one leaf:

TEXT

root = leaf

and special minimum rules apply.

Same principle as B-Tree.

## 25. MINIMUM LEAF OCCUPANCY

Leaf occupancy convention can vary.

A common rule:

**leaf nodes should be at least about half full.**

If maximum leaf entries are  $\lfloor q \rfloor$ , minimum may be specified as something like:

$\lceil \lfloor q/2 \rfloor \rceil$

depending on the definition used.

Again:

## GATE RULE

Use the exact minimum occupancy stated or implied by the question.

Do not transfer internal-node child formulas blindly to leaf entries.

## 26. INSERTION — B+ TREE

Basic insertion:

TEXT

1. Search correct leaf
2. Insert key in sorted order
3. If no overflow !' done
4. If leaf overflows !' split leaf
5. Copy separator key into parent
6. If parent overflows !' split internal node
7. Continue upward
8. Root split creates new root

## 27. LEAF SPLIT — IMPORTANT DIFFERENCE

Suppose leaf overflows:

TEXT

[10 | 20 | 30 | 40]

After split:

TEXT

[ 1 0 | 2 0 ] !' [ 3 0 | 4 0 ]

Parent gets a separator such as:

TEXT

30

But:

**30 remains in the leaf.**

So conceptually:

TEXT

```
      [30]
     /   \
[ 1 0 | 2 0 ] !' [ 3 0 | 4 0 ]
```

This is a crucial B+ Tree property.

## 28. B-TREE vs B+ TREE SPLIT IDEA

In a typical B-Tree split:

**median key is promoted, leaving the child.**

In B+ Tree leaf split:

**separator is generally copied upward, while the search key remains in the leaf.**

This distinction is often phrased:

TEXT

B - T r e e !' p r o m o t e

B + l e a f !' c o p y u p

Implementation conventions may vary in exact separator choice, but the leaf retains indexed entries.

## MEMORY TRICK

TEXT

B-Tree:  
MOVE UP

B+ leaf:  
COPY UP

Very useful exam memory.

---

## 29. INTERNAL NODE SPLIT

---

When a B+ Tree internal node overflows:

TEXT

```
[20 | 40 | 60 | 80]
```

it splits.

A separator is moved/promoted into the parent.

Unlike a leaf split, internal separator handling resembles the ordinary B-Tree behavior more closely.

---

## 30. ROOT SPLIT

---

If root overflows:

TEXT

```
old root
!' s p l i t
!' n e w   r o o t   c r e a t e d
```

Tree height increases by 1.

So B+ Tree grows upward while remaining balanced.

---

## 31. DELETION — BASIC INTUITION

---

Delete from leaf.

If leaf still has minimum required entries:

TEXT

```
done
```

If underflow:

TEXT

```
borrow/redistribute from sibling
```

or:

TEXT

```
merge with sibling
```

Parent separator keys may need adjustment.

If internal nodes underflow, similar redistribution/merge can propagate upward.

---

# MEMORY

TEXT

```
O v e r f l o w ! '   S p l i t

U n d e r f l o w ! '   B o r r o w   o r   M e r g e
```

Same high-level memory as B-Tree.

## PART — B-TREE vs B+ TREE

### 32. MASTER COMPARISON

| B-Tree                                              | B+ Tree                                            |
|-----------------------------------------------------|----------------------------------------------------|
| Data pointers may appear in internal and leaf nodes | Actual record pointers concentrated in leaves      |
| Search may stop at internal node                    | Search generally reaches leaf                      |
| Internal nodes larger per indexed record            | Internal nodes usually smaller                     |
| Lower fan-out than comparable B+ Tree               | Usually higher fan-out                             |
| Leaves not necessarily linked                       | Leaves linked                                      |
| Range traversal less convenient                     | Excellent range traversal                          |
| Keys typically stored once in tree path structure   | Separator keys may repeat in internal + leaf nodes |

### 33. WHY B+ TREE RANGE SEARCH IS BETTER

Suppose we want:

TEXT

```
1 0 0  "d   k e y  "d   1 0 0 0
```

B+ Tree:

TEXT

```
Search for 100
           !"
Find its leaf
           !"
Follow next-leaf pointers
           !"
Stop after 1000
```

No repeated root traversal.

That's the key advantage.

## 34. WHY B+ TREE INTERNAL FAN-OUT IS HIGH

Internal nodes don't need record pointers for actual data records.

So more:

TEXT

<Key, ChildPointer>

pairs fit per block.

Example:

TEXT

Higher fan-out  
! ' fewer internal nodes  
! ' lower tree height  
! ' fewer I/Os

## 35. HEIGHT CALCULATION — B+ TREE

Suppose:

TEXT

f = internal fan-out  
q = leaf capacity  
N = number of indexed records

First determine number of leaf blocks:

$$L_0 = \left\lceil \frac{N}{q} \right\rceil$$

Then build parent levels.

If fan-out = f:

$$L_1 = \left\lceil \frac{L_0}{f} \right\rceil$$

Then:

$$L_2 = \left\lceil \frac{L_1}{f} \right\rceil$$

Continue until:

TEXT

1 root

This is the safest GATE method.

## 36. SOLVED HEIGHT EXAMPLE

Given:

TEXT

Records = 1,000,000  
Leaf capacity = 100  
Internal fan-out = 100

### Leaf nodes

$\lceil \frac{1,000,000}{100} \rceil = 10,000$

### Parent level

$\lceil \frac{10,000}{100} \rceil = 100$

### Next level

$\lceil \frac{100}{100} \rceil = 1$

So structure:

TEXT

Root            1 node  
Level 1       100 nodes  
Leaves       10,000 nodes

Total levels:

TEXT

3

If root is counted as level 1.

Search reaches a leaf in:

TEXT

3 node accesses

if all on disk.

## 37. IF DATA RECORD NEEDS SEPARATE ACCESS

If each leaf entry is:

TEXT

<Key, RecordPointer>

and actual record is in a separate data block:

TEXT

3 B+ tree node accesses  
+  
1 data-block access  
=  
4 disk accesses

Again, question wording matters.

---

## 38. MAXIMUM RECORD CAPACITY FOR GIVEN HEIGHT

---

Suppose B+ Tree has:

TEXT

internal fan-out =  $f$   
leaf capacity =  $q$

and there are:

TEXT

$h$  internal edges from root to leaf

Maximum number of leaf nodes:

$$\lceil f^h \rceil$$

Each leaf stores up to:

$$\lceil q \rceil$$

entries.

Therefore:

$$\lceil \boxed{N_{\max} = qf^h} \rceil$$

under this convention.

---

## 39. EXAMPLE

---

Fan-out:

$$\lceil f=100 \rceil$$

Leaf capacity:

$$\lceil q=80 \rceil$$

Height root-to-leaf edges:

$$\lceil h=2 \rceil$$

Maximum leaves:

$$\lceil 100^2=10,000 \rceil$$

Maximum records:

$$\lceil 80 \times 10,000 \rceil$$

$$\lceil \boxed{800,000} \rceil$$

---

## 40. MINIMUM RECORD CAPACITY

---

This depends more strongly on:

- root minimum



- internal minimum fan-out
- leaf minimum occupancy

So do not memorize one universal formula unless the question specifies occupancy rules.

The safe approach is:

TEXT

1. Root minimum children
2. Non-root internal minimum children
3. Minimum number of leaves
4. Minimum entries per leaf
5. Multiply

## 41. SOLVED MINIMUM-CAPACITY STYLE PROBLEM

Suppose:

TEXT

Max internal children = 6  
Min non-root children = 3  
Root min children = 2  
Max leaf entries = 10  
Min leaf entries = 5  
Height = 2 edges

Tree:

TEXT

Root  
! "  
Internal level  
! "  
Leaves

Minimum:

Root has:

TEXT

2 children

Each internal child has minimum:

TEXT

3 leaf children

Therefore minimum leaves:

$\lceil 2 \times 3 = 6 \rceil$

Each leaf has minimum:

TEXT

5 entries

So minimum records:

$6 \times 5 = 30$

$\boxed{30}$

---

## 42. RANGE QUERY DISK ACCESS

---

Suppose first matching key requires:

TEXT

3 index I/Os

Then the qualifying range spans:

TEXT

20 leaf blocks

Because leaf blocks are linked, total index/leaf cost may be roughly:

TEXT

path to first leaf  
+  
subsequent sequential leaf blocks

For example:

TEXT

3 to reach first leaf  
+ 19 additional leaf reads

= 22 if the first leaf was already counted among those 3.

Always avoid double-counting the first leaf.

---

## & RANGE-QUERY TRAP

---

If search path already includes the first leaf, don't add that same leaf again.

---

## 43. B+ TREE AS MULTILEVEL INDEX

---

Recall multilevel index:

TEXT

Root  
!"  
Upper index  
!"  
Lower index  
!"  
Data

B+ Tree can be thought of as a dynamic, balanced multilevel index where:

- nodes split automatically
- tree remains balanced
- leaves hold actual index entries
- leaves are linked

This connection is extremely important.

## 44. B+ TREE vs HASH INDEX

| B+ Tree                   | Hash                          |
|---------------------------|-------------------------------|
| Equality search good      | Equality search excellent     |
| Range search excellent    | Range search poor             |
| Ordered keys              | No useful global order        |
| Logarithmic search        | Near constant expected lookup |
| Sequential traversal easy | Sequential order difficult    |

## MEMORY

TEXT

Equality only

!' Hash may be excellent

Equality + Range

!' B+ Tree excellent

## 45. B+ TREE vs ORDERED FILE

Ordered file gives sorted records but suffers from expensive insertion/reorganization.

B+ Tree keeps:

TEXT

sorted index

+

balanced dynamic splitting

So it handles insertions much better while retaining ordered access.

## 46. GATE NUMERICAL — FULL PROBLEM

Suppose:

#### TEXT

Block size = 1024 bytes  
Search key = 8 bytes  
Child pointer = 8 bytes  
Record pointer = 8 bytes  
Leaf next pointer = 8 bytes  
Records = 100,000

Find:

1. internal fan-out
2. leaf capacity
3. number of leaf blocks
4. tree levels

### Step 1 — Internal fan-out

Internal:

$$\lfloor \frac{8p + 8(p-1)}{1024} \rfloor$$

$$\lfloor \frac{16p - 8}{1024} \rfloor$$

$$\lfloor \frac{16p}{1032} \rfloor$$

$$\lfloor \frac{p}{64.5} \rfloor$$

So:

$$\boxed{f=64}$$

### Step 2 — Leaf capacity

$$\lfloor \frac{q(8+8) + 8}{1024} \rfloor$$

$$\lfloor \frac{16q}{1016} \rfloor$$

$$\lfloor \frac{q}{63.5} \rfloor$$

So:

$$\boxed{q=63}$$

### Step 3 — Leaf blocks

$$\left\lceil \frac{100000}{63} \right\rceil$$

$$63 \times 1587:$$

$$99981$$

So one more leaf needed:

$$\boxed{1588}$$

### Step 4 — First internal level

$\lceil 1588/64 \rceil$

$64 \times 24$ :

TEXT  
1536

Need:

$25$

nodes.

## Step 5 — Root

25 children fit in one internal node because maximum fan-out = 64.

So:

TEXT

|                 |   |         |
|-----------------|---|---------|
| R o o t         | ! | 1       |
| I n t e r n a l | ! | 2 5     |
| L e a v e s     | ! | 1 5 8 8 |

Tree levels:

$\boxed{3}$

## 47. SEARCH COST FOR PREVIOUS TREE

If all three nodes/pages on path are disk-resident:

TEXT

|      |                 |
|------|-----------------|
| Root |                 |
| !    | i n t e r n a l |
| !    | l e a f         |

= 3 I/Os to find leaf entry.

If actual data record is stored separately:

TEXT  
+1

Total:

$\boxed{4}$

If root cached:

TEXT  
3

including final data block.

---

## 48. GATE NUMERICAL — RANGE

---

Suppose:

TEXT

B+ tree height = 3 levels  
root not cached

A range query requires:

TEXT

10 consecutive leaf blocks

To reach first leaf:

TEXT

3 I/Os

This includes first leaf.

Additional leaves:

TEXT

9

Therefore index leaf I/Os:

$\lceil 3+9=12 \rceil$

If record pages must also be fetched separately, additional cost depends on clustering/data layout.

---

## 49. CLUSTERED vs UNCLUSTERED B+ INDEX — IMPORTANT INTUITION

---

Suppose range query returns 1000 records.

If the B+ index is **clustered**:

TEXT

nearby search keys  
!' nearby data records

so only a relatively small number of data blocks may be needed.

If **unclustered**:

TEXT

nearby index entries  
!' data records scattered

possibly many random data-page accesses.

This is why clustered B+ Tree indexes are especially good for ranges.

---

## 50. GATE TRAPS

---

### & pTrap 1 — Data in internal nodes

---

Typical B+ Tree:

TEXT

```
internal = navigation  
leaf = record pointers/data entries
```

Don't confuse with B-Tree.

---

### & pTrap 2 — Search usually reaches leaf

---

Even if separator key appears in an internal node, actual record pointer is usually at the leaf.

---

### & pTrap 3 — Duplicate separator

---

Same key may appear in internal and leaf node.

Normal.

---

### & pTrap 4 — Leaves linked

---

This is a defining practical advantage for range/sequential traversal.

---

### & pTrap 5 — Internal and leaf capacities differ

---

Use separate equations.

---

### & pTrap 6 — Order definition

---

Always resolve what “order” means.

---

### & pTrap 7 — Leaf split

---

Separator is typically copied upward, while key remains in leaf.

---

### & pTrap 8 — B-Tree vs B+ Tree

---

B-Tree search can finish at internal node.

B+ Tree record search generally reaches leaf.

---

## & pTrap 9 — Root cached

May reduce disk I/O by 1.

## & pTrap 10 — Extra data access

If leaf stores only record pointer, fetching actual record can require another I/O.

## & pTrap 11 — Range scan count

Don't double-count first leaf already reached during tree traversal.

## 51. PYQ-ORIENTED B+ TREE METHOD

When you see a B+ Tree numerical:

### STEP 1 — Identify node layout

Write:

TEXT

Internal:

what does each key/pointer occupy?

Leaf:

what does each entry occupy?

Is there a next-leaf pointer?

### STEP 2 — Calculate internal fan-out

Usually:

$$\lfloor \frac{pP+(p-1)K}{B} \rfloor$$

### STEP 3 — Calculate leaf capacity

Usually:

$$\lfloor \frac{q(K+P_r)+P_n}{B} \rfloor$$

### STEP 4 — Calculate leaves

$$\lceil \frac{N}{q} \rceil$$

### STEP 5 — Build internal levels

Repeatedly:

$$\lceil \frac{\text{Current}}{p} \rceil$$

until:



TEXT

1 root

## STEP 6 — Disk accesses

Count tree levels.

Then ask:

TEXT

Is root cached?

Does leaf contain record or record pointer?

## STEP 7 — Range queries

After first leaf:

TEXT

follow linked leaves

Do not repeat root traversal.

## 52. B-TREE vs B+ TREE FINAL TABLE

| Feature                           | B-Tree            | B+ Tree                        |
|-----------------------------------|-------------------|--------------------------------|
| Balanced                          | Yes               | Yes                            |
| Multiway                          | Yes               | Yes                            |
| Record pointers in internal nodes | Possible          | Typically no                   |
| Record pointers at leaves         | Yes/possible      | Yes                            |
| Search may stop internally        | Yes               | Usually no                     |
| Leaves linked                     | Not required      | Yes                            |
| Separator duplicated at leaf      | Usually no        | Can be yes                     |
| Fan-out                           | High              | Usually higher                 |
| Range query                       | Good              | Excellent                      |
| Sequential traversal              | Less convenient   | Excellent                      |
| Database usage                    | Used conceptually | Very common indexing structure |

## NOTEBOOK NOTES

## B+ TREE

Balanced multiway index tree.

### TEXT

Internal nodes:  
search keys + child pointers

Leaf nodes:  
search keys + record pointers

Leaves linked together

## INTERNAL FAN-OUT

If:

### TEXT

p pointers  
p" 1 k e y s

then:

$$\lfloor \frac{p}{p-1} \rfloor \leq B$$

Largest p = fan-out.

## LEAF CAPACITY

If:

### TEXT

q entries  
each = key + record pointer  
one next-leaf pointer

then:

$$\lfloor \frac{q(K+P_r)+P_n}{B} \rfloor$$

## NUMBER OF LEAVES

$$\boxed{\text{Leaves} = \left\lceil \frac{N}{\text{fanout}} \right\rceil}$$

## UPPER LEVELS

$$\text{Next} = \left\lceil \frac{\text{CurrentNodes}}{\text{Fanout}} \right\rceil$$

Repeat until one root.

## SEARCH

TEXT

Root

!' i n t e r n a l   n o d e s

!' l e a f

!' r e c o r d   p o i n t e r

Possible extra data-block access.

## RANGE SEARCH

TEXT

Find first leaf

!' f o l l o w   l i n k e d   l e a v e s

## INSERT

TEXT

Insert into leaf

!' o v e r f l o w ?

!' s p l i t

!' c o p y   s e p a r a t o r   u p w a r d

Internal overflow:

TEXT

split + promote

## MEMORY PAGE

TEXT

B+ TREE

INTERNAL = GUIDE

LEAF = DATA POINTERS

LEAVES = LINKED

RANGE = FAST

And:

TEXT

Internal fan-out:

p P + ( p " 1 ) K " d B

Leaf:

q ( K + R e c o r d P t r ) + N e x t P t r " d B

Leaves:  
 $\text{ceil}(N/q)$

Upper levels:  
 $\text{ceil}(\text{nodes}/\text{fanout})$   
until 1

---

## 30-SECOND REVISION

---

### TEXT

B+ Tree = balanced multiway index

All leaves same depth

Internal nodes:  
separator keys + child pointers

Leaves:  
key + record pointer

Leaves linked sequentially

B+ search generally reaches leaf

Range search:  
find first leaf, then follow links

Internal fan-out and leaf capacity  
must be calculated separately

Leaf split:  
key remains in leaf  
separator copied upward

B-Tree:  
data may be internal

B+ Tree:  
data pointers concentrated at leaves

B+ usually has higher fan-out  
and is excellent for ranges

---

## CHECK YOUR UNDERSTANDING

---

### Q1

In a typical B+ Tree, where are actual record pointers primarily stored?

A. Root only B. Internal nodes only C. Leaf nodes D. Every node must store them

---

### Q2

Why are leaf nodes linked together?

---

### Q3

What is the main structural difference between B-Tree and B+ Tree regarding data/record pointers?

---

### Q4

Given:

TEXT

Block size = 1024 bytes  
Key = 8 bytes  
Child pointer = 4 bytes

Internal node contains  $p$  pointers and  $p-1$  keys.

Find maximum internal fan-out.

---

### Q5

Given:

TEXT

Block = 1024 bytes  
Key = 8 bytes  
Record pointer = 8 bytes  
Next-leaf pointer = 8 bytes

Find maximum number of leaf entries.

---

### Q6

A B+ Tree has:

TEXT

100,000 records  
leaf capacity = 100  
fan-out = 50

Find:

1. Number of leaf nodes
  2. Number of nodes in next level
  3. Number of root nodes
  4. Total tree levels
- 

### Q7 — Important

During a B+ Tree leaf split, if separator key 50 is copied into the parent, does 50 normally disappear from the leaf?

---

### Q8

For a range query, after finding the first matching leaf, why don't we need to search from the root for every next key?

---

We have now completed:

---

File Organization !' Ordered Indexing !' Multilevel Index !' B-Tree !' B+ Tree.

Next we enter another **high-weight GATE DBMS block**:

## TOPIC 15 — TRANSACTIONS + ACID + TRANSACTION STATES+ SCHEDULES

We will cover:

**transaction concept !' read/write operations !' begin/commit/abort !' transaction states !' ACID !' serial vs concurrent schedules !' interleaving !' lost update !' dirty read !' unrepeatable read !' incorrect summary !' foundations for serializability.**

Type ``NEXT``.

USER

NEXT

ASSISTANT

## TOPIC 15 — TRANSACTIONS + ACID + TRANSACTION STATES+ SCHEDULES

We now enter another **very important GATE DBMS block**: **Transactions and Concurrency Control**, which is explicitly part of your syllabus.

This entire block will eventually cover:

**transactions !' read/write operations !' ACID !' transaction states !' schedules !' anomalies !' serializability !' precedence graph !' recoverability !' locks !' 2PL !' deadlocks !' timestamp ordering**

Today we build the foundation.

### 1. WHAT IS A TRANSACTION?

A **transaction** is a logical unit of database work.

Example:

**Transfer ₹ 1000 from account A to account B.**

This is not one database operation.

It may involve:

TEXT

1. Read balance of A
2. Subtract 1000
3. Write new balance of A
4. Read balance of B
5. Add 1000
6. Write new balance of B

All these operations together form **one transaction**.

## MEMORY TRICK

**Transaction = One complete logical job**

Examples:

TEXT

Bank transfer  
Ticket booking  
Order placement  
ATM withdrawal  
Student registration

## 2. WHY TRANSACTIONS ARE NEEDED

Suppose:

TEXT

A = 1 5 0 0 0  
B = 1 3 0 0 0

We transfer:

TEXT

1 1 0 0 0 from A to B

Correct final values:

TEXT

A = 1 4 0 0 0  
B = 1 4 0 0 0

But imagine the system crashes after:

TEXT

A = 1 4 0 0 0

and before B receives 1 1 0 0 0 .

Then:

TEXT

A = 1 4 0 0 0  
B = 1 3 0 0 0

1 1 0 0 0 effectively disappears.

A proper transaction system must prevent such inconsistent outcomes.

### 3. BASIC TRANSACTION OPERATIONS

For GATE schedules, transactions are usually represented using:

TEXT

```
read(X)
write(X)
```

Often abbreviated:

TEXT

```
r(X)
w(X)
```

Example:

TEXT

T1:

```
r1(A)
A = A - 1000
w1(A)

r1(B)
B = B + 1000
w1(B)

commit1
```

### 4. READ OPERATION

$\backslash[ r\_i(X) \backslash]$

means:

**Transaction  $\backslash(T\_i\backslash)$  reads database item X.**

Example:

TEXT

```
r1(A)
```

means transaction T1 reads A.

Usually the value moves conceptually:

TEXT

```
Database A
! "
Transaction's local variable
```

### 5. WRITE OPERATION



$\forall w_i(X)$

means:

**Transaction  $\backslash(T_i)$  writes a value to database item X.**

Example:

```
TEXT
w1(A)
```

means T1 updates database item A.

## 6. READ-MODIFY-WRITE

Most updates conceptually look like:

```
TEXT
read(X)
modify local X
write(X)
```

Example:

```
TEXT
r1(A)

A = A + 100

w1(A)
```

In schedule problems, the arithmetic step is often omitted.

We mainly analyze:

```
TEXT
r1(A)
w1(A)
```

## 7. COMMIT

COMMIT means:

**Transaction has successfully completed and its effects should become permanent.**

Notation:

```
TEXT
C1
```

or:

TEXT

commit(T1)

After commit, transaction is considered successfully completed.

## MEMORY

**COMMIT = Confirm permanently**

## 8. ABORT / ROLLBACK

If transaction cannot complete correctly:

TEXT

ABORT

or:

TEXT

ROLLBACK

is performed.

Its incomplete changes must be undone as required by the recovery mechanism.

Memory:

**Abort = Cancel transaction**

## 9. TRANSACTION BOUNDARY

A transaction can conceptually look like:

TEXT

BEGIN TRANSACTION

read(A)

write(A)

read(B)

write(B)

COMMIT

or if something goes wrong:

TEXT

BEGIN

...

ABORT

---

## PART A — ACID PROPERTIES

---

### 10. ACID

---

The four fundamental transaction properties are:

TEXT

```
A !'  A t o m i c i t y
C !'  C o n s i s t e n c y
I !'  I s o l a t i o n
D !'  D u r a b i l i t y
```

This acronym is extremely important for GATE.

---

### 11. ATOMICITY

---

Atomicity means:

**A transaction is all or nothing.**

Either:

TEXT

all operations happen

or:

TEXT

none of its effects remain

---

### BANK TRANSFER EXAMPLE

---

Transaction:

TEXT

```
A = A - 1000
B = B + 1000
```

Atomicity prevents:

TEXT

```
A decreased
but
B not increased
```

If transaction fails halfway:

---

TEXT

rollback

restores database appropriately.

## MEMORY TRICK

**Atomicity = ALL or NOTHING**

## 12. CONSISTENCY

Consistency means:

**A transaction should take the database from one valid state to another valid state, assuming the transaction logic itself is correct.**

Example:

Before transfer:

TEXT

A + B = 1 8 0 0 0

After transfer:

TEXT

A + B = 1 8 0 0 0

Database constraints should remain satisfied.

Examples:

TEXT

Primary key uniqueness  
Foreign key validity  
Balance rules  
Domain constraints

## MEMORY TRICK

**Consistency = Constraints remain correct**

## 13. ISOLATION

Isolation means:

**Concurrent transactions should behave, from a correctness perspective, as though they are appropriately isolated from one another.**

A transaction should not see harmful intermediate effects of another transaction.

Example:

T1 is transferring money.

T2 should not observe an inconsistent intermediate state such as:

TEXT

A already reduced  
B not yet increased

depending on the concurrency-control level.

## MEMORY TRICK

**Isolation = Transactions don't disturb each other**

## 14. DURABILITY

Durability means:

**Once a transaction commits, its effects must survive failures.**

Suppose:

TEXT

T1 COMMIT

Then power failure occurs.

After recovery:

TEXT

T1's committed changes should still exist

## MEMORY TRICK

**Durability = Done means Durable**

or:

**Commit survives crash**

---

## 15. ACID TABLE

---

| Property    | Core meaning                                          |
|-------------|-------------------------------------------------------|
| Atomicity   | All or nothing                                        |
| Consistency | Valid state != valid state                            |
| Isolation   | Concurrent execution should not interfere incorrectly |
| Durability  | Committed changes survive failure                     |

---

## 16. GATE TRAPS — ACID

---

### & pTrap 1

---

Atomicity is not:

TEXT

Only one transaction at a time

That's closer to isolation/serial behavior.

Atomicity means:

TEXT

all-or-nothing

---

### & pTrap 2

---

Durability applies after:

TEXT

COMMIT

not merely after a write operation.

---

### & pTrap 3

---

Consistency is not purely provided by DBMS concurrency control.

Application transaction logic must also preserve business rules.

---

### & pTrap 4

---

Isolation does not necessarily mean DBMS literally executes transactions one by one.

---

It can execute concurrently while maintaining a correctness criterion such as serializability.

## PART B — TRANSACTION STATES

## 17. TRANSACTION STATES

A standard transaction-state model contains:

TEXT

- Active
- Partially Committed
- Committed
- Failed
- Aborted
- Terminated

Some textbooks simplify the final state naming, but these are the important ones.

## 18. STATEDIAGRAM

TEXT

[illegible]

A more useful exam flow:

TEXT

```
Active
|
| last operation executed
!"
Partially Committed
|
| commit safely recorded
!"
Committed

Active
|
failure
!"
Failed
```

```
|
rollback
!"
Aborted
```

## 19. ACTIVE STATE

Transaction is currently executing.

Example:

TEXT

```
r(A)
w(A)
r(B)
...
```

It remains active until:

- last operation completes, or
- failure occurs.

## 20. PARTIALLY COMMITTED

Transaction has executed its final statement but commit is not yet guaranteed to be durable.

This is an important distinction.

Example:

TEXT

```
last write completed
```

but the system may still need to ensure commit-related information reaches stable storage.

## & PGATE TRAP

**Finishing the last statement does not immediately mean fully committed.**

There can be a:

TEXT

```
Partially Committed
```

state.

## 21. COMMITTED

Transaction successfully completes and its effects become durable.



This is the success state.

---

## 22. FAILED

---

Transaction cannot continue because of:

TEXT

system crash  
logical error  
deadlock victim selection  
constraint failure  
hardware/software problem

It enters:

TEXT

Failed

before rollback completes.

---

## 23. ABORTED

---

After transaction's effects are rolled back, it enters:

TEXT

Aborted

From there it may:

- restart, or
- terminate permanently.

---

## 24. STATEMEMORY

---

TEXT

ACTIVE

!' d o i n g   w o r k

PARTIALLY COMMITTED

!' f i n i s h e d   w o r k ,   c o m m i t   n o t   y e t   f u l l y   s a f e

COMMITTED

!' s u c c e s s

FAILED

!' c a n n o t   c o n t i n u e

ABORTED

!' e f f e c t s   u n d o n e

## PART C — SCHEDULES

---

### 25. WHAT IS A SCHEDULE?

---

When multiple transactions execute, their read/write operations may be interleaved.

A **schedule** is the chronological ordering of operations from multiple transactions, while preserving the internal order of each transaction.

---

### 26. EXAMPLE TRANSACTIONS

---

Suppose:

**T1**

TEXT

r1(X)  
w1(X)

**T2**

TEXT

r2(Y)  
w2(Y)

Possible schedule:

TEXT

r1(X)  
w1(X)  
r2(Y)  
w2(Y)

Another:

TEXT

r1(X)  
r2(Y)  
w1(X)  
w2(Y)

Both preserve each transaction's own program order.

---

### 27. INTERNAL ORDER MUST BE PRESERVED

---

If T1 is:

TEXT

r1(X)  
w1(X)

schedule cannot place:

---

TEXT  
w1(X)

before:

TEXT  
r1(X)

because that changes T1 itself.

Interleaving may occur **between transactions**, not by rearranging the internal order of one transaction.

---

## 28. SERIAL SCHEDULE

A **serial schedule** executes one complete transaction before starting another.

Example:

TEXT  
T1:  
r1(X)  
w1(X)  
  
then T2:  
r2(X)  
w2(X)

Schedule:

TEXT  
r1(X)  
w1(X)  
C1  
r2(X)  
w2(X)  
C2

No interleaving.

---

## 29. ANOTHER SERIAL ORDER

Could also execute:

TEXT  
T2 completely  
then  
T1 completely

So with 2 transactions there are:

$2! = 2$

possible serial orders.

With n transactions:

$\boxed{n!}$

possible serial schedules, if considering only transaction order.

## 30. NON-SERIAL / CONCURRENT SCHEDULE

Operations interleave.

Example:

TEXT

```
r1(X)
r2(X)
w1(X)
w2(X)
```

This is non-serial.

But:

**Non-serial does NOT automatically mean incorrect.**

Some non-serial schedules are equivalent to serial schedules.

That is exactly what **serializability** studies next.

## MEMORY TRICK

TEXT

```
Serial
!' transaction by transaction

Concurrent
!' operations interleaved
```

## 31. WHY USE CONCURRENT EXECUTION?

Why not simply execute everything serially?

Because concurrency can improve:

- CPU utilization
- disk utilization
- throughput
- response time

Example:

TEXT

```
T1 waiting for disk
```

CPU can execute:

TEXT

T2

instead of sitting idle.

## 32. BENEFIT vs DANGER

Concurrency gives:

TEXT

higher performance

but may create:

TEXT

incorrect results

if operations interfere.

Therefore we need:

### Concurrency Control

Goal:

TEXT

Concurrency  
+  
Correctness

## PART D — ANOMALIES

### 33. COMMON CONCURRENCY ANOMALIES

Important GATE anomalies:

1. Lost update
2. Dirty read / temporary update
3. Unrepeatable read
4. Incorrect summary / inconsistent analysis

We'll understand each carefully.

### 34. LOST UPDATE

Suppose:

$\forall X=100$

Two transactions both update X.

## T1

TEXT

```
read X
X = X + 10
write X
```

## T2

TEXT

```
read X
X = X + 20
write X
```

Correct serial final result should be:

$\{ 130 \}$

regardless of order.

## 35. BAD INTERLEAVING

TEXT

```
Initial X = 100

r 1 ( X ) !'   T 1   r e a d s   1 0 0

r 2 ( X ) !'   T 2   r e a d s   1 0 0

T1 computes 110
w 1 ( X ) !'   d a t a b a s e   b e c o m e s   1 1 0

T2 computes 120
w 2 ( X ) !'   d a t a b a s e   b e c o m e s   1 2 0
```

Final:

$\{ 120 \}$

But correct result:

$\{ 130 \}$

T1's update disappeared.

This is:

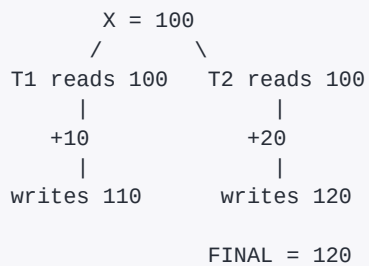
**Lost Update**

## MEMORY TRICK

**Lost update = One write overwrites another update.**

## 36. LOST UPDATE VISUAL

TEXT



The +10 is lost.

## 37. DIRTY READ

Dirty read occurs when:

**One transaction reads a value written by another transaction that has not committed yet.**

Example:

TEXT

Initial X = 100

T1:

TEXT

w1(X=200)

but T1 has not committed.

T2:

TEXT

r2(X)

reads:

TEXT

200

Then T1 aborts.

Database returns to:

TEXT

100

But T2 already used:

TEXT

200

which was never committed.

That value was **dirty**.

## MEMORY TRICK

**Dirty = Read uncommitted data**

## 38. DIRTY READ SCHEDULE

TEXT

w1(X)  
r2(X)  
A1

T2 read T1's value before T1 committed.

If T1 aborts, T2 has consumed invalid temporary data.

## 39. TEMPORARY UPDATE PROBLEM

Dirty read is also connected to the:

**Temporary Update / Uncommitted Dependency problem**

because T2 depends on T1's temporary value.

## 40. UNREPEATABLE READ

Transaction reads the same item twice but gets different values because another transaction modified and committed it in between.

Example:

TEXT

X = 100

Schedule:

TEXT

r1(X) !' reads 100  
  
r2(X)  
w2(X=200)



C2

r 1 ( X ) ! '   n o w   r e a d s   2 0 0

T1's two reads give:

TEXT

100  
then  
200

This is:

**Unrepeatable Read**

## MEMORY TRICK

**Read twice, result changed**

## 41. DIRTY vs UNREPEATABLE READ

| Dirty Read                       | Unrepeatable Read                         |
|----------------------------------|-------------------------------------------|
| Reads uncommitted value          | Reads committed changed value             |
| Writer may abort                 | Writer commits                            |
| Problem = invalid temporary data | Problem = same item changes between reads |

Very important distinction.

## 42. INCORRECT SUMMARY / INCONSISTENT ANALYSIS

Suppose transaction T1 calculates:

TEXT

Total = A + B

while T2 transfers money from A to B.

Initial:

TEXT

A = 1000  
B = 1000  
Total = 2000

T2 transfers:

TEXT

100 from A to B

Final should still total:

TEXT

2000

## 43. BAD INTERLEAVING

T1 reads:

TEXT

A = 1000

Then T2:

TEXT

A = 900

B = 1100

Then T1 reads:

TEXT

B = 1100

T1 calculates:

$\lfloor 1000 + 1100 = 2100 \rfloor$

But neither the old nor new consistent state had total 2100.

This is:

**Incorrect Summary / Inconsistent Analysis**

## MEMORY TRICK

**Summary mixed old + new values**

## 44. PHANTOM READ — BRIEF NOTE

Another anomaly seen in transaction theory is the **phantom read**:

A transaction repeats a predicate query such as:

SQL

```
SELECT *  
FROM EMPLOYEE  
WHERE Salary > 50000;
```

and another committed transaction inserts/deletes a row satisfying that predicate.

The second execution returns a different **set of rows**.

Example:

TEXT

```
F i r s t   q u e r y   ! '   1 0   r o w s  
Another transaction inserts matching row  
S e c o n d   q u e r y   ! '   1 1   r o w s
```

This is different from unrepeatable read, which focuses on the same existing row changing.

Phantom reads matter more when studying isolation levels, though they are not always emphasized in basic GATE questions.

## 45. ANOMALY COMPARISON

| Anomaly           | Key idea                                            |
|-------------------|-----------------------------------------------------|
| Lost Update       | One update overwritten                              |
| Dirty Read        | Read uncommitted data                               |
| Unrepeatable Read | Same row read twice gives different committed value |
| Incorrect Summary | Aggregate sees mixed database states                |
| Phantom Read      | Repeated predicate query sees different row set     |

## PART E — SCHEDULE NOTATION

### 46. COMMON GATE NOTATION

Example:

$\forall [S: r_1(X); r_2(X); w_1(X); w_2(X)]$

means:

TEXT

```
T1 reads X  
T2 reads X  
T1 writes X  
T2 writes X
```

Subscript identifies transaction.

## 47. COMMIT/ABORT IN SCHEDULE

TEXT

C1

means:

TEXT

T1 commits

TEXT

A1

means:

TEXT

T1 aborts

Example:

TEXT

w1(X)

r2(X)

C2

A1

T2 read an uncommitted T1 value and even committed before T1 aborted.

This is extremely problematic and will connect to **recoverability** later.

## 48. COMPLETE SCHEDULE

A schedule is **complete** when it includes all operations of the involved transactions and each transaction eventually:

TEXT

commits

or

aborts

depending on textbook terminology.

## 49. SERIAL DOES NOT MEAN SAME AS SERIALIZABLE

This is critical.

### Serial schedule

Actually executes:

TEXT

T1 completely  
then T2

## Serializable schedule

May be interleaved, but produces an effect equivalent to some serial schedule under the chosen equivalence criterion.

Therefore:

TEXT

Serial  
",  
Serializable

Every serial schedule is serializable.

But many serializable schedules are not serial.

## MEMORY

**Serial = physically one after another**

**Serializable = logically equivalent to one-after-another**

This distinction will dominate the next topic.

## 50. SERIAL SCHEDULE SAFETY

If each transaction individually preserves consistency, a serial execution generally preserves consistency because:

TEXT

consistent state  
!" T 1  
consistent state  
!" T 2  
consistent state

But serial schedules may have poor performance.

So DBMS aims for concurrent schedules with serial-like correctness.

## 51. WHY SERIALIZABILITY IS NEEDED

We want:

TEXT

Concurrency benefits  
+  
Correct serial behavior

Therefore:

**Allow interleaving, but only safe interleavings.**

This leads to:

TEXT

Conflict Serializability  
View Serializability

coming next.

## 52. GATE-STYLE EXAMPLE — IDENTIFY LOST UPDATE

Initial:

$\backslash$   $X=100$   $\backslash$

Schedule:

TEXT

$r_1(X)$   
 $r_2(X)$   
 $w_1(X)$   
 $w_2(X)$

Suppose:

TEXT

T1:  $X = X + 10$   
T2:  $X = X + 20$

Both read 100.

Final after T2 writes:

TEXT

120

instead of:

TEXT

130

Therefore:

$\backslash$   $\boxed{\text{Lost Update}}$   $\backslash$

## 53. GATE-STYLE EXAMPLE — DIRTY READ

Schedule:

TEXT

w1(X)  
r2(X)  
A1

T2 reads T1's uncommitted write.

T1 later aborts.

Therefore:

$\boxed{\text{Dirty Read}}$

## 54. GATE-STYLE EXAMPLE — UNREPEATABLE READ

TEXT

r1(X)  
w2(X)  
C2  
r1(X)

T1 reads X before and after T2's committed modification.

Different values possible.

Therefore:

$\boxed{\text{Unrepeatable Read}}$

## 55. GATE-STYLE EXAMPLE — INCORRECT SUMMARY

T1:

TEXT

read A  
read B  
calculate total

T2 modifies both A and B between those reads.

T1 may observe:

TEXT

old A  
+  
new B

Therefore:

$\boxed{\text{Incorrect Summary}}$

## 56. ACID vs ANOMALIES CONNECTION

#### TEXT

##### Atomicity

!' protects all-or-nothing behavior

##### Consistency

!' protects valid database states

##### Isolation

!' protects concurrent transactions  
from harmful interference

##### Durability

!' protects committed results  
from failure

Most concurrency anomalies primarily motivate:

#### TEXT

##### Isolation

+

##### Concurrency Control

## 57. CONCURRENCY CONTROL GOALS

Concurrency control techniques aim to achieve properties such as:

- serializability
- recoverability
- isolation
- avoiding harmful anomalies

Techniques later include:

#### TEXT

##### Locks

##### 2PL

##### Timestamp ordering

## 58. GATE TRAPS

### & pTrap 1 — Transaction

A transaction can contain many read/write operations.

Not just one SQL statement.

### & pTrap 2 — Commit

Commit makes successful transaction effects durable.



### & pTrap 3 — Partial commit

Last statement executed `` fully committed.

### & pTrap 4 — Serial vs serializable

Not the same.

### & pTrap 5 — Concurrent schedule

Interleaving does not automatically mean incorrect.

### & pTrap 6 — Lost update

Usually involves:

TEXT

both transactions read old value  
then later writes overwrite one another

### & pTrap 7 — Dirty read

Reader sees **uncommitted** write.

### & pTrap 8 — Unrepeatable read

Writer normally commits, and same item later reads differently.

### & pTrap 9 — Incorrect summary

Transaction sees mixture of states.

### & pTrap 10 — Isolation

Does not require literally serial execution.

Concurrency control can make concurrent schedules behave correctly.

## 59. PYQ-ORIENTED RECOGNITION

When GATE says:

**“Transaction either completes fully or has no effect”**

Think:

TEXT

Atomicity

When it says:

**“Committed changes survive crash”**

Think:

TEXT

Durability

When it says:

**“Database constraints remain valid”**

Think:

TEXT

Consistency

When it says:

**“Concurrent transactions appear isolated”**

Think:

TEXT

Isolation

When schedule shows:

TEXT

r1(X)

r2(X)

w1(X)

w2(X)

with both modifying X:

Think:

TEXT

Lost update

When:

TEXT

w1(X)  
r2(X)

before T1 commit:

Think:

TEXT

Dirty read

When:

TEXT

r1(X)  
...  
w2(X)  
C2  
...  
r1(X)

Think:

TEXT

Unrepeatable read

When aggregate reads old and new values:

Think:

TEXT

Incorrect summary

## 60. NOTEBOOK NOTES

### TRANSACTION

Logical unit of database work.

Basic operations:

TEXT

```
r _ i ( X ) ! '   T i   r e a d s   X
w _ i ( X ) ! '   T i   w r i t e s   X
C i               ! '   T i   c o m m i t s
A i               ! '   T i   a b o r t s
```

### ACID

#### TEXT

A !' A t o m i c i t y  
all or nothing

C !' C o n s i s t e n c y  
v a l i d   s t a t e !'   v a l i d   s t a t e

I !' I s o l a t i o n  
concurrent transactions  
don't interfere incorrectly

D !' D u r a b i l i t y  
committed changes survive failure

## TRANSACTION STATES

#### TEXT

Active  
!"

Partially Committed  
!"

Committed

Failure path:

#### TEXT

Active  
!"

Failed  
!"

Aborted

## SCHEDULE

Ordering of operations of multiple transactions while preserving each transaction's own operation order.

## SERIAL

#### TEXT

T1 completely  
then T2 completely

No interleaving.

## NON-SERIAL

Operations interleave.

May still be correct.

## SERIALIZABLE

Non-serial schedule whose effect is equivalent to some serial schedule.

TEXT

Every serial schedule  
!' s e r i a l i z a b l e

Not every serializable schedule  
!' s e r i a l

## ANOMALIES

### Lost Update

TEXT

One write overwrites another update

### Dirty Read

TEXT

Read uncommitted value

### Unrepeatable Read

TEXT

Same item read twice  
gives different committed values

### Incorrect Summary

TEXT

Aggregate uses mixture  
of old and new states

## MEMORY PAGE

TEXT

ACID

A = ALL or nothing

C = CORRECT state

I = ISOLATED transactions

D = DONE stays done

Transaction states:

#### TEXT

##### ACTIVE

```
!' P A R T I A L   C O M M I T  
!' C O M M I T
```

##### ACTIVE

```
!' F A I L E D  
!' A B O R T
```

#### Anomalies:

#### TEXT

##### LOST UPDATE

```
!' o v e r w r i t e
```

##### DIRTY READ

```
!' u n c o m m i t t e d
```

##### UNREPEATABLE

```
!' r e a d   t w i c e ,   v a l u e   c h a n g e s
```

##### INCORRECT SUMMARY

```
!' o l d   +   n e w   m i x e d
```

## 30-SECOND REVISION

#### TEXT

Transaction = one logical unit of work

$r1(X)$  = T1 reads X

$w1(X)$  = T1 writes X

##### ACID:

Atomicity = all or nothing

Consistency = valid state

Isolation = concurrent correctness

Durability = commit survives crash

##### States:

###### Active

```
!' P a r t i a l l y   C o m m i t t e d  
!' C o m m i t t e d
```

or

###### Active

```
!' F a i l e d  
!' A b o r t e d
```

##### Serial schedule:

one whole transaction at a time

##### Concurrent schedule:

interleaved operations

##### Lost update:

one update overwritten

##### Dirty read:

read uncommitted data

Unrepeatable read:  
same item changes between reads

Incorrect summary:  
mix of old/new database states

S e r i a l " ` S e r i a l i z a b l e

## CHECK YOUR UNDERSTANDING

### Q1

Which ACID property means:

**“Either all operations of a transaction occur or none of them do”?**

A. Consistency B. Atomicity C. Isolation D. Durability

### Q2

A transaction has executed its last statement, but its commit has not yet been safely completed.

What state is it in?

### Q3

Initial:

$\{ X=100 \}$

T1 adds 10 and T2 adds 20.

Schedule:

TEXT

r1(X)  
r2(X)  
w1(X)  
w2(X)

If both read 100, what anomaly can occur?

### Q4

Schedule:

TEXT

w1(X)  
r2(X)  
A1

What anomaly occurred?

## Q5

Schedule:

TEXT

r1(X)  
w2(X)  
C2  
r1(X)

What anomaly can T1 experience?

## Q6

What is the difference between:

TEXT

Serial schedule

and:

TEXT

Serializable schedule

?

## Q7 — Important GATE concept

With 4 transactions, how many possible **serial transaction orders** are there?

## Q8

Which ACID property is most directly related to ensuring that committed data survives a power failure?

Next is one of the **highest-scoring transaction topics in GATE**:

## TOPIC 16 — CONFLICT SERIALIZABILITY + PRECEDENCE GRAPH

We will learn:

**conflicting operations !' RW/WR/WW conflicts !' non-conflicting RR !' conflict equivalence !' conflict serializability !' precedence graph !' cycle detection !' equivalent serial order !' topological sorting !' difficult GATE schedule problems.**

Type ``NEXT``.

USER

NEXT



## TOPIC 16 — CONFLICT SERIALIZABILITY + PRECEDENCE GRAPH

This is one of the **most important and most frequently tested transaction topics in GATE DBMS**. It directly belongs to the **transactions and concurrency control** part of your syllabus.

The core flow is:

```
TEXT
Schedule
! "
Conflicting Operations
! "
Conflict Equivalence
! "
Precedence Graph
! "
Cycle?
! "
Serializable or Not
! "
Equivalent Serial Order
```

The biggest GATE skill here is:

**Given a schedule, quickly construct the graph and detect whether it has a cycle.**

### 1. WHY SERIALIZABILITY?

We already know:

- Serial schedules are safe but slow.
- Concurrent schedules are fast but can cause anomalies.

So DBMS tries to allow:

```
TEXT
interleaving
+
correctness
```

A schedule is serializable if its effect is equivalent to some serial execution.

Today we study:

**Conflict Serializability**

### 2. WHAT IS A CONFLICT?

Two operations **conflict** when all three conditions hold:

1. They belong to **different transactions**.
2. They operate on the **same data item**.

3. At least one of them is a **WRITE**.

This is the exact rule.

## MEMORY TRICK

Different Tx + Same Item + One Write = Conflict

Write this in your notebook.

## 3. POSSIBLE OPERATION PAIRS

For operations on the same data item X:

TEXT

r1(X) and r2(X)  
r1(X) and w2(X)  
w1(X) and r2(X)  
w1(X) and w2(X)

Which conflict?

## 4. READ–READ

$\{ r_1(X), r_2(X) \}$

Both only read.

No value is changed.

Therefore:

$\{ \boxed{\text{No conflict}} \}$

## 5. READ–WRITE

$\{ r_1(X), w_2(X) \}$

T1 reads X, T2 writes X.

Order matters.

Therefore:

$\{ \boxed{\text{Conflict}} \}$

## 6. WRITE–READ

$\{ w_1(X), r_2(X) \}$

T1 writes X, T2 reads X.

Order matters.

Therefore:

$\boxed{\text{Conflict}}$

## 7. WRITE-WRITE

$w_1(X), w_2(X)$

Both write same item.

Order matters because final value depends on which write occurs last.

Therefore:

$\boxed{\text{Conflict}}$

## MASTER CONFLICT TABLE

| Pair | Conflict? |
|------|-----------|
| R-R  | No        |
| R-W  | Yes       |
| W-R  | Yes       |
| W-W  | Yes       |

## MEMORY TRICK

Only R-R is peaceful.

Everything involving a write conflicts.

## 8. SAME DATA ITEM IS REQUIRED

Suppose:

TEXT

$r_1(X)$

$w_2(Y)$

Even though one operation is a write:

TEXT

X   "   Y

So they do **not** conflict.

Thus:

Same data item is mandatory.

## 9. SAME TRANSACTION DOES NOT COUNT

Suppose:

TEXT

$r1(X)$

$w1(X)$

These are both from T1.

They are not considered a conflict for serializability comparison between transactions because their internal order is fixed.

We care about conflicts between different transactions.

## 10. CONFLICTING OPERATIONS — QUICK TEST

For pair:

TEXT

$oi(X), oj(Y)$

ask:

### Q1

Different transactions?

If no !' no conflict.

### Q2

Same item?

If no !' no conflict.

### Q3

At least one write?

If no !' no conflict.

If all yes:

TEXT

CONFLICT

## 11. WHY ORDER OF CONFLICTS MATTERS

Suppose:

TEXT

w1(X)  
r2(X)

If T1 comes before T2, T2 reads T1's value.

If swapped:

TEXT

r2(X)  
w1(X)

T2 reads the old value.

Different behavior.

Therefore we cannot freely swap conflicting operations.

---

## 12. NON-CONFLICTING OPERATIONS CAN BE SWAPPED

Example:

TEXT

r1(X)  
r2(X)

Swapping them:

TEXT

r2(X)  
r1(X)

does not change database behavior.

Likewise:

TEXT

r1(X)  
w2(Y)

with X" Y can often be swapped.

This is the basis of conflict equivalence.

---

## PART B — CONFLICT EQUIVALENCE

---

### 13. CONFLICT EQUIVALENT SCHEDULES

Two schedules are conflict equivalent if:

1. They contain the same operations of the same transactions.
2. The relative order of every pair of conflicting operations is the same.

---

## SIMPLE EXAMPLE

---

Schedule S1:

```
TEXT
r1(X)
w1(X)
r2(Y)
w2(Y)
```

Schedule S2:

```
TEXT
r2(Y)
w2(Y)
r1(X)
w1(X)
```

T1 works only on X.

T2 works only on Y.

There are no conflicts between transactions.

Therefore we can swap the transactions' operations freely.

The schedules are conflict equivalent.

---

## 14. EXAMPLE WITH CONFLICT

---

Schedule:

```
TEXT
w1(X)
r2(X)
```

If another schedule has:

```
TEXT
r2(X)
w1(X)
```

they are **not conflict equivalent** because the order of a conflicting pair has reversed.

---

## MEMORY

---

**Conflict equivalence = Preserve conflict order**

Non-conflicting operations may move.

Conflicting ones cannot reverse.

---

---

## 15. CONFLICT SERIALIZABILITY

A schedule is **conflict serializable** if it is conflict equivalent to some serial schedule.

In simple language:

**Can we swap only non-conflicting operations and transform the schedule into a serial one?**

Doing this manually is possible but tedious.

So GATE uses a much easier method:

**Precedence Graph / Serialization Graph**

---

## PART C — PRECEDENCE GRAPH

### 16. WHAT IS A PRECEDENCE GRAPH?

For a schedule:

- One node for each transaction.
- Add a directed edge:

$T_i \rightarrow T_j$

if an operation of  $T_i$  occurs before and conflicts with an operation of  $T_j$  on the same data item.

Then:

**Schedule is conflict serializable iff the graph is acyclic.**

This is the most important rule.

---

## WRITE THIS IN A BOX

$\boxed{\text{Conflict Serializable} \iff \text{Precedence Graph has NO cycle}}$

---

## MEMORY TRICK

**Cycle = Cannot serialize**

**No cycle = Serializable**

---

## 17. HOW TO CREATE AN EDGE

Suppose schedule contains:

TEXT  
w1(X)  
...  
r2(X)

These conflict.

T1's operation occurs first.

So edge:

$\{ T_1 \rightarrow T_2 \}$

Suppose:

TEXT  
r2(Y)  
...  
w3(Y)

Then:

$\{ T_2 \rightarrow T_3 \}$

## 18. EDGE DIRECTION

Always:

**Earlier conflicting transaction !' Later conflicting transaction**

Not writer !' reader automatically.

Not lower-numbered !' higher-numbered.

It depends on schedule order.

## & pGATE TRAP

For:

TEXT  
r1(X)  
w2(X)

edge:

$\{ T1 \rightarrow T2 \}$

But if:



TEXT

w2(X)  
r1(X)

edge becomes:

$\backslash [T_2 \rightarrow T_1 \backslash]$

## 19. SIMPLE SERIALIZABLE EXAMPLE

Schedule:

TEXT

r1(X)  
w1(X)  
r2(X)  
w2(X)

Conflicts:

TEXT

w1(X) before r2(X)  
! ' T 1 ! ' T 2

TEXT

w1(X) before w2(X)  
! ' T 1 ! ' T 2

TEXT

r1(X) before w2(X)  
! ' T 1 ! ' T 2

Graph:

TEXT

T 1 % % % ! ' T 2

No cycle.

Therefore:

$\backslash [ \boxed{\text{Conflict Serializable}} \backslash ]$

Equivalent serial order:

TEXT

T1 then T2

## 20. SERIAL ORDER FROM GRAPH

If graph:

TEXT

T 1 !' T 2 !' T 3

then equivalent serial order:

TEXT

T1, T2, T3

This is a **topological ordering**.

## 21. WHAT IS TOPOLOGICAL ORDER?

For a directed acyclic graph, a topological order places every node before the nodes it points to.

If:

$\backslash T1 \rightarrow T2 \backslash$

then T1 must come before T2.

If:

$\backslash T1 \rightarrow T3 \backslash$

then T1 must come before T3.

## 22. SERIAL ORDER MAY NOT BE UNIQUE

Graph:

TEXT

```
      T1
     /  \
    /    \
   /      \
  /        \
 T2          T3
```

Edges:

TEXT

T 1 !' T 2  
T 1 !' T 3

Possible topological orders:

TEXT

T1, T2, T3

or:

TEXT

T1, T3, T2

Both valid.

So:

A conflict-serializable schedule may correspond to multiple serial orders.

### 23. NON-SERIALIZABLE EXAMPLE

Schedule:

TEXT

r1(X)  
w1(X)  
r2(X)  
w2(X)  
r2(Y)  
w2(Y)  
r1(Y)  
w1(Y)

Let's find conflicts.

On X:

TEXT

T1 operations before T2 conflicting operations

So:

$\nexists T1 \rightarrow T2$

On Y:

TEXT

T2 operations before T1 conflicting operations

So:

$\nexists T2 \rightarrow T1$

Graph:

TEXT

|       |          |     |
|-------|----------|-----|
| T 1   | % % % !' | T 2 |
| !'    |          |     |
| _____ |          |     |

Cycle:

$\nexists T1 \rightarrow T2 \rightarrow T1$

Therefore:

$\nexists \boxed{\text{Not conflict serializable}}$

### MEMORY

Opposite dependency directions between two transactions create an immediate 2-node cycle.

## PART D — COMPLETE GRAPH ALGORITHM

### 24. GATE ALGORITHM

Given schedule:

#### STEP 1

Create one node per transaction.

#### STEP 2

For every pair of operations:

- different transactions
- same item
- at least one write

identify conflict.

#### STEP 3

Whichever transaction's conflicting operation occurs first gets edge to the later transaction.

#### STEP 4

Remove duplicate edges.

#### STEP 5

Check for cycle.

#### STEP 6

If no cycle:

TEXT

Conflict Serializable

Find topological ordering for serial order.

If cycle:

TEXT

Not Conflict Serializable

### 25. FASTEST WAY — SCAN BY DATAITEM

Instead of comparing every pair blindly:

Group operations by data item.

Example schedule:

TEXT

r1(X)  
w2(Y)  
w1(X)  
r3(X)  
w2(X)  
r1(Y)

Analyze X separately.

Then Y separately.

This reduces mistakes.

---

## 26. SOLVED EXAMPLE 1

Schedule:

$\{ S: r_1(X); w_1(X); r_2(X); w_2(X) \}$

Transactions:

TEXT

T1  
T2

Conflicts:

**r1(X) vs w2(X)**

Different transactions, same X, one write.

Order:

TEXT

T1 before T2

Edge:

$\{ T1 \rightarrow T2 \}$

**w1(X) vs r2(X)**

Edge:

$\{ T1 \rightarrow T2 \}$

**w1(X) vs w2(X)**

Edge:

$\{ T1 \rightarrow T2 \}$

Graph:

TEXT

T 1 ! ' T 2

Acyclic.

Answer:

$\boxed{\text{text}\{\text{CSR, serial order}\}T1,T2}$

CSR = conflict serializable.

## 27. SOLVED EXAMPLE 2

Schedule:

$\boxed{r_1(X); r_2(X); w_1(X); w_2(X)}$

Find edges.

**r2(X) before w1(X)**

Conflict R–W.

So:

$T2 \rightarrow T1$

**r1(X) before w2(X)**

So:

$T1 \rightarrow T2$

Also:

TEXT

w1(X) before w2(X)

gives:

$T1 \rightarrow T2$

Graph:

TEXT

T 1 ! " T 2

Cycle.

Therefore:

$\boxed{\text{text}\{\text{Not conflict serializable}\}}$

This is also the classic lost-update-style schedule.

## 28. SOLVED EXAMPLE 3 — THREE TRANSACTIONS

Schedule:

TEXT

r1(X)  
w1(X)

r2(X)  
w2(X)

r3(X)  
w3(X)

Edges:

TEXT

T 1 !' T 2  
T 1 !' T 3  
T 2 !' T 3

Graph:

TEXT

T 1 !' T 2 !' T 3  
  \ \_ \_ \_ \_ \_ !'

No cycle.

Serial order:

$\boxed{T1, T2, T3}$

## 29. SOLVED EXAMPLE 4 — THREE TRANSACTIONS WITH CYCLE

Schedule:

TEXT

w1(X)  
r2(X)

w2(Y)  
r3(Y)

w3(Z)  
r1(Z)

From X:

$T1 \rightarrow T2$

From Y:

$T2 \rightarrow T3$

From Z:

$T3 \rightarrow T1$

Graph:

TEXT

T 1 !' T 2 !' T 3  
!' |  
|\_\_\_\_\_|

Cycle.

Therefore:

$\boxed{\text{Not CSR}}$

### 30. SOLVED EXAMPLE 5 — READ-READ TRAP

Schedule:

TEXT

r1(X)  
r2(X)  
r1(Y)  
r2(Y)

All cross-transaction pairs are R–R.

No conflicts.

Graph:

TEXT

T1 T2

No edges.

Acyclic.

Therefore schedule is conflict serializable.

Possible serial orders:

TEXT

T1, T2

or:

TEXT

T2, T1

### & pGATE TRAP

No edges does not mean “schedule is serial.”

It means it is conflict equivalent to **any compatible serial ordering** because no cross-transaction conflict constrains order.



## PART E — CONFLICT TYPES

---

### 31. READ–WRITE CONFLICT

---

Pattern:

```
TEXT
ri(X)
wj(X)
```

with  $\neg(i \neq j)$ .

If  $r_i$  occurs first:

$\neg [T_i \rightarrow T_j]$

---

### 32. WRITE–READ CONFLICT

---

Pattern:

```
TEXT
wi(X)
rj(X)
```

If  $w_i$  occurs first:

$\neg [T_i \rightarrow T_j]$

---

### 33. WRITE–WRITE CONFLICT

---

Pattern:

```
TEXT
wi(X)
wj(X)
```

If  $w_i$  first:

$\neg [T_i \rightarrow T_j]$

---

### 34. READ–READ

---

No edge.

Never add edge merely because two transactions access same item.

At least one must write.

---

## MEMORY

---

#### TEXT

```
R W !' e d g e
W R !' e d g e
W W !' e d g e
R R !' N O e d g e
```

## PART F — TOPLOGICAL SORT

### 35. FINDING SERIAL ORDER

Once graph is acyclic, perform a topological sort.

Easy method:

**Find transaction with indegree 0.**

Place it first.

Remove it and its outgoing edges.

Repeat.

### 36. EXAMPLE

Graph:

#### TEXT

```
T 1 !' T 2
T 1 !' T 3
T 2 !' T 4
T 3 !' T 4
```

Indegree:

#### TEXT

```
T1 = 0
T2 = 1
T3 = 1
T4 = 2
```

Start T1.

After removing T1:

#### TEXT

```
T2 and T3 both indegree 0
```

Either can come next.

Possible orders:

TEXT

T1, T2, T3, T4

or:

TEXT

T1, T3, T2, T4

## 37. UNIQUE SERIAL ORDER?

A DAG can have:

- one topological ordering
- multiple topological orderings

If at every step there is only one indegree-zero choice, the ordering is unique.

If multiple choices exist:

TEXT

multiple equivalent serial orders

This can appear in GATE.

## PART G — CONFLICT SERIALIZABILITY vs SERIAL SCHEDULE

### 38. EVERY SERIAL SCHEDULE IS CSR

A serial schedule already has a transaction order:

TEXT

T 1 !' T 2 !' T 3

Conflicts can only create edges consistent with that direction.

No reverse edges.

Therefore no cycle.

Hence:

$\boxed{\text{Every serial schedule is conflict serializable}}$

### 39. NOT EVERY CSR SCHEDULE IS SERIAL

Example:

TEXT

r1(X)  
r2(Y)  
w1(X)

w2(Y)

Interleaved:

TEXT  
not serial

But T1 and T2 operate on different items.

No conflicts.

So it is CSR.

Thus:

$\boxed{\text{Serial}} \subseteq \text{ConflictSerializable}$

---

## PART H — CONFLICT EQUIVALENCE BETWEEN TWO SCHEDULES

---

### 40. GATE MAY ASK

---

Are schedules S1 and S2 conflict equivalent?

Method:

#### Step 1

Check same transactions and same operations.

#### Step 2

List every conflicting operation pair.

#### Step 3

Check whether their relative order is identical in both schedules.

If all same:

TEXT  
Conflict equivalent

---

### 41. EXAMPLE

---

S1:

TEXT  
r1(X)  
w1(X)  
r2(X)  
w2(X)

S2:

TEXT

r1(X)  
w1(X)  
w2(X)  
r2(X)

Check conflicts.

In S1:

TEXT

w1(X) before w2(X)

Same in S2.

TEXT

w1(X) before r2(X)

Same.

TEXT

r1(X) before w2(X)

Same.

But S2 changed order between:

TEXT

w2(X)  
r2(X)

Both belong to T2 itself—such a change would violate T2's transaction program order if these are actual T2 operations, so S2 would not be a valid alternate schedule for the same transaction definition if T2 requires r2 before w2.

This reminds us:

**Internal transaction order must remain unchanged.**

## PART I — IMPORTANT GATE DETAIL: FINAL WRITE NOT USED HERE

Conflict serializability is based on **all conflict orders**.

It does not focus specifically on:

- who reads from whom
- final writer

Those ideas belong more directly to:

**View Serializability**

coming next.

For CSR:

TEXT

Conflicts + Graph + Cycle

is sufficient.

## PART J — WHY PRECEDENCE GRAPH WORKS

### 42. INTUITION

If graph contains:

$\{ T1 \rightarrow T2 \}$

then any conflict-equivalent serial schedule must place:

TEXT

T1 before T2

If also:

$\{ T2 \rightarrow T3 \}$

then:

TEXT

T1 before T2 before T3

No problem.

But if:

$\{ T3 \rightarrow T1 \}$

then requirements become:

TEXT

T1 before T2  
T2 before T3  
T3 before T1

Impossible.

Hence cycle means no serial ordering can satisfy all conflict constraints.

## MEMORY

**Graph edge = “must come before.”**

Cycle = impossible ordering.

## PART K — GATE-STYLE HARDER PROBLEM

### 43. PROBLEM

Schedule:

TEXT

r1(A)  
r2(A)  
w2(A)  
r3(B)  
w1(B)  
r2(B)  
w3(A)

Find whether CSR.

### 44. ANALYZE ITEM A

Operations on A:

TEXT

r1(A)  
r2(A)  
w2(A)  
w3(A)

Conflicts:

**r1(A) before w2(A)**

\[ T1\rightarrow T2 \]

**r1(A) before w3(A)**

\[ T1\rightarrow T3 \]

**r2(A) before w3(A)**

\[ T2\rightarrow T3 \]

**w2(A) before w3(A)**

\[ T2\rightarrow T3 \]

### 45. ANALYZE ITEM B

Operations:

TEXT

r3(B)  
w1(B)  
r2(B)

Conflicts:

**r3(B) before w1(B)**

$\backslash [ T3 \rightarrow T1 \backslash ]$

**w1(B) before r2(B)**

$\backslash [ T1 \rightarrow T2 \backslash ]$

## 46. GRAPH

Edges:

TEXT

```
T 1  !'  T 2
T 1  !'  T 3
T 2  !'  T 3
T 3  !'  T 1
```

There is cycle:

TEXT

```
T 1  !'  T 3  !'  T 1
```

Therefore:

$\backslash [ \boxed{\text{Not conflict serializable}} \backslash ]$

Notice:

**We do not need to analyze anything further once a cycle is proven.**

GATE time-saving trick.

## 47. FAST CYCLE DETECTION

If you already find:

TEXT

```
T 1  !'  T 2
```

and later:

TEXT

```
T 2  !'  T 1
```

stop.

There is a cycle.

No need to complete the graph unless question explicitly asks for all edges.

## 48. TRANSITIVE EDGES NEED NOT BE ADDED MANUALLY



Suppose conflicts directly give:

TEXT

```
T 1  !'   T 2
T 2  !'   T 3
```

You don't need to additionally draw:

TEXT

```
T 1  !'   T 3
```

unless an actual direct conflict also creates it.

Cycle detection works without adding transitive closure.

## & pGATE TRAP

The precedence graph contains edges generated by direct conflicting operation order, not every logically implied transitive edge.

## 49. MULTIPLE CONFLICTS GIVE ONE EDGE

Suppose five conflicts all imply:

$\{ T1 \rightarrow T2 \}$

The graph still just needs one edge:

TEXT

```
T 1  !'   T 2
```

No need to draw five arrows.

## PART L — CONNECTION TO ANOMALIES

## 50. LOST UPDATE SCHEDULE

TEXT

```
r1(X)
r2(X)
w1(X)
w2(X)
```

Graph:

From:

TEXT

```
r2(X) before w1(X)
```

$\{ T2 \rightarrow T1 \}$

From:

TEXT

r1(X) before w2(X)

$\lceil T1 \rightarrow T2 \rceil$

Cycle.

Therefore schedule is not CSR.

This is why lost-update schedules are often non-serializable.

## 51. BUT ANOMALY CHECK "SERIALIZABILITY TEST"

Do not try to determine CSR only by recognizing anomalies.

The reliable method is:

TEXT

Conflict pairs  
!' Graph  
!' Cycle

A schedule may look harmless but still require formal graph analysis.

## PART M — GATE TRAPS

### & pTrap 1

R-R does not conflict.

### & pTrap 2

Different data items don't conflict.

### & pTrap 3

Operations in same transaction are not used to create graph edges.

### & pTrap 4

Edge direction = earlier conflicting operation !' later conflicting operation.

### & pTrap 5

Cycle means not CSR.

## & pTrap 6

Acyclic graph means CSR.

No exceptions for conflict serializability.

---

## & pTrap 7

Serializable does not mean serial.

---

## & pTrap 8

Equivalent serial order is a topological ordering, not necessarily transaction numerical order.

---

## & pTrap 9

More than one topological ordering may exist.

---

## & pTrap 10

Duplicate conflicts between same transaction pair create only one directed edge.

---

## & pTrap 11

Do not add edge just because transactions access same variable.

Need at least one write.

---

## & pTrap 12

A schedule may be **not conflict serializable yet still view serializable**.

We study that next.

This is very important.

---

## PART N — PYQ SOLVING METHOD

When GATE gives a schedule:

### STEP 1

Write transactions as graph nodes.

### STEP 2

Circle/mark all writes.

Writes are the key because every conflict needs a write.

---

### STEP 3

For each written data item, compare with reads/writes by other transactions.

### STEP 4

Draw earlier !' later edge.

### STEP 5

Immediately look for:

TEXT

```
T i !' T j
and
T j !' T i
```

or longer cycles.

### STEP 6

If cycle:

TEXT

Not CSR

Stop.

### STEP 7

If no cycle:

Topologically sort.

That gives equivalent serial order(s).

## NOTEBOOK NOTES

### CONFLICT

Two operations conflict iff:

TEXT

1. Different transactions
2. Same data item
3. At least one WRITE

### CONFLICT TABLE

TEXT

```
R R !' N o
R W !' Y e s
W R !' Y e s
```

W W !' Y e s

Memory:

TEXT

Only RR is peaceful.

## CONFLICT EQUIVALENCE

Two schedules preserve the relative order of every conflicting pair.

## CONFLICT SERIALIZABLE

Schedule is conflict equivalent to a serial schedule.

## PRECEDENCE GRAPH

TEXT

Node = transaction

For conflict:

TEXT

earlier  $T_i$  operation  
before later  $T_j$  operation

!' e d g e  $T_i$  !'  $T_j$

## MAIN TEST

$\lceil \boxed{\text{Acyclic}} \text{ graph } \iff \text{Conflict} \text{ Serializable} \rceil$

TEXT

C y c l e !' N o t C S R  
N o c y c l e !' C S R

## SERIAL ORDER

Any topological ordering of precedence graph.

## MEMORY PAGE

TEXT

CONFLICT?

Different Tx?

Same Item?

One Write?

Y E S   t o   a l l   ! '   c o n f l i c t

Then:

TEXT

E a r l i e r   ! '   L a t e r

Finally:

TEXT

GRAPH

NO CYCLE

! '   S E R I A L I Z A B L E

CYCLE

! '   N O T   S E R I A L I Z A B L E

---

## 30-SECOND REVISION

---

TEXT

Conflict:

different transactions

same data item

at least one write

RR = no conflict

RW = conflict

WR = conflict

WW = conflict

Precedence graph:

node = transaction

Edge:

earlier conflicting transaction

! '   l a t e r   t r a n s a c t i o n

Conflict serializable

iff graph is acyclic

Cycle = not CSR

Equivalent serial order

= topological order

Serial schedule

! '   a l w a y s   C S R

CSR schedule

! '   n e e d   n o t   b e   s e r i a l

No edges

!' C S R , p o s s i b l y m a n y s e r i a l o r d e r s

## CHECK YOUR UNDERSTANDING

### Q1

Do these conflict?

$\{ r_1(X), r_2(X) \}$

Why?

### Q2

Do these conflict?

$\{ r_1(X), w_2(Y) \}$

Why?

### Q3

For:

TEXT

$w_2(X)$

...

$r_1(X)$

what graph edge is created?

A.  $T_1 \rightarrow T_2$  B.  $T_2 \rightarrow T_1$  C. No edge

### Q4

Schedule:

TEXT

$r_1(X)$

$w_1(X)$

$r_2(X)$

$w_2(X)$

Is it conflict serializable? What serial order does it correspond to?

### Q5

Schedule:

TEXT

r1(X)  
r2(X)  
w1(X)  
w2(X)

Find the graph edges and determine whether it is conflict serializable.

## Q6

Graph:

TEXT

|     |    |     |
|-----|----|-----|
| T 1 | !' | T 2 |
| T 1 | !' | T 3 |
| T 2 | !' | T 4 |
| T 3 | !' | T 4 |

Give two possible equivalent serial orders.

## Q7 — Most important

Complete:

A schedule is conflict serializable iff its precedence graph is \_\_\_\_\_.

## Q8 — GATE trap

Can a schedule that is **not conflict serializable** still be serializable under another equivalence concept?

Next:

## TOPIC 17 — VIEW SERIALIZABILITY + BLIND WRITES

This is where many students get confused.

We will cover:

read-from relation !' initial reads !' final writes !' view equivalence !' view serializability !' blind writes !' CSR vs VSR !' why every CSR is VSR but not every VSR is CSR !' GATE shortcuts and tricky schedules.

Type `NEXT`.

USER

NEXT

ASSISTANT



## TOPIC 17 — VIEW SERIALIZABILITY + BLIND WRITES

Now we move to the second major serializability concept:

### View Serializability

This topic is usually more confusing than conflict serializability because there is **no simple cycle test** like the precedence graph.

The key ideas are:

**read-from relation !' initial reads !' final writes !' view equivalence !' view serializability !' blind writes !' CSR vs VSR**

## 1. WHY VIEW SERIALIZABILITY?

Conflict serializability is strict.

Sometimes a schedule fails the conflict-serializability test because its precedence graph has a cycle.

But the schedule may still produce the same meaningful read/write behavior as some serial schedule.

Such schedules may be:

### View Serializable

So:

$\boxed{\text{Conflict} \setminus \text{Serializable} \subseteq \text{View} \setminus \text{Serializable}}$

More precisely:

**Every conflict-serializable schedule is view-serializable.**

But:

**Some view-serializable schedules are not conflict-serializable.**

This difference often occurs because of:

### Blind Writes

## 2. FIRST UNDERSTAND “WHO READS FROM WHOM”

Suppose schedule:

TEXT

w1(X)

r2(X)

T1 writes X.

Then T2 reads X.

Assuming no other write to X occurs between them:

**T2 reads the value written by T1.**

We say:

$\text{\textit{T}_2 reads X from \textit{T}_1}$

or:

TEXT

$r_2(X) \text{ reads-from } w_1(X)$

This read-from relationship is very important in view equivalence.

### 3. INITIAL READ

Suppose:

TEXT

$r_1(X)$

occurs before any transaction writes X.

Then T1 reads the:

**Initial value of X**

We can think of the database before transactions as a special source:

TEXT

T0

So conceptually:

$\text{\textit{T}_1 reads X from initial database}$

### MEMORY

TEXT

No earlier write to X  
!' initial read

### 4. FINAL WRITE

Suppose several transactions write X:

TEXT

w1(X)  
w2(X)  
w3(X)

If:

TEXT

w3(X)

is the last write on X in the schedule, then:

**T3 performs the final write on X.**

View equivalence requires final writers to match.

## 5. VIEW EQUIVALENCE

Two schedules S and S' are **view equivalent** if they satisfy three conditions.

For every data item X:

### Condition 1 — Initial Read

If a transaction reads the initial value of X in S, it must also read the initial value of X in S'.

### Condition 2 — Read-From Relation

If:

$\forall [T_i]$

reads X from:

$\forall [T_j]$

in S, then  $T_i$  must read X from  $T_j$  in S'.

### Condition 3 — Final Write

The transaction performing the final write on X must be the same in both schedules.

These are the three key conditions.

## MEMORY TRICK

Remember:

**I-R-F**

TEXT

I !' Initial read  
R !' Read - from  
F !' Final write

Two schedules are view equivalent if they preserve:

```
TEXT
IRF
```

## 6. VIEW SERIALIZABILITY

A schedule is **view serializable** if it is view equivalent to some serial schedule.

So the process is:

```
TEXT
Given non-serial schedule
    !"
Find a serial order
    !"
Check IRF conditions
    !"
If all match
!' View Serializable
```

## 7. SIMPLE EXAMPLE

Schedule:

```
TEXT
r1(X)
w1(X)
r2(X)
w2(X)
```

Here:

- T1 reads initial X.
- T2 reads X written by T1.
- T2 is final writer of X.

This schedule is clearly equivalent to serial:

```
TEXT
T 1 !' T 2
```

So it is:

```
TEXT
Conflict Serializable '
View Serializable '
```

## 8. EVERY CSR IS VSR

Why?

Conflict equivalence preserves the order of all conflicting operations.

That automatically preserves:

- who reads initial value
- who reads from whom
- final writer

Therefore:

$\boxed{CSR \rightarrow VSR}$

## 9. BUT VSR DOES NOT IMPLY CSR

There are schedules where:

TEXT

Precedence graph has cycle

so they are not conflict serializable.

But they are still view equivalent to some serial schedule.

This typically happens because of:

### Blind Writes

## PART A — BLIND WRITES

### 10. WHAT IS A BLIND WRITE?

A **blind write** occurs when a transaction writes a data item without first reading that item.

Example:

TEXT

T1:  
w1(X)

No:

TEXT

r1(X)

before it.

So T1 writes X blindly.

## MEMORY TRICK

**Blind write = WRITE without READ**

## 11. EXAMPLE

Transaction:

TEXT

T1:  
w1(X)  
w1(Y)

These are blind writes if T1 never reads X or Y before writing.

Blind writes are important because conflict serializability treats write-write order very strictly.

But view serializability mainly cares about:

TEXT

who reads what  
+  
who writes finally

So some intermediate blind writes can become irrelevant to the final observable view.

## 12. CLASSIC VSR BUT NOT CSR EXAMPLE

Consider schedule:

$\{ S: w_1(X); w_2(X); w_1(X) \}$

Let's analyze.

## 13. CONFLICT SERIALIZABILITY TEST

Operations are:

TEXT

w1(X)  
w2(X)  
w1(X)

First:

TEXT

w1(X) before w2(X)

gives:

$\{ T1 \rightarrow T2 \}$

Then:

TEXT

w2(X) before later w1(X)

gives:

$\lceil T2 \rightarrow T1 \rceil$

Graph:

TEXT

T 1 ! " T 2

Cycle.

Therefore:

$\lceil \boxed{\text{Not Conflict Serializable}} \rceil$

## 14. VIEW SERIALIZABILITY TEST

Does anyone read X?

No.

So:

TEXT

Initial-read condition  
!' irrelevant

TEXT

Read-from condition  
!' irrelevant

Final write on X is:

TEXT

w1(X)

So final writer is T1.

Now consider serial order:

TEXT

T 2 ! ' T 1

Operations serially:

TEXT

w2(X)  
w1(X)

Final writer is again:

TEXT

T1

No reads need to be preserved.

Thus schedule is view equivalent to serial:

TEXT

T 2 !' T 1

Therefore:

$\boxed{\text{View Serializable}}$

but not conflict serializable.

This is the classic idea.

## 15. WHY BLIND WRITES CREATE THIS DIFFERENCE

In:

TEXT

w1(X)  
w2(X)  
w1(X)

the first write by T1 is overwritten later.

No transaction reads it.

So from the view perspective, that first write has no observable effect.

Conflict serializability still sees:

TEXT

T 1 !' T 2

and:

TEXT

T 2 !' T 1

creating a cycle.

View serializability can ignore the practical irrelevance of that overwritten blind write.

## 16. IMPORTANT RELATION

$\boxed{\text{Conflict Serializable} \subset \text{View Serializable}}$

Strict inclusion because there exist:



TEXT

VSR but not CSR

schedules.

## 17. IF THERE ARE NO BLIND WRITES?

A very useful theoretical fact:

**If a schedule has no blind writes, then view serializability and conflict serializability coincide for practical testing of such schedules.**

In common GATE framing:

**A schedule that is view serializable but not conflict serializable must involve blind writes.**

This is a powerful shortcut.

## MEMORY

TEXT

VSR but not CSR?

! ' Look for blind writes

## PART B — INITIAL READ CONDITION

### 18. EXAMPLE

Schedule S1:

TEXT

r1(X)

w2(X)

T1 reads the initial value of X.

Suppose S2:

TEXT

w2(X)

r1(X)

Now T1 reads the value written by T2.

So S1 and S2 are **not view equivalent**.

Why?

Because initial-read condition changed.

---

## 19. INITIAL READ RULE

---

If:

TEXT  
ri(X)

reads initial X in one schedule,  
then in a view-equivalent schedule:

TEXT  
ri(X)

must also read initial X.

---

## PART C — READ-FROM CONDITION

---

### 20. EXAMPLE

---

Schedule S1:

TEXT  
w1(X)  
r2(X)

T2 reads from T1.

Now consider S2:

TEXT  
w3(X)  
w1(X)  
r2(X)

If T1's write is still the most recent write before r2(X), T2 still reads from T1.

Fine.

But if:

TEXT  
w1(X)  
w3(X)  
r2(X)

then T2 reads from T3.

That changes read-from relation.

Therefore not view equivalent.

---

## MEMORY

Same reader must read from same writer.

## PART D — FINAL WRITE CONDITION

### 21. EXAMPLE

S1:

TEXT

w1(X)

w2(X)

Final writer:

TEXT

T2

S2:

TEXT

w2(X)

w1(X)

Final writer:

TEXT

T1

Therefore S1 and S2 are not view equivalent.

Even though there are no reads.

### 22. FINAL WRITE RULE

For every data item X:

Same transaction must perform the final write in both schedules.

## PART E — COMPLETE VSR CHECK

### 23. GATE METHOD

Given a schedule S:

## STEP 1

Identify for each read:

TEXT

Does it read initial value?  
or  
Which transaction's write does it read from?

## STEP 2

Identify final writer for each data item.

## STEP 3

Try possible serial transaction orders.

## STEP 4

For each candidate serial order, check:

TEXT

Initial reads same?  
Read-from same?  
Final writers same?

If yes:

TEXT

View Serializable

---

## 24. WHY VSR IS HARDER THAN CSR

CSR has:

TEXT

Precedence graph  
!' c y c l e t e s t

Very efficient.

VSR does not have an equally simple general graph test.

Checking view serializability can be computationally harder.

For GATE, schedules are generally small enough to reason manually.

---

## PART F — SOLVED EXAMPLE 1

Schedule:

TEXT

r1(X)  
w1(X)  
r2(X)

w2(X)

### Read information

r1(X) :

TEXT

reads initial X

r2(X) :

TEXT

reads from T1

Final writer:

TEXT

T2

Serial order:

TEXT

T 1 !' T 2

preserves all conditions.

Therefore:

$\boxed{\text{VSR}}$

And graph is acyclic, so it is also CSR.

## PART G — SOLVED EXAMPLE 2

Schedule:

TEXT

w1(X)

w2(X)

w1(X)

We already saw:

TEXT

C S R '

because graph cycle.

But:

- no reads
- final writer T1

Serial:

TEXT

T 2 !' T 1

has final writer T1.

Therefore:

$\boxed{\text{VSR but not CSR}}$

## PART H — SOLVED EXAMPLE 3

Schedule:

TEXT

w1(X)

r2(X)

w2(X)

Read relation:

TEXT

T2 reads X from T1

Final writer:

TEXT

T2

Serial candidate:

TEXT

T 1 !' T 2

In this serial order:

- T2 reads T1's X
- T2 final writes X

Everything matches.

Therefore:

TEXT

V S R '

Graph:

TEXT

T 1 !' T 2

SO:

TEXT

C S R ' '

## PART I — SOLVED EXAMPLE 4

Schedule:

TEXT

$r1(X)$

$w2(X)$

$w1(X)$

Let's inspect.

**$r1(X)$**

No earlier write.

So T1 reads initial X.

### Final writer

Last write:

TEXT

$w1(X)$

So T1 final writer.

Could serial order be:

TEXT

T 2 !' T 1

?

Then T2 writes X before T1 executes.

T1's read would see T2's value.

But in original schedule T1 reads initial value.

So initial-read condition breaks.

Could serial be:

TEXT

T 1 !' T 2

?

Then final writer would be T2.

But original final writer is T1.

Breaks final-write condition.

Therefore no serial order works.

So:

$\boxed{\text{Not View Serializable}}$

## 25. IMPORTANT INSIGHT

For two transactions, you can often test only:

TEXT

T 1 !' T 2

and:

TEXT

T 2 !' T 1

against the IRF conditions.

If neither works:

TEXT

not VSR

## PART J — THREE TRANSACTIONS EXAMPLE

## 26. SCHEDULE

TEXT

w1(X)  
w2(X)  
w3(X)  
w1(X)

No reads.

Final writer:

TEXT

T1

For view equivalence, all we need is a serial order ending in:

TEXT

T1

For example:

TEXT

T 2 !' T 3 !' T 1



Final writer T1.  
So schedule is VSR.  
Conflict graph?  
Conflicts include:

```
TEXT
T 1  !'  T 2
T 1  !'  T 3
T 2  !'  T 3
T 2  !'  T 1
T 3  !'  T 1
```

Cycles exist.  
Therefore:

```
TEXT
V S R  '
C S R  '
```

Again due to blind writes.

## PART K — VSR vs CSR COMPARISON

### 27. MASTER TABLE

| Conflict Serializability             | View Serializability                             |
|--------------------------------------|--------------------------------------------------|
| Based on conflicting operation order | Based on read/write view                         |
| Uses precedence graph                | No simple general cycle test                     |
| Easier to test                       | Harder to test                                   |
| More restrictive                     | More general                                     |
| Preserves all conflict order         | Preserves initial reads, read-from, final writes |
| CSR implies VSR                      | VSR need not imply CSR                           |
| Blind writes can cause difference    | Can allow some blind-write schedules             |

## MEMORY

```
TEXT
CSR :
CONFLICTS

VSR :
IRF
```

Where:

TEXT

I = Initial read  
R = Read-from  
F = Final write

## 28. SERIALIZABILITY RELATION

Think of sets:

TEXT

View Serializable  
% % % % % % % % % % % % % % % % % % % % % %  
% %  
% C o n f l i c t S e r i a l i z a b l e  
% % % % % % % % % % % % % % % %  
% % % % % % % % % % % % % % % %  
% % % % % % % % % % % % % % % %  
% % % % % % % % % % % % % % % %  
% % % % % % % % % % % % % % % %

Thus:

$\mathcal{V}[\text{CSR} \setminus \text{subseq} \text{VSR}]$

and generally strict:

$\mathcal{V}[\text{CSR} \setminus \text{subset} \text{VSR}]$

## PART L — CONFLICT EQUIVALENT vs VIEW EQUIVALENT

### 29. CONFLICT EQUIVALENCE

Requires:

Same relative order of every conflicting operation pair.

Very strict.

### 30. VIEW EQUIVALENCE

Only requires same observable read/write behavior:

TEXT

same initial readers  
same read-from relationships  
same final writers

Therefore less restrictive.

## 31. EXAMPLE OF VIEW EQUIVALENT BUT NOT CONFLICT EQUIVALENT

S:

TEXT

w1(X)  
w2(X)  
w1(X)

Serial:

TEXT

w2(X)  
w1(X)

Final writer same.

No reads.

View equivalent.

But conflict orders differ because the original includes an earlier:

TEXT

w1(X) before w2(X)

and later:

TEXT

w2(X) before w1(X)

Hence not conflict equivalent.

## PART M — BLIND WRITE DETECTION

### 32. GATE SHORTCUT

For every:

TEXT

w<sub>i</sub>(X)

look backward within transaction T<sub>i</sub>.

Has T<sub>i</sub> previously executed:

TEXT

r<sub>i</sub>(X)

?

If no:

TEXT  
blind write

### 33. EXAMPLE

T1:

TEXT  
r1(A)  
w1(A)  
w1(B)

w1(A) :

TEXT  
not blind

because T1 read A first.

w1(B) :

TEXT  
blind

because T1 did not read B first.

### & pGATE TRAP

Blind write depends on whether **the same transaction** reads that item first.

It doesn't matter whether another transaction read it.

## PART N — IMPORTANT THEORETICAL FACTS

### 34. FACT 1

$\boxed{\text{Serial} \rightarrow \text{CSR}}$

### 35. FACT 2

$\boxed{\text{CSR} \rightarrow \text{VSR}}$

### 36. FACT 3

$\boxed{\text{VSR} \rightarrow \text{CSR}}$

## 37. OVERALL RELATION

$\boxed{\text{Serial} \subseteq \text{ConflictSerializable} \subseteq \text{ViewSerializable}}$

With strict inclusion possible.

## MEMORY LADDER

TEXT

```
Serial
  |
! "
Conflict Serializable
  |
! "
View Serializable
```

Moving downward gives a more general class.

## PART O — GATE-STYLE QUESTION

### 38. QUESTION

Schedule:

$\{ S: w_1(X); w_2(X); w_1(X) \}$

Which is correct?

A. Neither CSR nor VSR B. CSR but not VSR C. VSR but not CSR D. Both CSR and VSR

We know:

Graph:

TEXT

```
T1 !' T2
T2 !' T1
```

so not CSR.

No reads.

Final writer T1.

Serial T2! T1 preserves final write.

Therefore:

$\boxed{C}$

## PART P — GATE-STYLE QUESTION 2

Schedule:

TEXT

r1(X)  
w2(X)  
w1(X)

Check VSR.

Original:

TEXT

T1 reads initial X  
Final writer = T1

Serial T 1! T 2 :

TEXT

initial read preserved  
but final writer = T2

fails.

Serial T 2! T 1 :

TEXT

final writer preserved  
but T1 no longer reads initial X

fails.

Therefore:

$\lnot \boxed{\text{Not VSR}}$

Since CSR implies VSR, it also cannot be CSR.

---

## PART Q — GATE-STYLE QUESTION 3

Schedule:

TEXT

r1(X)  
w1(X)  
w2(X)

Read:

TEXT

T1 reads initial X

Final writer:

TEXT

T2

Serial:

TEXT

T 1 !' T 2

preserves both.

Therefore VSR.

Graph:

TEXT

T 1 !' T 2

so CSR too.

Answer:

TEXT

C S R '  
V S R '

---

## PART R — FINAL WRITE ACROSS MULTIPLE ITEMS

---

### 39. IMPORTANT

---

View equivalence checks final writer **for every data item separately**.

Example:

TEXT

w1(X)  
w2(Y)  
w3(X)  
w1(Y)

Final writer:

TEXT

X !' T 3  
Y !' T 1

Any serial schedule considered view equivalent must preserve both:

TEXT

T3 final on X  
T1 final on Y

This may impose contradictory ordering constraints.

---

### 40. EXAMPLE

---

If T3 must be after all other writers of X:

TEXT

. . . !' T 3

and T1 must be after all other writers of Y:

TEXT

. . . !' T 1

depending on reads and writes, a serial order might or might not exist.

So VSR can still fail even with blind writes.

---

## PART 5 — VIEW SERIALIZABILITY FAST METHOD

---

When schedule has only 2–3 transactions:

### STEP 1

Write:

TEXT

Initial readers:

X !' ?

Y !' ?

### STEP 2

Write:

TEXT

Read-from:

T2 reads X from T1

...

### STEP 3

Write:

TEXT

Final writer:

X !' T ?

Y !' T ?

### STEP 4

Try possible serial orders.

With 2 transactions:

TEXT

T1, T2

T2, T1



With 3:

up to:

$\lceil 3!/6 \rceil$

but IRF constraints usually eliminate most quickly.

---

## 41. USE CONSTRAINTS TO AVOID TRYING ALL ORDERS

---

Suppose:

TEXT

T2 reads X from T1

Then in candidate serial order:

TEXT

T1 must occur before T2

and no intervening writer may change the relevant read source.

If final writer X = T3:

TEXT

T3 must come after other relevant X writers

These constraints quickly narrow possible serial orders.

---

## PART T — CONFLICT GRAPH CAN HELP FIRST

---

A good exam strategy:

**First test CSR.**

Why?

Precedence graph is easy.

If graph is acyclic:

TEXT

CSR

Then automatically:

TEXT

VSR

No need for separate VSR analysis.

Only if graph has a cycle and question asks about VSR:

---

TEXT

check blind writes + IRF

This saves time.

## GATE SHORTCUT

TEXT

Graph acyclic?

YES

!' C S R

!' a u t o m a t i c a l l y V S R

Graph cyclic?

!' N o t C S R

!' V S R s t i l l p o s s i b l e

!' c h e c k b l i n d w r i t e s / I R F

Excellent exam strategy.

## PART U — GATE TRAPS

### & pTrap 1

Cycle proves only:

TEXT

Not conflict serializable

It does **not** automatically prove:

TEXT

Not view serializable

### & pTrap 2

Every CSR is VSR.

### & pTrap 3

Not every VSR is CSR.

### & pTrap 4

Blind write:

TEXT

write without prior read  
by same transaction

## & pTrap 5

View equivalence does not preserve every conflict order.

It preserves:

TEXT

Initial read  
Read-from  
Final write

## & pTrap 6

Final writer must be checked separately for every item.

## & pTrap 7

If T1 reads initial X in original schedule, it must also read initial X in equivalent serial schedule.

## & pTrap 8

Same final database value alone is not sufficient for view equivalence.

You must preserve read relationships too.

## & pTrap 9

Two schedules can produce the same final values accidentally yet not be view equivalent.

## & pTrap 10

VSR testing is generally harder than CSR testing.

Use CSR test first.

## NOTEBOOK NOTES

## BLIND WRITE

TEXT

$w_i(X)$

without prior:

TEXT

$r_i(X)$

in transaction  $T_i$ .

Memory:

TEXT

WRITE without READ

## VIEW EQUIVALENCE

Two schedules must preserve:

### I — Initial read

Same transaction reads initial X.

### R — Read-from

If  $T_i$  reads X from  $T_j$  in one schedule, same in other.

### F — Final write

Same transaction performs final write on X.

Mnemonic:

TEXT

IRF

## VIEW SERIALIZABLE

Schedule is view equivalent to some serial schedule.

## RELATION

$\boxed{\text{Serial}} \subseteq \text{CSR} \subseteq \text{VSR}$

## IMPORTANT

TEXT

CSR !' always VSR

VSR !' may not be CSR

Difference usually involves:

TEXT

Blind writes

## CSR vs VSR MASTER TABLE

| CSR                           | VSR                                   |
|-------------------------------|---------------------------------------|
| Conflict based                | View based                            |
| Graph test                    | IRF test                              |
| Cycle !' fail CSR             | Cycle does not automatically fail VSR |
| Easier                        | Harder                                |
| More restrictive              | More general                          |
| Blind writes cause difficulty | Can accept some blind-write schedules |

## MEMORY PAGE

TEXT

CSR

= CONFLICT ORDER

= GRAPH

= NO CYCLE

TEXT

VSR

= IRF

I !' Initial read  
R !' Read - from  
F !' Final write

and:

TEXT

VSR but not CSR?

!' BLIND WRITE likely

## 30-SECOND REVISION

TEXT

View equivalence preserves:

1. Initial reads

- 2. Read-from relationships
- 3. Final writers

Mnemonic = IRF

View serializable:  
view equivalent to some serial schedule

Blind write:  
write X without reading X first

Every CSR is VSR

But some VSR schedules are not CSR

Classic:

```
w1(X)
w2(X)
w1(X)
```

Graph cycle !' not CSR

No reads + final writer T1

Serial T2!' T1 matches  
!' VSR

Exam method:

Check CSR first.

If acyclic !' automatically VSR.  
If cyclic !' inspect blind writes + IRF.

## CHECK YOUR UNDERSTANDING

### Q1

What are the three conditions required for view equivalence?

### Q2

What is a blind write?

### Q3

True or False:

**Every view-serializable schedule is conflict serializable.**

### Q4

True or False:

**Every conflict-serializable schedule is view serializable.**

## Q5

Schedule:

TEXT

w1(X)  
w2(X)  
w1(X)

Determine:

1. Conflict serializable?
2. View serializable?
3. Equivalent serial order if view serializable.

## Q6

Schedule:

TEXT

r1(X)  
w2(X)  
w1(X)

Can it be view equivalent to:

TEXT

T 1 !' T 2

or:

TEXT

T 2 !' T 1

?

Hence determine whether it is VSR.

## Q7 — GATE shortcut

A precedence graph contains a cycle. What can you conclude immediately?

A. Schedule is neither CSR nor VSR B. Schedule is not CSR, but VSR may still be possible C. Schedule is VSR D. Schedule is serial

Next is another extremely important GATE block:

## TOPIC 18 — RECOVERABILITY + CASCADING ROLLBACK + CASCADELESS + STRICT SCHEDULES

We will learn:

recoverable vs unrecoverable !' commit dependencies !' dirty reads !' cascading rollback !' cascadeless schedules !' strict schedules !' exact hierarchy !' schedule classification !' GATE traps and solved schedules.

Type ``NEXT``.

USER

NEXT

ASSISTANT

## TOPIC 18 — RECOVERABILITY + CASCADELESS + STRICT SCHEDULES

This is another very important GATE topic under **Transactions and Concurrency Control**. It connects directly to dirty reads, commits, aborts, and safe recovery.

The main progression is:

TEXT

Recoverable

!"

Cascadeless

!"

Strict

And the key hierarchy is:

$\boxed{\text{Strict} \rightarrow \text{Cascadeless} \rightarrow \text{Recoverable}}$

But the reverse directions do **not** always hold.

## 1. WHY RECOVERABILITY MATTERS

Suppose:

TEXT

T1 writes X

T2 reads that X

If T2 commits before T1 commits, and then T1 aborts, we have a serious problem.

Why?

Because T2 committed using a value that later became invalid.

So we need rules about:

- who may read whose value
- when commits are allowed
- what happens if a writer aborts

This is what recoverability studies.

## 2. READ-FROM DEPENDENCY

Suppose:



TEXT

w1(X)  
r2(X)

and no other write to X occurs between them.

Then:

**T2 reads X from T1.**

We write conceptually:

TEXT

T2 depends on T1

because T2 used T1's value.

This dependency becomes crucial for commit ordering.

### 3. RECOVERABLE SCHEDULE

A schedule is **recoverable** if:

**Whenever Tj reads a value written by Ti, Tj commits only after Ti commits.**

Formally, if:

$\forall T_j \text{ reads from } T_i$

then:

$\forall C_i < C_j$

That means:

TEXT

writer commits first  
reader commits later

### MEMORY TRICK

**Recoverable = Writer commits before Reader**

Very important.

### 4. SIMPLE RECOVERABLE EXAMPLE

Schedule:

TEXT

w1(X)  
r2(X)  
C1  
C2

T2 reads X from T1.

Commit order:

TEXT

C1 before C2

Correct.

Therefore:

$\boxed{\text{Recoverable}}$

## 5. UNRECOVERABLE SCHEDULE

Consider:

TEXT

w1(X)  
r2(X)  
C2  
A1

T2 reads T1's uncommitted value.

Then:

TEXT

T2 commits

before T1 commits.

Later T1 aborts.

Now T2 has permanently committed based on invalid data.

This schedule is:

$\boxed{\text{Unrecoverable}}$

## MEMORY

### Reader commits before Writer = Danger

If reader commits first, and writer aborts later, we cannot safely undo the reader after commit.

## 6. EXACT RECOVERABILITY TEST

For every read:

TEXT  
rj(X)

find which write it reads from.

If it reads from:

TEXT  
wi(X)

then require:

TEXT  
Ci before Cj

for recoverability.

## 7. IMPORTANT — READING UNCOMMITTED DATA DOES NOT AUTOMATICALLY MEAN UNRECOVERABLE

Consider:

TEXT  
w1(X)  
r2(X)  
C1  
C2

T2 read before T1 committed.

So dirty read occurred.

But T2 waited to commit until after T1 committed.

Therefore schedule is:

TEXT  
R e c o v e r a b l e '

even though it allowed dirty read.

This distinction is very important.

## PART A — CASCADING ROLLBACK

### 8. WHAT IS CASCADING ROLLBACK?

Suppose:

TEXT

T1 writes X  
T2 reads X  
T3 reads a value written by T2

If T1 aborts:

TEXT

T2 must abort

because T2 used invalid data.

Then:

TEXT

T3 may also have to abort

because it depended on T2.

This chain is called:

### Cascading Rollback

## 9. EXAMPLE

Schedule:

TEXT

w1(X)  
r2(X)  
w2(Y)  
r3(Y)  
A1

Dependencies:

TEXT

T2 depends on T1  
T3 depends on T2

If T1 aborts:

TEXT

T2 must rollback

Then because T3 used T2's value:

TEXT

T3 must rollback

So rollback cascades:

TEXT

T 1 !' T 2 !' T 3

## MEMORY TRICK

**Cascade = One abort causes more aborts**

## 10. WHY CASCADING ROLLBACK IS BAD

It causes:

- wasted work
- repeated rollback
- complex recovery
- poor performance

So we prefer schedules that avoid reading uncommitted values.

That leads to:

**Cascadeless schedules**

## PART B — CASCADELESS SCHEDULE

### 11. DEFINITION

A schedule is **cascadeless** if:

**A transaction reads a value written by another transaction only after that writer commits.**

So if:

TEXT

wi(X)  
rj(X)

then we require:

TEXT

Ci before rj(X)

## WRITE THIS IN A BOX

For:

$\text{[ } T_j \text{ reads } X \text{ from } T_i \text{ ]}$

Cascadeless requires:

$\text{[ } \boxed{C_i < r_j(X)} \text{ ]}$

Not just:

TEXT

$C_i < C_j$

The writer must commit **before the reader reads**.

## MEMORY TRICK

**Cascadeless = Commit before Read**

Compare:

TEXT

Recoverable

! ' C o m m i t   b e f o r e   C o m m i t

Cascadeless

! ' C o m m i t   b e f o r e   R e a d

This is a fantastic shortcut.

## 12. CASCADELESS EXAMPLE

TEXT

$w_1(X)$

$C_1$

$r_2(X)$

$C_2$

T2 reads T1's value only after T1 commits.

Therefore:

TEXT

No dirty read

No cascading rollback

So:

$\text{[ } \boxed{\text{Cascadeless}} \text{ ]}$

## 13. RECOVERABLE BUT NOT CASCADELESS

Schedule:

TEXT

w1(X)  
r2(X)  
C1  
C2

T2 reads before T1 commits.

So:

TEXT

Dirty read occurs

Therefore not cascadeless.

But T2 commits after T1.

So recoverable.

Thus:

$\boxed{\text{Recoverable} \text{ but not Cascadeless }}$

This is a classic GATE pattern.

---

## 14. WHY EVERY CASCADELESS SCHEDULE IS RECOVERABLE

---

If writer must commit before reader even reads:

TEXT

$C_i < r_j(X)$

then naturally:

TEXT

$C_i < C_j$

because reader's commit occurs after its read.

So:

$\boxed{\text{Cascadeless} \rightarrow \text{Recoverable}}$

---

## PART C — STRICT SCHEDULE

---

### 15. DEFINITION OF STRICT

---

A schedule is **strict** if:

**Once  $T_i$  writes  $X$ , no other transaction may read or write  $X$  until  $T_i$  commits or aborts.**

Suppose:

TEXT

w1(X)

Then before T1 commits/aborts, another transaction cannot do:

TEXT

r2(X)

and cannot do:

TEXT

w2(X)

## MEMORY

**Strict = After WRITE, nobody touches X**

until writer finishes.

This is stronger than cascadeless.

## 16. STRICT EXAMPLE

TEXT

w1(X)

C1

r2(X)

w2(X)

C2

After T1 writes X:

TEXT

T2 waits

until T1 commits.

Therefore strict.

## 17. NOT STRICT EXAMPLE — DIRTY READ

TEXT

w1(X)

r2(X)

C1

C2

T2 reads X before T1 commits.



Violation.

Therefore not strict.

## 18. NOT STRICT EXAMPLE — DIRTY WRITE

TEXT

w1(X)  
w2(X)  
C1  
C2

T2 writes X before T1 commits.

Even though there is no read:

**Strictness is violated.**

This is a crucial difference between strict and cascadeless.

## PART D — DIRTY WRITE

### 19. WHAT IS DIRTY WRITE?

A dirty write occurs when:

**One transaction overwrites a value written by another transaction that has not committed yet.**

Example:

TEXT

w1(X)  
w2(X)

before T1 commits.

Suppose T1 then aborts.

What value should recovery restore?

This can become difficult because T2 already overwrote T1's value.

Strict schedules prevent this.

## MEMORY

TEXT

Dirty Read  
!' read uncommitted value  
  
Dirty Write

!' o v e r w r i t e   u n c o m m i t t e d   v a l u e

## 20. CASCADELESS vs STRICT

Cascadeless prevents:

TEXT  
reading uncommitted writes

But it does not necessarily prevent:

TEXT  
writing over uncommitted writes

Strict prevents both.

## MASTER TABLE

| Property    | Prevent dirty read? | Prevent dirty write? |
|-------------|---------------------|----------------------|
| Recoverable | No                  | No                   |
| Cascadeless | Yes                 | Not necessarily      |
| Strict      | Yes                 | Yes                  |

## 21. STRICT IMPLIES CASCADELESS

If after:

TEXT  
 $w_i(X)$

no one can read X before  $T_i$  finishes, then naturally no transaction can read  $T_i$ 's uncommitted value.

Therefore:

$\boxed{\text{Strict} \rightarrow \text{Cascadeless}}$

And since:

$\boxed{\text{Cascadeless} \rightarrow \text{Recoverable}}$

we get:

$\boxed{\text{Strict} \rightarrow \text{Cascadeless} \rightarrow \text{Recoverable}}$

## 22. HIERARCHY

Write this permanently in your notes:

```
TEXT
STRICT
  ! "
CASCADELESS
  ! "
RECOVERABLE
```

or:

$\boxed{\text{Strict} \subseteq \text{Cascadeless} \subseteq \text{Recoverable}}$

as sets of schedules.

Meaning strict is the strongest/restrictive class.

## & REVERSE DOES NOT HOLD

```
TEXT
Recoverable !> Cascadeless
```

and:

```
TEXT
Cascadeless !> Strict
```

We will see examples.

## 23. EXAMPLE — RECOVERABLE BUT NOT CASCADELESS

```
TEXT
w1(X)
r2(X)
C1
C2
```

Recoverable:

```
TEXT
C1 before C2 '
```

Cascadeless:

```
TEXT
Did C1 happen before r2(X)?
No.
```

Therefore:

$\boxed{R \not\subseteq CL}$

## 24. EXAMPLE — CASCADELESS BUT NOT STRICT

Schedule:

```
TEXT
w1(X)
w2(X)
C1
C2
```

There are no reads.

Therefore no transaction reads an uncommitted value.

So under the read-based cascadeless condition:

```
TEXT
C a s c a d e l e s s '
```

But T2 writes X before T1 completes.

Strict requires no read **or** write.

Therefore:

```
TEXT
S t r i c t '
```

Thus:

$\boxed{\text{Cascadeless} \setminus \text{but} \setminus \text{not} \setminus \text{Strict}}$

This is an important theoretical example.

## 25. STRICT BUT WHAT ABOUT SERIALIZABILITY?

Important:

**Strictness and serializability are different properties.**

Strict deals with:

```
TEXT
recovery safety
```

Serializability deals with:

```
TEXT
concurrent correctness / serial equivalence
```

A schedule can satisfy one property and not necessarily the other unless a concurrency-control protocol guarantees both.

So never assume:

TEXT

S t r i c t ! ' C o n f l i c t S e r i a l i z a b l e

as a general schedule-property implication.

## 26. RECOVERABILITY vs SERIALIZABILITY

Another critical distinction:

### Serializability

Asks:

Is concurrent behavior equivalent to some serial schedule?

### Recoverability

Asks:

Are commit/read dependencies safe if failures happen?

Different questions.

## MEMORY

TEXT

Serializable

! ' C O R R E C T E X E C U T I O N O R D E R

Recoverable

! ' S A F E C O M M I T O R D E R

## 27. EXAMPLE

Schedule may be conflict serializable:

TEXT

T 1 ! ' T 2

but if T2 reads T1's uncommitted value and commits before T1:

TEXT

w1(X)

r2(X)

C2

C1

then it is:

TEXT

C o n f l i c t   S e r i a l i z a b l e   p o s s i b l y   '  
R e c o v e r a b l e   '

Let's check graph:

TEXT

w1(X) before r2(X)  
!'   T 1   !'   T 2

acyclic.

So CSR.

But:

TEXT

C2 before C1

unrecoverable.

This demonstrates independence.

## 28. IMPORTANT GATE EXAMPLE

Schedule:

TEXT

w1(X)  
r2(X)  
C2  
C1

### Serializability

Edge:

$\backslash [ T1 \rightarrow T2 \backslash ]$

No cycle.

So CSR.

### Recoverability

T2 reads from T1.

Required:

TEXT

C1 before C2

But actual:

TEXT

C2 before C1

So unrecoverable.

Therefore:

$\boxed{\text{CSR} \setminus \text{but} \setminus \text{Unrecoverable}}$

Do not mix the concepts.

## PART E — RIGOROUS SCHEDULE

### 29. RIGOROUS SCHEDULE — USEFUL GATE CONCEPT

A schedule is often called **rigorous** if:

**No other transaction may read or write an item written by  $T_i$  until  $T_i$  commits/aborts, and lock discipline is even stronger in the locking formulation.**

In common transaction-theory hierarchy, rigorous schedules are at least as restrictive as strict schedules under typical definitions.

You may see:

TEXT

Rigorous

! ' S t r i c t

! ' C a s c a d e l e s s

! ' R e c o v e r a b l e

But exact textbook definitions can vary slightly.

For GATE, focus primarily on:

TEXT

Recoverable

Cascadeless

Strict

and use the question's stated definitions if rigorous appears.

## PART F — COMPLETE CLASSIFICATION METHOD

### 30. GATE METHOD

Given a schedule:

#### STEP 1 — Find read-from relations

For every:

TEXT

$rw(X)$

identify which transaction's write produced the value.

---

## STEP 2 — Recoverable?

If  $T_j$  reads from  $T_i$ :

check:

TEXT

$C_i < C_j$  ?

If every dependency satisfies:

TEXT

YES

recoverable.

---

## STEP 3 — Cascadeless?

For each read-from  $T_i \rightarrow T_j$ :

check:

TEXT

$C_i < rw(X)$  ?

If yes for all:

TEXT

cascadeless

---

## STEP 4 — Strict?

For every:

TEXT

$w_i(X)$

look until  $T_i$  commits/aborts.

Does another transaction:

TEXT

read  $X$   
or  
write  $X$

during that interval?

---



If yes:

TEXT  
not strict

If no for all:

TEXT  
strict

---

## 31. MEMORY ALGORITHM

TEXT  
RECOVERABLE:  
Writer COMMIT  
before  
Reader COMMIT

TEXT  
CASCADELESS:  
Writer COMMIT  
before  
Reader READ

TEXT  
STRICT:  
Writer finishes  
before  
any other READ/WRITE of X

This is probably the most important note of this entire topic.

---

## PART G — SOLVED PROBLEM 1

Schedule:

TEXT  
w1(X)  
r2(X)  
C1  
C2

T2 reads from T1.

**Recoverable?**

TEXT  
C1 before C2

Yes.

### Cascadeless?

Need:

TEXT

C1 before r2(X)

No.

### Strict?

After w1(X), T2 reads X before C1.

No.

Result:

TEXT

Recoverable '  
Cascadeless '  
Strict '

## 32. SOLVED PROBLEM 2

TEXT

w1(X)  
C1  
r2(X)  
C2

### Recoverable?

Yes.

### Cascadeless?

Yes, because writer commits before read.

### Strict?

After w1(X), nobody accesses X until C1.

Yes.

Result:

$\boxed{\text{Strict}}$

and therefore also:

TEXT

Cascadeless  
Recoverable

## 33. SOLVED PROBLEM 3

TEXT

w1(X)  
w2(X)  
C1  
C2

No read dependency.

### Recoverable?

No read-from dependency causing commit constraint.

So recoverability is not violated.

### Cascadeless?

No dirty read.

So cascadeless under the standard read-based definition.

### Strict?

T2 writes X before T1 commits.

Violation.

Result:

TEXT

R e c o v e r a b l e '   
C a s c a d e l e s s '   
S t r i c t '

## 34. SOLVED PROBLEM 4

TEXT

w1(X)  
r2(X)  
C2  
A1

T2 reads from T1.

T2 commits before T1 commits.

In fact T1 aborts.

Therefore:

$\boxed{\text{Unrecoverable}}$

Also:

TEXT

Not cascadeless  
Not strict

## 35. SOLVED PROBLEM 5 — THREE TRANSACTIONS

Schedule:

TEXT

w1(X)  
r2(X)  
w2(Y)  
r3(Y)  
C1  
C2  
C3

Dependencies:

TEXT

T2 reads from T1  
T3 reads from T2

Recoverable requirements:

TEXT

C1 before C2  
C2 before C3

Actual:

TEXT

C 1   !'   C 2   !'   C 3

So:

TEXT

R e c o v e r a b l e   '

Cascadeless?

T2 read X before C1.

So:

TEXT

No

Thus cascading rollback could occur before commits.

## 36. CASCADING ROLLBACK IN PREVIOUS EXAMPLE

If T1 aborts before commit:

TEXT

T2 must abort

because T2 used T1's dirty value.

Then:

```
TEXT
T3 must abort
```

because T3 used T2's value.

Hence:

```
TEXT
T1 abort
!"
T2 abort
!"
T3 abort
```

classic cascading rollback.

---

## PART H — COMMIT DEPENDENCY GRAPH INTUITION

---

### 37. READ-FROM CREATES COMMIT ORDER

---

If:

```
TEXT
T2 reads from T1
```

then recoverability imposes:

$\backslash [ T1 \rightarrow T2 ]$

in **commit ordering**.

This is not the same as a precedence graph edge, although they may look similar.

Don't confuse:

```
TEXT
Conflict graph
```

with:

```
TEXT
Read-from/commit dependency
```

---

### 38. PRECEDENCE GRAPH vs RECOVERABILITY

---

Precedence graph considers:

TEXT

RW, WR, WW conflicts

Recoverability specifically considers:

TEXT

who READS whose WRITE

Then examines commit ordering.

So:

TEXT

WW conflict

may matter for serializability but not create a read-from dependency.

---

## PART I — DIRTY READ / CASCADING CONNECTION

---

### 39. DIRTY READ

---

Pattern:

TEXT

wi(X)  
rj(X)

before Ci.

If Ti aborts:

TEXT

Tj has consumed invalid data

and may need rollback.

Thus dirty reads create risk of cascading rollback.

Cascadeless schedules completely prevent dirty reads.

---

### 40. STRICTNESS AND RECOVERY

---

Strict schedules are particularly desirable because they simplify undo/redo recovery.

Why?

If Ti writes X, no other transaction can alter or read X until Ti finishes.

So an abort can safely restore the old value without worrying that another transaction already depended on/overwrote it.

## PART J — FINAL HIERARCHY

## 41. SET RELATION

TEXT

Recoverable

```
% % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
%  
%      C a s c a d e l e s s                               %  
%      % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
%      %                                                                    %    %  
%      S t r i c t                                          %          %  
%      % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
%      % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
%      % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
% % % % % % % % % % % % % % % % % % % % % % % % % % % % %
```

Therefore:

$$\boxed{\text{Strict} \subseteq \text{Cascadeless} \subseteq \text{Recoverable}}$$

## MEMORY LADDER

TEXT

STRICT

Most restrictive

11

## CASCADELESS

!"

RECOVERABLE

Least restrictive of the three

## PART K — GATE TRAPS

## & pTrap 1

Recoverable does not mean no dirty reads.

## & bTrap 2

Cascadeless prevents dirty reads.

### & bTrap 3

Strict prevents dirty reads **and dirty writes.**

## & pTrap 4

Recoverable condition is:

TEXT

Writer commit before Reader commit

not writer commit before reader reads.

## & pTrap 5

Cascadeless condition is:

TEXT

Writer commit before Reader reads

## & pTrap 6

Strict condition applies to:

TEXT

both reads and writes by other transactions

after an uncommitted write.

## & pTrap 7

Serializability and recoverability are different.

A schedule can be CSR and unrecoverable.

## & pTrap 8

Cascadeless does not automatically mean strict.

Dirty writes can distinguish them.

## & pTrap 9

Every strict schedule is recoverable.

## & pTrap 10

If no transaction reads from another, recoverability constraints may be vacuously satisfied, but strictness can still fail due to dirty writes.



## 42. GATE QUICK CLASSIFICATION TABLE

| Schedule pattern  | Recoverable | Cascadeless | Strict |
|-------------------|-------------|-------------|--------|
| w1(X) C1 r2(X) C2 | '           | '           | '      |
| w1(X) r2(X) C1 C2 | '           | '           | '      |
| w1(X) r2(X) C2 C1 | '           | '           | '      |
| w1(X) w2(X) C1 C2 | '           | '           | '      |

Memorize the logic, not only the table.

## NOTEBOOK NOTES

### RECOVERABLE

If Tj reads from Ti:

$\forall \boxed{C_i < C_j}$

Memory:

TEXT

Writer COMMIT  
before  
Reader COMMIT

### UNRECOVERABLE

Reader commits before writer:

TEXT

wi(X)  
rj(X)  
Cj  
...  
Ci/Ai

Danger.

### CASCADING ROLLBACK

One transaction aborts:

TEXT

dependent transactions  
also must abort

## CASCADELESS

If Tj reads from Ti:

$$\forall \boxed{C_i < r_j(X)}$$

Memory:

TEXT

Writer COMMIT  
before  
Reader READ

No dirty read.

## STRICT

After:

$$\forall w_i(X)$$

no other transaction can:

TEXT

read X  
or  
write X

until Ti commits/aborts.

Memory:

After write, nobody touches X.

## HIERARCHY

$$\forall \boxed{\text{Strict} \rightarrow \text{Cascadeless} \rightarrow \text{Recoverable}}$$

Reverse does not necessarily hold.

## MEMORY PAGE

TEXT

RECOVERABLE  
C before C

CASCADELESS  
C before R

STRICT  
Finish before R/W

More explicitly:

#### TEXT

Recoverable:  
Writer COMMIT  
before Reader COMMIT

Cascadeless:  
Writer COMMIT  
before Reader READ

Strict:  
Writer COMMIT/ABORT  
before another transaction READS or WRITES X

## 30-SECOND REVISION

#### TEXT

Recoverable:  
if T2 reads from T1,  
T1 commits before T2 commits

Cascadeless:  
T1 commits before T2 reads T1's value

Strict:  
after T1 writes X,  
nobody else reads/writes X  
until T1 commits/aborts

Dirty read:  
read uncommitted data

Dirty write:  
overwrite uncommitted data

Cascading rollback:  
one abort causes dependent aborts

Hierarchy:  
Strict  
!' Cascadeless  
!' Recoverable

Serializable and recoverable  
are different properties.

## CHECK YOUR UNDERSTANDING

### Q1

Schedule:

#### TEXT

w1(X)  
r2(X)  
C1  
C2

Classify it as:

1. Recoverable?
  2. Cascadeless?
  3. Strict?
- 

## Q2

Schedule:

TEXT

w1(X)  
C1  
r2(X)  
C2

Classify it.

---

## Q3

Schedule:

TEXT

w1(X)  
r2(X)  
C2  
C1

Is it recoverable?

---

## Q4

What is cascading rollback?

---

## Q5

Which of the following prevents **dirty reads but may still allow dirty writes**?

A. Recoverable B. Cascadeless C. Strict D. Unrecoverable

---

## Q6

Complete the hierarchy:

[ \\_ \\_ \\_ \\_ \\_ \rightarrow \\_ \\_ \\_ \\_ \\_ \rightarrow Recoverable ]

---

## Q7 — Important GATE trap

Schedule:

TEXT

w1(X)  
w2(X)  
C1

C2

Can it be cascadeless but not strict? Explain why.

Q8

True or False:

**Every conflict-serializable schedule is automatically recoverable.**

Next is the major practical concurrency-control topic:

## TOPIC 19 — LOCKS + SHARED/EXCLUSIVE LOCKS + TWO-PHASE LOCKING (2PL)

We will cover:

**lock concept !' S-lock !' X-lock !' compatibility matrix !' lock/unlock operations !' lock conversion !' basic 2PL !' growing/shrinking phase !' lock point !' conflict serializability guarantee !' strict 2PL !' rigorous 2PL !' deadlock possibility !' GATE schedule problems.**

Type `NEXT`.

USER

NEXT

ASSISTANT

## TOPIC 19 — LOCKS + SHARED/EXCLUSIVE LOCKS + TWO-PHASE LOCKING (2PL)

Now we move into the main practical concurrency-control mechanism used to enforce safe concurrent execution.

The flow is:

TEXT

Locks

!"

Shared / Exclusive Locks

!"

Compatibility

!"

Lock / Unlock Rules

!"

2PL

!"

Strict 2PL

!"

Rigorous 2PL

!"

Deadlocks

This topic is very important for GATE because questions often mix:

- lock compatibility
- legal/illegal lock requests
- growing/shrinking phases
- whether a schedule follows 2PL
- what 2PL guarantees
- whether 2PL can deadlock

---

## 1. WHY DO WE NEED LOCKS?

Suppose T1 is updating X.

At the same time T2 also accesses X.

Without control, we can get:

TEXT

lost update  
dirty read  
dirty write

So DBMS uses a locking mechanism.

Before accessing an item, a transaction may need to acquire a lock.

---

## 2. WHAT IS A LOCK?

A **lock** is a permission that controls access to a data item.

For example:

TEXT

T1 requests lock on X

If allowed:

TEXT

T1 can access X

If not:

TEXT

T1 must wait

Locks help coordinate concurrent transactions.

---

## 3. TWO MAIN LOCK TYPES

The most important are:

TEXT

Shared Lock  
Exclusive Lock

Notation:

TEXT

S(X)  
X(X)

or sometimes:

TEXT

lock-S(X)  
lock-X(X)

Do not confuse data item X with lock type X.

## 4. SHARED LOCK — S LOCK

A shared lock is used mainly for:

### Reading

If transaction wants to read X:

TEXT

S-lock(X)  
read(X)

Multiple transactions may simultaneously hold shared locks on the same item.

Example:

TEXT

T1: S(X)  
T2: S(X)

Both can read X.

## MEMORY TRICK

**Shared = Share for reading**

Many readers can coexist.

## 5. EXCLUSIVE LOCK — X LOCK

An exclusive lock is needed for:

## Writing

If T1 has X-lock on X:

TEXT

T1 can read/write X

but no other transaction may simultaneously hold:

TEXT

S-lock(X)

or

X-lock(X)

on that same item.

## MEMORY TRICK

Exclusive = Only one transaction gets it

## 6. LOCK COMPATIBILITY MATRIX

This is extremely important.

| Existing lock | Requested S | Requested X |
|---------------|-------------|-------------|
| S             | Yes         | No          |
| X             | No          | No          |

So:

TEXT

```
S + S !' c o m p a t i b l e
S + X !' i n c o m p a t i b l e
X + S !' i n c o m p a t i b l e
X + X !' i n c o m p a t i b l e
```

## MEMORY TRICK

Only S-S can coexist.

Everything involving X with another transaction is incompatible.

## 7. EXAMPLE



Suppose:

TEXT

T1 holds S(X)

Then T2 requests:

TEXT

S(X)

Allowed.

But if T2 requests:

TEXT

X(X)

not allowed until T1 releases S(X).

---

## 8. ANOTHER EXAMPLE

Suppose:

TEXT

T1 holds X(X)

Then:

TEXT

T2 requests S(X)

No.

And:

TEXT

T2 requests X(X)

also no.

So X-lock blocks everyone else.

---

## 9. WHY WRITE REQUIRES EXCLUSIVE LOCK

Imagine:

TEXT

T1 writing X

T2 reading X

T2 might see an intermediate/uncommitted value.

---

Or:

TEXT

T1 writing X  
T2 writing X

one write may overwrite the other.

Exclusive locking prevents such simultaneous conflicting accesses.

## 10. LOCK / UNLOCK NOTATION

A transaction might look like:

TEXT

T1:

S-lock(A)  
read(A)  
unlock(A)

X-lock(B)  
write(B)  
unlock(B)

However, whether locking/unlocking at those points follows 2PL depends on phase rules.

## PART A — LOCK CONVERSION

### 11. LOCK UPGRADE

Suppose T1 already has:

TEXT

S(X)

and later needs to write X.

It can request conversion:

TEXT

S ( X ) !' X ( X )

This is called:

**Lock Upgrade**

### 12. LOCK DOWNGRADE

Suppose T1 has:

TEXT

X(X)

and later only needs read access.

It may convert:

TEXT

X ( X ) ! ' S ( X )

This is a:

**Lock Downgrade**

The timing of upgrade/downgrade matters under 2PL.

## MEMORY

TEXT

S ! ' X = U p g r a d e

X ! ' S = D o w n g r a d e

## PART B — TWO-PHASE LOCKING

### 13. WHAT IS 2PL?

**Two-Phase Locking**, or 2PL, divides a transaction's lock behavior into two phases:

1. Growing phase
2. Shrinking phase

### 14. GROWING PHASE

During growing phase:

**Transaction may acquire locks.**

But:

**It may not release locks.**

So:

TEXT

Lock

Lock

Lock  
Lock

allowed.

No unlock yet.

## 15. SHRINKING PHASE

Once a transaction releases its first lock:

**Shrinking phase begins.**

During shrinking phase:

TEXT  
UnLock  
UnLock  
UnLock

allowed.

But:

**No new lock may be acquired.**

## WRITE THIS IN A BOX

TEXT  
GROWING:  
Acquire locks  
No unlock  
  
SHRINKING:  
Release locks  
No new locks

## MEMORY TRICK

**Grow first, shrink later.**

Just like:

TEXT  
take locks  
then  
give locks back

Never return and take new locks again.

## 16. TWO PHASES VISUALLY

TEXT

LOCK POINT

!"

Growing Phase

Shrinking Phase

lock(A)

lock(B)

lock(C)

unlock(A)

unlock(B)

unlock(C)

Once first unlock happens, no further lock acquisition.

## 17. LOCK POINT

The **lock point** of a transaction is the instant when it obtains its final lock.

After that, it may eventually start releasing locks.

The ordering of lock points can help determine serialization order under 2PL.

## 18. VALID 2PL EXAMPLE

TEXT

T1:

S(A)

X(B)

read(A)

write(B)

unlock(A)

unlock(B)

Lock acquisition sequence:

TEXT

S(A)

X(B)

Then unlocks:

TEXT

unlock(A)

unlock(B)

No new lock after first unlock.

So:

$\boxed{\text{Follows 2PL}}$

## 19. INVALID 2PL EXAMPLE

TEXT

T1:

S(A)  
read(A)  
unlock(A)

X(B)  
write(B)  
unlock(B)

After:

TEXT

unlock(A)

shrinking phase begins.

But then transaction acquires:

TEXT

X(B)

This violates 2PL.

Therefore:

$\boxed{\text{Not 2PL}}$

## MEMORY

**Unlock once !' never lock again**

Excellent quick test.

## 20. LOCK UPGRADE UNDER 2PL

Upgrade:

TEXT

S ( X ) !' X ( X )

requires acquiring additional lock privilege.

So upgrade must occur in:

## Growing phase

## 21. LOCK DOWNGRADE UNDER 2PL

Downgrade:

TEXT

```
X ( X ) !' S ( X )
```

releases some privilege.

So it belongs to:

## Shrinking phase

After downgrade, no new lock can be acquired in basic 2PL.

## MEMORY

TEXT

```
U p g r a d e !' G r o w i n g  
D o w n g r a d e !' S h r i n k i n g
```

## PART C — WHAT DOES 2PL GUARANTEE?

## 22. BIGGEST GATE FACT

Basic Two-Phase Locking guarantees:

$\boxed{\text{Conflict Serializability}}$

Any schedule generated by transactions obeying 2PL is conflict serializable.

This is one of the most important concurrency-control facts.

## 23. WHY?

Very roughly:

The growing/shrinking discipline ensures conflicting lock acquisition orders create a consistent ordering among transactions.

That corresponds to an acyclic precedence relation.

So:

TEXT

2PL

! ' C o n f l i c t   S e r i a l i z a b l e

## 24. DOES EVERY CSR SCHEDULE FOLLOW 2PL?

No.

This is an important one-way implication.

$\boxed{2PL \rightarrow CSR}$

But:

$\boxed{CSR \nrightarrow 2PL}$

There are conflict-serializable schedules that cannot be produced under a particular 2PL locking sequence.

## MEMORY

**2PL guarantees CSR, but CSR doesn't guarantee 2PL.**

## PART D — DOES 2PL GUARANTEE RECOVERABILITY?

### 25. BASIC 2PL DOES NOT NECESSARILY GUARANTEE STRICTNESS

Basic 2PL allows a transaction to release an exclusive lock before commit, provided it has entered shrinking phase.

Then another transaction might read/write that uncommitted value.

Example conceptually:

TEXT

T1 gets X-lock(X)

T1 writes X

T1 unlocks X

T2 gets S-lock(X)

T2 reads X

T1 later aborts

This can create dirty reads/cascading rollback.

Therefore:

**Basic 2PL guarantees conflict serializability, but not necessarily strictness/cascadeless recovery.**

This is why strict 2PL is important.



## PART E — STRICT TWO-PHASE LOCKING

### 26. STRICT 2PL

In Strict 2PL:

**All exclusive/write locks are held until the transaction commits or aborts.**

Shared locks may be released earlier according to the specific implementation, provided the 2PL rules are obeyed.

Core rule:

TEXT

X-locks held until COMMIT/ABORT

### MEMORY

**Strict 2PL = Keep WRITE locks till end**

### 27. WHY STRICT 2PL HELPS

If T1 writes X and holds its X-lock until commit:

TEXT

T2 cannot read X  
T2 cannot write X

until T1 finishes.

Therefore it prevents:

TEXT

dirty read  
dirty write

on T1's written data.

This produces strict schedules.

### 28. STRICT 2PL GUARANTEES

Strict 2PL provides:

TEXT

Conflict serializability  
+  
Strict schedules

Therefore also:

TEXT

Cascadeless  
Recoverable

because:

[ Strict \rightarrow Cascadeless \rightarrow Recoverable ]

## 29. BASIC 2PL vs STRICT 2PL

| Basic 2PL                               | Strict 2PL                              |
|-----------------------------------------|-----------------------------------------|
| Growing then shrinking                  | Same 2PL discipline                     |
| Locks can begin releasing before commit | X-locks retained until commit/abort     |
| Guarantees CSR                          | Guarantees CSR                          |
| May allow cascading rollback            | Prevents dirty reads from written items |
| Not necessarily strict                  | Strict                                  |

## PART F — RIGOROUS 2PL

### 30. RIGOROUS 2PL

In **Rigorous 2PL**:

Both shared locks and exclusive locks are held until commit/abort.

So:

TEXT

S-locks held till end  
X-locks held till end

## MEMORY

TEXT

Strict 2PL  
!' X l o c k s t i l l e n d

Rigorous 2PL  
!' A L L l o c k s t i l l e n d

## 31. STRICT vs RIGOROUS

| Strict 2PL                       | Rigorous 2PL                    |
|----------------------------------|---------------------------------|
| X-locks till commit/abort        | S and X locks till commit/abort |
| Shared locks may release earlier | Nothing released before end     |
| Strong                           | Stronger                        |

## 32. RIGOROUS 2PL AND SERIAL ORDER

Because locks are held until commit, serialization order tends to correspond closely to commit order under rigorous locking.

This simplifies reasoning and recovery.

For GATE, remember the main distinction:

TEXT

S t r i c t ! ' w r i t e l o c k s r e t a i n e d

R i g o r o u s ! ' a l l l o c k s r e t a i n e d

## PART G — CONSERVATIVE/ STATIC2PL

### 33. CONSERVATIVE2PL

Another variant is:

**Transaction acquires all required locks before it starts execution.**

If it cannot acquire all of them:

TEXT

i t w a i t s

before doing any work.

Also called:

TEXT

S t a t i c 2 P L

### 34. WHY CONSERVATIVE2PL IS SPECIAL

Because a transaction never holds some locks while waiting to acquire other required locks after starting.

This helps prevent:

## Deadlocks

So:

$\boxed{\text{Conservative} \rightarrow \text{2PL} \rightarrow \text{Deadlock} \rightarrow \text{Free}}$

assuming all required locks are known in advance.

## MEMORY

**Conservative = Collect everything first**

## 35. DRAWBACK

A transaction may not know all required locks beforehand.

It can also reduce concurrency because it reserves many resources early.

So it is not always practical.

## PART H — 2PL VARIANTS MASTER TABLE

| Protocol         | Main rule                          | CSR? | Deadlock possible? | Strict recovery?       |
|------------------|------------------------------------|------|--------------------|------------------------|
| Basic 2PL        | Growing then shrinking             | Yes  | Yes                | Not necessarily        |
| Conservative 2PL | Acquire all locks before start     | Yes  | No                 | Not necessarily strict |
| Strict 2PL       | Hold X-locks till commit/abort     | Yes  | Yes                | Yes                    |
| Rigorous 2PL     | Hold S & X locks till commit/abort | Yes  | Yes                | Yes                    |

This table is highly GATE-relevant.

## PART I — DEADLOCK POSSIBILITY

## 36. DOES 2PL PREVENT DEADLOCK?

No.

This is a major GATE trap.

Basic/strict 2PL can create deadlocks.

## 37. DEADLOCK EXAMPLE

T1:

TEXT

X-lock(A)

T2:

TEXT

X-lock(B)

Then:

T1 requests:

TEXT

X-lock(B)

but B held by T2.

So T1 waits.

T2 requests:

TEXT

X-lock(A)

but A held by T1.

So T2 waits.

Now:

TEXT

T1 waits for T2  
T2 waits for T1

Deadlock.

---

## VISUAL

TEXT

T1 holds A  
T1 wants B

!"

T2

T2 holds B  
T2 wants A

!"

T1

Cycle in waiting.

---

## MEMORY

2PL gives serializability, not deadlock freedom.

We study deadlocks fully next.

## PART J — LOCK REQUEST PROCESS

### 38. WHEN LOCK REQUEST IS COMPATIBLE

Suppose T1 has:

TEXT  
S(X)

T2 asks:

TEXT  
S(X)

Compatible.

Grant immediately.

### 39. WHEN INCOMPATIBLE

T1 has:

TEXT  
S(X)

T2 requests:

TEXT  
X(X)

Not compatible.

T2 is:

TEXT  
blocked / waiting

until conflicting lock is released.

### 40. MULTIPLE SHARED LOCK HOLDERS

Suppose:

TEXT

T1: S(X)  
T2: S(X)  
T3: S(X)

all can hold simultaneously.

Now T4 requests:

TEXT

X(X)

T4 must wait until:

TEXT

T1, T2, T3

all release their shared locks.

---

## 41. UPGRADE PROBLEM

Suppose:

TEXT

T1 holds S(X)  
T2 holds S(X)

Now T1 requests upgrade:

TEXT

S ( X ) ! ' X ( X )

It cannot get X-lock while T2 still holds S(X).

So T1 must wait for T2.

This can contribute to deadlocks in some locking patterns.

---

## PART K — GATE-STYLE PROBLEM 1

Transaction:

TEXT

T1:  
S(A)  
S(B)  
unlock(A)  
X(C)

Does T1 follow 2PL?

No.

Why?

After:

TEXT

unlock(A)

shrinking phase begins.

Then T1 acquires:

TEXT

X(C)

New lock in shrinking phase.

Violation.

$\boxed{\text{Not 2PL}}$

---

## PART L — GATE-STYLE PROBLEM 2

TEXT

T1:  
S(A)  
X(B)  
X(C)  
unlock(A)  
unlock(B)  
unlock(C)

All locks acquired before first unlock.

Therefore:

$\boxed{\text{2PL}}$

---

## PART M — GATE-STYLE PROBLEM 3

TEXT

T1:  
S(A)  
upgrade A to X(A)  
unlock(A)

Upgrade occurs before any unlock.

So upgrade is in growing phase.

Valid under 2PL.

---

## PART N — GATE-STYLE PROBLEM 4



TEXT

T1:  
X(A)  
downgrade A to S(A)  
X(B)

Downgrade effectively starts shrinking phase.

After that T1 attempts:

TEXT

X(B)

new lock.

Violation.

Therefore:

$\boxed{\text{Not 2PL}}$

---

## PART O — LOCK POINT AND SERIAL ORDER

---

### 42. LOCK POINT CONCEPT

---

Suppose:

TEXT

T1 final lock acquired at time 5  
T2 final lock acquired at time 9

Under 2PL, lock-point ordering can be used to derive a serialization order:

TEXT

T1 before T2

in the standard theorem.

This is useful theoretically.

For most GATE schedule questions, however, precedence graphs remain the easiest way to verify CSR.

---

## PART P — SERIALIZABILITY RELATION

---

### 43. IMPORTANT SET RELATION

---

Think:

TEXT

Schedules generated by 2PL  
!"  
Conflict Serializable schedules

! "  
View Serializable schedules

So:

$\lceil 2PL \rceil \rightarrow \text{CSR} \rightarrow \text{VSR} \rceil$

But reverse implications do not generally hold.

## PART Q — STRICT 2PL vs STRICT SCHEDULE

### 44. DON'T CONFUSE THEM

#### Strict schedule

A property of a schedule:

no transaction reads/writes an item written by an uncommitted transaction.

#### Strict 2PL

A protocol that generates strict schedules by holding X-locks until commit/abort.

So:

TEXT  
Strict 2PL  
! ' p r o d u c e s   s t r i c t   s c h e d u l e

They are related but not identical terms.

## PART R — GATE TRAPS

### & pTrap 1

Only S-S is compatible.

### & pTrap 2

X-lock is needed for writing.

### & pTrap 3

After first unlock in basic 2PL:

TEXT  
no new locks

## & pTrap 4

Lock upgrade belongs in growing phase.

## & pTrap 5

Lock downgrade begins/is part of shrinking phase.

## & pTrap 6

2PL guarantees:

TEXT

Conflict Serializability

not view serializability only.

Since CSR implies VSR, of course it is VSR too.

## & pTrap 7

Basic 2PL does not automatically give strict schedules.

## & pTrap 8

Strict 2PL holds X-locks until commit/abort.

## & pTrap 9

Rigorous 2PL holds **all locks** until commit/abort.

## & pTrap 10

2PL does not prevent deadlock.

## & pTrap 11

Conservative 2PL can prevent deadlock because all locks are acquired before execution.

## & pTrap 12

CSR does not imply a schedule follows 2PL.

## PART 5 — PYQ RECOGNITION

If question says:

**“Transaction cannot acquire any new lock after it has released one lock”**

Think:

TEXT

2PL

If:

**“All write locks retained until commit”**

Think:

TEXT

Strict 2PL

If:

**“All shared and exclusive locks retained until commit”**

Think:

TEXT

Rigorous 2PL

If:

**“All locks acquired before transaction begins”**

Think:

TEXT

Conservative / Static 2PL

If:

**“Can 2PL deadlock?”**

Answer:

TEXT

Yes

except conservative 2PL avoids it.

---

## NOTEBOOK NOTES

---

# LOCK TYPES

## Shared Lock — S

TEXT

Read access  
Multiple S-locks can coexist

## Exclusive Lock — X

TEXT

Write access  
No other S/X lock allowed

# COMPATIBILITY

TEXT

|   |   |   |  |   |
|---|---|---|--|---|
|   | S | X |  |   |
| S |   | ' |  | ' |
| X |   | ' |  | ' |

Memory:

Only S–S compatible.

# 2PL

## Growing phase

TEXT

Acquire locks  
No unlock

## Shrinking phase

TEXT

Release locks  
No new lock

Memory:

Unlock once !' lock never again.

# LOCK CONVERSION

TEXT

S! ' X  
Upgrade  
Growing phase

TEXT

X! ' S  
Downgrade  
Shrinking phase

---

## BASIC 2PL

$\boxed{2PL \rightarrow \text{Conflict} \rightarrow \text{Serializable}}$

But:

TEXT

Deadlock possible  
Cascading rollback possible

---

## STRICT 2PL

TEXT

Hold all X-locks  
until commit/abort

Guarantees:

TEXT

CSR  
+  
Strict  
+  
Cascadeless  
+  
Recoverable

---

## RIGOROUS 2PL

TEXT

Hold all S and X locks  
until commit/abort

---

## CONSERVATIVE 2PL

---

TEXT

Acquire all needed locks  
before execution begins

Advantage:

TEXT

Deadlock-free

---

## MEMORY PAGE

TEXT

LOCKS:

S = Shared = Read  
X = Exclusive = Write

Only S-S compatible

TEXT

2PL:

GROW  
Lock Lock Lock

SHRINK  
Unlock Unlock Unlock

TEXT

Strict 2PL:  
X locks till END

Rigorous:  
ALL locks till END

Conservative:  
ALL locks BEFORE START

And the most important fact:

TEXT

2PL  
!' C S R  
but  
!' D e a d l o c k p o s s i b l e

---

## 30-SECOND REVISION

TEXT

S-lock:  
read

can share with S

X-lock:

write

cannot share

Compatibility:

SS yes

SX no

XS no

XX no

2PL:

Growing = acquire locks

Shrinking = release locks

After first unlock:

no new lock

Upgrade S! X :

growing

Downgrade X! S :

shrinking

Basic 2PL:

guarantees conflict serializability

but deadlock possible

Strict 2PL:

X-locks held till commit/abort

Rigorous 2PL:

all locks held till commit/abort

Conservative 2PL:

all locks acquired before start

! deadlock - free

## CHECK YOUR UNDERSTANDING

### Q1

Complete the lock compatibility matrix:

| Held | Request S | Request X |
|------|-----------|-----------|
| S    | ?         | ?         |
| X    | ?         | ?         |

### Q2

A transaction performs:

TEXT

S(A)

X(B)



```
unlock(A)
X(C)
```

Does it obey 2PL?

### Q3

Which phase allows new locks?

A. Growing B. Shrinking C. Both D. Neither

### Q4

True or False:

**Every schedule generated using 2PL is conflict serializable.**

### Q5

True or False:

**2PL guarantees freedom from deadlock.**

### Q6

What is the difference between:

TEXT

Strict 2PL

and:

TEXT

Rigorous 2PL

?

### Q7

Which 2PL variant acquires all required locks before execution begins?

### Q8 — Important GATE trap

Is every conflict-serializable schedule necessarily generated by 2PL?

Next:

## TOPIC 20 — DEADLOCKS + WAIT-FOR GRAPH

We will cover:

**deadlock definition !' necessary conditions !' lock-based deadlock !' wait-for graph !' cycle detection !' deadlock prevention !' wait-die !' wound-wait !' timeout !' detection/recovery !' victim selection !' starvation !' GATE problems.**

Type `NEXT`.

USER

NEXT

ASSISTANT

## TOPIC 20 — DEADLOCKS + WAIT-FOR GRAPH

Now we continue with the deadlock part of concurrency control.

The main flow is:

TEXT

Locks

!"

Waiting

!"

Circular Waiting

!"

Deadlock

!"

Wait-For Graph

!"

Detection / Prevention / Recovery

For GATE, the highest-value ideas are:

- definition of deadlock
- four necessary conditions
- wait-for graph
- cycle detection
- wait-die
- wound-wait
- timeout
- victim selection
- starvation

## 1. WHAT IS A DEADLOCK?

A **deadlock** occurs when two or more transactions wait indefinitely for one another to release resources.

Simple example:

TEXT

T1 holds A  
T2 holds B

T1 wants B  
T2 wants A

So:

TEXT

T1 waits for T2  
T2 waits for T1

Neither can proceed.

That is a deadlock.

## 2. SIMPLE DEADLOCK EXAMPLE

Suppose:

**T1**

TEXT

X-lock(A)  
...  
requests X-lock(B)

**T2**

TEXT

X-lock(B)  
...  
requests X-lock(A)

Now:

TEXT

A held by T1  
B held by T2

T1 waits for B.

T2 waits for A.

Diagram:

TEXT

```
T1 ----waits for----> T2
^                      |
|                      |
% - - - - - w a i t s   f o r - - - - - %
```

Deadlock.

---

## MEMORY TRICK

**Deadlock = Everyone waits, nobody moves.**

---

## 3. WHY 2PL CAN CAUSE DEADLOCK

Recall that 2PL allows transactions to:

TEXT

```
hold old locks
+
request new locks
```

during the growing phase.

Therefore:

TEXT

```
T1 holds A, waits for B
T2 holds B, waits for A
```

is possible.

Hence:

$\lceil \boxed{2PL \text{ does not guarantee deadlock freedom}} \rceil$

---

## PART A — FOUR NECESSARY CONDITIONS

A deadlock requires four classical conditions.

These are often called the **Coffman conditions**:

1. Mutual Exclusion
2. Hold and Wait
3. No Preemption
4. Circular Wait

All four must hold for a deadlock to be possible.

---

## 4. MUTUAL EXCLUSION

At least one resource must be non-shareable.

Example:

TEXT

```
X-lock(A)
```

Only one transaction can hold an exclusive lock on A.

If all resources were freely shareable, transactions wouldn't block one another.

## MEMORY

**Mutual exclusion = one resource, one owner**

## 5. HOLD AND WAIT

A transaction:

**Holds one resource while waiting for another.**

Example:

TEXT

T1 holds A  
T1 waits for B

## MEMORY

**Hold one, wait for another.**

## 6. NO PREEMPTION

A resource cannot simply be forcibly taken away under the ordinary locking model.

If T1 holds A:

TEXT

T2 cannot just steal A

T1 must release it normally, or the DBMS must abort T1 through a recovery/prevention policy.

## 7. CIRCULAR WAIT

A circular chain exists.

Example:

TEXT

T1 waits for T2  
T2 waits for T3  
T3 waits for T1

Diagram:

TEXT

```
T 1  !'   T 2  !'   T 3
!'      |
% % % % % % % % % %
```

This is circular wait.

## MEMORY TRICK — M H N C

Remember:

TEXT

```
M !'   M u t u a l   e x c l u s i o n
H !'   H o l d   a n d   w a i t
N !'   N o   p r e e m p t i o n
C !'   C i r c u l a r   w a i t
```

You can remember:

**MHNC**

Or simply memorize the four in order.

## 8. IMPORTANT FACT

Deadlock prevention works by ensuring that **at least one** of these four conditions can never hold.

You don't need to eliminate all four.

Breaking even one is sufficient.

## PART B — WAIT-FOR GRAPH

### 9. WHAT IS A WAIT-FOR GRAPH?

A **Wait-For Graph (WFG)** represents which transaction is waiting for which other transaction.

Nodes:

TEXT

Transactions

Edge:

$T_i \rightarrow T_j$

means:

**$T_i$  is waiting for a resource currently held by  $T_j$ .**

## 10. EDGE DIRECTION

Very important:

TEXT

WAITING transaction  
!"  
HOLDING transaction

So:

$\boxed{\text{Waiter} \rightarrow \text{Holder}}$

## MEMORY TRICK

I wait for YOU !' I point to YOU

## 11. EXAMPLE

T1 holds A.

T2 requests A.

Then:

TEXT

T2 waits for T1

Graph:

$T2 \rightarrow T1$

Not:

$T1 \rightarrow T2$

## & pGATE TRAP

Wait-for graph direction is not:

TEXT

holder !' waiter

It is:

TEXT

waiter !' holder

## 12. DEADLOCK TEST USING WAIT-FOR GRAPH

For systems where each wait relationship is represented directly among transactions:

$\boxed{\text{Cycle} \Rightarrow \text{WFG} \Rightarrow \text{Deadlock}}$

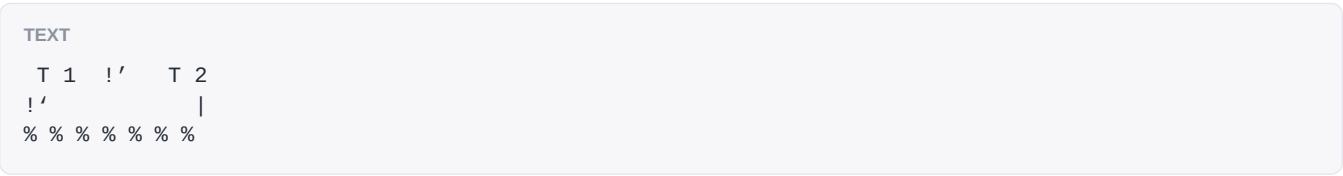
This is the key GATE test for standard database lock deadlocks.

## 13. SIMPLE EXAMPLE

Suppose:

```
TEXT
T1 waits for T2
T2 waits for T1
```

WFG:



Cycle.

Therefore:

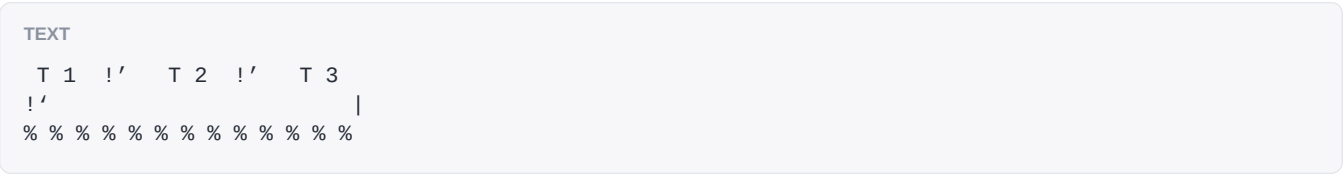
$\boxed{\text{Deadlock}}$

## 14. THREE-TRANSACTION EXAMPLE

Suppose:

```
TEXT
T1 waits for T2
T2 waits for T3
T3 waits for T1
```

Graph:



Cycle.

Deadlock.

## 15. ACYCLIC WAIT-FOR GRAPH

Suppose:



TEXT

T 1 !' T 2 !' T 3

but T3 is not waiting.

No cycle.

Therefore:

TEXT

No deadlock currently

T3 can finish, release locks, T2 proceeds, then T1 proceeds.

## PART C — PRECEDENCE GRAPH vs WAIT-FOR GRAPH

This is a massive GATE trap.

### 16. PRECEDENCE GRAPH

Used for:

**Conflict serializability.**

Edge:

TEXT

Earlier conflicting transaction  
!' later conflicting transaction

Cycle means:

TEXT

Not conflict serializable

### 17. WAIT-FOR GRAPH

Used for:

**Deadlock detection.**

Edge:

TEXT

Waiting transaction  
!' transaction holding needed resource

Cycle means:

TEXT

Deadlock

## MASTER COMPARISON

| Precedence Graph                 | Wait-For Graph          |
|----------------------------------|-------------------------|
| Serializability                  | Deadlock                |
| Based on conflicting operations  | Based on lock waiting   |
| Edge = earlier !' later conflict | Edge = waiter !' holder |
| Cycle = not CSR                  | Cycle = deadlock        |

## MEMORY

TEXT

PRECEDENCE GRAPH

!' SERIALIZABILITY

WAIT-FOR GRAPH

!' DEADLOCK

Never mix them.

## PART D — DEADLOCK DETECTION

### 18. DETECTION METHOD

A DBMS may allow deadlocks to occur and periodically check for them.

Process:

TEXT

1. Build wait-for graph
2. Check for cycle
3. If cycle exists !' deadlock
4. Select victim
5. Abort/rollback victim
6. Release its locks

### 19. EXAMPLE

Graph:

TEXT

|     |    |     |
|-----|----|-----|
| T 1 | !' | T 2 |
| T 2 | !' | T 3 |
| T 3 | !' | T 1 |

Deadlock.

Suppose DBMS chooses:

TEXT

T2

as victim.

Then:

TEXT

Abort T2  
Release T2's locks

This breaks the cycle.

Other transactions can continue.

## PART E — DEADLOCK RECOVERY

### 20. VICTIM SELECTION

If a deadlock is detected, DBMS may abort one or more transactions.

The selected transaction is called:

**Victim**

Possible criteria:

- amount of work already done
- number of locks held
- number of resources still needed
- transaction age
- rollback cost
- priority
- number of previous rollbacks

Goal:

**Minimize total recovery cost.**

### 21. ROLLBACK

Victim transaction may be:

TEXT

fully rolled back

or in some systems:

TEXT

rolled back to a safe point

Then its locks are released.

This allows other transactions to proceed.

## 22. STARVATION

Suppose the same transaction repeatedly becomes the victim.

Example:

TEXT

```
T3 deadlocks
!' abort T3
!' restart T3
!' deadlocks again
!' abort T3 again
```

T3 may never finish.

This is:

**Starvation**

## MEMORY

**Deadlock = everyone stuck together**

**Starvation = one transaction keeps losing**

Important distinction.

## 23. AVOIDING STARVATION

Possible approaches:

- increase priority after each rollback
- use timestamps/age
- avoid selecting same victim repeatedly

---

## PART F — DEADLOCK PREVENTION

---

Instead of detecting deadlock after it occurs, DBMS can prevent it.

Methods include:

- conservative 2PL
  - resource ordering
  - timestamp-based schemes such as wait-die and wound-wait
- 

### 24. CONSERVATIVE 2PL

---

Recall:

**Transaction acquires all required locks before execution starts.**

If any lock unavailable:

TEXT

transaction waits without holding partial set of locks

This breaks:

TEXT

Hold and Wait

Therefore deadlock is prevented.

---

### 25. RESOURCE ORDERING

---

Suppose all data items have a fixed order:

TEXT

$A < B < C < D$

Transactions must request locks only in increasing order.

Example:

Allowed:

TEXT

A !' C !' D

Not allowed:

TEXT

C !' A

This prevents circular wait.

---

---

## MEMORY

Global lock order breaks circular waiting.

---

## PART G — TIMESTAMP-BASED DEADLOCK PREVENTION

Now we reach two common theoretical protocols:

1. Wait-Die
2. Wound-Wait

Both use transaction timestamps.

---

## 26. TRANSACTION TIMESTAMP

Suppose:

TEXT

$TS(T1) < TS(T2)$

Then:

TEXT

T1 is older

T2 is younger

Smaller timestamp = older transaction.

This assumption is standard.

---

## MEMORY

TEXT

Smaller TS

! ' o l d e r   t r a n s a c t i o n

---

## PART H — WAIT-DIE

---

## 27. WAIT-DIE RULE

Suppose transaction  $T_i$  requests a lock held by  $T_j$ .

**If  $T_i$  is older than  $T_j$ :**

TEXT

Older requester  
!' W A I T

If Ti is younger than Tj:

TEXT

Younger requester  
!' D I E / A B O R T

So:

$\boxed{\text{Older} \setminus \text{waits}, \text{Younger} \setminus \text{dies}}$

## MEMORY TRICK

**WAIT-DIE: Old waits, young dies**

This is the easiest mnemonic.

## 28. WAIT-DIE EXAMPLE 1

Suppose:

TEXT

T1 older  
T2 younger

T1 requests resource held by T2.

Older requests from younger.

Rule:

TEXT

T1 waits

## 29. WAIT-DIE EXAMPLE 2

T2 younger requests resource held by older T1.

Rule:

TEXT

T2 dies / aborts

## 30. WHY WAIT-DIE PREVENTS DEADLOCK

Waiting edges follow an age direction:

TEXT

o l d e r ! ' y o u n g e r

because only older transactions are allowed to wait for younger ones.

You cannot build a circular chain of strictly decreasing/increasing age relationships.

Thus deadlocks are prevented.

## 31. RESTART TIMESTAMP

Typically, an aborted transaction restarts using its **original timestamp**, not a brand-new younger timestamp.

Why?

To reduce starvation.

Over time, it retains its age priority.

## PART I — WOUND-WAIT

### 32. WOUND-WAIT RULE

Again  $T_i$  requests a lock held by  $T_j$ .

**If  $T_i$  is older than  $T_j$ :**

Older requester does not wait.

Instead:

TEXT

Older wounds younger  
!' younger holder aborts

**If  $T_i$  is younger than  $T_j$ :**

TEXT

Younger requester waits

So:

$\{ \text{Older} \setminus \text{wounds}, \text{Younger} \setminus \text{waits} \}$

## MEMORY TRICK

**WOUND-WAIT: Old wounds, young waits**



### 33. WOUND-WAIT EXAMPLE

T1 older.  
T2 younger holds X.  
T1 requests X.  
Rule:

TEXT

T1 wounds T2

Meaning:

TEXT

abort T2  
release X  
give X to T1

### 34. SECOND EXAMPLE

T1 older holds X.  
T2 younger requests X.  
Rule:

TEXT

T2 waits

### PART J — WAIT-DIE vs WOUND-WAIT

### 35. MASTER TABLE

| Situation                               | Wait-Die       | Wound-Wait      |
|-----------------------------------------|----------------|-----------------|
| Older requests resource held by younger | Older waits    | Younger aborted |
| Younger requests resource held by older | Younger aborts | Younger waits   |

### MEMORY TABLE

TEXT

WAIT-DIE:  
O L D ! ' W A I T  
Y O U N G ! ' D I E  
  
WOUND-WAIT:  
O L D ! ' W O U N D

Y O U N G ! ' W A I T

This is worth memorizing exactly.

## 36. COMMON CONFUSION

Both are based on age.

But:

### Wait-Die

Non-preemptive style:

TEXT

older may wait  
younger aborts itself

### Wound-Wait

Preemptive style:

TEXT

older can force younger holder to abort

## 37. WHICH TRANSACTION IS FAVORED?

Both protocols generally favor older transactions.

Why?

Older transactions have usually done more work and should be allowed to eventually finish.

## PART K — TIMEOUT

### 38. TIMEOUT METHOD

Simpler deadlock handling:

**If transaction waits too long, abort it.**

Example:

TEXT

T1 waits > 10 seconds  
!' r o l l b a c k T 1

Advantage:

TEXT  
simple

Disadvantage:

TEXT  
transaction may be aborted even when there was no deadlock

because the delay might simply be long contention.

## MEMORY

Timeout guesses deadlock from waiting time.

Not exact like cycle detection.

## 39. FALSE ABORTS

Suppose T1 waits because T2 is just performing a long computation.

No cycle exists.

But timeout expires.

T1 aborts unnecessarily.

So timeout can produce:

TEXT  
false deadlock suspicion

## PART L — DEADLOCK DETECTION FREQUENCY

### 40. HOW OFTEN SHOULD DBMS CHECK?

If deadlocks are frequent:

TEXT  
check often

If rare:

TEXT  
less frequent detection

Detection itself has overhead.

This is a design tradeoff.

For GATE, the main point is understanding detection through cycles.

## PART M — GATE-STYLE WAIT-FOR GRAPH PROBLEM

### 41. PROBLEM

Current lock state:

TEXT

T1 holds X(A), requests X(B)  
T2 holds X(B), requests X(C)  
T3 holds X(C), requests X(A)

Find wait-for graph.

T1 wants B held by T2:

$\{ T1 \rightarrow T2 \}$

T2 wants C held by T3:

$\{ T2 \rightarrow T3 \}$

T3 wants A held by T1:

$\{ T3 \rightarrow T1 \}$

Graph:

TEXT

T 1 ! ' T 2 ! ' T 3  
! '  
% % % % % % % % % % % %

Cycle.

Therefore:

$\{ \boxed{\text{Deadlock}} \}$

### 42. PROBLEM — NO DEADLOCK

TEXT

T1 waits for T2  
T2 waits for T3  
T3 holds resource but waits for nobody

Graph:

TEXT

T 1 ! ' T 2 ! ' T 3

No cycle.

Therefore:

$\neg \boxed{\text{No deadlock}}$

T3 can finish and release resource.

## PART N — MULTIPLE HOLDERS

### 43. SHARED LOCK CASE

Suppose:

TEXT

T1 holds S(X)

T2 holds S(X)

T3 requests:

TEXT

X(X)

T3 must wait for both T1 and T2.

WFG:

TEXT

T3 !' T1

T3 !' T2

because both are blocking T3's exclusive request.

### 44. IMPORTANT

A transaction may have multiple outgoing edges in a wait-for graph.

Example:

TEXT

T3 !' T1

T3 !' T2

This does not itself imply deadlock.

Only a cycle matters.

## PART O — LOCK UPGRADE DEADLOCK

### 45. CLASSIC UPGRADE DEADLOCK

Suppose:

TEXT

T1 holds S(X)  
T2 holds S(X)

Now both request upgrade:

TEXT

T1 wants X(X)  
T2 wants X(X)

T1 cannot upgrade until T2 releases S(X).

T2 cannot upgrade until T1 releases S(X).

So:

TEXT

T1 waits for T2  
T2 waits for T1

Deadlock.

This is why lock upgrades require careful handling.

---

## PART P — DEADLOCK vs STARVATION vs LIVELOCK

---

### 46. DEADLOCK

Transactions are permanently waiting in a cycle.

TEXT

T1 waits T2  
T2 waits T1

No progress.

---

### 47. STARVATION

System overall may progress, but one transaction repeatedly fails to get resources.

Example:

TEXT

T1 continually postponed

---

### 48. LIVELOCK — BRIEF IDEA

Transactions are not blocked permanently; they keep changing/retrying but still fail to make useful progress.

Example:

#### TEXT

T1 yields  
T2 yields  
T1 retries  
T2 retries  
...

Less central for this GATE DBMS syllabus, but know the distinction.

## PART Q — DEADLOCK PREVENTION vs AVOIDANCE vs DETECTION

### 49. PREVENTION

Design protocol so deadlock cannot happen.

Examples:

#### TEXT

Conservative 2PL  
Wait-Die  
Wound-Wait  
Resource ordering

### 50. DETECTION

Allow deadlock.

Then:

#### TEXT

Wait-for graph  
!' cycle detection  
!' abort victim

### 51. AVOIDANCE

General systems theory may use knowledge of future resource needs to avoid unsafe states.

The classic OS example is Banker's algorithm.

For DBMS lock questions, GATE more commonly focuses on:

#### TEXT

prevention  
+  
detection

rather than full avoidance algorithms.

## PART R — GATE TRAPS

---

### & pTrap 1

---

2PL does not imply deadlock-free.

---

### & pTrap 2

---

Cycle in wait-for graph means deadlock.

---

### & pTrap 3

---

Wait-for graph edge:

TEXT

```
w a i t e r ! '   h o l d e r
```

---

### & pTrap 4

---

Do not confuse WFG with precedence graph.

---

### & pTrap 5

---

Conservative 2PL prevents deadlock.

---

### & pTrap 6

---

Wait-Die:

TEXT

```
older waits  
younger aborts
```

---

### & pTrap 7

---

Wound-Wait:

TEXT

```
older aborts younger  
younger waits for older
```

---

### & pTrap 8

---

Smaller timestamp means older transaction.

---



---

### & pTrap 9

Timeout does not prove a deadlock existed.

---

### & pTrap 10

Deadlock and starvation are different.

---

### & pTrap 11

Acyclic WFG means no current deadlock.

---

### & pTrap 12

Shared locks can create multiple blocking holders for an X-lock request.

---

## PART S — PYQ SOLVING METHOD

When given lock table:

### STEP 1

For each blocked transaction, ask:

TEXT

Which transaction(s) hold the incompatible lock?

### STEP 2

Draw:

TEXT

w a i t e r ! ' h o l d e r

### STEP 3

Ignore transactions not involved in waiting unless needed.

### STEP 4

Check graph for cycle.

### STEP 5

If cycle:

TEXT

Dead lock

## STEP 6

If asked prevention rule, compare timestamps carefully.

## PART T — WAIT-DIE FAST METHOD

Suppose:

TEXT

TS(T1)=5

TS(T2)=10

So:

TEXT

T1 older

T2 younger

T1 requests lock held by T2:

TEXT

Old requests young

Wait-Die:

TEXT

OLD WAITS

T2 requests lock held by T1:

TEXT

Young requests old

Wait-Die:

TEXT

YOUNG DIES

## PART U — WOUND-WAIT FAST METHOD

Same ages.

T1 older requests lock held by T2:

TEXT

OLD WOUNDS

!' a b o r t T 2

T2 younger requests lock held by T1:

TEXT

YOUNG WAITS

## MEMORY PAGE

TEXT

DEADLOCK

= circular waiting

Four conditions:

TEXT

Mutual Exclusion

Hold and Wait

No Preemption

Circular Wait

Wait-for graph:

TEXT

Node = Transaction

Edge:

W A I T E R ! ' H O L D E R

Main rule:

TEXT

C y c l e ! ' D e a d l o c k

N o c y c l e ! ' N o c u r r e n t d e a d l o c k

Timestamp methods:

TEXT

WAIT-DIE:

Old waits

Young dies

WOUND-WAIT:

Old wounds

Young waits

## NOTEBOOK NOTES

### DEADLOCK

Set of transactions wait indefinitely for one another.

## FOUR NECESSARY CONDITIONS

### TEXT

1. Mutual exclusion
2. Hold and wait
3. No preemption
4. Circular wait

All four required.

## WAIT-FOR GRAPH

$T_i \rightarrow T_j$

means:

### TEXT

$T_i$  waits for  $T_j$

Cycle:

$\boxed{\text{Deadlock}}$

## DETECTION

### TEXT

```
Build WFG
!' find cycle
!' choose victim
!' rollback
!' release locks
```

## STARVATION

Same transaction repeatedly postponed/aborted and may never finish.

## CONSERVATIVE2PL

Acquire all locks before execution.

Breaks hold-and-wait.

Deadlock-free.

## WAIT-DIE

### TEXT

```
Older requester !' WAIT
Younger requester !' ABORT
```

---

## WOUND-WAIT

---

### TEXT

```
Older requester !' abort younger holder
Younger requester !' WAIT
```

---

## TIMEOUT

---

Wait too long:

### TEXT

```
abort
```

Simple but can abort without actual deadlock.

---

## 30-SECOND REVISION

---

### TEXT

Deadlock:  
transactions wait forever in cycle

Conditions:  
Mutual exclusion  
Hold and wait  
No preemption  
Circular wait

Wait-for graph:  
waiter !' holder

Cycle in WFG  
!' deadlock

Precedence graph:  
serializability  
Wait-for graph:  
deadlock

2PL:  
deadlock possible

Conservative 2PL:  
deadlock-free

Wait-Die:  
old waits  
young dies

Wound-Wait:  
old wounds  
young waits

Timeout:  
simple but may abort falsely

Starvation:  
one transaction repeatedly denied progress

## CHECK YOUR UNDERSTANDING

### Q1

Complete the four necessary deadlock conditions:

1. Mutual exclusion
2. \_\_\_\_\_
3. No preemption
4. \_\_\_\_\_

### Q2

If T2 waits for a lock held by T1, what edge appears in the wait-for graph?

A.  $T1 \nrightarrow T2$  B.  $T2 \nrightarrow T1$

### Q3

Graph:

TEXT

|     |    |     |
|-----|----|-----|
| T 1 | !' | T 2 |
| T 2 | !' | T 3 |
| T 3 | !' | T 1 |

What can you conclude?

### Q4

Does basic 2PL prevent deadlock?

### Q5

Suppose:

TEXT

T1 older  
T2 younger

Under **Wait-Die**, T2 requests a resource held by T1.

What happens?

### Q6

Under **Wound-Wait**, an older T1 requests a resource held by younger T2.

What happens?

## Q7

What is the difference between deadlock and starvation?

## Q8 — Important GATE trap

Which graph detects:

1. Conflict serializability?
2. Deadlock?

Next is the final major concurrency-control mechanism:

## TOPIC 21 — TIMESTAMP ORDERING PROTOCOL

We will cover:

transaction timestamps !' read timestamp !' write timestamp !' basic timestamp-ordering rules !' when a read is allowed/rejected !' when a write is allowed/rejected !' rollback !' Thomas Write Rule !' deadlock freedom !' starvation possibility !' GATE numericals.

Type `NEXT`.

USER

NEXT

ASSISTANT

## TOPIC 21 — TIMESTAMP ORDERING PROTOCOL

Now we study the other major concurrency-control approach in GATE:

### Timestamp Ordering (TO)

Unlike locking, timestamp ordering does not make transactions wait for locks. Instead, it forces conflicting operations to obey a predefined transaction order.

The flow is:

TEXT

```
Transaction Timestamp
    !"
Read Timestamp / Write Timestamp
    !"
Read Rule
    !"
Write Rule
    !"
Allow or Abort
    !"
Thomas Write Rule
```

The highest-value GATE points are:

- `TS(Ti)`
- `read_TS(X)`
- `write_TS(X)`
- exact read/write conditions
- when transaction aborts
- deadlock freedom
- starvation possibility
- Thomas Write Rule

## 1. BASIC IDEA

Every transaction gets a unique timestamp when it starts.

Example:

TEXT

```
TS(T1) = 5
TS(T2) = 10
TS(T3) = 20
```

Smaller timestamp means older transaction.

So:

TEXT

```
T1 oldest
T2 younger
T3 youngest
```

## MEMORY TRICK

**Smaller TS = Older**

This is the same convention we used in Wait-Die and Wound-Wait.

## 2. WHAT DOES TIMESTAMP ORDERING TRY TO DO?

Suppose:

$\forall [TS(T_1) < TS(T_2)]$

Timestamp ordering wants conflicting database operations to behave as if:

TEXT

```
T1
before
T2
```



in timestamp order.

So:

**Older transactions should logically appear before younger transactions.**

If an operation would violate this required order, the DBMS may abort the transaction.

### 3. WHY NO LOCKS?

Timestamp ordering does not normally use the waiting mechanism of lock-based protocols.

Instead:

```
TEXT
Operation arrives
    !"
Check timestamps
    !"
Allowed?
  YES  !'  e x e c u t e
  NO   !'  a b o r t / r e s t a r t
```

Therefore transactions do not form lock-wait cycles.

Important result:

$\boxed{\text{Basic Timestamp Ordering is deadlock-free}}$

### 4. INFORMATION STORED FOR EACH DATAITEM

For each database item X, maintain:

#### Read Timestamp

$\text{read\_TS}(X)$

or:

$\text{RTS}(X)$

This is:

**Largest timestamp of any transaction that has successfully read X.**

#### Write Timestamp

$\text{write\_TS}(X)$

or:

$\text{WTS}(X)$

This is:

**Largest timestamp of any transaction that has successfully written X.**

## MEMORY

TEXT

$RTS(X)$   
= youngest/latest timestamp transaction  
that has READ X

$WTS(X)$   
= youngest/latest timestamp transaction  
that has WRITTEN X

“Largest timestamp” here means youngest transaction under our convention.

## 5. INITIAL VALUES

Initially, before any transaction accesses X:

TEXT

$RTS(X) = 0$   
 $WTS(X) = 0$

or some minimal timestamp value.

Exact initialization may vary, but GATE usually gives the required values.

## PART A — READ RULE

### 6. TRANSACTION $T_i$ WANTS TO READ X

Suppose:

$\lceil TS(T_i) = TS_i \rceil$

$T_i$  requests:

$\lceil r_i(X) \rceil$

We compare:

$\lceil TS_i \rceil$

with:

$\lceil WTS(X) \rceil$

### 7. READ IS REJECTED WHEN

If:

$\lceil \boxed{TS(T_i) < WTS(X)} \rceil$

then  $T_i$  is too old to read the current value.

Why?

A younger transaction has already written  $X$ .

Allowing older  $T_i$  to read that newer value would violate timestamp ordering.

So:

TEXT

$T_i$  aborts / restarts

## MEMORY TRICK

For READ:

**Old transaction cannot read a value written by the future.**

Mathematically:

$\lceil TS_i < WTS(X) \rceil \rightarrow \text{Abort}$

## 8. READ IS ALLOWED WHEN

If:

$\lceil TS(T_i) \geq WTS(X) \rceil$

then read is allowed.

After successful read:

$\lceil RTS(X) = \max(RTS(X), TS(T_i)) \rceil$

## WRITE THIS IN A BOX

### READ RULE

For:

$\lceil r_i(X) \rceil$

if:

$\lceil \boxed{TS_i < WTS(X)} \rceil$

! abort  $T_i$ .

Otherwise:

TEXT

read  $X$

$RTS(X) = \max(RTS(X), TS_i)$

## 9. READ EXAMPLE

Suppose:

TEXT

$TS(T1) = 10$   
 $WTS(X) = 20$

T1 wants:

TEXT

$r1(X)$

Check:

$\nmid 10 < 20 \nmid$

Yes.

Meaning transaction timestamp 20 already wrote X.

T1 is older and would be reading a value produced by a transaction that should logically come after it.

Therefore:

$\nmid \boxed{T1 \text{ aborts}} \nmid$

## 10. ALLOWED READ EXAMPLE

Suppose:

TEXT

$TS(T2) = 30$   
 $RTS(X) = 15$   
 $WTS(X) = 20$

T2 wants to read X.

Check:

$\nmid 30 < 20? \nmid$

No.

Read allowed.

Update:

$\nmid RTS(X) = \max(15, 30) = 30 \nmid$

Final:

TEXT

$RTS(X) = 30$   
 $WTS(X) = 20$

## PART B — WRITE RULE

### 11. TRANSACTION $T_i$ WANTS TO WRITE $X$

For:

$\forall w_i(X) \forall$

we must check **two** things:

1. Has a younger transaction already read  $X$ ?
2. Has a younger transaction already written  $X$ ?

So compare  $TS(T_i)$  with:

TEXT

$RTS(X)$

$WTS(X)$

### 12. FIRST WRITE CHECK — READ TIMESTAMP

If:

$\forall \boxed{TS_i < RTS(X)} \forall$

then a younger transaction has already read the old value of  $X$ .

If older  $T_i$  now writes  $X$ , that would violate timestamp order.

Therefore:

TEXT

$T_i$  aborts

## INTUITION

Suppose:

TEXT

$T_2$  younger

already read  $X$ .

Now older  $T_1$  tries to write  $X$ .

In the required serial order:

TEXT

$T_1$  should have happened before  $T_2$

So  $T_2$  should have seen  $T_1$ 's written value.

But it already read before that write occurred.

Too late.

Hence abort T1.

### 13. SECOND WRITE CHECK — WRITE TIMESTAMP

Under **basic timestamp ordering**, if:

$$\boxed{TS_i < WTS(X)}$$

then a younger transaction has already written X.

Older Ti's write is arriving too late.

Therefore basic TO says:

TEXT  
Ti aborts

### 14. BASIC WRITE RULE

For:

$$w_i(X)$$

if either:

$$TS_i < RTS(X)$$

or:

$$TS_i < WTS(X)$$

then:

$$\boxed{\text{Abort } T_i}$$

Otherwise:

TEXT  
write X  
 $WTS(X) = TS_i$

### MEMORY TRICK

For WRITE:

**Compare with BOTH R and W.**

TEXT  
READ operation  
!' check WTS only  
  
WRITE operation  
!' check RTS + WTS

This is one of the best shortcuts in the topic.

## 15. BASIC RULE TABLE

| Operation | Abort condition                                |
|-----------|------------------------------------------------|
| ri(X)     | $\neg(TS_i < WTS(X))$                          |
| wi(X)     | $\neg(TS_i < RTS(X)) \vee \neg(TS_i < WTS(X))$ |

If allowed:

TEXT

Read !' update RTS

Write !' update WTS

## MEMORY PAGE

TEXT

READ:

Check WRITE timestamp

WRITE:

Check READ + WRITE timestamps

Easy:

**R checks W**

**W checks R + W**

## 16. WRITE EXAMPLE 1

Suppose:

TEXT

$TS(T1)=10$

$RTS(X)=20$

$WTS(X)=5$

T1 wants:

TEXT

$w1(X)$

Check:

$\lceil 10 < 20 \rceil$

Yes.

A younger transaction with timestamp 20 already read X.

Therefore:

$\lceil \boxed{T1 \text{ aborts}} \rceil$

---

## 17. WRITE EXAMPLE 2

Suppose:

TEXT

$TS(T1)=10$

$RTS(X)=5$

$WTS(X)=20$

T1 wants to write.

Check:

$\lceil 10 < RTS(X)=5? \rceil$

No.

Check:

$\lceil 10 < WTS(X)=20? \rceil$

Yes.

Under **basic TO**:

$\lceil \boxed{\text{Abort } T1} \rceil$

Later we will see Thomas Write Rule changes this second case.

---

## 18. WRITE EXAMPLE 3

Suppose:

TEXT

$TS(T3)=50$

$RTS(X)=30$

$WTS(X)=40$

T3 wants to write X.

Check:

$\lceil 50 < 30? \rceil$

No.

$\lceil 50 < 40? \rceil$

No.

Allowed.



Then:

$\neg [WTS(X)=50]$

---

## PART C — STEP-BY-STEP GATE PROBLEM

---

### 19. GIVEN

Initially:

TEXT

$RTS(X)=0$

$WTS(X)=0$

$TS(T1)=10$

$TS(T2)=20$

Operations:

TEXT

$r1(X)$

$w1(X)$

$r2(X)$

$w2(X)$

---

### 20. $r1(X)$

Check:

$\neg [TS(T1)=10 < WTS(X)=0?]$

No.

Allowed.

Update:

$\neg [RTS(X)=10]$

State:

TEXT

$RTS(X)=10$

$WTS(X)=0$

---

### 21. $w1(X)$

Check:

$\neg [10 < RTS(X)=10?]$

No.

$\neg [10 < WTS(X)=0?]$

No.

Allowed.

Update:

$\lceil \text{WTS}(X)=10 \rceil$

State:

TEXT

$\text{RTS}(X)=10$

$\text{WTS}(X)=10$

## 22. $r_2(X)$

$\lceil 20 < \text{WTS}(X)=10? \rceil$

No.

Allowed.

Update:

$\lceil \text{RTS}(X)=20 \rceil$

State:

TEXT

$\text{RTS}(X)=20$

$\text{WTS}(X)=10$

## 23. $w_2(X)$

Check:

$\lceil 20 < \text{RTS}(X)=20? \rceil$

No.

$\lceil 20 < \text{WTS}(X)=10? \rceil$

No.

Allowed.

Final:

TEXT

$\text{RTS}(X)=20$

$\text{WTS}(X)=20$

No abort.

The operations follow timestamp order:

TEXT

T1 older  
before  
T2 younger

## PART D — OUT-OF-ORDER EXAMPLE

### 24. GIVEN

TEXT

$TS(T1)=10$   
 $TS(T2)=20$

$RTS(X)=0$   
 $WTS(X)=0$

Schedule:

TEXT

$w2(X)$   
 $r1(X)$

### 25. $w2(X)$

Check:

$\nexists 20 < 0?$

No.

Allowed.

Set:

$\nexists WTS(X)=20$

### 26. $r1(X)$

Check:

$\nexists TS(T1)=10 < WTS(X)=20$

Yes.

So:

$\nexists \boxed{T1 \text{ aborts}}$

Why?

T1 is older than T2 but is trying to read a value already written by younger T2.

This violates timestamp order.

---

## PART E — ANOTHER IMPORTANT WRITE EXAMPLE

---

### 27. GIVEN

TEXT

$TS(T1)=10$   
 $TS(T2)=20$

Schedule:

TEXT

$r2(X)$   
 $w1(X)$

Initial timestamps zero.

---

### 28. $r2(X)$

Allowed.

Set:

$\{ RTS(X)=20 \}$

---

### 29. $w1(X)$

Check:

$\{ TS(T1)=10 < RTS(X)=20 \}$

Yes.

Therefore:

$\{ \boxed{T1 \text{ aborts}} \}$

Why?

T2 already read the old value.

If T1's write were allowed, then in timestamp order T1 should have written before T2's read.

But it didn't.

---

## PART F — THOMAS WRITE RULE

---

### 30. WHY THOMAS WRITE RULE?

Consider:

---

TEXT

$TS(T_1)=10$   
 $WTS(X)=20$

T1 wants:

TEXT

$w_1(X)$

A younger transaction already wrote X.

Under basic TO:

TEXT

abort T1

But ask:

**Is T1's old write actually useful anymore?**

A younger write already overwrote the logical future value.

If T1's write would immediately be obsolete, perhaps we can simply ignore it instead of aborting.

That's the idea behind:

**Thomas Write Rule**

## 31. THOMAS WRITE RULE

For:

$\forall w_i(X)$

first check:

$\forall TS_i < RTS(X)$

If yes:

TEXT

Abort  $T_i$

This remains a real conflict.

But if:

$\forall TS_i < WTS(X)$

then instead of aborting:

TEXT

Ignore the write

because it is obsolete.

## WRITE THIS IN A BOX

### THOMAS WRITE RULE

For  $w_i(X)$ :

#### Case 1

$\neg [TS_i < RTS(X)]$

!' abort.

#### Case 2

$\neg [TS_i < WTS(X)]$

!' ignore obsolete write.

#### Otherwise

!' perform write and set:

$\neg [WTS(X) = TS_i]$

## 32. BASIC TO vs THOMAS RULE

| Situation              | Basic TO | Thomas Write Rule |
|------------------------|----------|-------------------|
| $\neg [TS_i < RTS(X)]$ | Abort    | Abort             |
| $\neg [TS_i < WTS(X)]$ | Abort    | Ignore write      |
| Otherwise              | Write    | Write             |

This is extremely GATE-relevant.

## MEMORY TRICK

Thomas forgives an obsolete WRITE.

It does not forgive a write that contradicts a younger transaction's prior read.

## 33. THOMAS EXAMPLE

Suppose:

TEXT

$TS(T_1) = 10$

$RTS(X) = 5$

$WTS(X) = 20$

T1 wants:

TEXT  
w1(X)

Check:

$\lfloor 10 < \text{RTS}(X) = 5? \rfloor$

No.

Check:

$\lfloor 10 < \text{WTS}(X) = 20? \rfloor$

Yes.

### Basic TO

TEXT  
Abort T1

### Thomas Write Rule

TEXT  
Ignore w1(X)  
Continue T1

The old write is obsolete.

---

## 34. IMPORTANT — DO NOT UPDATE WTS WHEN WRITE IS IGNORED

If Thomas rule ignores:

TEXT  
w1(X)

then:

TEXT  
WTS(X)

remains the timestamp of the younger successful write.

Example:

TEXT  
WTS(X)=20

stays 20.

Do not reset it to 10.

## 35. THOMAS WRITE RULE AND SERIALIZABILITY

Basic timestamp ordering ensures conflict-serializable schedules in timestamp order.

Thomas Write Rule allows some schedules that are:

**view serializable but not necessarily conflict serializable,**

because obsolete writes can be ignored.

This connects directly to our earlier:

```
TEXT
blind writes
+
view serializability
```

discussion.

## MEMORY CONNECTION

```
TEXT
Basic TO
!' Conflict Serializability

Thomas Write Rule
!' allows broader View-Serializable behavior
```

This is a useful conceptual GATE distinction.

## PART G — DEADLOCK

### 36. DOES TIMESTAMP ORDERING DEADLOCK?

Basic timestamp ordering does not make transactions wait for locks.

Instead:

```
TEXT
allow
or
abort
```

Therefore no wait-for cycle arises.

So:

$\boxed{\text{Timestamp Ordering is deadlock-free}}$

### 37. BUT CAN STARVATION OCCUR?

Yes.



Suppose a transaction repeatedly aborts because conflicting younger transactions continually execute before it.  
It may restart repeatedly.

So:

TEXT

Deadlock !' No

Starvation !' Possible

depending on restart/priority policy.

## MEMORY

TO trades waiting for rollback.

Locking:

TEXT

may WAIT

!' deadlock possible

Timestamp ordering:

TEXT

may ABORT

!' no deadlock

## PART H — LOCKING vs TIMESTAMP ORDERING

### 38. COMPARISON

| Lock-Based Protocol               | Timestamp Ordering                                   |
|-----------------------------------|------------------------------------------------------|
| Uses locks                        | Uses timestamps                                      |
| Transactions may wait             | Usually abort rather than wait                       |
| Deadlock possible                 | Deadlock-free                                        |
| 2PL ensures CSR                   | Basic TO ensures timestamp-order serializability     |
| May block transactions            | More restarts possible                               |
| Strict 2PL good recovery behavior | Basic TO may need additional recovery considerations |

### 39. ORDER IS FIXED IN ADVANCE

A major difference:

2PL does not necessarily predetermine the serial order before operations execute.

Timestamp ordering does.

If:

$\forall [TS(T1) < TS(T2) < TS(T3)]$

the desired serialization order is:

TEXT

T 1 !' T 2 !' T 3

Any operation violating this order is rejected/handled.

---

## PART I — MULTIPLE DATA ITEMS

### 40. IMPORTANT

Each item maintains separate:

TEXT

RTS

WTS

Example:

TEXT

RTS(X)=20

WTS(X)=10

RTS(Y)=5

WTS(Y)=30

An operation on X uses X's timestamps only.

Do not use Y's timestamp information.

---

### 41. GATE PROBLEM

Given:

TEXT

TS(T1)=10

TS(T2)=20

Initially:

RTS(X)=0

WTS(X)=0

RTS(Y)=0

WTS(Y)=0

Schedule:

TEXT

$r_2(X)$   
 $w_1(Y)$   
 $w_1(X)$

## 42. $r_2(X)$

$\lceil 20 < 0? \rceil$

No.

Allowed.

$\lceil \text{RTS}(X) = 20 \rceil$

## 43. $w_1(Y)$

Check Y:

TEXT

$TS_1 = 10$   
 $\text{RTS}(Y) = 0$   
 $\text{WTS}(Y) = 0$

Allowed.

Set:

$\lceil \text{WTS}(Y) = 10 \rceil$

## 44. $w_1(X)$

Now X:

TEXT

$\text{RTS}(X) = 20$   
 $\text{WTS}(X) = 0$

Check:

$\lceil 10 < 20 \rceil$

Yes.

So:

$\lceil \boxed{T_1 \text{ aborts}} \rceil$

Notice  $w_1(Y)$  succeeded earlier, but transaction abort requires recovery of its effects as needed.

## PART J — COMPLETE GATE NUMERICAL

## 45. GIVEN

TEXT

$TS(T1)=5$   
 $TS(T2)=10$   
 $TS(T3)=15$

Initially:  
 $RTS(X)=0$   
 $WTS(X)=0$

Operations:

TEXT

$r1(X)$   
 $w2(X)$   
 $r3(X)$   
 $w1(X)$

Use basic TO.

## 46. $r1(X)$

Check:

$\neg [5 < 0?]$

No.

Allowed.

$\neg [RTS(X)=5]$

## 47. $w2(X)$

Check:

$\neg [10 < RTS(X)=5?]$

No.

$\neg [10 < WTS(X)=0?]$

No.

Allowed.

$\neg [WTS(X)=10]$

State:

TEXT

$RTS=5$   
 $WTS=10$

## 48. r3(X)

Check:

$\{ 15 < WTS(X) = 10 \}$

No.

Allowed.

Update:

$\{ RTS(X) = 15 \}$

State:

TEXT

$RTS = 15$

$WTS = 10$

## 49. w1(X)

T1 has:

$\{ TS = 5 \}$

Check:

$\{ 5 < RTS(X) = 15 \}$

Yes.

So:

$\{ \boxed{T1} \text{ aborts} \}$

No need to check further.

## PART K — READ TIMESTAMP UPDATE TRAP

### 50. RTS IS MAXIMUM

Suppose:

TEXT

$RTS(X) = 30$

Transaction timestamp 20 successfully reads X.

Would RTS become 20?

No.

Use:

$\{ RTS(X) = \max(30, 20) = 30 \}$

So it stays 30.

---

## MEMORY

**RTS never decreases.**

Similarly:

**WTS records latest successful write timestamp and does not move backward under standard successful operations.**

---

## PART L — RESTARTS

### 51. WHAT HAPPENS AFTER ABORT?

A transaction may restart with a new timestamp under the basic timestamp-ordering implementation.

A new larger timestamp makes it younger and may allow it to execute without violating previous timestamp order.

This differs from Wait-Die/Wound-Wait, where keeping the original timestamp is often used to avoid starvation.

---

## & pGATE TRAP

Do not automatically apply the restart-timestamp rule of Wait-Die to timestamp ordering.

They are different protocols.

Use the protocol stated by the question.

---

## PART M — STRICT TIMESTAMP ORDERING

### 52. BASIC TO AND RECOVERY

Basic TO controls operation ordering but can potentially allow another transaction to access a value written by an uncommitted transaction unless additional rules are imposed.

Variants such as:

**Strict Timestamp Ordering**

delay certain reads/writes until earlier writers commit/abort, improving recoverability.

For your GATE core preparation, prioritize the **basic TO read/write rules + Thomas Write Rule** unless the question explicitly asks strict TO.

---

## PART N — GATE TRAPS

## & pTrap 1

Smaller timestamp = older transaction.

## & pTrap 2

Each item has:

TEXT  
RTS(X)  
WTS(X)

not one global RTS/WTS.

## & pTrap 3

Read checks:

TEXT  
WTS only

## & pTrap 4

Write checks:

TEXT  
RTS  
+  
WTS

## & pTrap 5

For read:

$\forall [TS_i < WTS(X)]$

!' abort.

## & pTrap 6

For basic write:

$\forall [TS_i < RTS(X)]$

or:

$\forall [TS_i < WTS(X)]$

!' abort.

& pTrap 7

Thomas Write Rule changes only the:

$\lceil TS_i < WTS(X) \rceil$

case into:

TEXT

ignore write

not abort.

& pTrap 8

Thomas does NOT ignore the write when:

$\lceil TS_i < RTS(X) \rceil$

That case still requires abort.

& pTrap 9

Ignored Thomas write does not update WTS.

& pTrap 10

Basic timestamp ordering is deadlock-free.

& pTrap 11

Deadlock-free does not mean starvation-free.

& pTrap 12

RTS uses maximum timestamp of successful readers.

PART O — PYQ-ORIENTED FAST METHOD

When given a timestamp problem:

STEP 1

Create table:

| Item | RTS | WTS |
|------|-----|-----|
| X    | ... | ... |
| Y    | ... | ... |



---

## STEP 2

Write transaction timestamps:

TEXT

T1 = ...

T2 = ...

T3 = ...

---

## STEP 3

For every READ:

Check:

$\neg [TS_i < WTS(X)? \neg]$

Yes:

TEXT

Abort

No:

TEXT

Allow

$RTS = \max(RTS, TS_i)$

---

## STEP 4

For every WRITE:

Check first:

$\neg [TS_i < RTS(X)? \neg]$

Yes:

TEXT

Abort

Otherwise check:

$\neg [TS_i < WTS(X)? \neg]$

**Basic TO:**

TEXT

Abort

**Thomas:**

TEXT

Ignore write

Otherwise:

TEXT

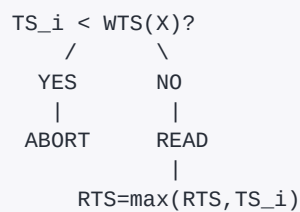
Execute write

WTS=TS<sub>i</sub>

## 53. DECISION FLOWCHART

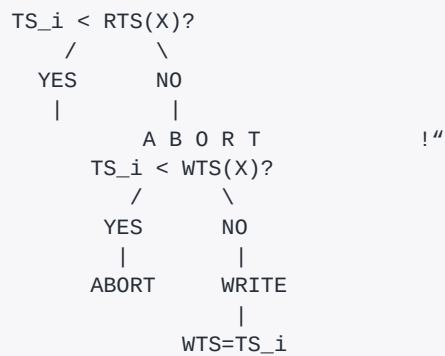
### READ $ri(X)$

TEXT



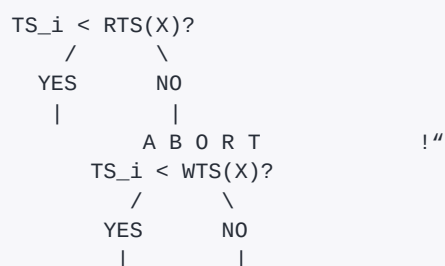
### WRITE $wi(X)$ — BASIC TO

TEXT



## 54. THOMAS FLOW

TEXT



## NOTEBOOK NOTES

### TIMESTAMP

Each transaction  $T_i$  gets:

$\{ TS(T_i) \}$

Smaller timestamp:

TEXT

older transaction

### FOR EACH ITEM X

TEXT

$RTS(X)$

= largest TS that successfully read X

$WTS(X)$

= largest TS that successfully wrote X

### READ RULE

For:

$\{ r_i(X) \}$

if:

$\{ \boxed{TS_i < WTS(X)} \}$

!' abort.

Otherwise:

$\{ RTS(X) = \max(RTS(X), TS_i) \}$

### BASIC WRITE RULE

For:

$\{ w_i(X) \}$

if:

$\{ TS_i < RTS(X) \}$

or:

$\{ TS_i < WTS(X) \}$

!' abort.

Otherwise:

$\lceil \text{WTS}(X) = \text{TS}_i \rceil$

## THOMAS WRITE RULE

TEXT

```
TS_i < RTS(X)
!' A b o r t
```

```
TS_i < WTS(X)
!' I g n o r e   o b s o l e t e   w r i t e
```

```
Otherwise
!' W r i t e
```

## PROPERTIES

TEXT

Basic TO

```
!' s e r i a l i z a b l e   i n   t i m e s t a m p   o r d e r
!' d e a d l o c k - f r e e
!' s t a r v a t i o n / r e s t a r t s   p o s s i b l e
```

## MEMORY PAGE

TEXT

R operation

```
!' c h e c k   W
```

W operation

```
!' c h e c k   R   +   W
```

More exactly:

TEXT

READ:

TS < WTS

```
!' A B O R T
```

TEXT

WRITE:

TS < RTS

```
!' A B O R T
```

TS < WTS

```
!' A B O R T   i n   b a s i c   T O
```

```
!' I G N O R E   u n d e r   T h o m a s
```

And:

## 30-SECOND REVISION

TEXT

Timestamp Ordering:

Smaller TS = older

Each item X:

RTS(X)

WTS(X)

READ ri(X):

if TS<sub>i</sub> < WTS(X)

! ' a b o r t

else

! ' r e a d

! ' R T S = m a x ( R T S , T S \_ i )

WRITE wi(X):

if TS<sub>i</sub> < RTS(X)

! ' a b o r t

else if TS<sub>i</sub> < WTS(X):

B a s i c T O ! ' a b o r t

T h o m a s ! ' i g n o r e w r i t e

else

! ' w r i t e

! ' W T S = T S \_ i

Basic T0:

deadlock-free

But:

restarts/starvation possible

Shortcut:

READ checks W

WRITE checks R+W

## CHECK YOUR UNDERSTANDING

### Q1

Suppose:

TEXT

TS(T1)=10

WTS(X)=15

T1 requests:

TEXT

$r1(X)$

What happens?

---

## Q2

Suppose:

TEXT

$TS(T2)=20$

$RTS(X)=10$

$WTS(X)=15$

T2 requests:

TEXT

$w2(X)$

Is it allowed?

---

## Q3

Suppose:

TEXT

$TS(T1)=10$

$RTS(X)=20$

$WTS(X)=5$

T1 requests  $w1(X)$  .

What happens?

---

## Q4

Under **basic timestamp ordering**:

TEXT

$TS(T1)=10$

$RTS(X)=5$

$WTS(X)=20$

T1 requests  $w1(X)$  .

What happens?

---

## Q5

For the exact same Q4 situation, what happens under **Thomas Write Rule**?

---

## Q6

True or False:

Basic timestamp ordering can produce deadlocks.

### Q7

What does:

$\lceil \text{RTS}(X)=30 \rceil$

mean?

### Q8 — Important

Complete:

TEXT

READ checks \_\_\_\_\_ timestamp(s)

WRITE checks \_\_\_\_\_ timestamp(s)

We have now covered the major **transaction + concurrency-control chain**:

**Transactions !' ACID !' schedules/anomalies !' conflict serializability !' view serializability !' recoverability !' locks/2PL !' deadlocks !' timestamp ordering.**

Next:

## TOPIC 22 — TRANSACTIONS & CONCURRENCY CONTROL MASTER REVISION + MIXED GATE PROBLEMS

We will combine all these concepts so you can distinguish them quickly:

**CSR vs VSR !' recoverable vs cascadeless vs strict !' 2PL variants !' wait-for graph vs precedence graph !' wait-die vs wound-wait !' timestamp rules !' mixed GATE-style schedule questions.**

Type **`NEXT`**.

USER

NEXT

ASSISTANT

## TOPIC 22 — TRANSACTIONS & CONCURRENCY CONTROL MASTER REVISION + MIXED GATE PROBLEMS

Now we will compress the entire transaction/concurrency block into one **exam-ready revision system**.

The goal is to help you quickly answer questions that mix:

- conflict serializability

- view serializability
- recoverability
- cascadeless/strict
- 2PL
- deadlocks
- wait-for graph
- wait-die/wound-wait
- timestamp ordering

## PART A — MASTER CONCEPT MAP

### TEXT

#### TRANSACTION

!"

#### ACID

!"

#### SCHEDULES

!"

#### SERIALIZABILITY

%%% Conflict Serializability  
%%% View Serializability

#### RECOVERY SAFETY

%%% Recoverable  
%%% Cascadeless  
%%% Strict

#### CONCURRENCY CONTROL

%%% Locks  
%      %%% 2PL  
%      %%% Basic  
%      %%% Strict  
%      %%% Rigorous  
%      %%% Conservative  
%  
%%% Timestamp Ordering

## 1. FIRST DISTINCTION: SERIALIZABILITY vs RECOVERABILITY

This is one of the biggest GATE confusions.

### Serializability asks:

Is the concurrent execution logically equivalent to some serial execution?

### Recoverability asks:

If transactions depend on each other's uncommitted data, are commit/abort orders safe?

So:



TEXT

Serializability  
!' correctness of execution order

Recoverability  
!' safety of commit/recovery order

They are different properties.

## 2. CONFLICT SERIALIZABILITY — ONE-LINE RULE

Two operations conflict if:

TEXT

Different transactions  
+  
Same data item  
+  
At least one WRITE

Conflict table:

TEXT

|   |   |          |
|---|---|----------|
| R | R | !' N o   |
| R | W | !' Y e s |
| W | R | !' Y e s |
| W | W | !' Y e s |

Then build precedence graph.

Edge:

TEXT

Earlier conflicting transaction  
!' Later transaction

Main rule:

$\boxed{\text{Acyclic precedence graph} \iff \text{Conflict Serializable}}$

## MEMORY

TEXT

CSR  
!' C O N F L I C T S  
!' G R A P H  
!' N O C Y C L E

## 3. VIEW SERIALIZABILITY — ONE-LINE RULE

View equivalence preserves:

TEXT

I !' I n i t i a l   r e a d s  
R !' R e a d - f r o m   r e l a t i o n  
F !' F i n a l   w r i t e s

Mnemonic:

## IRF

A schedule is view serializable if it is view equivalent to some serial schedule.

Relationship:

$\boxed{CSR \rightarrow VSR}$

but:

$\boxed{VSR \rightarrow CSR}$

Difference often comes from:

TEXT

Blind Writes

## 4. BLIND WRITE

A transaction performs:

TEXT

$w_i(X)$

without first doing:

TEXT

$r_i(X)$

Blind writes are the classic reason a schedule can be:

TEXT

VSR  
but not CSR

## 5. RECOVERABILITY HIERARCHY

The most important hierarchy:

$\boxed{\text{Strict} \rightarrow \text{Cascadeless} \rightarrow \text{Recoverable}}$

Memory:

#### TEXT

Recoverable:  
Writer COMMIT before Reader COMMIT

Cascadeless:  
Writer COMMIT before Reader READ

Strict:  
Writer finishes before another transaction READS or WRITES that item

## 6. MASTER RECOVERY TABLE

| Property    | Requirement                                                           |
|-------------|-----------------------------------------------------------------------|
| Recoverable | $\forall (C_i < C_j)$ if $T_j$ reads from $T_i$                       |
| Cascadeless | $\forall (C_i < r_j(X))$                                              |
| Strict      | After $\forall (w_i(X))$ , no other R/W(X) until $T_i$ commits/aborts |

## MEMORY TRICK

#### TEXT

Recoverable = C before C

Cascadeless = C before R

Strict = Finish before R/W

## 7. DIRTY READ vs DIRTY WRITE

### Dirty Read

#### TEXT

$w_1(X)$   
 $r_2(X)$

before  $T_1$  commits.

$T_2$  reads uncommitted X.

### Dirty Write

#### TEXT

$w_1(X)$   
 $w_2(X)$

before  $T_1$  commits.

T2 overwrites uncommitted X.

Cascadeless prevents:

TEXT  
Dirty Read

Strict prevents:

TEXT  
Dirty Read + Dirty Write

---

## 8. 2PL MASTER RULE

Two phases:

### Growing

TEXT  
Acquire locks  
No unlock

### Shrinking

TEXT  
Release locks  
No new lock

Shortcut:

**After first unlock, never acquire another lock.**

---

## 9. 2PL GUARANTEE

$\boxed{2PL \rightarrow \text{Conflict Serializable}} \rightarrow \boxed{\text{Serializable}}$

But:

$\boxed{CSR \rightarrow 2PL}$

And:

**Basic 2PL can deadlock.**

---

## 10. LOCK COMPATIBILITY

| Held | Request S | Request X |
|------|-----------|-----------|
| S    | Yes       | No        |
| X    | No        | No        |

Memory:

Only S-S is compatible.

## 11. 2PL VARIANTS

### Basic 2PL

TEXT

Grow  
then shrink

Guarantees:

TEXT

CSR

Deadlock:

TEXT

Possible

### Strict 2PL

TEXT

Hold X-locks until commit/abort

Guarantees:

TEXT

CSR  
Strict schedule  
Cascadeless  
Recoverable

### Rigorous 2PL

TEXT

Hold S and X locks until commit/abort

Stronger than strict 2PL.

---

## Conservative 2PL

TEXT

Acquire all needed locks before execution starts

Major advantage:

TEXT

Deadlock-free

---

## 12. DEADLOCK — MASTER RULE

Deadlock:

TEXT

Transactions wait forever in a cycle

Four necessary conditions:

TEXT

1. Mutual exclusion
2. Hold and wait
3. No preemption
4. Circular wait

---

## 13. WAIT-FOR GRAPH

Node:

TEXT

Transaction

Edge:

TEXT

W a i t e r ! '   H o l d e r

Main test:

$\nexists \text{ Cycle in WaitForGraph } \iff \text{ Deadlock }$

---

## 14. PRECEDENCE GRAPH vs WAIT-FOR GRAPH

| Precedence Graph         | Wait-For Graph   |
|--------------------------|------------------|
| Conflict serializability | Deadlock         |
| Conflict ordering        | Lock waiting     |
| Earlier !' Later         | Waiter !' Holder |
| Cycle = Not CSR          | Cycle = Deadlock |

This distinction is extremely important.

## 15. WAIT-DIE

Timestamp-based deadlock prevention.

Smaller TS = older transaction.

Rule:

TEXT

```
Older requester !' WAIT
Younger requester !' DIE / ABORT
```

Memory:

**Old waits, young dies**

## 16. WOUND-WAIT

Rule:

TEXT

```
Older requester !' WOUND younger holder
Younger requester !' WAIT
```

Memory:

**Old wounds, young waits**

## 17. WAIT-DIE vs WOUND-WAIT

| Situation               | Wait-Die    | Wound-Wait  |
|-------------------------|-------------|-------------|
| Old requests from young | Wait        | Abort young |
| Young requests from old | Abort young | Wait        |

## 18. TIMESTAMP ORDERING — MASTER RULE

Each transaction has:

$\{ TS(T_i) \}$

Each data item X maintains:

TEXT  
RTS(X)  
WTS(X)

Where:

TEXT  
RTS = largest timestamp that successfully read X  
WTS = largest timestamp that successfully wrote X

## 19. READ RULE

For:

$\{ r_i(X) \}$

check:

$\{ TS_i < WTS(X)? \}$

If yes:

TEXT  
Abort

Otherwise:

TEXT  
Allow read  
 $RTS(X) = \max(RTS(X), TS_i)$

## MEMORY

**READ checks WTS**

## 20. WRITE RULE — BASIC TO

For:

$\{ w_i(X) \}$

check:

$\{ TS_i < RTS(X)? \}$



or:

$\neg [TS_i < WTS(X)]$

If either yes:

TEXT

Abort

Otherwise:

TEXT

Write  
 $WTS(X) = TS_i$

Memory:

**WRITE checks RTS + WTS**

## 21. THOMAS WRITE RULE

For write:

**If**

$\neg [TS_i < RTS(X)]$

! abort.

**Else if**

$\neg [TS_i < WTS(X)]$

! ignore obsolete write.

**Else**

! execute write.

Memory:

**Thomas forgives old writes.**

## 22. DEADLOCK: LOCKING vs TIMESTAMP

TEXT

2PL

! transactions may wait  
! deadlock possible

Timestamp Ordering

! allow or abort  
! deadlock-free

But TO can cause:

TEXT

repeated aborts / starvation

## PART B — MASTER RELATIONSHIPS

### 23. SERIALIZABILITY SETS

$\boxed{\text{Serial} \subseteq \text{CSR} \subseteq \text{VSR}}$

### 24. RECOVERY SETS

$\boxed{\text{Strict} \subseteq \text{Cascadeless} \subseteq \text{Recoverable}}$

### 25. PROTOCOL RELATION

TEXT

2PL

!' C S R

!' V S R

Strict 2PL additionally gives:

TEXT

Strict

!' C a s c a d e l e s s

!' R e c o v e r a b l e

## PART C — HOW TO IDENTIFY QUESTION TYPE IN 5 SECONDS

If question gives:

TEXT

$r_1(X), w_2(X), \dots$

and asks serializability:

**Build precedence graph.**

If question includes:

TEXT

C1, C2, A1

and asks recoverability:

**Track who reads from whom + commit order.**

If question gives:

TEXT

S-lock / X-lock / unlock

**Think 2PL + compatibility.**

If question says:

TEXT

T1 waits for T2

**Wait-for graph !' deadlock.**

If question gives:

TEXT

TS(T1), RTS(X), WTS(X)

**Timestamp ordering.**

If question mentions:

TEXT

blind writes

**View serializability / Thomas Write Rule.**

## PART D — MIXED GATE PROBLEM 1

Schedule:

TEXT

r1(X)  
w1(X)  
r2(X)  
w2(X)  
C1  
C2

Find:

1. CSR?
2. Recoverable?
3. Cascadeless?
4. Strict?

---

## 26. CSR

Conflicts all go:

$\lceil T1 \rightarrow T2 \rceil$

No cycle.

So:

$\lceil \boxed{\text{CSR}} \rceil$

---

## 27. RECOVERABLE

T2 reads X from T1.

Commit order:

TEXT  
C1 before C2

So:

$\lceil \boxed{\text{Recoverable}} \rceil$

---

## 28. CASCADELESS

Did T1 commit before:

TEXT  
r2(X)

?

No.

So:

$\lceil \boxed{\text{Not Cascadeless}} \rceil$

---

## 29. STRICT

T2 reads and writes X before T1 commits.

So:

$\lceil \boxed{\text{Not Strict}} \rceil$

Final:

TEXT

```
C S R '
R e c o v e r a b l e '
C a s c a d e l e s s '
S t r i c t '
```

## PART E — MIXED PROBLEM 2

Schedule:

TEXT

```
w1(X)
r2(X)
C2
C1
```

### 30. CSR

Conflict:

$\lceil T1 \rightarrow T2 \rceil$

Acyclic.

So:

TEXT

```
C S R '
```

### 31. RECOVERABILITY

T2 reads from T1.

But:

TEXT

```
C2 before C1
```

So:

TEXT

```
R e c o v e r a b l e '
```

Therefore:

$\lceil \boxed{\text{CSR}} \text{ but } \text{Unrecoverable} \rceil$

This is a classic exam trap.

## PART F — MIXED PROBLEM 3

Schedule:

TEXT

$w_1(x)$   
 $w_2(x)$   
 $w_1(x)$

### 32. CSR

Edges:

TEXT

$T_1 \quad !' \quad T_2$   
 $T_2 \quad !' \quad T_1$

Cycle.

So:

TEXT

C S R   '

### 33. VSR

No reads.

Final writer:

TEXT

T1

Serial order:

TEXT

$T_2 \quad !' \quad T_1$

has same final writer.

Thus:

TEXT

V S R   '

Final:

$\boxed{\text{VSR} \text{ but not CSR }}$

## PART G — MIXED PROBLEM 4

Suppose:

TEXT

T1 holds X(A)  
T2 holds X(B)  
T1 requests X(B)  
T2 requests X(A)

Wait-for graph:

TEXT

T 1   !'   T 2  
T 2   !'   T 1

Cycle.

Therefore:

\[ \boxed{Deadlock} \]

## PART H — MIXED PROBLEM 5

Suppose:

TEXT

TS(T1)=5  
TS(T2)=10

T2 holds X.

T1 requests X.

Under Wait-Die:

TEXT

T1 older  
!'   w a i t s

Under Wound-Wait:

TEXT

T1 older  
!'   w o u n d s / a b o r t s   T 2

## PART I — MIXED PROBLEM 6

Given:

TEXT

TS(T1)=10  
RTS(X)=20

WTS(X)=5

T1 requests:

TEXT

w1(X)

Basic TO:

$\boxed{10 < 20}$

so:

$\boxed{\text{Abort } T1}$

Thomas rule also aborts because:

TEXT

TS < RTS

Thomas only forgives:

TEXT

TS < WTS

case.

---

## PART J — MIXED PROBLEM 7

---

Given:

TEXT

TS(T1)=10

RTS(X)=5

WTS(X)=20

T1 writes X.

Basic TO:

TEXT

TS < WTS

!' A b o r t

Thomas:

TEXT

TS < WTS

!' I g n o r e w r i t e

Classic distinction.

---



## PART K — MIXED PROBLEM 8

Transaction:

TEXT

S(A)  
X(B)  
unlock(A)  
S(C)

Does it obey 2PL?

After:

TEXT

unlock(A)

shrinking phase begins.

Then:

TEXT

S(C)

new lock acquired.

Violation.

$\boxed{\text{Not } 2\text{PL}}$

## PART L — MIXED PROBLEM 9

Schedule:

TEXT

w1(X)  
C1  
r2(X)  
w2(X)  
C2

Recoverable?

TEXT

Yes

Cascadeless?

TEXT

Yes

Strict?

After w1(X), nobody touches X before C1.

TEXT

Yes

Therefore:

$\boxed{\text{Strict}}$

and automatically cascadeless and recoverable.

## PART M — MIXED PROBLEM 10

Graph:

TEXT

|     |    |     |
|-----|----|-----|
| T 1 | !' | T 2 |
| T 1 | !' | T 3 |
| T 2 | !' | T 4 |
| T 3 | !' | T 4 |

Cycle?

No.

Therefore schedule is CSR.

Possible serial orders:

TEXT

T1, T2, T3, T4

or:

TEXT

T1, T3, T2, T4

## PART N — GATE TRAPS MASTER LIST

### & pTrap 1

RR is not a conflict.

### & pTrap 2

Cycle in precedence graph means:

TEXT

Not CSR

not necessarily:

TEXT

Not VSR

### & pTrap 3

Cycle in wait-for graph means:

TEXT

Dead Lock

### & pTrap 4

Recoverable does not mean cascadeless.

### & pTrap 5

Cascadeless does not mean strict.

### & pTrap 6

Strict implies both.

### & pTrap 7

CSR does not automatically imply recoverable.

### & pTrap 8

2PL guarantees CSR, not deadlock freedom.

### & pTrap 9

Strict 2PL holds X-locks till commit.

### & pTrap 10

Rigorous 2PL holds all locks till commit.

### & pTrap 11

Conservative 2PL is deadlock-free.

## & pTrap 12

Wait-Die:

TEXT

Old waits  
Young dies

## & pTrap 13

Wound-Wait:

TEXT

Old wounds  
Young waits

## & pTrap 14

Read operation in TO checks only WTS.

## & pTrap 15

Write operation checks RTS and WTS.

## & pTrap 16

Thomas changes only obsolete-write case.

## & pTrap 17

Smaller timestamp means older.

## PART O — MASTER ONE-PAGE NOTEBOOK SHEET

### SERIALIZABILITY

TEXT

Conflict:  
Different Tx  
Same item  
One write

TEXT

RR no  
RW yes

WR yes  
WW yes

TEXT

Precedence graph:  
e a r l i e r   c o n f l i c t   ! '   l a t e r   c o n f l i c t

TEXT

C y c l e   ! '   N o t   C S R  
N o   c y c l e   ! '   C S R

## VIEW SERIALIZABILITY

TEXT

IRF

Initial Read  
Read-from  
Final Write

TEXT

C S R   ! '   V S R  
V S R   ! >   C S R

Blind writes create difference.

## RECOVERY

TEXT

Recoverable:  
Cwriter < Creader

Cascadeless:  
Cwriter < Read

Strict:  
writer finish before another R/W

Hierarchy:

TEXT

Strict  
!"  
Cascadeless  
!"  
Recoverable

## 2PL

#### TEXT

Growing:  
Locks only

Shrinking:  
Unlocks only

#### TEXT

After first unlock  
!' n o n e w l o c k

#### TEXT

2 P L !' C S R  
Deadlock possible

## 2PL VARIANTS

#### TEXT

Strict:  
X locks till end

Rigorous:  
All locks till end

Conservative:  
All locks before start  
!' d e a d l o c k - f r e e

## DEADLOCK

#### TEXT

Wait-for graph:  
W a i t e r !' H o l d e r

#### TEXT

C y c l e !' D e a d l o c k

#### TEXT

Wait-Die:  
Old waits  
Young dies

Wound-Wait:  
Old wounds  
Young waits

## TIMESTAMP ORDERING

#### TEXT

READ:

TS < WTS

!' A b o r t

#### TEXT

WRITE:

TS < RTS

!' A b o r t

TS < WTS

!' B a s i c : A b o r t

!' T h o m a s : I g n o r e

## 30-SECOND REVISION

#### TEXT

CSR:

graph + no cycle

VSR:

IRF

Recoverable:

C before C

Cascadeless:

C before R

Strict:

finish before R/W

2PL:

grow then shrink

Strict 2PL:

X locks till commit

Rigorous:

all locks till commit

Conservative:

all locks before start

Wait-for graph:

w a i t e r !' h o l d e r

c y c l e !' d e a d l o c k

Wait-Die:

old waits, young dies

Wound-Wait:

old wounds, young waits

Timestamp:

Read checks WTS

Write checks RTS + WTS

Thomas:

o b s o l e t e w r i t e !' i g n o r e

---

## CHECK YOUR UNDERSTANDING — MIXED

---

### Q1

Schedule:

TEXT

r1(X)  
r2(X)  
w1(X)  
w2(X)

Is it conflict serializable?

---

### Q2

Schedule:

TEXT

w1(X)  
r2(X)  
C1  
C2

Classify:

1. Recoverable?
  2. Cascadeless?
  3. Strict?
- 

### Q3

Which graph detects deadlock?

A. Precedence graph B. Wait-for graph

---

### Q4

Which graph detects conflict serializability?

---

### Q5

Under Wait-Die:

TEXT

Older requests a resource held by younger

What happens?

---

### Q6

Under Wound-Wait:

---



TEXT

Younger requests a resource held by older

What happens?

## Q7

Given:

TEXT

$TS(T1)=10$   
 $RTS(X)=5$   
 $WTS(X)=20$

T1 writes X.

What happens under:

1. Basic TO
2. Thomas Write Rule

## Q8

True or False:

**Every strict schedule is recoverable.**

## Q9

True or False:

**Every conflict-serializable schedule follows 2PL.**

## Q10

Complete:

$\{ \text{Serial} \subseteq \text{seq} \subseteq \text{2PL} \subseteq \text{CR} \subseteq \text{Serializable} \}$

We have now completed the full **Transactions + Concurrency Control** portion.

Next we can move into the **final DBMS consolidation phase**:

## TOPIC 23 — COMPLETE DBMS MASTER REVISION

This will combine the entire syllabus into:

**ER Model !' Relational Model !' RA !' TRC !' SQL !' Integrity Constraints !' FD/Keys !' Normalization !' File Organization !' Indexing !' B/B+ Trees !' Transactions !' Serializability !' Recovery !' Concurrency Control**

with:

- master formula sheet

- comparison tables
- GATE traps
- last-day revision notes
- high-frequency question patterns
- mixed conceptual questions
- mixed numerical problems
- weak-area checklist

Type **`NEXT`**.

USER

NEXT

ASSISTANT

## TOPIC 23 — COMPLETE DBMS MASTER REVISION

This is the **full GATE DBMS consolidation sheet** based on the syllabus and the sequence we completed.

Use this as:

- final revision notes
- last-day sheet
- formula sheet
- trap sheet
- question-recognition guide

## 1. ER MODEL — MASTER REVISION

### Entity

A real-world object.

Examples:

TEXT

Student  
Employee  
Course  
Department

### Entity Set

Collection of similar entities.

TEXT

STUDENT  
EMPLOYEE

## Attributes

Types:

### TEXT

Simple  
Composite  
Single-valued  
Multivalued  
Derived  
Key

## Examples

### TEXT

Name !' simple  
Address(Street, City, PIN)  
!' composite  
PhoneNumbers  
!' multivalued  
Age from DOB  
!' derived

## 2. ER NOTATION

### TEXT

|                  |                             |
|------------------|-----------------------------|
| Rectangle        | !' Entity                   |
| Ellipse          | !' Attribute                |
| Diamond          | !' Relationship             |
| Underlined attr  | !' Key                      |
| Double ellipse   | !' Multivalued              |
| Dashed ellipse   | !' Derived                  |
| Double rectangle | !' Weak entity              |
| Double diamond   | !' Identifying relationship |

## 3. CARDINALITY

### TEXT

1 : 1  
1 : N  
N : 1  
M : N

Memory:

Cardinality tells how many entities can participate.

## 4. PARTICIPATION

TEXT

Total  
Partial

Total:

**Every entity must participate.**

Partial:

**Participation optional.**

## 5. WEAK ENTITY

Weak entity:

- cannot be uniquely identified by its own attributes alone
- depends on owner entity
- has partial key
- uses identifying relationship

Key often becomes:

TEXT

Owner Primary Key  
+  
Partial Key

## 6. ER !' RELATIONAL MAPPING

### Strong entity

TEXT

One entity !' one relation

### Composite attribute

Store components.

### Multivalued attribute

Create separate relation.

### 1:N

Put key of **1-side** into **N-side** as foreign key.

Memory:

1:N !' FK goes to N

## M:N

Create separate relationship relation.

## Weak entity

Store:

TEXT

Owner key + partial key + weak attributes

## 7. RELATIONAL MODEL

A relation is conceptually a table.

Terms:

TEXT

R e l a t i o n            !'   T a b l e  
T u p l e                !'   R o w  
A t t r i b u t e        !'   C o l u m n  
D o m a i n             !'   A l l o w e d   v a l u e s

## 8. DEGREE vs CARDINALITY

$\boxed{\text{Degree} = \text{Number of attributes}}$

$\boxed{\text{Cardinality} = \text{Number of tuples}}$

Memory:

TEXT

D e g r e e   !'   c o l u m n s  
C a r d i n a l i t y   !'   r o w s

## 9. SCHEMA vs INSTANCE

### Schema

Structure/design.

Example:

TEXT

STUDENT(RollNo, Name, Dept)

## Instance

Actual rows currently stored.

Memory:

**Schema changes rarely, instance changes often.**

## 10. KEYS MASTER TABLE

| Key           | Meaning                            |
|---------------|------------------------------------|
| Superkey      | Uniquely identifies tuple          |
| Candidate key | Minimal superkey                   |
| Primary key   | Chosen candidate key               |
| Alternate key | Candidate key not chosen           |
| Composite key | Multiple attributes                |
| Foreign key   | References key in another relation |

## 11. PRIME ATTRIBUTE

Attribute belonging to **at least one candidate key**.

Non-prime:

**Attribute belonging to no candidate key.**

Very important for 2NF/3NF.

## 12. INTEGRITY CONSTRAINTS

TEXT

Domain constraint  
Key constraint  
Entity integrity  
Referential integrity

### Entity Integrity

Primary-key value cannot be NULL.

## Referential Integrity

Foreign-key value must either:

- match referenced key
- or be NULL if NULL is allowed

## 13. RELATIONAL ALGEBRA — OPERATORS

Basic:

TEXT

```
σ Selection
π Projection
⋈ Union
− Difference
× Cartesian Product
ρ Rename
```

Derived:

TEXT

```
∩ Intersection
⋈ Join
÷ Division
```

## 14. SELECTION

$\sigma_{\text{condition}}(R)$

Chooses rows.

Does not normally change number of columns.

## 15. PROJECTION

$\pi_{\text{attributes}}(R)$

Chooses columns.

Relational algebra removes duplicate tuples.

## 16. CARTESIAN PRODUCT

If:

TEXT

```
|R| = m
|S| = n
```

then:

$$\boxed{|R \times S| = mn}$$

Degree:

$$\boxed{\deg(R \times S) = \deg(R) + \deg(S)}$$

## 17. NATURAL JOIN DEGREE

If:

TEXT

$\deg(R) = p$   
 $\deg(S) = q$

and they share  $k$  common attributes:

$$\boxed{\deg(R \bowtie S) = p + q - k}$$

## 18. DIVISION

Division answers:

**FOR ALL / EVERY**

Example:

**Students who completed all compulsory courses.**

Memory:

TEXT

ALL  
 EVERY  
 !' D I V I S I O N / d o u b l e N O T E X I S T S

## 19. RA OPTIMIZATION SHORTCUT

Push selections downward:

TEXT

⋈

as early as possible.

Why?

Reduces intermediate relation size.

## 20. TRC



General form:

$\forall t \mid P(t)$

TRC is:

**Declarative.**

RA is:

**Procedural.**

## 21. QUANTIFIERS

TEXT

" !' T h e r e e x i s t s  
" !' F o r a l l

Very useful equivalence:

$\forall x (P(x) \rightarrow Q(x)) \equiv \neg \exists x (P(x) \wedge \neg Q(x))$

Memory:

**ALL = No counterexample**

## 22. FREE vs BOUND VARIABLE

A variable under:

TEXT

"  
"

is bound.

Others can be free/output variables.

## 23. SAFE TRC

Safe expressions return finite results based on database values.

Unsafe example:

TEXT

$\{t \mid \text{NOT STUDENT}(t)\}$

could refer to an unbounded universe.

## 24. RA vs TRC

For safe expressions:

`[ \boxed{RA} and \ TRC have equivalent expressive power ]`

## 25. SQL — LOGICAL PROCESSING ORDER

Important sequence:

```
TEXT
FROM
WHERE
GROUP BY
HAVING
SELECT
DISTINCT
ORDER BY
```

Mnemonic:

**FWGHSDO**

## 26. WHERE vs HAVING

```
TEXT
WHERE
!' filters rows before grouping

HAVING
!' filters groups after grouping
```

## 27. AGGREGATES + NULL

Functions:

```
TEXT
COUNT
SUM
AVG
MIN
MAX
```

Important:

```
TEXT
COUNT ( * ) !' counts rows

COUNT(column)
!' ignores NULLs
```

Most aggregate functions ignore NULLs.

## 28. SQL JOIN TYPES

TEXT

INNER JOIN  
LEFT OUTER JOIN  
RIGHT OUTER JOIN  
FULL OUTER JOIN  
CROSS JOIN  
NATURAL JOIN  
SELF JOIN

## 29. OUTER JOIN TRAP

Condition in:

TEXT

ON

VS:

TEXT

WHERE

can produce different results after outer join.

WHERE may eliminate NULL-extended rows.

## 30. EXISTS

SQL

EXISTS(subquery)

TRUE if subquery returns at least one row.

TEXT

EXISTS(empty)  
!' F A L S E

NOT EXISTS(empty)  
!' T R U E

## 31. ANY / ALL SHORTCUTS

For a non-empty, non-NULL set:

$\forall [x > \text{ANY}(S)] \text{ iff } x > \text{MIN}(S)$

$\forall x < \text{ANY}(S) \iff x < \text{MAX}(S)$

$\forall x > \text{ALL}(S) \iff x > \text{MAX}(S)$

$\forall x < \text{ALL}(S) \iff x < \text{MIN}(S)$

Also:

TEXT

= ANY "H I N

< > ALL "H N O T I N

with NULL caveats.

## 32. NULL — SQL THREE-VALUED LOGIC

SQL uses:

TEXT

TRUE

FALSE

UNKNOWN

Example:

SQL

X = NULL

does not behave like normal equality.

Use:

SQL

X IS NULL

## 33. NOT IN + NULL TRAP

Suppose subquery returns:

TEXT

1, 2, NULL

Then:

SQL

x NOT IN (...)

may become UNKNOWN for values not matching 1 or 2.

This is a classic GATE trap.

`NOT EXISTS` is often safer conceptually.

---

## 34. SET OPERATIONS

```
TEXT
UNION
UNION ALL
INTERSECT
EXCEPT
```

`UNION` removes duplicates.

`UNION ALL` keeps duplicates.

Relations/queries must be union compatible.

---

## 35. DDL / DML

### DDL

```
TEXT
CREATE
ALTER
DROP
TRUNCATE
```

### DML

```
TEXT
INSERT
UPDATE
DELETE
```

`SELECT` often called DQL.

---

## 36. DELETE vs TRUNCATE vs DROP

```
TEXT
DELETE
!' removes selected / all rows
!' table remains

TRUNCATE
!' removes all rows quickly
!' table remains

DROP
!' removes table object
```

Transaction/rollback behavior can vary by DBMS implementation, so follow question assumptions.

---

## 37. FUNCTIONAL DEPENDENCY

$X \rightarrow Y$

means:

If two tuples agree on X, they must agree on Y.

X:

TEXT

Determinant

Y:

TEXT

Dependent

## 38. TRIVIAL FD

$X \rightarrow Y$

is trivial when:

$Y \subseteq X$

## 39. COMPLETELY NONTRIVIAL

$X \cap Y = \emptyset$

## 40. ARMSTRONG AXIOMS

Mnemonic:

RAT

TEXT

Reflexivity  
Augmentation  
Transitivity

## 41. DERIVED FD RULES

TEXT

Union  
Decomposition  
Pseudotransitivity

## 42. ATTRIBUTE CLOSURE

To compute  $\overline{X^+}$ :

TEXT

1. Start with X
2. Apply FDs whose LHS is contained in closure
3. Add RHS
4. Repeat until no change

If:

$\overline{X^+} = R$

then X is a superkey.

## 43. CANDIDATE KEY

Candidate key:

TEXT

Superkey  
+  
Minimal

Shortcut:

**Attributes never appearing on RHS often must appear in every candidate key.**

But always verify using closure.

## 44. SUPERKEY COUNT SHORTCUT

If there is exactly one candidate key of size  $k$  in a relation of  $n$  attributes:

$\boxed{2^{n-k}}$

superkeys contain that candidate key.

Do not blindly use if multiple candidate keys exist.

## 45. MINIMAL / CANONICAL COVER

Conditions:

TEXT

S !' Single RHS  
E !' no Extraneous attribute  
R !' no Redundant FD

Mnemonic:

## SER

Algorithm:

### TEXT

Split RHS  
Remove extraneous LHS attributes  
Remove redundant FDs  
Optionally combine same LHS

## 46. NORMAL FORM HIERARCHY

$\boxed{BCNF \rightarrow 3NF \rightarrow 2NF \rightarrow 1NF}$

## 47. 1NF

**Atomic values.**

Memory:

**One value per cell**

## 48. 2NF

Requires:

1. 1NF
2. No non-prime attribute partially dependent on a candidate key.

Partial dependency:

**dependency on proper subset of composite candidate key.**

## GATE SHORTCUT

If every candidate key is single attribute:

**Partial dependency is impossible.**

So a 1NF relation automatically satisfies 2NF.

But it may still violate 3NF.

## 49. 3NF — EXACT TEST



For every nontrivial FD:

$\forall X \rightarrow A$

3NF holds if:

$\forall \{ X \text{ is superkey OR } A \text{ is prime} \}$

Memory:

**3NF = Superkey OR Prime**

## 50. BCNF

For every nontrivial:

$\forall X \rightarrow Y$

require:

$\forall \{ X \text{ is a superkey} \}$

No prime-attribute exception.

## 51. 3NF vs BCNF

| 3NF                                                   | BCNF                                |
|-------------------------------------------------------|-------------------------------------|
| X superkey OR RHS prime                               | X must be superkey                  |
| Less strict                                           | More strict                         |
| Dependency-preserving decomposition always obtainable | Dependency preservation may be lost |

## 52. CLASSIC 3NF BUT NOT BCNF

Relation:

$R(A,B,C)$

FDs:

$AB \rightarrow C$

$C \rightarrow B$

Candidate keys:

TEXT

AB  
AC

All attributes prime.

$C \nrightarrow B$

- C not superkey !' violates BCNF
- B prime !' satisfies 3NF

Thus:

$\boxed{3NF \text{ but not } BCNF}$

## 53. HIGHEST NORMAL FORM FAST METHOD

TEXT

1. Find candidate keys
2. Mark prime/non-prime
3. Check partial dependency !' 2NF
4. Check X superkey OR RHS prime !' 3NF
5. Check every determinant superkey !' BCNF

## 54. DECOMPOSITION

Good decomposition wants:

TEXT

Lossless Join  
Dependency Preservation

## 55. BINARY LOSSLESS TEST

For:

TEXT

$R \rightarrow R_1, R_2$

Let:

$X = R_1 \cap R_2$

Lossless iff:

$\boxed{X \rightarrow R_1 \text{ OR } X \rightarrow R_2}$

in  $(F^+)$ .

Memory:

**Common attributes must determine one side.**

## 56. DEPENDENCY PRESERVATION

Dependencies can be enforced using individual decomposed relations without joining them.

Formally:

$\bigcup F_i = F$

## 57. BCNF DECOMPOSITION

For violating FD:

$X \rightarrow Y$

decompose into roughly:

$R_1 = X \cup Y$

$R_2 = R - (Y - X)$

Repeat.

BCNF decomposition:

TEXT

Lossless  
Dependency preserving! not always

## 58. 3NF SYNTHESIS

Typical algorithm:

TEXT

1. Find minimal cover
2. Create relation for FD groups
3. If no relation contains a candidate key, add key relation
4. Remove redundant subset relations

Standard result:

TEXT

3NF  
Lossless  
Dependency preserving

## 59. FILE ORGANIZATION

Storage:

TEXT

Table  
! Records  
! Blocks / Pages  
! Disk

I/O occurs primarily in blocks/pages.

## 60. BLOCKING FACTOR — UNSPANNED

Given:

TEXT

B = block size  
R = record size

$$\lceil \boxed{bfr = \left\lfloor \frac{BR}{r} \right\rfloor} \rceil$$

## 61. NUMBER OF BLOCKS — UNSPANNED

If:

TEXT

r = number of records

$$\lceil \boxed{b = \left\lceil \frac{r}{bfr} \right\rceil} \rceil$$

Memory:

Fit !' FLOOR

Need !' CEILING

## 62. SPANNED FILE

If records may cross block boundaries and overhead ignored:

$$\lceil \boxed{b = \left\lceil \frac{rR}{B} \right\rceil} \rceil$$

## 63. UNUSED SPACE PER FULL BLOCK

$$\lceil \boxed{B - bfr \cdot R} \rceil$$

## 64. FILE ORGANIZATION TYPES

### Heap

TEXT

Fast inserts  
Equality search without index !' linear

Average successful search:

$$\lceil \approx b/2 \rceil$$

Unsuccessful:

$\lceil b \rceil$

## Ordered/Sequential

TEXT

Good range access  
Insertion more expensive

Binary block search approximately:

$\lceil \lceil \log_2 b \rceil \rceil$

## Hash

TEXT

Excellent equality  
Poor ranges

## 65. INDEX BASICS

Index entry:

$\langle \text{SearchKey}, \text{Pointer} \rangle$

Search key need not be candidate key.

## 66. PRIMARY INDEX

Data physically ordered on a unique/key field.

Typically:

TEXT

Sparse  
One index entry per data block

## 67. CLUSTERING INDEX

Data physically ordered on non-key field.

Repeated values clustered together.

## 68. SECONDARY INDEX

Index on non-ordering field.

Typically:

TEXT

Dense

Can have many secondary indexes.

## 69. DENSE vs SPARSE

### Dense

Entry for every record/search-key occurrence as required.

### Sparse

Only some entries, often one per data block.

Requires ordered data for straightforward searching.

## 70. INDEX ENTRY SIZE

$$R_i = \text{KeySize} + \text{PointerSize}$$

## 71. INDEX BLOCKING FACTOR

$$bfr_i = \left\lfloor \frac{B(R_i)}{R_i} \right\rfloor$$

## 72. INDEX BLOCKS

$$b_i = \left\lceil \frac{\text{NumberOfIndexEntries}}{bfr_i} \right\rceil$$

## 73. MULTILEVEL INDEX

Repeat:

$$\text{NextLevelBlocks} = \left\lceil \frac{\text{CurrentBlocks}}{\text{fanout}} \right\rceil$$

until:

TEXT

1 root block

Search roughly:

TEXT

index levels + data block

depending on caching/storage layout.

## 74. B-TREE — CORE

Using convention:

$m = \text{maximum children}$

Then:

$\boxed{\text{MaxChildren} = m}$

$\boxed{\text{MaxKeys} = m - 1}$

Non-root:

$\boxed{\text{MinChildren} = \left\lceil \frac{m}{2} \right\rceil}$

$\boxed{\text{MinKeys} = \left\lceil \frac{m}{2} \right\rceil - 1}$

Root is special.

All leaves same depth.

## 75. B-TREE HEIGHT FORMULAS

If height  $(h)$  = number of edges root to leaf:

Maximum keys:

$\boxed{N_{\max} = m^{h+1} - 1}$

Let:

$t = \left\lceil \frac{m}{2} \right\rceil$

Minimum keys for  $(h \geq 1)$ :

$\boxed{N_{\min} = 2t^h - 1}$

## 76. B-TREE INTERNAL BLOCK FORMULA

If internal node has:

TEXT

p child pointers  
p-1 keys

then:

$\boxed{pP + (p-1)K \leq B}$

Largest integer p gives max fan-out under that layout.

## & B-TREE GOLDEN TRAP

Always verify what the question means by:

order

It may mean:

TEXT  
max children

or sometimes:

TEXT  
max keys

Never assume blindly.

---

## 77. B+ TREE

---

Internal nodes:

TEXT  
separator keys + child pointers

Leaves:

TEXT  
search keys + record pointers

Leaves are linked.

Memory:

TEXT  
Internal = GUIDE  
Leaf = DATA POINTERS  
Leaves = LINKED

---

## 78. B+ TREE RANGE QUERY

---

TEXT  
Find first leaf  
!' follow leaf links

Excellent for:

TEXT  
BETWEEN  
<  
>  
range scans  
ordered traversal

---



## 79. B-TREE vs B+ TREE

| B-Tree                               | B+ Tree                             |
|--------------------------------------|-------------------------------------|
| Record pointers may occur internally | Record pointers primarily at leaves |
| Search may stop internally           | Search generally reaches leaf       |
| Leaves not necessarily linked        | Leaves linked                       |
| Good range access                    | Excellent range access              |
| Lower fan-out generally              | Higher internal fan-out generally   |

## 80. B+ INTERNAL FAN-OUT

Usually:

$$\lceil \frac{pP+(p-1)K}{B} \rceil$$

## 81. B+ LEAF CAPACITY

If:

TEXT

q entries

Key K

Record pointer Pr

Next-leaf pointer Pn

then:

$$\lceil \frac{q(K+P_r)+P_n}{B} \rceil$$

## 82. B+ TREE HEIGHT METHOD

First leaves:

$$\lceil L_0 = \left\lceil \frac{N}{q} \right\rceil \rceil$$

Then:

$$\lceil L_1 = \left\lceil \frac{L_0}{f} \right\rceil \rceil$$

repeat until 1 root.

This iterative method is safest.

## 83. TRANSACTION

Logical unit of database work.

Operations:

TEXT

ri(X)  
wi(X)  
Ci  
Ai

## 84. ACID

TEXT

A !' Atomicity !' all or nothing  
C !' Consistency !' valid state  
I !' Isolation !' concurrent correctness  
D !' Durability !' commit survives failure

Memory:

**All Correct Is Durable**

## 85. TRANSACTION STATES

TEXT

Active  
!' Partially Committed  
!' Committed

Failure:

TEXT

Active  
!' Failed  
!' Aborted

## 86. ANOMALIES

### Lost Update

One update overwritten.

### Dirty Read

Read uncommitted value.

### Unrepeatable Read

Same row read twice, different committed value.

## Incorrect Summary

Aggregate sees mixture of old/new state.

## Phantom

Repeated predicate query returns changed set of rows.

## 87. SERIAL vs SERIALIZABLE

TEXT

Serial

! ' a c t u a l l y o n e t r a n s a c t i o n a f t e r a n o t h e r

Serializable

! ' m a y i n t e r l e a v e b u t e q u i v a l e n t t o s e r i a l

## 88. CONFLICT RULE

Operations conflict iff:

TEXT

Different transactions

Same item

At least one WRITE

Table:

TEXT

|   |   |   |       |
|---|---|---|-------|
| R | R | ! | n o   |
| R | W | ! | y e s |
| W | R | ! | y e s |
| W | W | ! | y e s |

## 89. PRECEDENCE GRAPH

Node:

TEXT

Transaction

Edge:

TEXT

Earlier conflicting transaction

! ' l a t e r t r a n s a c t i o n

Main result:

$\neg \boxed{\text{NoCycle} \iff \text{ConflictSerializable}}$

Equivalent serial order:

**Topological order.**

## 90. VIEW SERIALIZABILITY

Preserve:

TEXT

```
I !'   I n i t i a l   r e a d s
R !'   R e a d - f r o m
F !'   F i n a l   w r i t e s
```

Mnemonic:

**IRF**

Relationship:

$\boxed{CSR \rightarrow VSR}$

but not reverse.

Blind writes can produce:

TEXT

VSR but not CSR

## 91. RECOVERABILITY

If  $T_j$  reads from  $T_i$ :

$\boxed{C_i < C_j}$

Writer commits before reader commits.

## 92. CASCADELESS

If  $T_j$  reads from  $T_i$ :

$\boxed{C_i < r_j(X)}$

Writer commits before reader reads.

No dirty reads.

## 93. STRICT

After:

$\boxed{w_i(X)}$

no other transaction may read/write X until Ti commits/aborts.

Prevents:

TEXT

dirty reads  
dirty writes

Hierarchy:

$\boxed{\text{Strict} \rightarrow \text{Cascadeless} \rightarrow \text{Recoverable}}$

## 94. LOCKS

Shared:

TEXT

S-lock  
!' read

Exclusive:

TEXT

X-lock  
!' write

Compatibility:

|   | S | X |
|---|---|---|
| S | ' | ' |
| X | ' | ' |

Memory:

Only S-S compatible

## 95. BASIC 2PL

TEXT

Growing:  
acquire  
no release

Shrinking:  
release  
no new acquisition

Main result:

$\boxed{2PL \rightarrow CSR}$

Deadlock possible.

## 96. 2PL VARIANTS

TEXT

Strict 2PL

!' hold X - locks till commit / abort

Rigorous 2PL

!' hold S and X locks till commit / abort

Conservative 2PL

!' acquire all locks before execution

!' deadlock - free

## 97. DEADLOCK CONDITIONS

TEXT

Mutual exclusion

Hold and wait

No preemption

Circular wait

All four needed.

## 98. WAIT-FOR GRAPH

Edge:

TEXT

Waiter !' Holder

Cycle:

$\boxed{\text{Deadlock}}$

## 99. PRECEDENCE vs WAIT-FOR

TEXT

Precedence Graph

!' Serializability

Wait-For Graph

!' Deadlock

Do not mix them.

## 100. WAIT-DIE

Smaller timestamp = older.

TEXT

```
Old requester !' WAIT
Young requester !' DIE
```

Memory:

**Old waits, young dies**

## 101. WOUND-WAIT

TEXT

```
Old requester !' WOUND younger
Young requester !' WAIT
```

Memory:

**Old wounds, young waits**

## 102. TIMESTAMP ORDERING

Each item has:

TEXT

```
RTS(X)
WTS(X)
```

## 103. TO READ RULE

For:

$\forall r_i(X) \setminus$

if:

$\forall \boxed{TS_i < WTS(X)} \setminus$

!' abort.

Otherwise:

$\forall RTS(X) = \max(RTS(X), TS_i) \setminus$

Memory:

**READ checks W**

---

## 104. TO WRITE RULE

---

For:

$\forall [w_i(X)]$

basic TO:

If:

$\forall [TS_i < RTS(X)]$

or:

$\forall [TS_i < WTS(X)]$

!' abort.

Otherwise execute.

Memory:

**WRITE checks R + W**

---

## 105. THOMAS WRITE RULE

---

For write:

TEXT

$TS < RTS$

!' A b o r t

$TS < WTS$

!' I g n o r e o b s o l e t e w r i t e

Otherwise

!' E x e c u t e

Memory:

**Thomas forgives obsolete writes.**

---

## 106. TO PROPERTIES

---

Basic timestamp ordering:

TEXT

Deadlock-free

Serializable in timestamp order

Repeated abort/starvation possible

---



## MASTER FORMULA SHEET

### File

$\lfloor bfr = \left\lfloor \frac{BR}{r} \right\rfloor \rfloor$

$\lfloor b = \left\lceil \frac{r}{bfr} \right\rceil \rfloor$

Spanned:

$\lfloor b = \left\lceil \frac{rR}{B} \right\rceil \rfloor$

### Index

$\lfloor R_i = \text{Key} + \text{Pointer} \rfloor$

$\lfloor bfr_i = \left\lfloor \frac{B}{R_i} \right\rfloor \rfloor$

$\lfloor b_i = \left\lceil \frac{\text{entries}}{bfr_i} \right\rceil \rfloor$

### B-Tree

$\lfloor \text{MaxKeys} = m - 1 \rfloor$

$\lfloor \text{MinChildren} = \left\lceil \frac{m}{2} \right\rceil \rfloor$

$\lfloor \text{MinKeys} = \left\lceil \frac{m}{2} \right\rceil - 1 \rfloor$

$\lfloor N_{\text{max}} = m^{h+1} - 1 \rfloor$

$\lfloor N_{\text{min}} = 2 \left( \left\lceil \frac{m}{2} \right\rceil \right)^{h-1} \rfloor$

### B+ Leaf Count

$\lfloor \text{Leaves} = \left\lceil \frac{N_q}{n} \right\rceil \rfloor$

## MASTER GATE SHORTCUT SHEET

#### TEXT

1:N ER mapping

! ' F K goes to N

ALL / EVERY

! ' D i v i s i o n / N O T E X I S T S

3NF

! ' S u p e r k e y O R P r i m e

BCNF

! ' S u p e r k e y O N L Y

Lossless binary decomposition

! ' C o m m o n a t t r i b u t e s d e t e r m i n e o n e s i d e

Records/block

!' F L O O R

Blocks needed

!' C E I L I N G

Primary index

!' t y p i c a l l y   s p a r s e

Secondary index

!' t y p i c a l l y   d e n s e

B-tree

!' B r o a d   +   b a l a n c e d

B+

!' l e a v e s   l i n k e d

Conflict

!' D i f f e r e n t   T x   +   S a m e   I t e m   +   O n e   W r i t e

CSR

!' P r e c e d e n c e   g r a p h   +   N O   c y c l e

VSR

!' I R F

Recoverable

!' C   b e f o r e   C

Cascadeless

!' C   b e f o r e   R

Strict

!' F i n i s h   b e f o r e   R / W

2PL

!' G r o w   t h e n   s h r i n k

WFG

!' W a i t e r   !'   H o l d e r

Wait-Die

!' O l d   w a i t s ,   y o u n g   d i e s

Wound-Wait

!' O l d   w o u n d s ,   y o u n g   w a i t s

TO Read

!' c h e c k   W

TO Write

!' c h e c k   R   +   W

Thomas

!' o b s o l e t e   w r i t e   i g n o r e d

## TOP 20 GATE TRAPS

1. Search key need not be candidate key.
2. Projection removes duplicates in relational algebra.
3. SQL normally allows duplicates unless DISTINCT.

4. COUNT(column) ignores NULL.
5. NOT IN + NULL is dangerous.
6. Attributes never on RHS are useful candidate-key clues, not complete proof.
7. All attributes prime does not imply BCNF.
8. Single-attribute key guarantees no partial dependency, not necessarily 3NF.
9. BCNF decomposition may lose dependency preservation.
10. Primary index "primary key.
11. Sparse primary index entries often equal data blocks, not records.
12. B/B+ order definition must be checked.
13. Height vs number of levels can differ by 1.
14. RR is not a conflict.
15. Cycle in precedence graph means not CSR, not necessarily not VSR.
16. CSR does not imply recoverable.
17. Recoverable does not imply cascadeless.
18. 2PL can deadlock.
19. Cycle in WFG means deadlock; WFG is not precedence graph.
20. Thomas Rule ignores only obsolete write based on WTS; TS<RTS still aborts.

---

## LAST-DAYNORMALIZATION METHOD

---

Whenever you see an FD/normalization question:

TEXT

STEP 1

Find candidate keys

STEP 2

Mark prime and non-prime attributes

STEP 3

Check partial dependency

! ' 2 N F

STEP 4

For every X! ' A :

X superkey OR A prime?

! ' 3 N F

STEP 5

For every nontrivial X! ' Y :

X superkey?

! ' B C N F

STEP 6

If decomposition:

check lossless + dependency preservation

---

## LAST-DAYSERIALIZABILITY METHOD

---

Given schedule:

TEXT

STEP 1

Mark WRITES

STEP 2

For each same item:

RW / WR / WW conflicts

STEP 3

E a r l i e r ! ' l a t e r e d g e

STEP 4

Cycle?

Y E S ! ' N o t C S R

N O ! ' C S R

STEP 5

If needed:

topological order = serial order

If graph has cycle but VSR asked:

TEXT

Check blind writes

Check IRF

---

## LAST-DAYRECOVERY METHOD

---

If Tj reads from Ti:

### Recoverable

TEXT

Ci before Cj?

### Cascadeless

TEXT

Ci before rj?

### Strict

After wi(X):

TEXT

Any other R/W(X) before Ci/Ai?

If yes !' not strict.

---

## LAST-DAYTIMESTAMP METHOD

---

Write table:

| Item | RTS | WTS |
|------|-----|-----|
| X    | ... | ... |

Then:

TEXT

READ:

TS < WTS?

Y E S ! ' a b o r t

N O ! ' R T S = m a x

WRITE:

TS < RTS?

Y E S ! ' a b o r t

otherwise TS < WTS?

B a s i c ! ' a b o r t

T h o m a s ! ' i g n o r e

o t h e r w i s e ! ' w r i t e

## 30 HIGH-VALUE CONCEPTUAL QUESTIONS

Try these without looking back.

1. Can a search key contain duplicate values?
2. What is the difference between relation degree and cardinality?
3. Does relational algebra projection retain duplicates?
4. Which operator naturally expresses “for all”?
5. What does `COUNT(A)` do with NULLs?
6. What is the difference between WHERE and HAVING?
7. Why is `NOT IN` dangerous with NULL?
8. What is a candidate key?
9. What does  $\setminus(X^+=R)$  imply?
10. State the 3NF condition.
11. State the BCNF condition.
12. Can a 3NF relation fail BCNF?
13. Does BCNF decomposition always preserve dependencies?
14. State binary lossless-decomposition test.
15. What is the unspanned blocking-factor formula?
16. Why is primary index generally sparse?
17. Why is secondary index generally dense?
18. Why are B+ leaves linked?
19. What is the main difference between B-tree and B+ tree?
20. What is a transaction?
21. State ACID.
22. Which pairs conflict among RR/RW/WR/WW?
23. What does a cycle in precedence graph mean?

24. What does VSR preserve?
25. What is a blind write?
26. State `strict !0 cascade less !0 recoverable`
27. What does basic 2PL guarantee?
28. Does 2PL prevent deadlock?
29. What does an edge in WFG mean?
30. What is the difference between basic TO and Thomas Write Rule?
- 

## 20 MIXED GATE-STYLE PROBLEMS

---

### Q1 — Relational Algebra

---

If:

$$\forall [R]=20, \text{quad } |S|=30 \]$$

what is:

$$\forall [R \text{ times } S]? \]$$

---

### Q2 — Natural Join Degree

---

If:

TEXT

$$\begin{aligned} \deg(R) &= 5 \\ \deg(S) &= 7 \end{aligned}$$

and there are 2 common join attributes, find degree of natural join.

---

### Q3 — Candidate Key

---

For:

$$\forall [R(A,B,C,D) \]$$

FDs:

$$\forall [A \rightarrow B \]$$

$$\forall [B \rightarrow C \]$$

$$\forall [A \rightarrow D \]$$

find a candidate key.

---

### Q4 — 3NF

---

If:

$$\forall [X \rightarrow A \]$$

where X is not a superkey but A is prime, does this FD violate 3NF?

---

---

## Q5 — BCNF

---

Same Q4: does it violate BCNF?

---

## Q6 — Lossless

---

Decompose:

$\bowtie R(A,B,C) \bowtie$

into:

TEXT

R1(A, B)  
R2(B, C)

Given:

$\bowtie B \rightarrow C \bowtie$

Is decomposition lossless?

---

## Q7 — File Blocks

---

TEXT

Block size = 1000 bytes  
Record size = 120 bytes  
Records = 1000

Unspanned.

Find BFR and number of blocks.

---

## Q8 — Sparse Index

---

Data file contains:

TEXT

5000 data blocks

Primary sparse index has one entry per data block.

If index block stores 100 entries, find number of index blocks.

---

## Q9 — B-Tree

---

Order  $\bowtie(m=6)$ , using max-children convention.

Find:

---

TEXT

max keys  
min non-root children  
min non-root keys

## Q10 — B-Tree Capacity

Order:

$\lfloor m=4 \rfloor$

height:

$\lfloor h=2 \rfloor$

Find maximum keys.

## Q11 — B+ Tree

TEXT

100,000 records  
leaf capacity = 100  
fan-out = 50

Find number of leaves and total levels.

## Q12 — Conflict

Do:

TEXT

$r1(x)$   
 $r2(x)$

conflict?

## Q13 — CSR

Schedule:

TEXT

$r1(x)$   
 $r2(x)$   
 $w1(x)$   
 $w2(x)$

Construct edges and determine CSR.

## Q14 — Recoverability



TEXT

w1(X)  
r2(X)  
C1  
C2

Classify recoverable/cascadeless/strict.

### Q15 — Strictness

TEXT

w1(X)  
w2(X)  
C1  
C2

Can it be cascadeless but not strict?

### Q16 — 2PL

TEXT

S(A)  
X(B)  
unlock(A)  
S(C)

Does transaction obey 2PL?

### Q17 — Deadlock

TEXT

T1 waits for T2  
T2 waits for T3  
T3 waits for T1

Deadlock?

### Q18 — Wait-Die

T1 older, T2 younger.

T2 requests a resource held by T1.

What happens?

### Q19 — Timestamp Read

TEXT

TS(T1)=10  
WTS(X)=20

T1 requests `r1(X)` .

What happens?

## Q20 — Thomas Rule

TEXT

TS(T1)=10  
RTS(X)=5  
WTS(X)=20

T1 requests `w1(X)` .

What happens under Thomas Write Rule?

## ANSWER KEY — CHECK ONLY AFTER TRYING

TEXT

Q1 = 600  
Q2 = 10  
Q3 = A  
Q4 = No  
Q5 = Yes  
Q6 = Yes, because common B determines R2  
Q7 = BFR 8, blocks 125  
Q8 = 50  
Q9 = max keys 5, min children 3, min keys 2  
Q10 = 63  
Q11 = 1 0 0 0 leaves !' 2 0 parents !' 1 root !' 3 levels  
Q12 = No  
Q13 = T2!' T1 and T1!' T2 !' cycle !' not CSR  
Q14 = Recoverable yes; cascadeless no; strict no  
Q15 = Yes  
Q16 = No  
Q17 = Yes  
Q18 = Younger dies/aborts  
Q19 = Abort T1  
Q20 = Ignore obsolete write

## FINAL WEAK-AREA CHECKLIST

Before calling DBMS finished, you should be able to solve all of these without notes:

TEXT

%i ER cardinality + participation  
%i Weak entity mapping  
%i Keys and prime attributes  
%i RA query evaluation  
%i Division

```

%i   T R C   q u a n t i f i e r s
%i   G R O U P   B Y   /   H A V I N G
%i   C o r r e l a t e d   s u b q u e r i e s
%i   E X I S T S   /   N O T   E X I S T S
%i   A N Y   /   A L L
%i   N U L L   l o g i c
%i   A t t r i b u t e   c l o s u r e
%i   C a n d i d a t e   k e y s
%i   M i n i m a l   c o v e r
%i   H i g h e s t   n o r m a l   f o r m
%i   3 N F   v s   B C N F
%i   L o s s l e s s   d e c o m p o s i t i o n
%i   D e p e n d e n c y   p r e s e r v a t i o n
%i   B l o c k i n g - f a c t o r   n u m e r i c a l s
%i   P r i m a r y / s e c o n d a r y / d e n s e / s p a r s e   i n d e x e s
%i   M u l t i l e v e l - i n d e x   l e v e l s
%i   B - t r e e   o r d e r / h e i g h t   n u m e r i c a l s
%i   B +   i n t e r n a l / l e a f   c a p a c i t i e s
%i   A C I D
%i   A n o m a l i e s
%i   C o n f l i c t   g r a p h
%i   V i e w   s e r i a l i z a b i l i t y
%i   R e c o v e r a b i l i t y   h i e r a r c h y
%i   L o c k   c o m p a t i b i l i t y
%i   2 P L   v a r i a n t s
%i   W a i t - f o r   g r a p h s
%i   W a i t - D i e   /   W o u n d - W a i t
%i   T i m e s t a m p   o r d e r i n g
%i   T h o m a s   W r i t e   R u l e

```

## FINAL 60-SECOND DBMS REVISION

### TEXT

#### ER:

```

1 : N ! '   F K   o n   N
M : N ! '   s e p a r a t e   r e l a t i o n

```

#### Relational:

Degree=columns

Cardinality=rows

#### RA:

```

Ã   r o w s
À   c o l u m n s
÷   A L L

```

#### SQL:

FROM WHERE GROUP HAVING SELECT DISTINCT ORDER

#### FD:

C l o s u r e ! ' k e y s

#### 3NF:

Superkey OR Prime

#### BCNF:

Superkey only

#### Lossless:

Common determines one side

#### File:

BFR=floor(B/R)

Blocks=ceil(r/BFR)

Index:  
Primary sparse  
Secondary dense

B-tree:  
m a x k e y s = m" 1  
balanced

B+:  
data pointers at linked leaves

ACID:  
All Correct Is Durable

Conflict:  
same item + different Tx + one write

CSR:  
precedence graph acyclic

VSR:  
IRF

Recovery:  
S t r i c t ! ' C a s c a d e l e s s ! ' R e c o v e r a b l e

2PL:  
Grow then shrink  
!' C S R  
!' d e a d l o c k p o s s i b l e

WFG:  
W a i t e r ! ' H o l d e r  
cycle = deadlock

Wait-Die:  
old waits, young dies

Wound-Wait:  
old wounds, young waits

T0:  
Read checks W  
Write checks R+W

Thomas:  
obsolete write ignored

You have now completed the **full DBMS syllabus teaching + master revision**.

If you type **`NEXT`**, I'll move to the next useful phase:

## TOPIC 24 — DBMS GATE PRACTICE SET

I'll give you **GATE-style questions topic-by-topic without immediately revealing answers**, starting easy and progressing to difficult, so we can identify exactly where your DBMS weak areas remain.

USER

NEXT

## TOPIC 24 — DBMS GATE PRACTICE SET

---

Now we switch from **learning mode** to **exam-solving mode**.

I will not immediately reveal answers. First solve them yourself. After you respond, I can check your answers and explain only the mistakes.

We'll start with a **mixed set covering the complete DBMS syllabus**.

---

### SECTION A — RELATIONAL MODEL + KEYS

---

#### Q1

For relation:

$R(A, B, C, D)$

with FDs:

$A \rightarrow B$

$B \rightarrow C$

$A \rightarrow D$

Which of the following is a candidate key?

A. A B. B C. C D. AB

---

#### Q2

Suppose relation R has:

TEXT

8 attributes

and exactly one candidate key containing:

TEXT

3 attributes

How many superkeys does R have?

---

#### Q3

Which statement is correct?

A. Every superkey is a candidate key B. Every candidate key is a superkey C. Every foreign key is a candidate key D. Every primary key is a foreign key

---

### SECTION B — RELATIONAL ALGEBRA

---

#### Q4

Suppose:

$\forall |R|=20 \setminus$

$\forall |S|=30 \setminus$

What is:

$\setminus$

$R \times S$

$\setminus$

?

#### Q5

Suppose:

TEXT

$\deg(R) = 7$

$\deg(S) = 5$

R and S have 2 common attributes.

What is the degree of:

$\setminus R \bowtie S \setminus$

assuming natural join?

#### Q6

Which relational algebra operator is most naturally associated with queries containing:

“for all”

A. Selection B. Projection C. Division D. Cartesian product

## SECTION C — SQL

#### Q7

Consider:

SQL

```
SELECT COUNT(A)
FROM R;
```

Suppose R contains 10 rows, but A is NULL in 3 rows.

What is returned?

A. 10 B. 7 C. 3 D. NULL

---

### Q8

Which clause filters groups after aggregation?

A. WHERE B. HAVING C. GROUP BY D. ORDER BY

---

### Q9 — NULL Trap

Suppose a subquery returns:

TEXT

1  
2  
NULL

What important problem can occur with:

SQL

x NOT IN (subquery)

?

Explain in one line.

---

### Q10

Arrange the following in SQL logical processing order:

TEXT

SELECT  
WHERE  
FROM  
HAVING  
GROUP BY  
ORDER BY

---

## SECTION D — FUNCTIONAL DEPENDENCIES

---

### Q11

For:

$\{ R(A,B,C,D,E) \}$

and:

$F = \{ A \rightarrow B, B \rightarrow C, CD \rightarrow E \}$

find:

$\{ A^+ \}$

---

### Q12

For the same relation and FDs, is A a superkey?

---

---

**Q13**

Consider FD:

$\{ AB \rightarrow A \}$

Is this:

A. Trivial B. Non-trivial C. Completely non-trivial

---

**SECTION E — NORMALIZATION**

---

**Q14**

For every non-trivial FD:

$\{ X \rightarrow A \}$

3NF requires:

A. X must always be a candidate key B. X must be a superkey OR A must be prime C. A must be non-prime D. X must not be a superkey

---

**Q15**

BCNF requires:

A. Every RHS attribute must be prime B. Every determinant of a non-trivial FD must be a superkey C. Every candidate key must be single attribute D. No foreign keys are allowed

---

**Q16 — High-Value Question**

Given:

$\{ R(A,B,C) \}$

FDs:

$\{ AB \rightarrow C \}$

$\{ C \rightarrow B \}$

Candidate keys are:

TEXT

AB  
AC

What is the highest normal form?

A. 1NF B. 2NF C. 3NF D. BCNF

---

**SECTION F — DECOMPOSITION**

---

**Q17**

Relation:

---



$\lceil R(A,B,C) \rceil$

decomposed into:

TEXT

$R_1(A, B)$

$R_2(B, C)$

Given:

$\lceil B \rightarrow C \rceil$

Is the decomposition lossless?

---

### Q18

For binary decomposition into  $R_1$  and  $R_2$ , write the lossless join condition using:

$\lceil R_1 \cap R_2 \rceil$

---

## SECTION G — FILE ORGANIZATION

---

### Q19

Given:

TEXT

Block size = 1000 bytes

Record size = 120 bytes

Records = 1000

Assume unspanned records.

Find:

1. Blocking factor
  2. Number of blocks
- 

### Q20

For a heap file having:

TEXT

1000 blocks

approximately how many block accesses are needed on average for a successful linear search?

---

## SECTION H — INDEXING

---

### Q21

A file has:

---

TEXT

20,000 data blocks

A sparse primary index contains one entry per data block.

If each index block can hold:

TEXT

100 index entries

how many index blocks are required?

## Q22

Which statement is generally correct?

A. Primary index is normally dense B. Secondary index is normally sparse C. Primary index is commonly sparse D. Search keys must always be candidate keys

## Q23

What is the major difference between:

TEXT

Primary Index

and:

TEXT

Secondary Index

in terms of physical ordering of the data file?

## SECTION I — B-TREE

Assume:

**Order  $\lfloor(m)\rfloor$  = maximum number of children.**

## Q24

For B-tree order:

$\lfloor m=6 \rfloor$

find:

1. Maximum keys per node
2. Minimum children of a non-root internal node
3. Minimum keys of a non-root node

### Q25

A B-tree has:

$\lfloor m=4 \rfloor$

and height:

$\lfloor h=2 \rfloor$

where height means number of edges from root to leaf.

Find maximum possible number of keys.

---

### Q26

What is the most important thing you must verify when a GATE question says only:

**“B-tree of order 5”**

?

---

## SECTION J — B+ TREE

---

### Q27

Why are leaf nodes of a B+ tree linked?

A. To reduce root size B. To support efficient range/sequential access C. To prevent deadlock D. To support hashing

---

### Q28

A B+ tree stores:

TEXT

100,000 records  
Leaf capacity = 100 entries  
Internal fan-out = 50

Find:

1. Number of leaf nodes
  2. Number of nodes immediately above leaves
  3. Total number of tree levels
- 

### Q29

In a typical B+ tree, actual record pointers are primarily stored in:

A. Root B. Internal nodes C. Leaf nodes D. Every node equally

---

## SECTION K — TRANSACTIONS + ACID

---

### Q30

Which ACID property means:

**“Committed changes survive a system crash”?**

A. Atomicity B. Consistency C. Isolation D. Durability

### Q31

Which ACID property means:

**“Either all operations of a transaction occur or none of them”?**

### Q32

A transaction has executed its final operation, but commit has not yet been made fully durable.

What state is it in?

## SECTION L — CONCURRENCY ANOMALIES

### Q33

Schedule:

TEXT

r1(X)  
r2(X)  
w1(X)  
w2(X)

Both transactions modify X based on what they read.

Which anomaly is likely?

### Q34

Schedule:

TEXT

w1(X)  
r2(X)  
A1

What anomaly occurs?

### Q35

Schedule:

TEXT

r1(X)  
w2(X)  
C2  
r1(X)

What anomaly can T1 experience?

## SECTION M — CONFLICT SERIALIZABILITY

### Q36

Which pair does **not** conflict?

A. r1(X), w2(X) B. w1(X), r2(X) C. w1(X), w2(X) D. r1(X), r2(X)

### Q37

Schedule:

TEXT

r1(X)  
w1(X)  
r2(X)  
w2(X)

Construct the precedence graph.

Is it conflict serializable?

If yes, give serial order.

### Q38 — Important

Schedule:

TEXT

r1(X)  
r2(X)  
w1(X)  
w2(X)

Find all graph-edge directions needed and determine whether the schedule is CSR.

## SECTION N — VIEW SERIALIZABILITY

### Q39

What are the three things preserved by view equivalence?

Use the mnemonic:

TEXT

IRF

### Q40

What is a blind write?

### Q41

Schedule:

TEXT

w1(X)

w2(X)

w1(X)

Determine:

1. CSR?
2. VSR?
3. If VSR, equivalent serial order?

## SECTION O — RECOVERABILITY

### Q42

Schedule:

TEXT

w1(X)

r2(X)

C1

C2

Classify:

1. Recoverable?
2. Cascadeless?
3. Strict?

### Q43

Schedule:

TEXT

w1(X)

C1

r2(X)

C2

Classify the same three properties.

#### Q44

Complete:

$\boxed{\_ \_ \_ \_ \rightarrow \_ \_ \_ \_ \rightarrow \text{Recoverable}} \}$

## SECTION P — LOCKING + 2PL

#### Q45

Fill the compatibility matrix:

| Held | Request S | Request X |
|------|-----------|-----------|
| S    | ?         | ?         |
| X    | ?         | ?         |

#### Q46

Transaction:

TEXT

S(A)  
X(B)  
unlock(A)  
S(C)

Does it follow basic 2PL?

Explain in one line.

#### Q47

What does basic 2PL guarantee?

A. Deadlock freedom B. Conflict serializability C. View serializability but not conflict serializability D. No transaction aborts

#### Q48

Differentiate in one line:

TEXT

Strict 2PL  
Rigorous 2PL  
Conservative 2PL

## SECTION Q — DEADLOCK

### Q49

In a wait-for graph:

$\backslash [T_i \rightarrow T_j \backslash]$

means what?

---

### Q50

Wait-for graph:

TEXT

|     |    |     |
|-----|----|-----|
| T 1 | !' | T 2 |
| T 2 | !' | T 3 |
| T 3 | !' | T 1 |

What does this imply?

---

### Q51

What are the four necessary conditions for deadlock?

---

### Q52

Which 2PL variant is deadlock-free because it acquires all required locks before execution?

---

## SECTION R — WAIT-DIE / WOUND-WAIT

---

Suppose:

TEXT

|    |         |
|----|---------|
| T1 | older   |
| T2 | younger |

### Q53

Under Wait-Die:

T2 requests a resource held by T1.

What happens?

---

### Q54

Under Wait-Die:

T1 requests a resource held by T2.

What happens?

---

### Q55

Under Wound-Wait:

T1 requests a resource held by T2.

---



What happens?

---

### Q56

Under Wound-Wait:

T2 requests a resource held by T1.

What happens?

---

## SECTION S — TIMESTAMP ORDERING

---

### Q57

Given:

TEXT

$TS(T1)=10$   
 $WTS(X)=20$

T1 requests:

TEXT

$r1(X)$

What happens under basic timestamp ordering?

---

### Q58

Given:

TEXT

$TS(T1)=10$   
 $RTS(X)=20$   
 $WTS(X)=5$

T1 requests:

TEXT

$w1(X)$

What happens?

---

### Q59

Given:

TEXT

$TS(T1)=10$   
 $RTS(X)=5$   
 $WTS(X)=20$

T1 requests:

TEXT

w1(X)

What happens under:

1. Basic TO
2. Thomas Write Rule

## Q60 — Final Memory Question

Complete:

TEXT

TO READ

!' c h e c k s \_ \_ \_ \_ \_

TO WRITE

!' c h e c k s \_ \_ \_ \_ \_ + \_ \_ \_ \_ \_

## HOW TO ATTEMPT THIS

Don't try to write long explanations.

You can answer compactly like:

TEXT

1-A

2-32

3-B

4-600

5-10

...

For numerical/graph questions, give the final result plus one line if needed.

## Target

TEXT

50+/60 correct

!' D B M S f o u n d a t i o n s t r o n g

45-49

!' g o o d , r e v i s e w e a k t o p i c s

35-44

!' c o n c e p t s u n d e r s t o o d b u t n e e d s P Y Q p r a c t i c e

Below 35

!' r e v i s e w e a k c h a p t e r s b e f o r e h e a v y P Y Q s

Do **not** look back at the notes while doing the first attempt if possible. This set is meant to reveal what you genuinely remember.